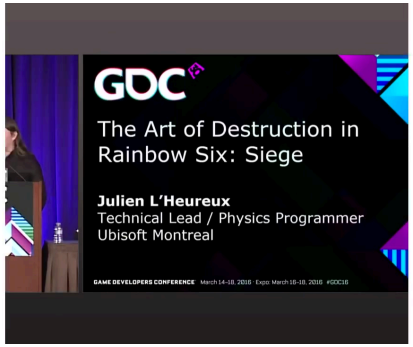
001. GDC 오프닝

영상 00:00–00:05 · 장면 캡처 00:02

이 장면의 한국어 자막



00:04–00:17

안녕하세요, 여러분. 휴대기기를 꺼 주시거나 무음으로 설정해 주세요. 발표가 끝난 뒤 설문에도 응답해 주시면 감사하겠습니다.

002. The Art of Destruction in Rainbow Six: Siege

영상 00:05–00:35 · 장면 캡처 00:07

이 장면의 한국어 자막

00:04–00:17

안녕하세요, 여러분. 휴대기기를 꺼 주시거나 무음으로 설정해 주세요. 발표가 끝난 뒤 설문에도 응답해 주시면 감사하겠습니다.

00:17–00:33

좋습니다. 다들 준비되셨나요? 저는 유비소프트 몬트리올의 테크니컬 리드 줄리앙입니다. 오늘 발표 제목은 '파괴의 기술'입니다.

00:33–00:49

레인보우 식스를 플레이해 보신 분 계신가요? 꽤 많네요. 아시다시피 레인보우 식스 시즈에는 파괴 요소가 많고, 게임플레이의 핵심 기능이라고 해도 과언이 아닙니다.

003. The Context

영상 00:35–01:05 · 장면 캡처 00:37

이 장면의 한국어 자막

00:33–00:49

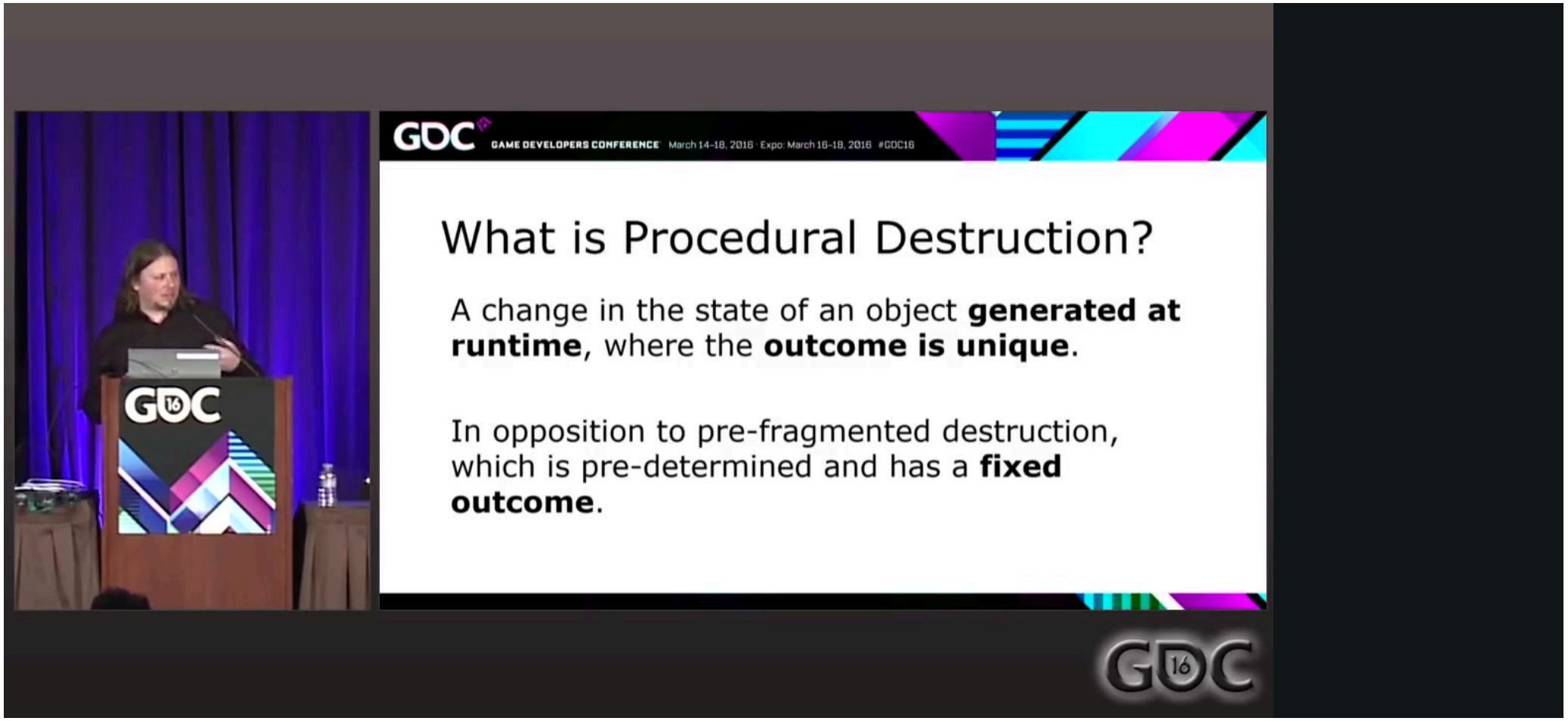
레인보우 식스를 플레이해 보신 분 계신가요? 꽤 많네요. 아시다시피 레인보우 식스 시즈에는 파괴 요소가 많고, 게임플레이의 핵심 기능이라고 해도 과언이 아닙니다.

00:49–01:05

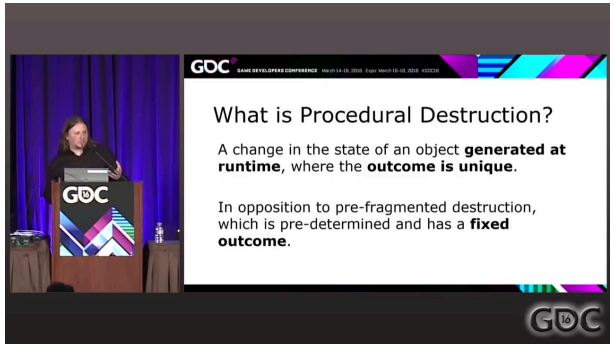
그렇다면 이걸 어떻게 만들었을까요? 그 과정을 말씀드리겠습니다. 아주 소박한 시작에서 출발해 훌륭한 기능으로 발전시켰죠. 먼저 절차적 파괴가 뭔지 보겠습니다.

004. What is Procedural Destruction?

영상 01:05–01:45 · 장면 캡처 01:07

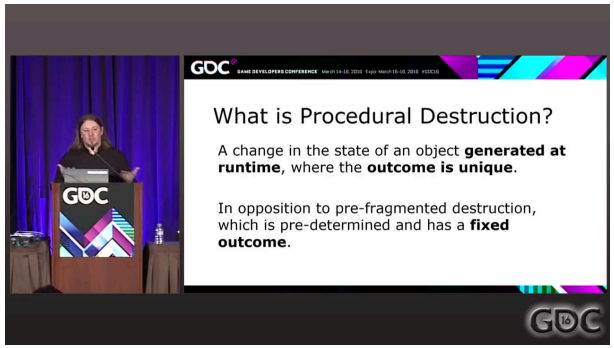


이 장면의 한국어 자막



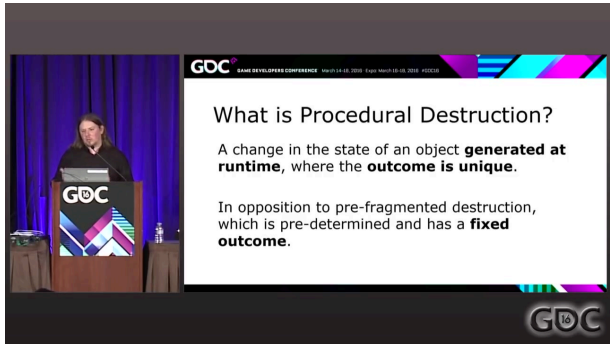
01:05–01:21

절차적 파괴에는 여러 기술이 있지만, 일반화하면 런타임에 물체의 형태를 생성하거나 변경하는 기술입니다. 입력에 따라 서로 다른 결과를 만들어낼 수 있죠.



01:21–01:36

그 결과는 매번 고유할 수 있습니다. 대부분의 게임에서 쓰는 사전 분할 파괴와는 반대입니다. 사전 분할 방식에서는 물체가 언제나 같은 조각으로 부서집니다.

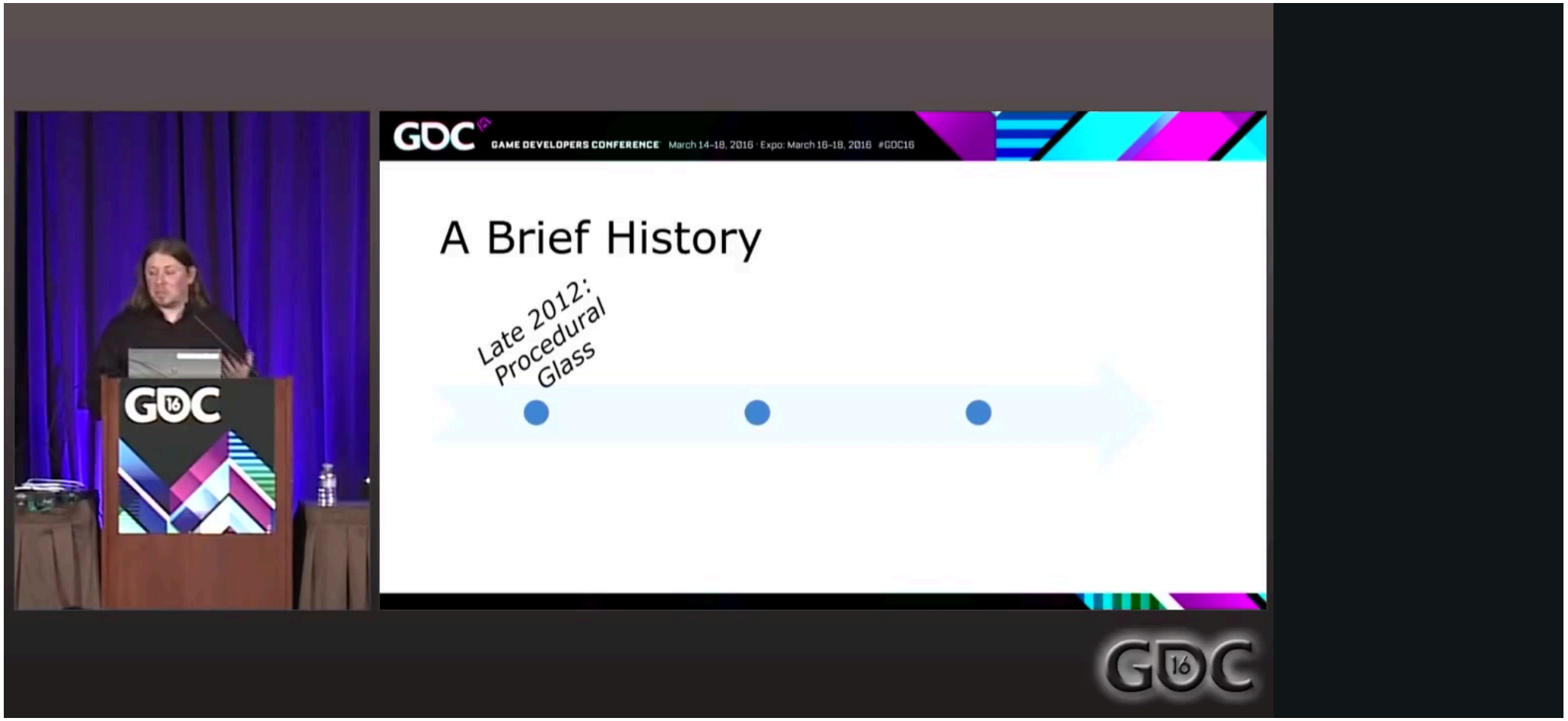


01:36–01:53

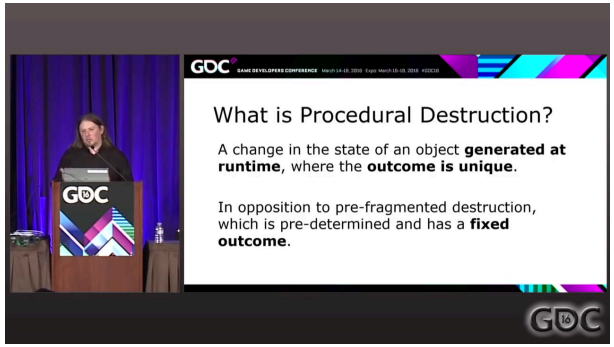
차로 들이받든, 야구방망이로 치든, 총알을 쏘든 결과가 같죠. 이야기를 거슬러 올라가면 2012년 말, 레인보우 식스를 위한 절차적 유리 파괴를 개발하라는 과제를 받았습니다.

005. A Brief History: Procedural Glass

영상 01:45–01:55 · 장면 캡처 01:47



이 장면의 한국어 자막



01:36–01:53

차로 들이받든, 야구방망이로 치든, 총알을 쏘든 결과가 같죠. 이야기를 거슬러 올라가면 2012년 말, 레인보우 식스를 위한 절차적 유리 파괴를 개발하라는 과제를 받았습니다.

01:53–02:10

이전에도 여러 파괴 기술을 만들었지만, 이번에는 이런 결과물을 냈습니다. 단순해 보여도 실제로 런타임에 새로운 지오메트리를 생성하고, 노멀과 데칼을 활용해 외형을 개선했습니다.

006. Procedural Glass Prototype

영상 01:55–02:00 · 장면 캡처 01:57

이 장면의 한국어 자막

01:53–02:10

이전에도 여러 파괴 기술을 만들었지만, 이번에는 이런 결과물을 냈습니다. 단순해 보여도 실제로 런타임에 새로운 지오메트리를 생성하고, 노멀과 데칼을 활용해 외형을 개선했습니다.

007. 유리 파괴 시연

영상 02:00–02:20 · 장면 캡처 02:02

이 장면의 한국어 자막

01:53–02:10

이전에도 여러 파괴 기술을 만들었지만, 이번에는 이런 결과물을 냈습니다. 단순해 보여도 실제로 런타임에 새로운 지오메트리를 생성하고, 노멀과 데칼을 활용해 외형을 개선했습니다.

02:10–02:26

벌써 3년 넘게 지난 일이네요. 이 결과는 프로젝트의 전환점이 됐습니다. 팀에서 아주 좋아했고, 다른 재질에도 적용하기 시작했죠. 그러자 한 가지 가능성이 보였습니다.

008. Procedural Glass Prototype: 시연 후

영상 02:20–02:30 · 장면 캡처 02:22

이 장면의 한국어 자막

02:10–02:26

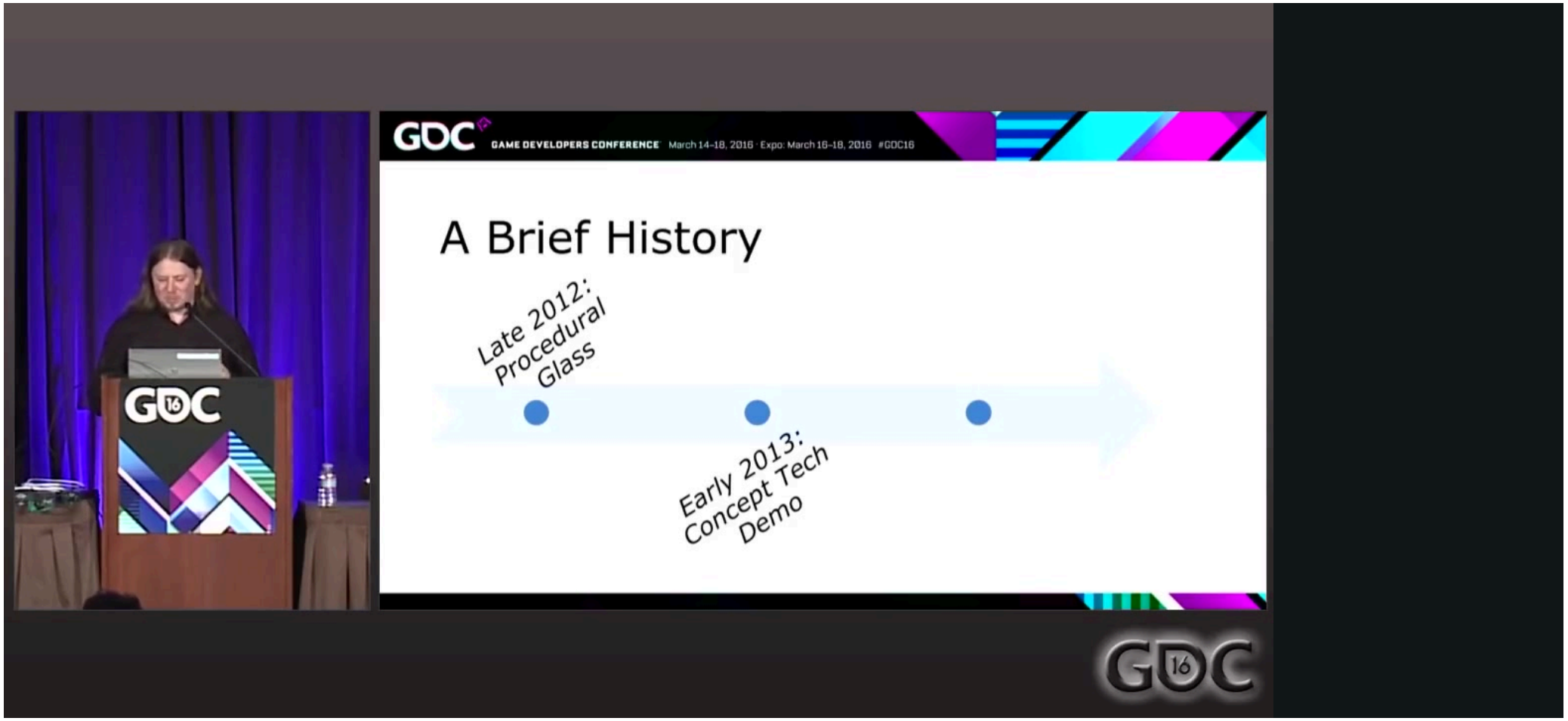
벌써 3년 넘게 지난 일이네요. 이 결과는 프로젝트의 전환점이 됐습니다. 팀에서 아주 좋아했고, 다른 재질에도 적용하기 시작했죠. 그러자 한 가지 가능성이 보였습니다.

02:26–02:42

파괴를 중심으로 게임을 만들 수도 있겠다는 생각이었죠. 그래서 인력과 자원을 지원받아 2013년 초에 기술 콘셉트 데모를 만들었습니다. 꽤 멋진 데모였어요.

009. A Brief History: Concept Tech Demo

영상 02:30–02:35 · 장면 캡처 02:32



이 장면의 한국어 자막

02:26–02:42

파괴를 중심으로 게임을 만들 수도 있겠다는 생각이었죠. 그래서 인력과 자원을 지원받아 2013년 초에 기술 콘셉트 데모를 만들었습니다. 꽤 멋진 데모였어요.

010. Concept Tech Demo

영상 02:35–02:45 · 장면 캡처 02:37

이 장면의 한국어 자막

02:26–02:42

파괴를 중심으로 게임을 만들 수도 있겠다는 생각이었죠. 그래서 인력과 자원을 지원받아 2013년 초에 기술 콘셉트 데모를 만들었습니다. 꽤 멋진 데모였어요.

02:42–02:57

몬트리올 스튜디오에서도 오랫동안 파괴 기술의 기준이 되었고, 많은 사람이 참여했습니다. 3년 전 데모라는 걸 생각하면 지금 봐도 꽤 괜찮아 보이죠.

011. 기술 데모: 절차적 파괴

영상 02:45–03:00 · 장면 캡처 02:47

이 장면의 한국어 자막

02:42–02:57

몬트리올 스튜디오에서도 오랫동안 파괴 기술의 기준이 되었고, 많은 사람이 참여했습니다. 3년 전 데모라는 걸 생각하면 지금 봐도 꽤 괜찮아 보이죠.

02:57–03:15

당시에는 3D 불리언 연산을 활용한 파괴가 있었고, 나무와 벽돌, 창문, 바닥, 천장까지 부술 수 있었습니다. 폭발과 총격, 다양한 파티클 효과도 있었죠.

012. 기술 데모: 재질의 반응

영상 03:00–03:25 · 장면 캡처 03:02

이 장면의 한국어 자막

02:57–03:15

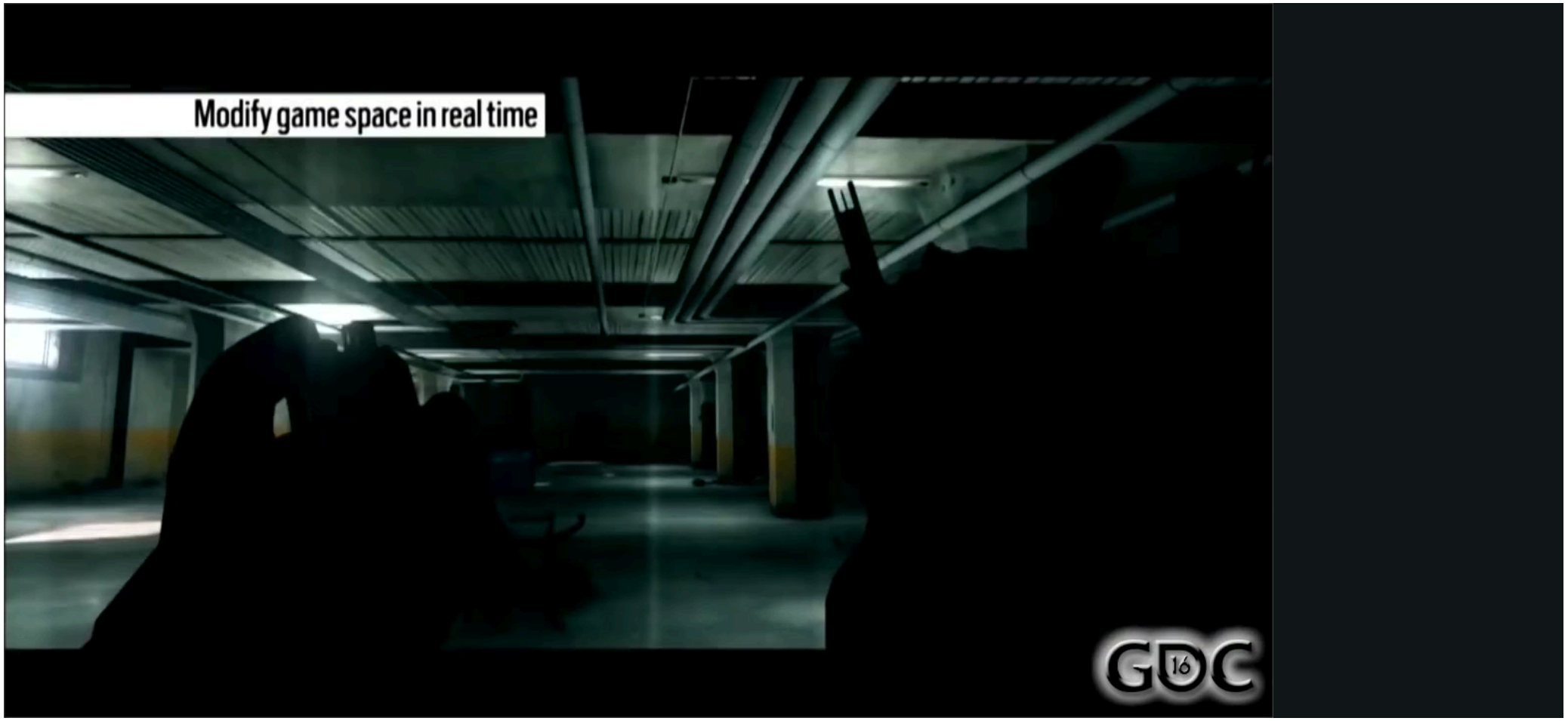
당시에는 3D 불리언 연산을 활용한 파괴가 있었고, 나무와 벽돌, 창문, 바닥, 천장까지 부술 수 있었습니다. 폭발과 총격, 다양한 파티클 효과도 있었죠.

03:15–03:29

상당히 멋졌습니다. 사전 분할 파괴도 함께 활용했고, 잘 작동했죠. 하지만 당시에는 60FPS를 달성하지 못했고, 안정적이지도 않았으며, 온라인을 지원하지도 않았습니다.

013. 기술 데모: 공간의 변화

영상 03:25–03:55 · 장면 캡처 03:27



이 장면의 한국어 자막

03:15–03:29

상당히 멋졌습니다. 사전 분할 파괴도 함께 활용했고, 잘 작동했죠. 하지만 당시에는 60FPS를 달성하지 못했고, 안정적이지도 않았으며, 온라인을 지원하지도 않았습니다.

03:29–03:46

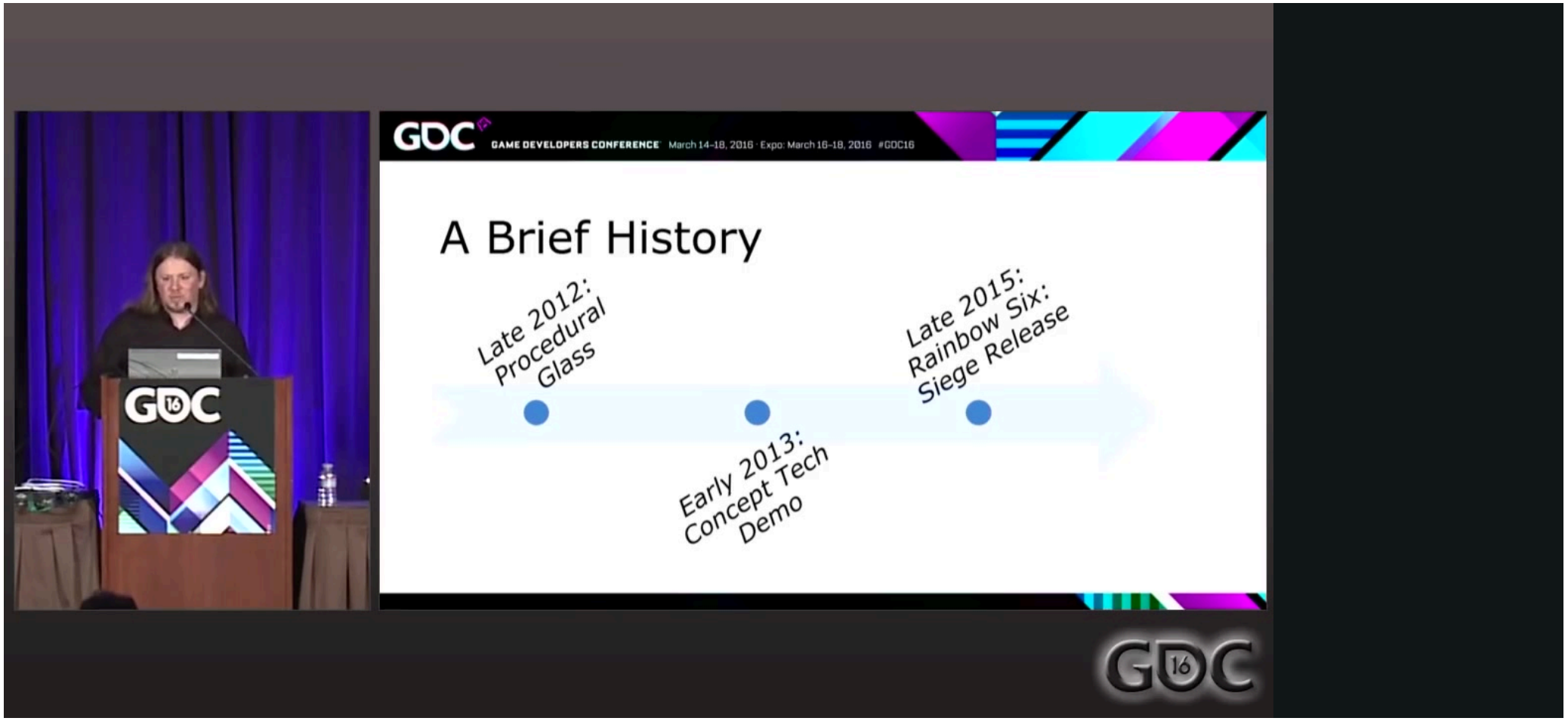
제작 도구도 사실상 없었습니다. 실제 게임 기능으로 만들려면 해야 할 일이 많았죠. 화면처럼 멀쩡한 공간에서 총을 쏘고 파괴하면 완전히 달라진 모습이 됩니다.

03:46–04:03

상상력을 자극하지 않나요? 이제 2015년 말로 넘어가 보죠. 레인보우 식스 시즈가 출시됐고 좋은 성과를 거뒀습니다. 파괴 기능도 훌륭하게 작동했죠.

014. A Brief History: Siege Release

영상 03:55–04:15 · 장면 캡처 03:57



이 장면의 한국어 자막

03:46–04:03

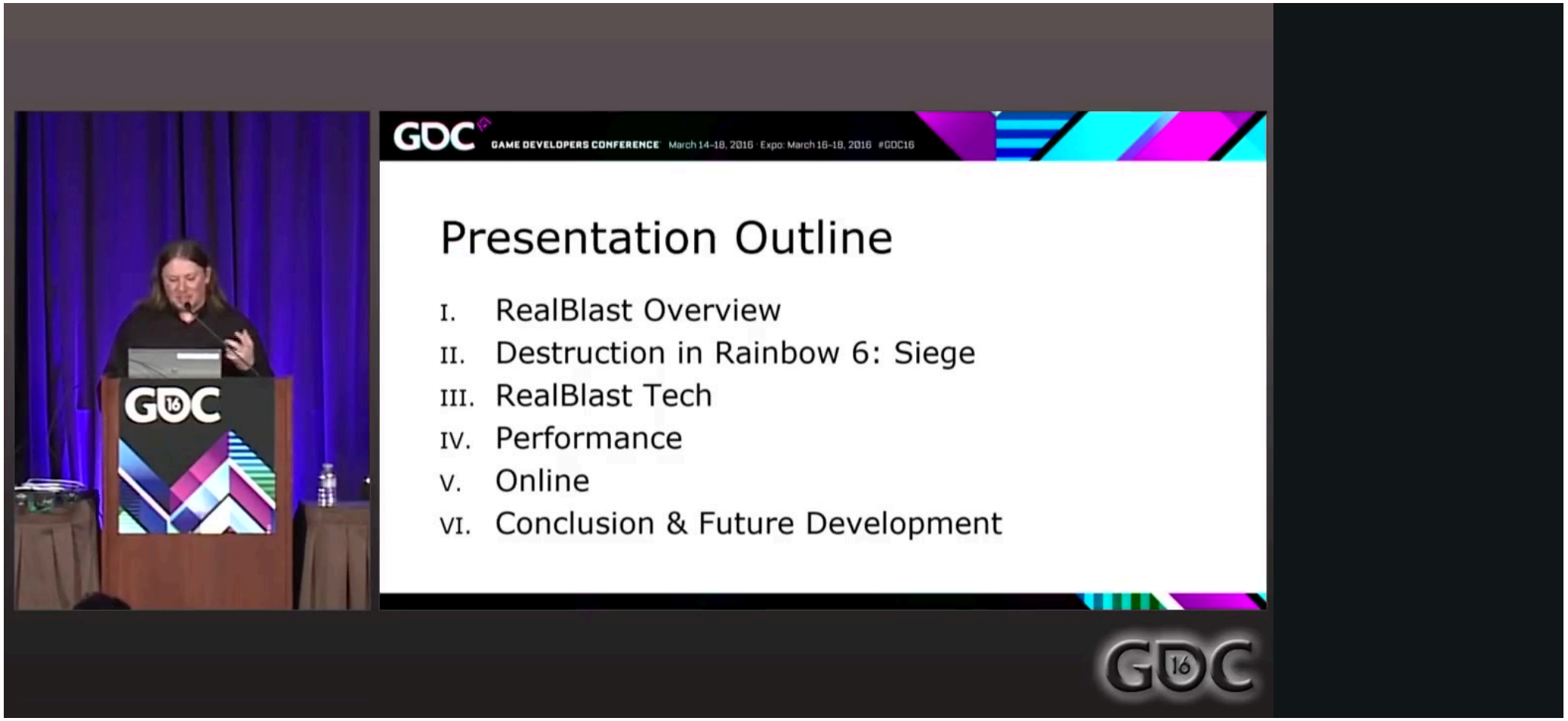
상상력을 자극하지 않나요? 이제 2015년 말로 넘어가 보죠. 레인보우 식스 시즈가 출시됐고 좋은 성과를 거뒀습니다. 파괴 기능도 훌륭하게 작동했죠.

04:03–04:20

흥미롭게도 그 3년 동안 기술의 기본 개념 자체는 크게 바뀌지 않았습니다. 하지만 실제 게임에 넣으려면 넘어야 할 단계가 많았고, 오늘은 바로 그 이야기를 하겠습니다.

015. Presentation Outline

영상 04:15–04:45 · 장면 캡처 04:17



이 장면의 한국어 자막

04:03–04:20

흥미롭게도 그 3년 동안 기술의 기본 개념 자체는 크게 바뀌지 않았습니다. 하지만 실제 게임에 넣으려면 넘어야 할 단계가 많았고, 오늘은 바로 그 이야기를 하겠습니다.

04:20–04:35

먼저 저희 팀과 제품인 리얼블라스트를 소개하고, 파괴가 레인보우 식스 시즈의 게임플레이에 어떤 영향을 주었는지 살펴보겠습니다. 다른 기능에도 적용할 수 있는 관점이죠.

04:35–04:50

그다음 다양한 파괴 기술을 자세히 알아보고, 성능과 온라인 동기화를 다루겠습니다. 시즈에서는 이 두 가지가 특히 중요합니다. 마지막으로 결론을 말씀드리죠.

016. RealBlast Overview

영상 04:45–04:50 · 장면 캡처 04:47

이 장면의 한국어 자막

04:35–04:50

그다음 다양한 파괴 기술을 자세히 알아보고, 성능과 온라인 동기화를 다루겠습니다. 시즈에서는 이 두 가지가 특히 중요합니다. 마지막으로 결론을 말씀드리죠.

017. RealBlast 팀

영상 04:50–04:55 · 장면 캡처 04:52

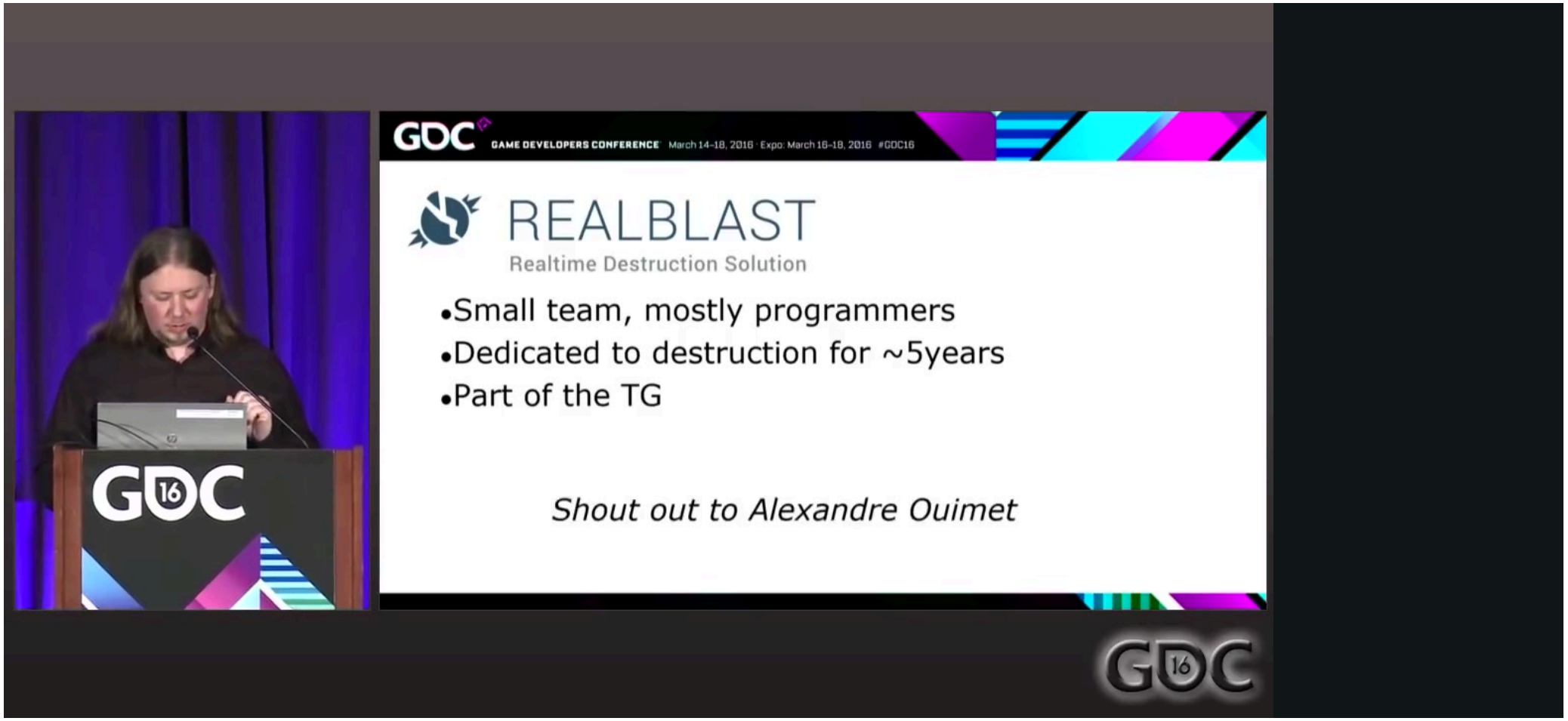
이 장면의 한국어 자막

04:50–05:05

리얼블라스트는 주로 프로그래머로 구성된 작은 팀입니다. 5년 넘게 개발해 왔고 기술 그룹에 속해 있습니다. 발표 준비를 도와준 동료 알렉시스에게도 감사드립니다.

018. RealBlast 팀: 협업

영상 04:55–05:05 · 장면 캡처 04:57



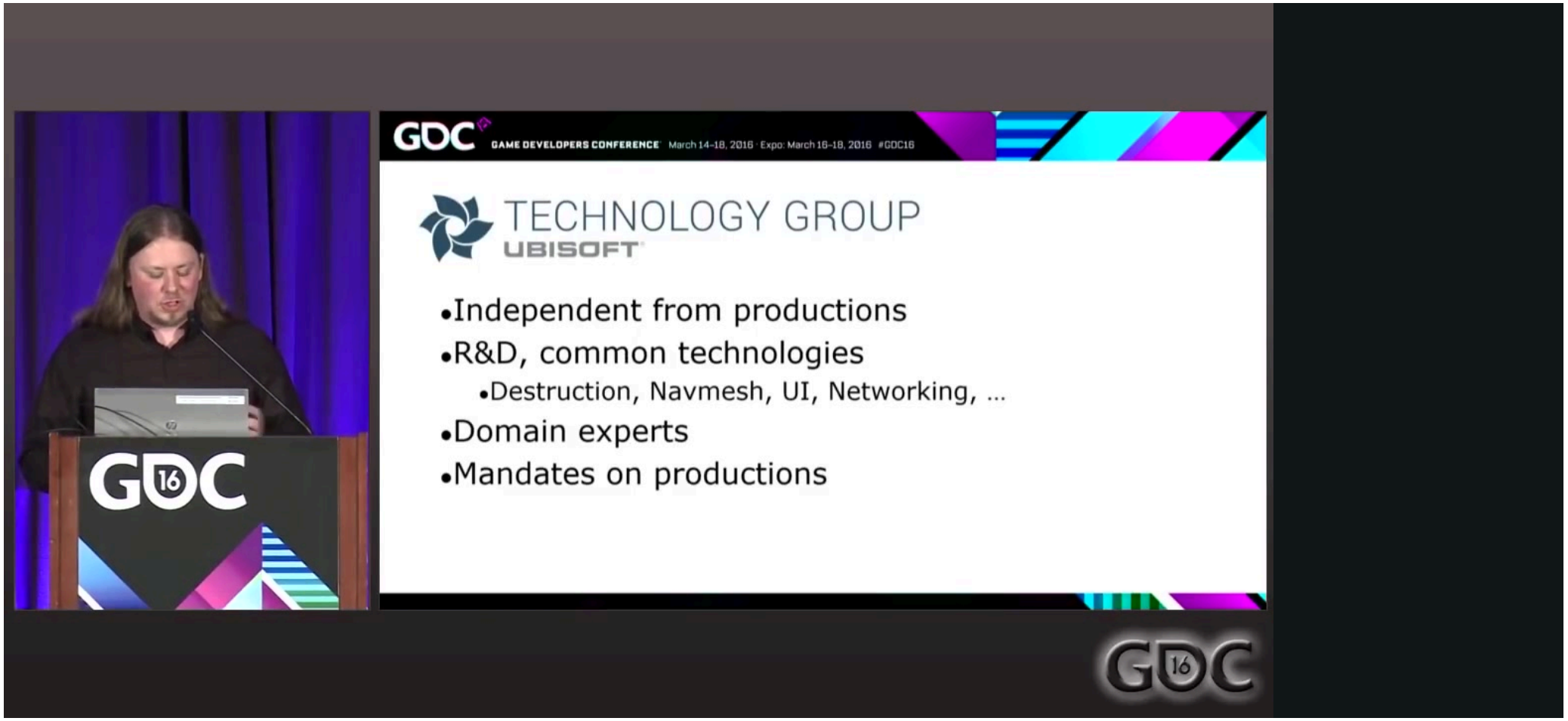
이 장면의 한국어 자막

04:50–05:05

리얼블라스트는 주로 프로그래머로 구성된 작은 팀입니다. 5년 넘게 개발해 왔고 기술 그룹에 속해 있습니다. 발표 준비를 도와준 동료 알렉시스에게도 감사드립니다.

019. Technology Group

영상 05:05–05:35 · 장면 캡처 05:07



이 장면의 한국어 자막

05:05–05:21

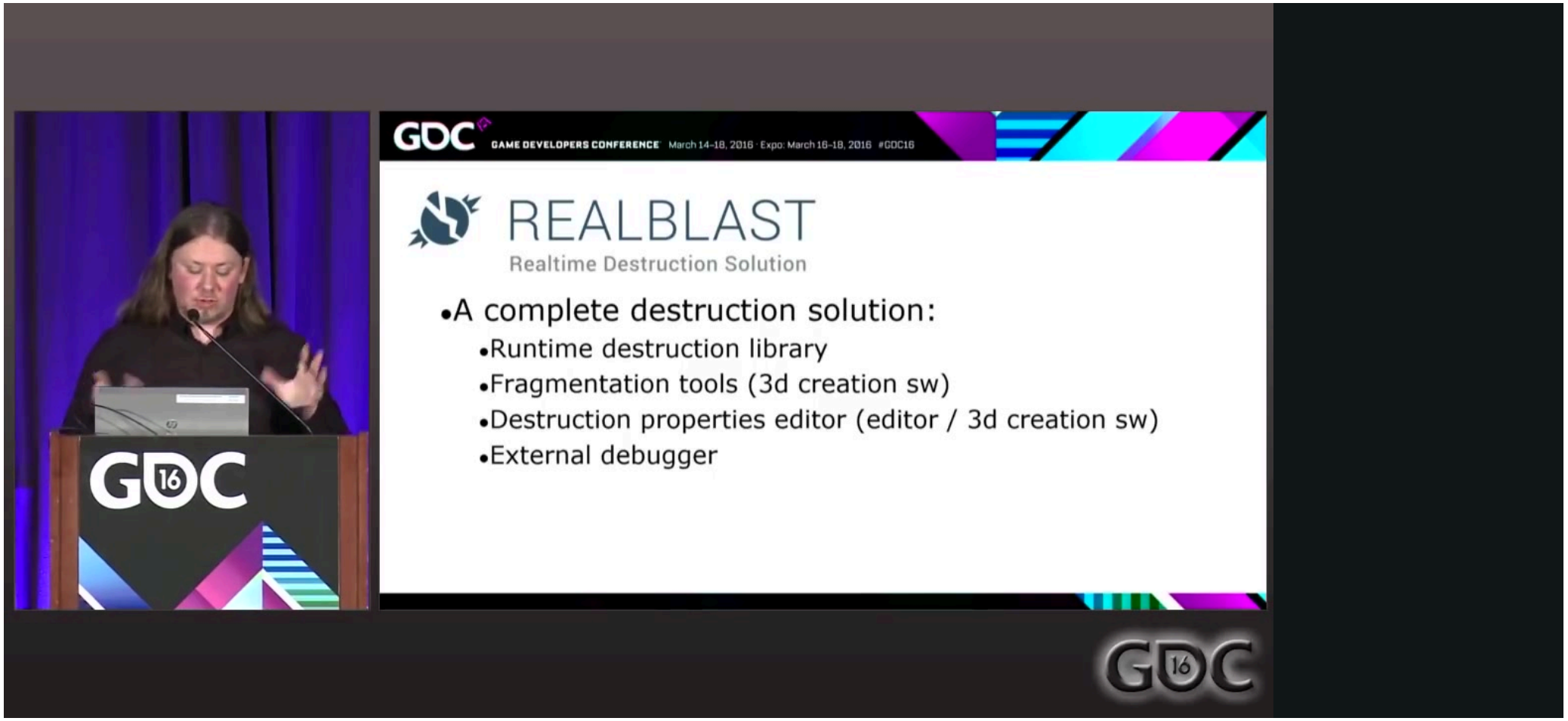
유비소프트의 기술 그룹은 여러 제작팀과 별도로 연구개발을 수행하고, 공통 기술을 개발해 팀 간 공유를 촉진하는 조직입니다. 파괴나 동적 내비게이션 메시 같은 분야에 전문성을 쌓죠.

05:21–05:38

네트워킹과 애니메이션 등도 연구합니다. 이렇게 축적한 전문 지식을 바탕으로 각 게임 제작팀의 요청을 수행하고, 궁극적으로 더 크고 좋은 게임을 만들도록 돕습니다.

020. RealBlast: 솔루션과 도구

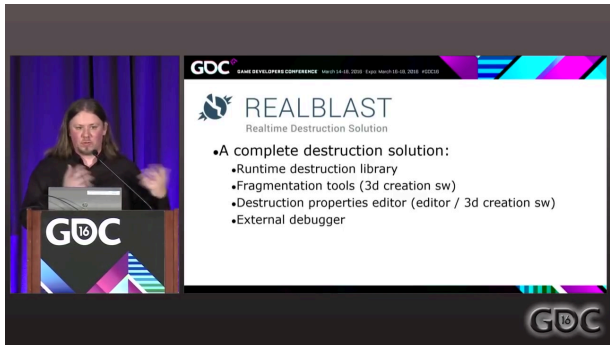
영상 05:35–06:30 · 장면 캡처 05:37



이 장면의 한국어 자막

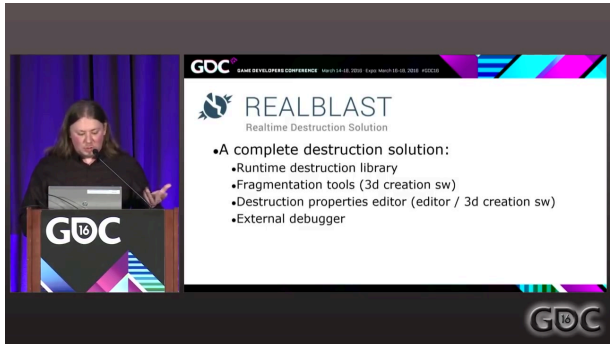
05:21–05:38

네트워킹과 애니메이션 등도 연구합니다. 이렇게 축적한 전문 지식을 바탕으로 각 게임 제작팀의 요청을 수행하고, 궁극적으로 더 크고 좋은 게임을 만들도록 돕습니다.



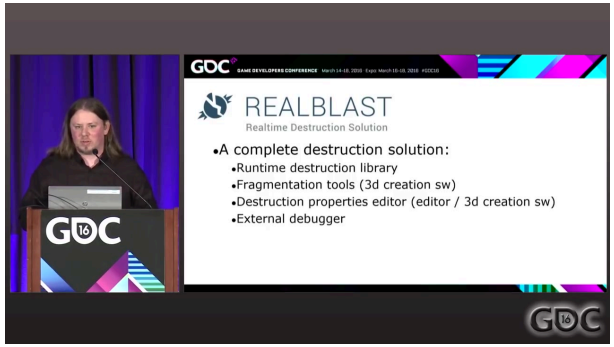
05:38–05:53

리얼블라스트는 완전한 파괴 솔루션입니다. 사전 분할 파괴와 절차적 파괴를 런타임에 모두 지원합니다. 엔진 기능뿐 아니라 콘텐츠 제작용 도구도 포함하죠.



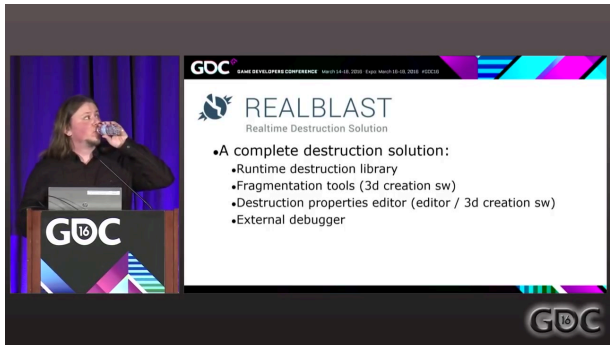
05:53–06:09

3ds Max나 Maya에서 쓰는 조각 분할 도구, 물리 및 파괴 기법을 설정하는 속성 편집기, 상태를 추적하는 외부 디버거도 있습니다. 복잡한 지오메트리를 디버깅할 때는 이런 도구가 매우 중요합니다.



06:09–06:25

무엇이 잘못됐는지 눈에 보이지 않으면 디버깅이 정말 어렵거든요. 다만 도구 세 가지에 대한 자세한 설명은 오늘 발표에서 생략하고, 관심 있는 분을 위해 유비소프트 부스에서 다루겠습니다.

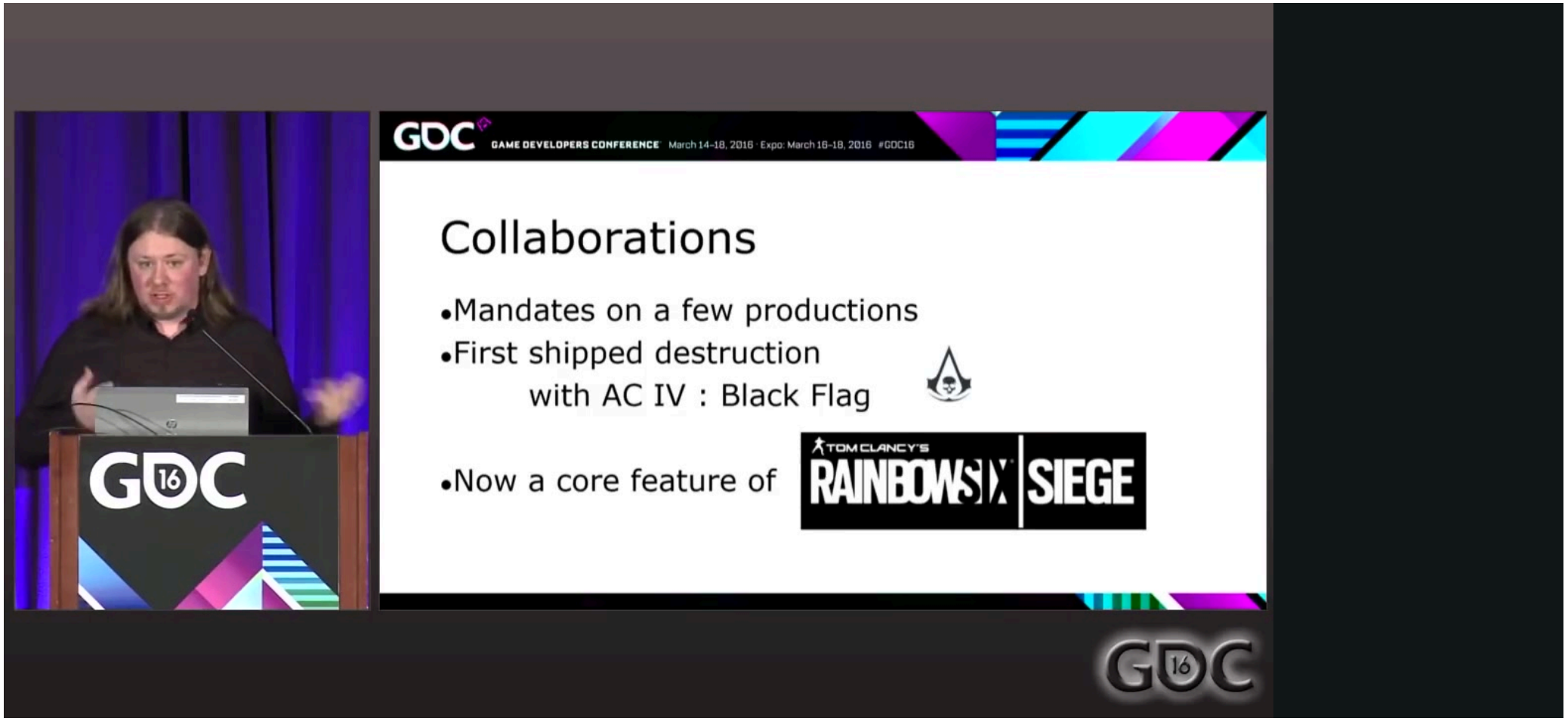


06:25–06:41

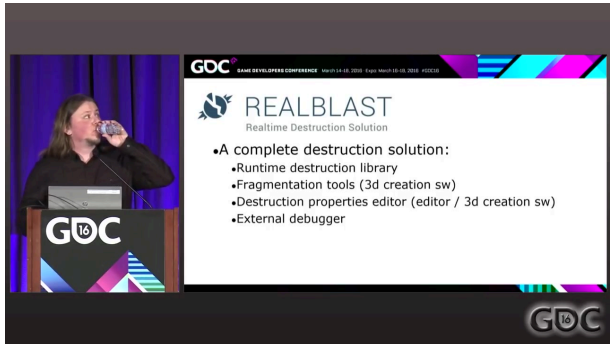
협업 이야기로 돌아가면, 여러 프로젝트에 기술을 적용했고 성공한 것도, 그렇지 않은 것도 있었습니다. 실제 출시작에 처음 파괴 기술을 적용한 건 어쌔신 크리드 4: 블랙 플래그였습니다.

021. Collaborations

영상 06:30–06:55 · 장면 캡처 06:32



이 장면의 한국어 자막



06:25–06:41

협업 이야기로 돌아가면, 여러 프로젝트에 기술을 적용했고 성공한 것도, 그렇지 않은 것도 있었습니다. 실제 출시작에 처음 파괴 기술을 적용한 건 어쌔신 크리드 4: 블랙 플래그였습니다.

06:41–06:58

해상 요새에서 사용했고 반응도 좋았습니다. 그리고 최근에는 레인보우 식스 시즈의 핵심 기능이 되었죠. 게임을 모르시는 분들을 위해 플레이 방식을 간단하게 설명하겠습니다.

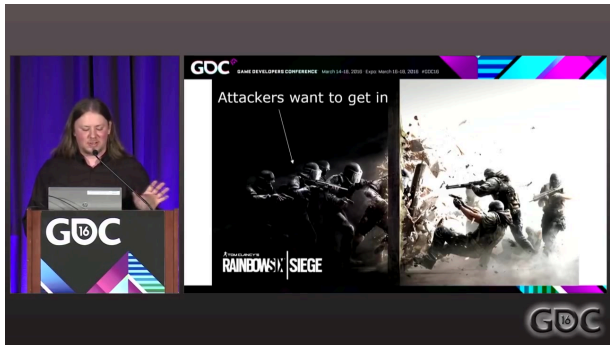
022. Destruction in Rainbow Six: Siege

영상 06:55–07:00 · 장면 캡처 06:57

이 장면의 한국어 자막

06:41–06:58

해상 요새에서 사용했고 반응도 좋았습니다. 그리고 최근에는 레인보우 식스 시즈의 핵심 기능이 되었죠. 게임을 모르시는 분들을 위해 플레이 방식을 간단하게 설명하겠습니다.

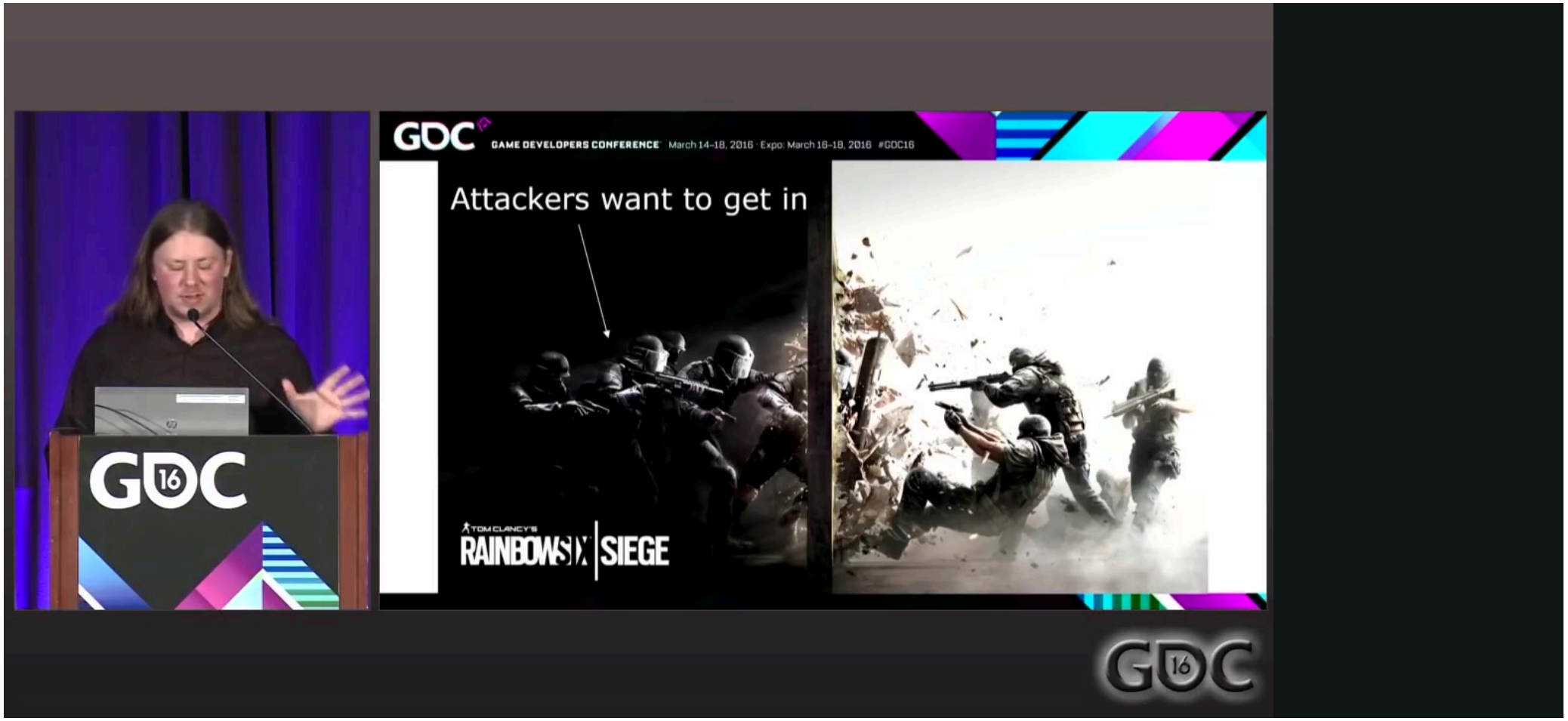


06:58–07:13

한쪽에는 공격팀이 있습니다. 안으로 진입해 적을 제압하거나 인질을 구출해야 하고, 파괴를 비롯한 다양한 수단을 활용합니다. 반대편에는 방어팀이 있죠.

023. 공격팀

영상 07:00–07:10 · 장면 캡처 07:02



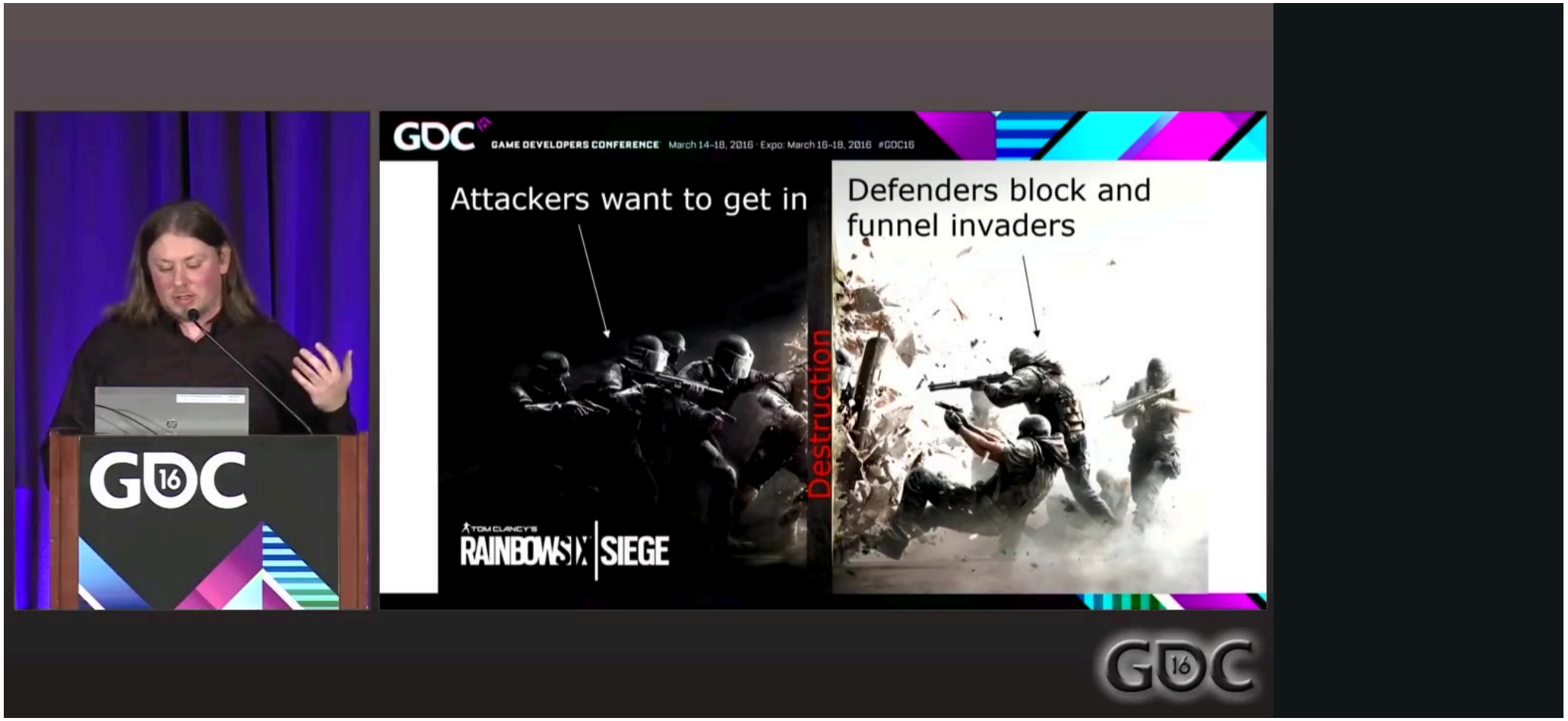
이 장면의 한국어 자막

06:58–07:13

한쪽에는 공격팀이 있습니다. 안으로 진입해 적을 제압하거나 인질을 구출해야 하고, 파괴를 비롯한 다양한 수단을 활용합니다. 반대편에는 방어팀이 있죠.

024. 공격팀과 방어팀

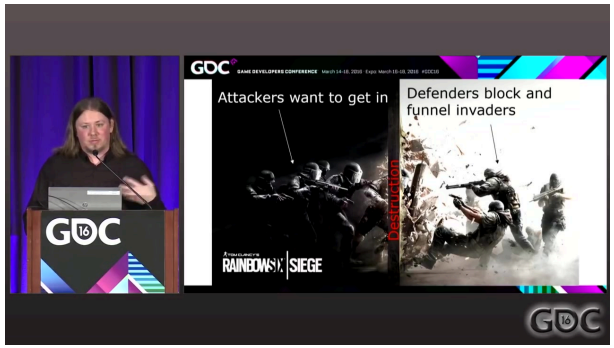
영상 07:10–07:20 · 장면 캡처 07:12



이 장면의 한국어 자막

06:58–07:13

한쪽에는 공격팀이 있습니다. 안으로 진입해 적을 제압하거나 인질을 구출해야 하고, 파괴를 비롯한 다양한 수단을 활용합니다. 반대편에는 방어팀이 있죠.

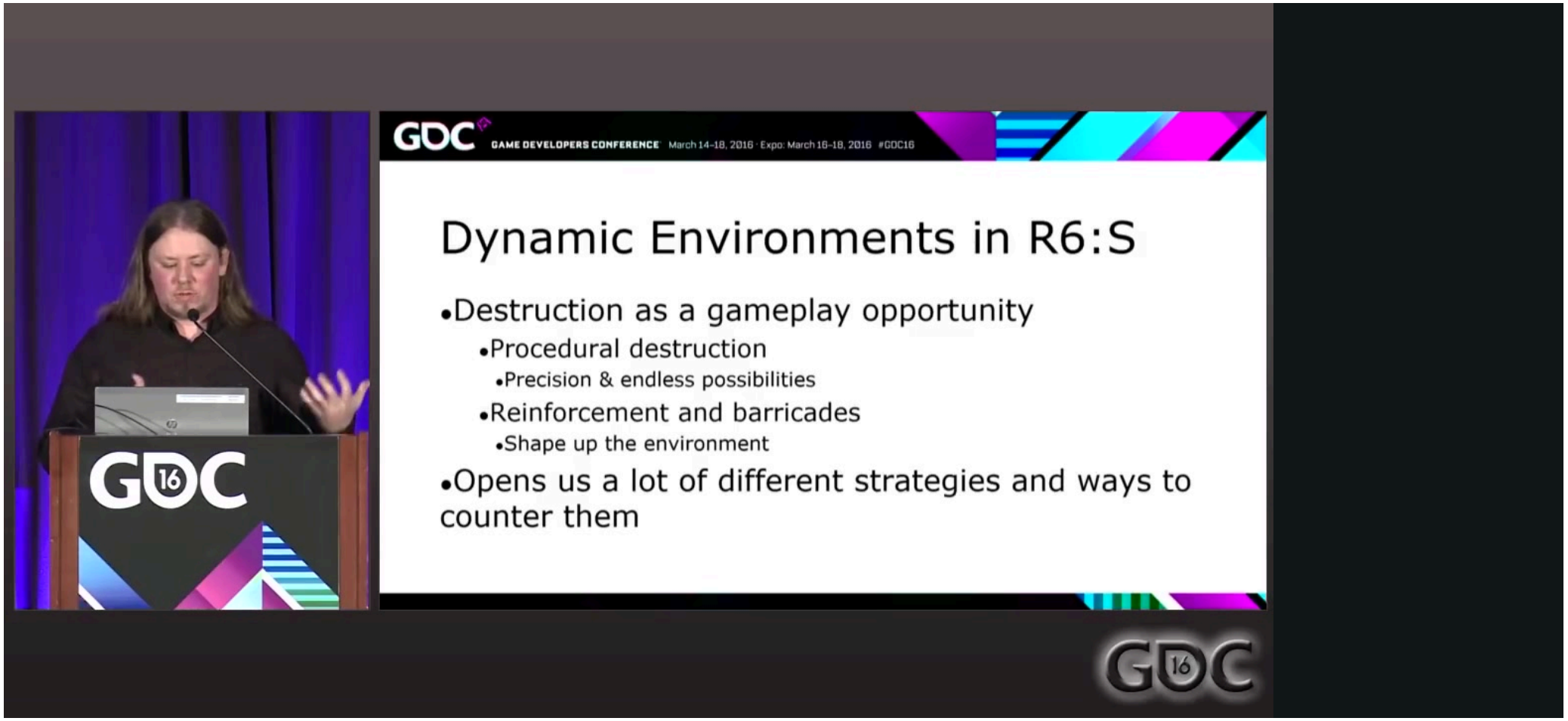


07:13–07:30

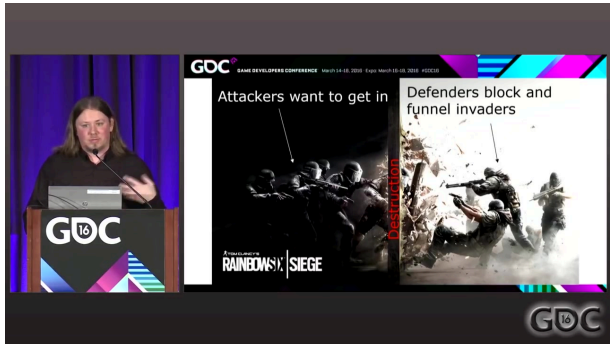
방어팀은 공격자를 막고 진입로를 통제합니다. 두 진영의 역할이 비대칭적입니다. 공격팀에는 파괴가 새로운 경로를 만들거나 사격 구멍을 뚫는 수단이 됩니다.

025. Dynamic Environments in R6:S

영상 07:20–07:55 · 장면 캡처 07:22

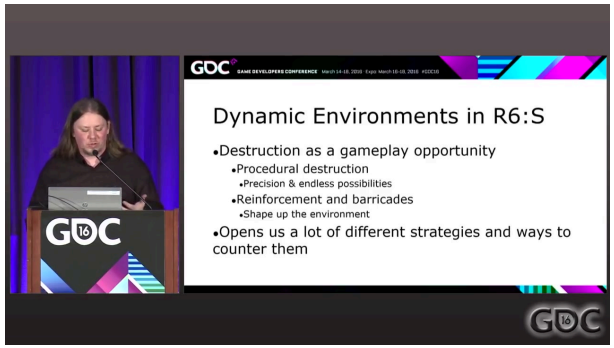


이 장면의 한국어 자막



07:13–07:30

방어팀은 공격자를 막고 진입로를 통제합니다. 두 진영의 역할이 비대칭적입니다. 공격팀에는 파괴가 새로운 경로를 만들거나 사격 구멍을 뚫는 수단이 됩니다.



07:30–07:47

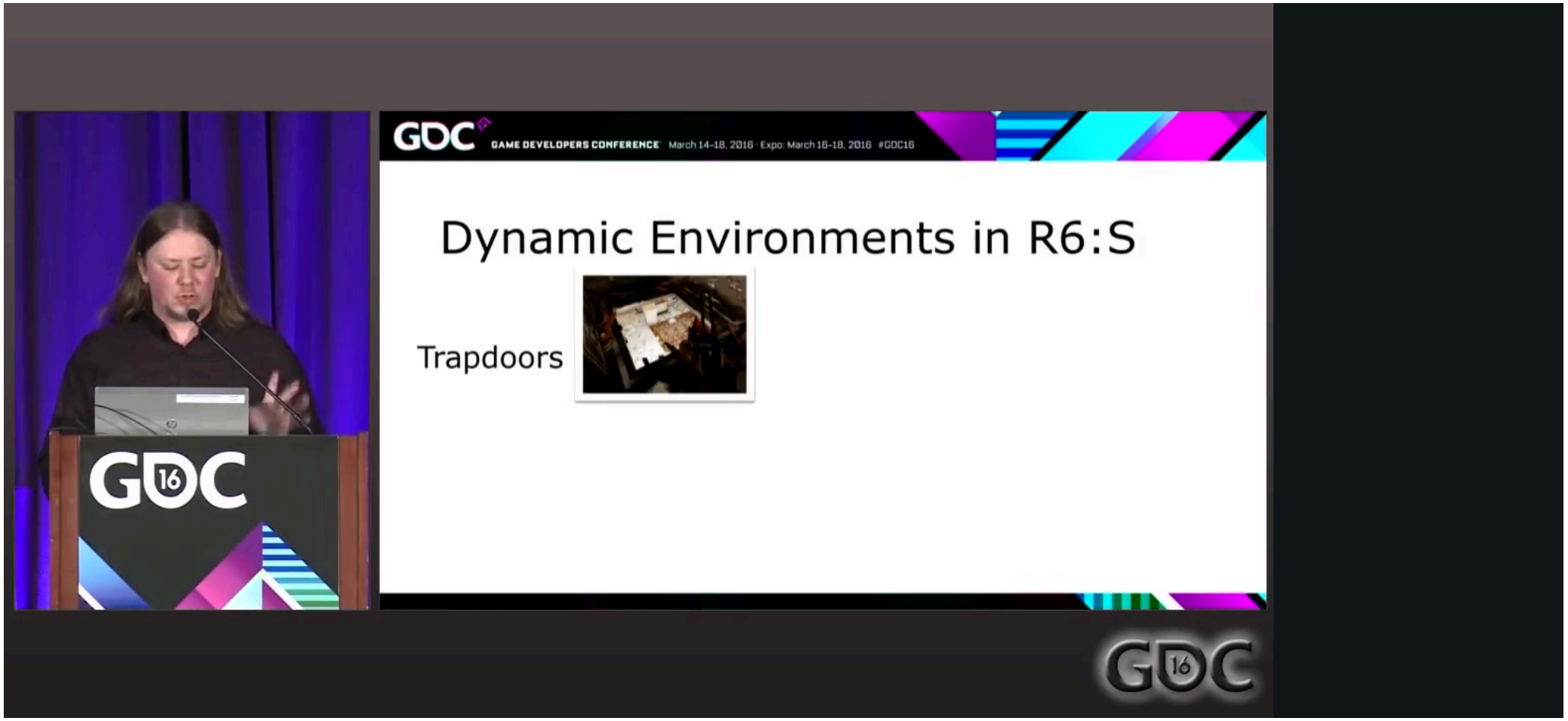
반면 방어팀은 바리케이드를 설치하고 벽을 강화해 공격자의 진입을 막습니다. 이런 선택지가 게임 플레이를 풍부하게 하고 매 판을 다르게 만들어 줍니다.

07:47–08:04

게임의 파괴 가능한 환경에는 몇 가지 대표적인 형태가 있습니다. 수직 이동을 제어하는 해치, 시야를 가리고 파괴 시 소리를 내는 바리케이드가 있죠.

026. Dynamic Environments: Trapdoors

영상 07:55–08:00 · 장면 캡처 07:57



이 장면의 한국어 자막

07:47–08:04

게임의 파괴 가능한 환경에는 몇 가지 대표적인 형태가 있습니다. 수직 이동을 제어하는 해치, 시야를 가리고 파괴 시 소리를 내는 바리케이드가 있죠.

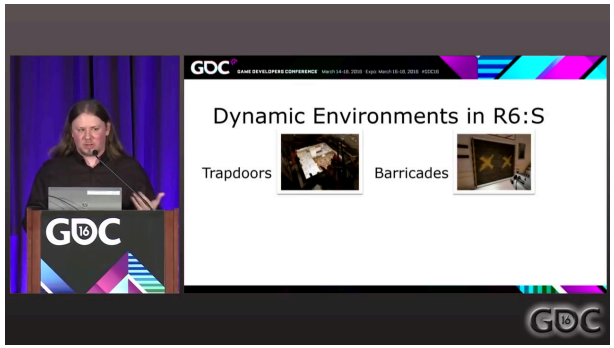
027. Dynamic Environments: Barricades

영상 08:00–08:10 · 장면 캡처 08:02

이 장면의 한국어 자막

07:47–08:04

게임의 파괴 가능한 환경에는 몇 가지 대표적인 형태가 있습니다. 수직 이동을 제어하는 해치, 시야를 가리고 파괴 시 소리를 내는 바리케이드가 있죠.



08:04–08:21

총알이나 수류탄은 통과하지만 플레이어는 지나갈 수 없는 바닥도 있습니다. 그리고 시즈를 상징하는 파괴 요소라면 역시 뚫고 지나갈 수 있는 벽일 겁니다.

028. Dynamic Environments: LOS Floors

영상 08:10–08:15 · 장면 캡처 08:12

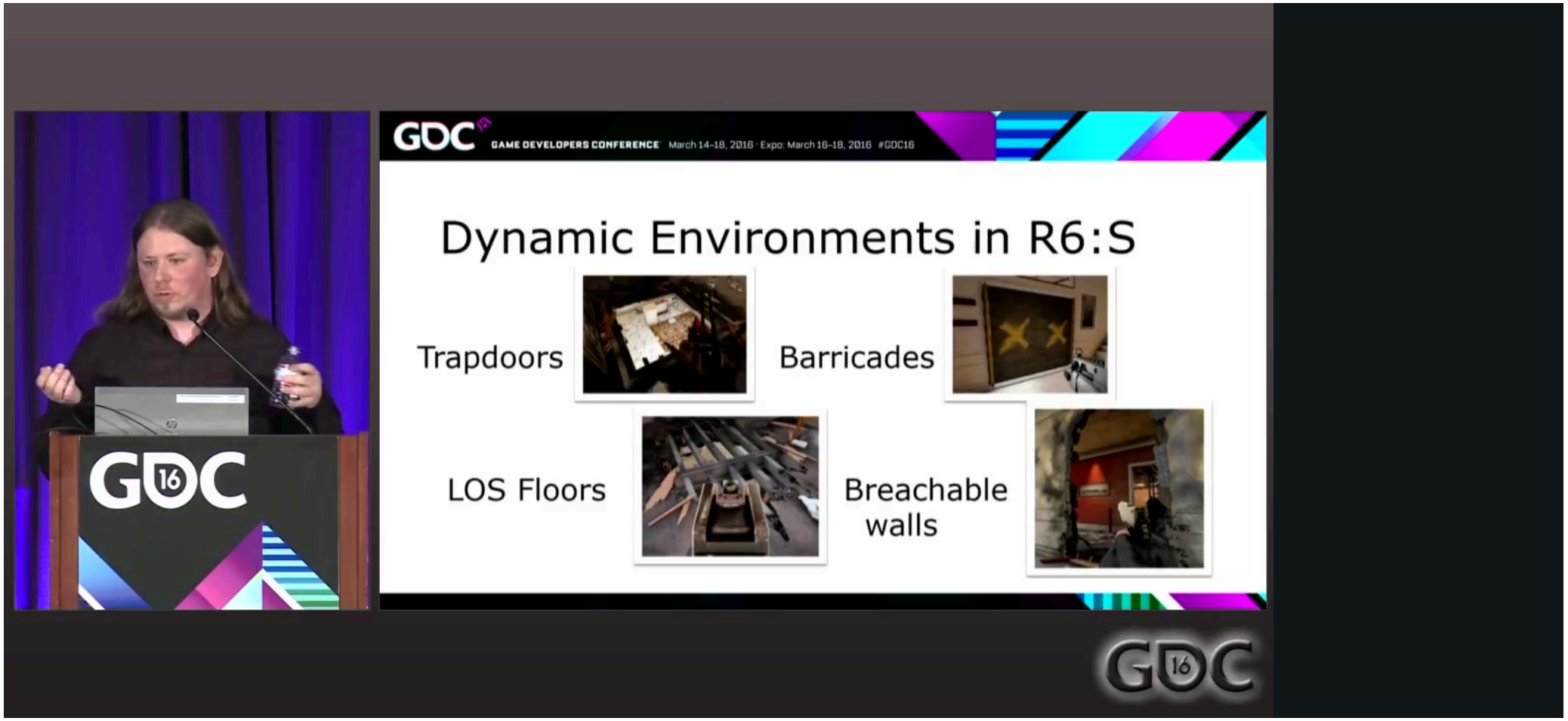
이 장면의 한국어 자막

08:04–08:21

총알이나 수류탄은 통과하지만 플레이어는 지나갈 수 없는 바닥도 있습니다. 그리고 시즈를 상징하는 파괴 요소라면 역시 뚫고 지나갈 수 있는 벽일 겁니다.

029. Dynamic Environments: Breachable Walls

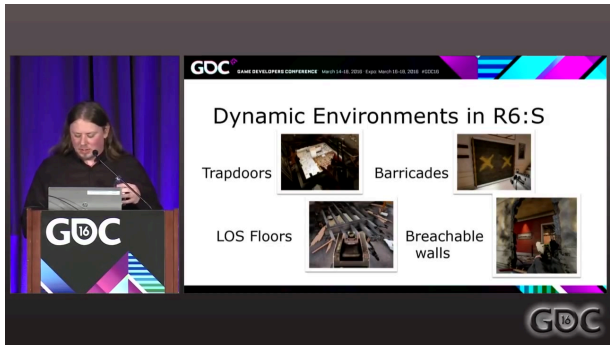
영상 08:15–08:30 · 장면 캡처 08:17



이 장면의 한국어 자막

08:04–08:21

총알이나 수류탄은 통과하지만 플레이어는 지나갈 수 없는 바닥도 있습니다. 그리고 시즈를 상징하는 파괴 요소라면 역시 뚫고 지나갈 수 있는 벽일 겁니다.

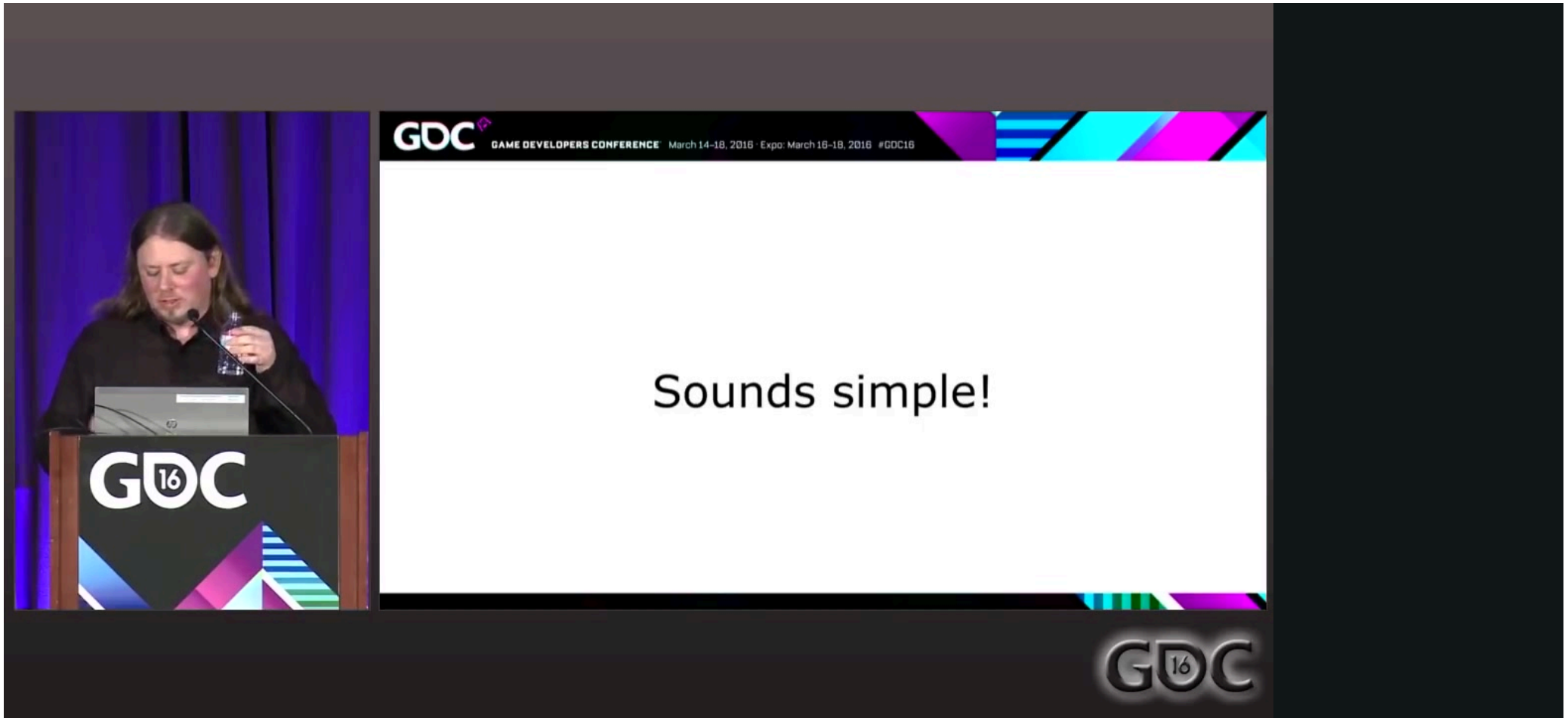


08:21–08:34

이렇게 들으면 기존 파괴 기술을 게임에 넣기만 하면 될 것 같죠. 3년 전에도 잘 작동했으니 그냥 붙이면 되지 않을까요?

030. Sounds simple!

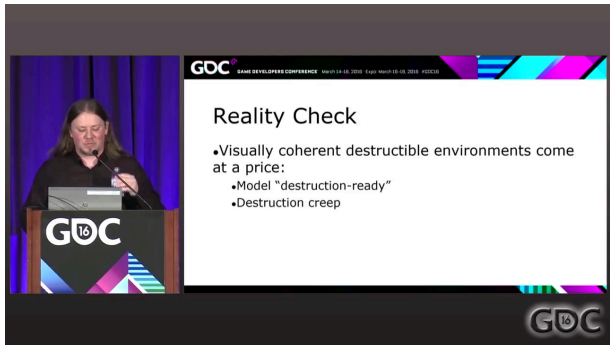
영상 08:30–08:35 · 장면 캡처 08:32



이 장면의 한국어 자막

08:21–08:34

이렇게 들으면 기존 파괴 기술을 게임에 넣기만 하면 될 것 같죠. 3년 전에도 잘 작동했으니 그냥 붙이면 되지 않을까요?

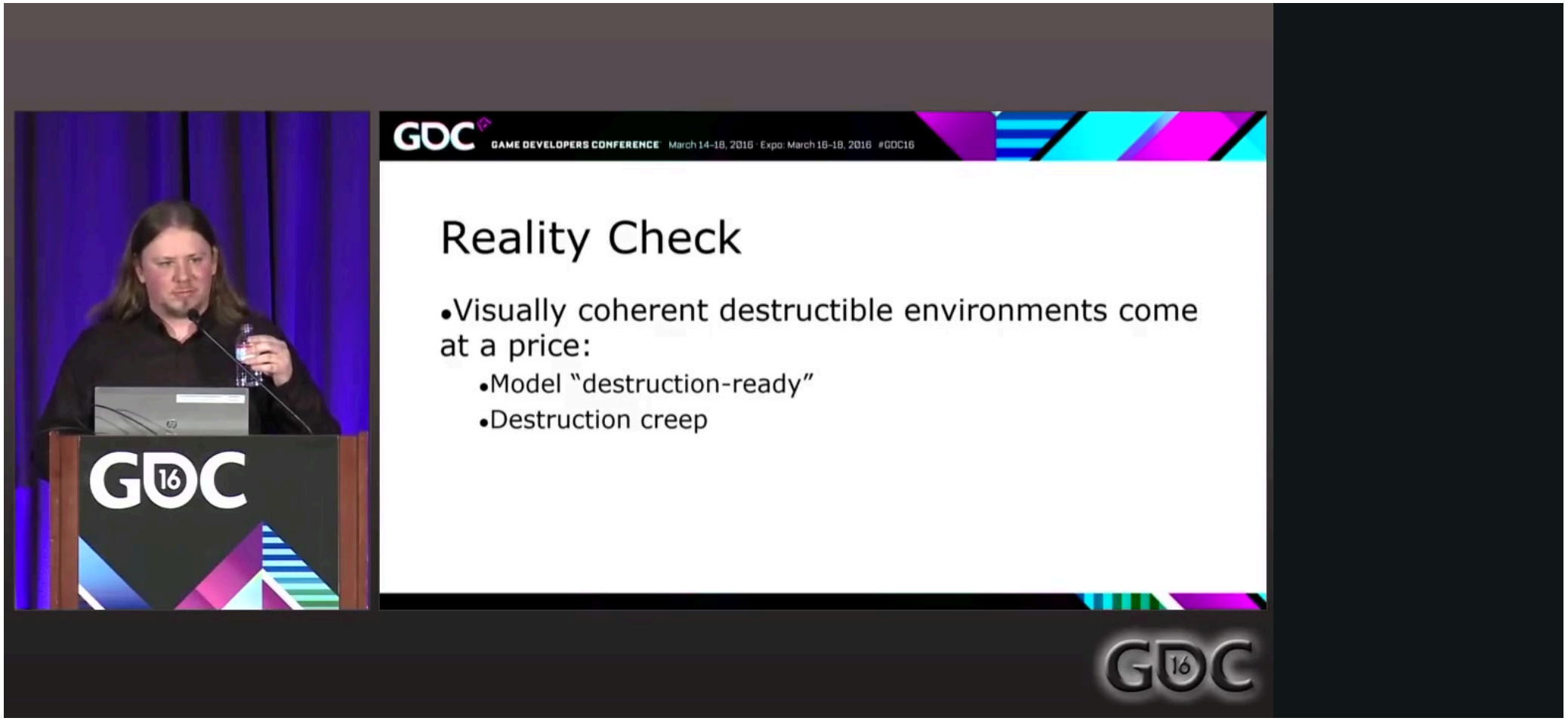


08:34–08:48

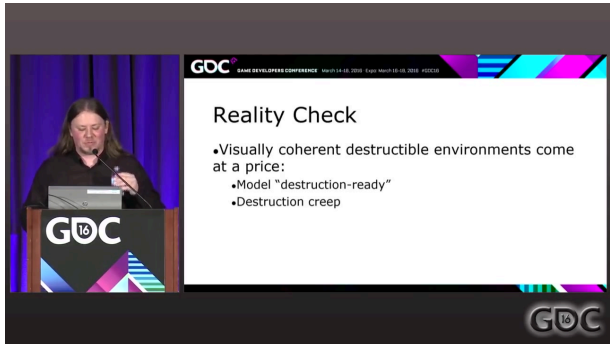
하지만 그렇게 간단하지 않습니다. 잠깐 물 좀 마시겠습니다. 실제로는 꽤 큰 대가를 치러야 합니다.

031. Reality Check: 제작 비용

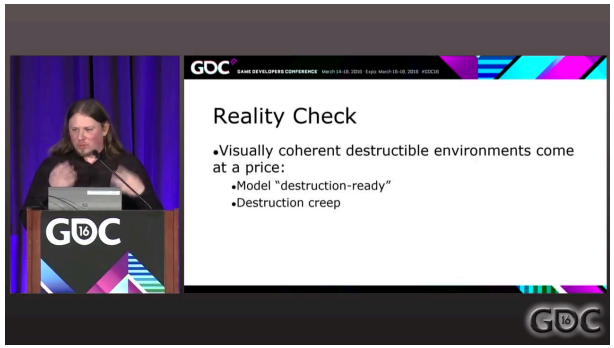
영상 08:35–09:25 · 장면 캡처 08:37



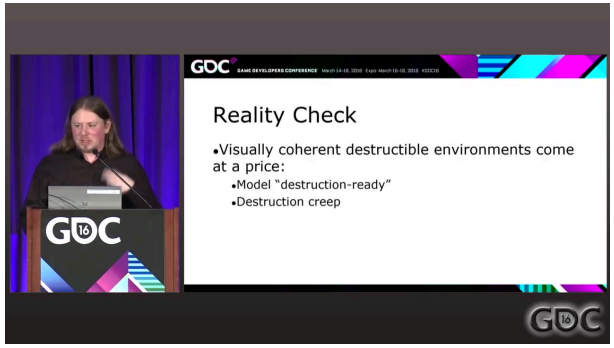
이 장면의 한국어 자막



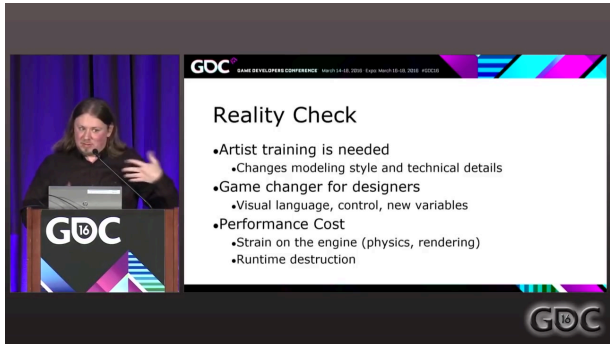
08:34–08:48
하지만 그렇게 간단하지 않습니다. 잠깐 물 좀 마시겠습니다. 실제로는 꽤 큰 대가를 치러야 합니다.



08:48–09:06
일관성 있고 플레이어가 납득할 만한 환경을 만들려면 파괴를 염두에 두고 모델링해야 합니다. 이제 벽 너머나 천장 안까지 보이니까, 원래 보이지 않던 내부도 만들어야 하죠.



09:06–09:21
예전에는 모델링하지 않던 부분까지 작업해야 하므로 아티스트의 부담이 늘어납니다. 물리적으로도 그럴듯하게 표현해야 하니, 작업 방식과 지식도 달라져야 합니다.

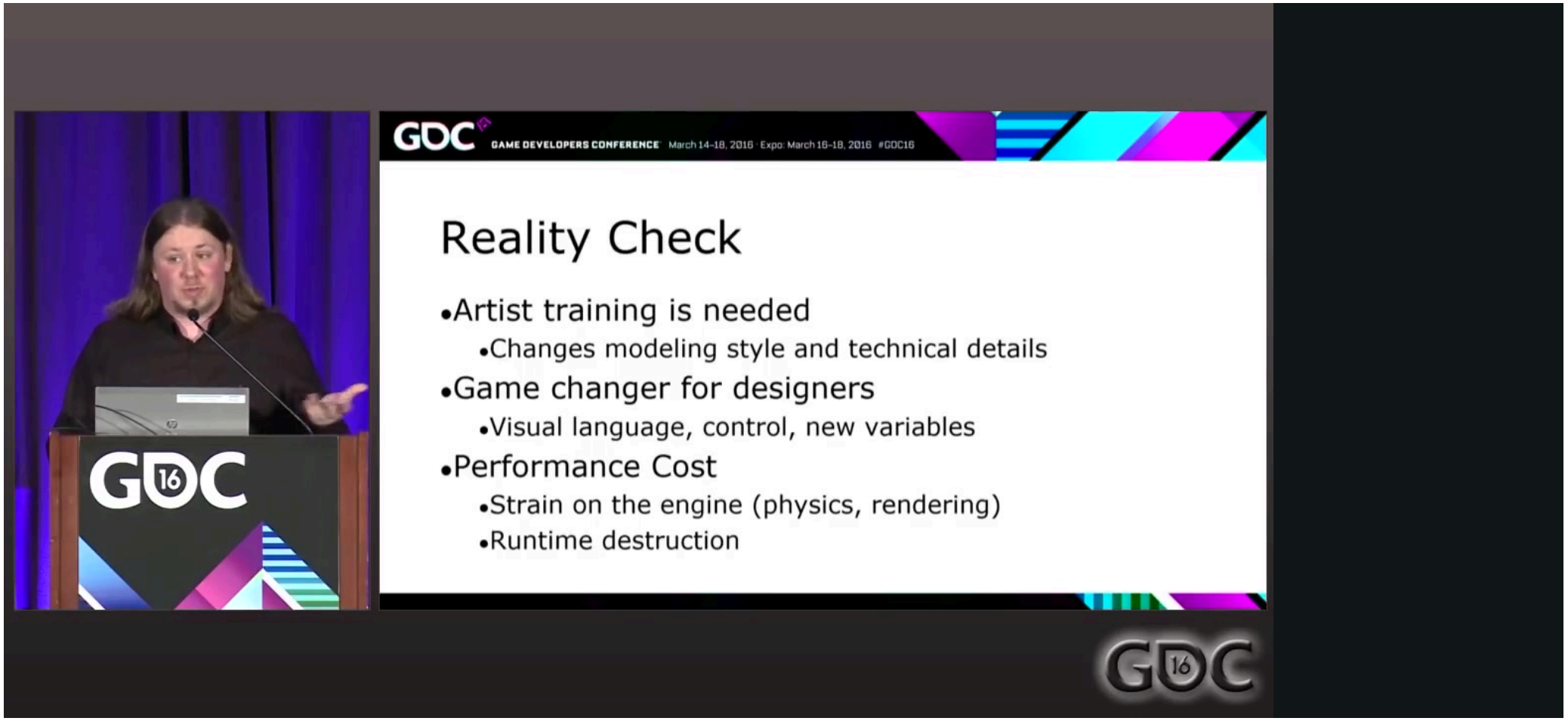


09:21–09:37

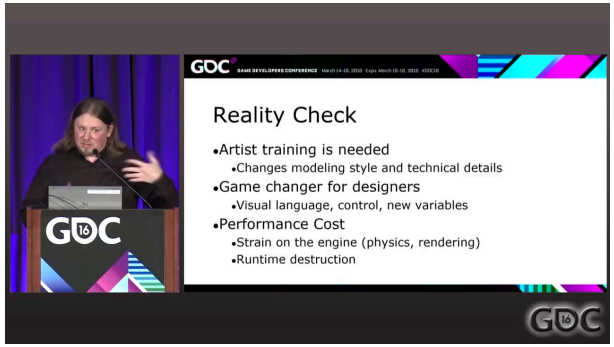
아티스트에게 물리를 교육해야 할 수도 있고, 디자이너에게도 큰 변화입니다. 파괴의 시각적 언어를 새로 정의해야 하면서도, 한편으로는 새로운 게임플레이 변수를 얻는 셈이죠.

032. Reality Check: 제작 방식과 성능

영상 09:25–09:55 · 장면 캡처 09:27

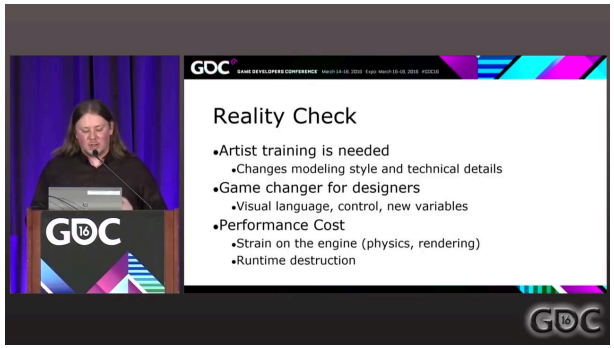


이 장면의 한국어 자막



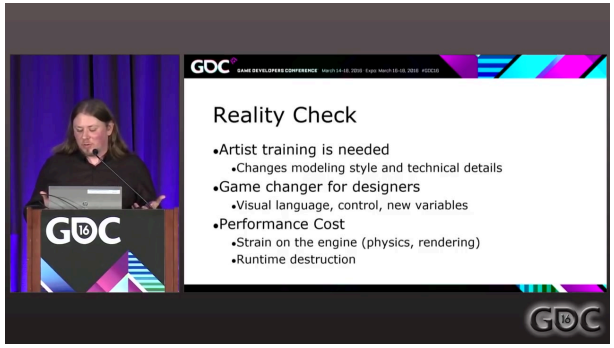
09:21–09:37

아티스트에게 물리를 교육해야 할 수도 있고, 디자이너에게도 큰 변화입니다. 파괴의 시각적 언어를 새로 정의해야 하면서도, 한편으로는 새로운 게임플레이 변수를 얻는 셈이죠.



09:37–09:54

물론 공짜는 아닙니다. 동적 오브젝트가 늘어나 물리 비용이 증가하고, 여러 이유로 렌더링 비용도 커집니다. 런타임에 파괴를 처리하는 계산 비용도 추가됩니다.



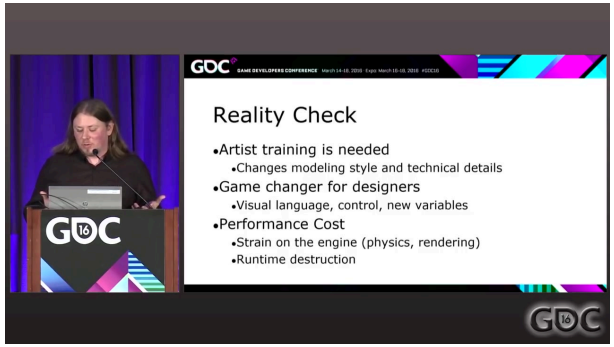
09:54–10:09

그 비용을 감수하고 파괴 시스템을 도입한다고 합시다. 다른 게임 시스템과는 어떻게 연결할까요? 저희가 사용한 방법은 사실 간단합니다.

033. The R6:S Game Ecosystem

영상 09:55–10:05 · 장면 캡처 09:57

이 장면의 한국어 자막



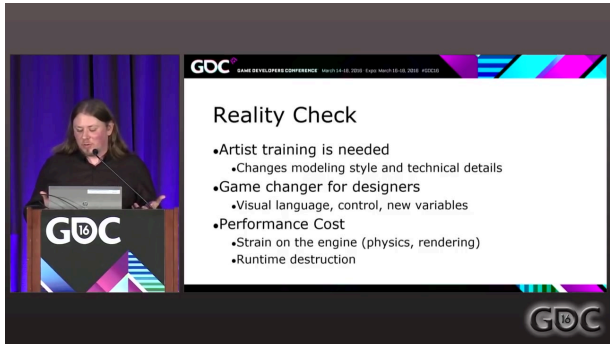
09:54–10:09

그 비용을 감수하고 파괴 시스템을 도입한다고 합시다. 다른 게임 시스템과는 어떻게 연결할까요? 저희가 사용한 방법은 사실 간단합니다.

034. Destruction Event System

영상 10:05–10:45 · 장면 캡처 10:07

이 장면의 한국어 자막

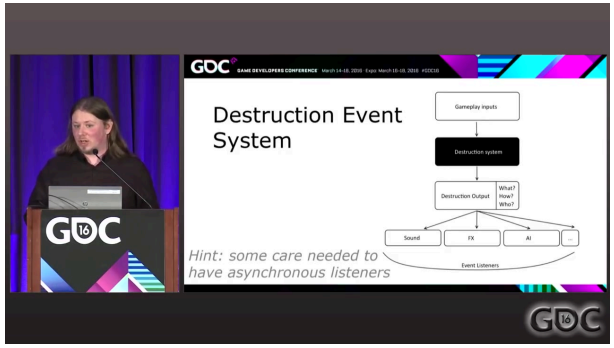


09:54–10:09

그 비용을 감수하고 파괴 시스템을 도입한다고 합시다. 다른 게임 시스템과는 어떻게 연결할까요? 저희가 사용한 방법은 사실 간단합니다.

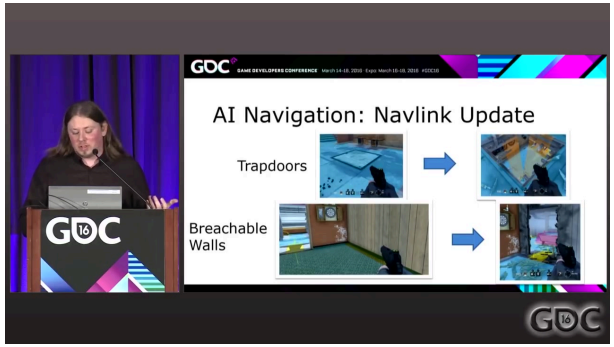
10:09–10:26

게임이 입력을 파괴 시스템에 전달하면 파괴가 발생합니다. 파괴 내부는 블랙박스여도 괜찮지만, 다시 게임에 돌려주는 출력 정보는 각 시스템에 필요한 형태로 구조화해야 합니다.



10:26–10:43

무엇이 부서졌는지, 어떤 무기를 썼는지, 파편이 지금 어디 있는지 등 다양한 정보를 전달합니다. 사운드, AI를 비롯한 게임의 여러 시스템이 이 정보를 사용하죠.



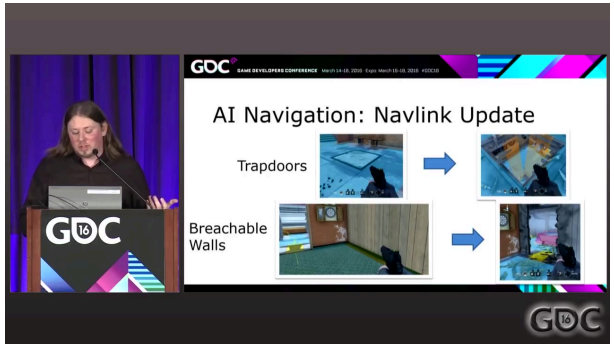
10:43–10:59

예를 들어 AI 내비게이션은 새로운 구멍이나 파괴된 벽에 맞춰 갱신해야 합니다. 벽을 뚫었는데 AI가 열린 길 앞에서 멈춰 있어서는 안 되니까요.

035. AI Navigation: Navlink Update

영상 10:45–10:55 · 장면 캡처 10:47

이 장면의 한국어 자막

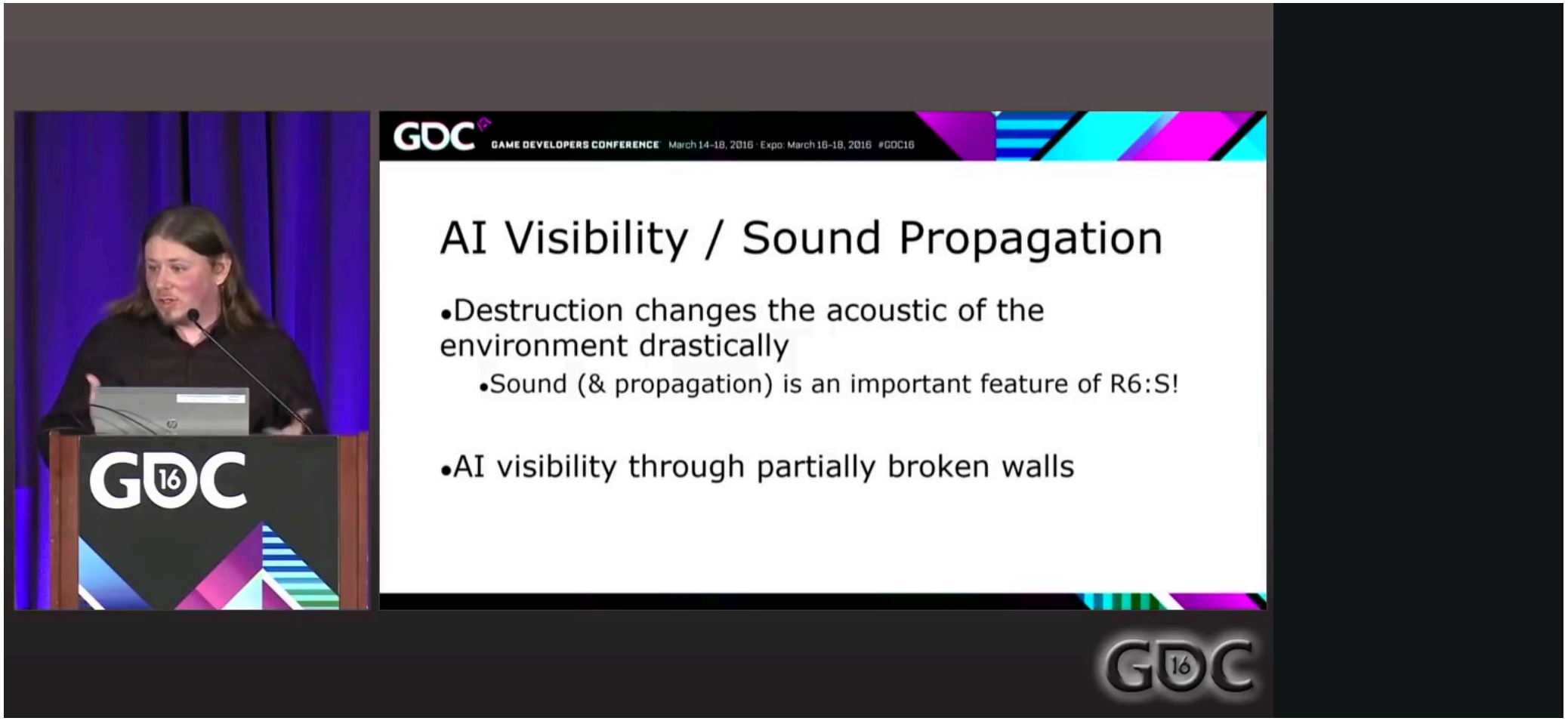


10:43–10:59

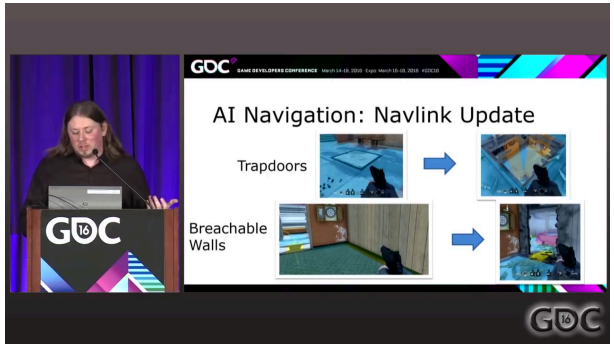
예를 들어 AI 내비게이션은 새로운 구멍이나 파괴된 벽에 맞춰 갱신해야 합니다. 벽을 뚫었는데 AI가 열린 길 앞에서 멈춰 있어서는 안 되니까요.

036. AI Visibility / Sound Propagation

영상 10:55–11:35 · 장면 캡처 10:57



이 장면의 한국어 자막



10:43–10:59

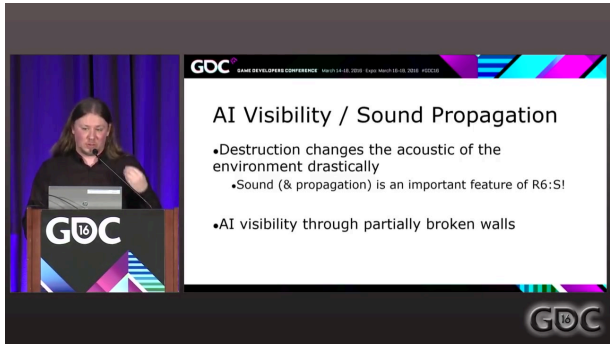
예를 들어 AI 내비게이션은 새로운 구멍이나 파괴된 벽에 맞춰 갱신해야 합니다. 벽을 뚫었는데 AI가 열린 길 앞에서 멈춰 있어서는 안 되니까요.

10:59–11:14

이 정보는 시야 판정에도 활용됩니다. AI가 지나치게 자주 시야를 재계산하거나, 벽 너머의 적을 너무 쉽게 알아차리는 걸 막을 수 있습니다. 사운드 전파에도 쓰이고요.

11:14–11:30

레인보우 식스 시즈에서 사운드 전파는 대단히 중요한 기능입니다. 벽에 구멍이 얼마나, 어디에 뚫렸느냐에 따라 방의 음향 특성이 달라집니다. 플레이어는 그 소리에 의존하죠.

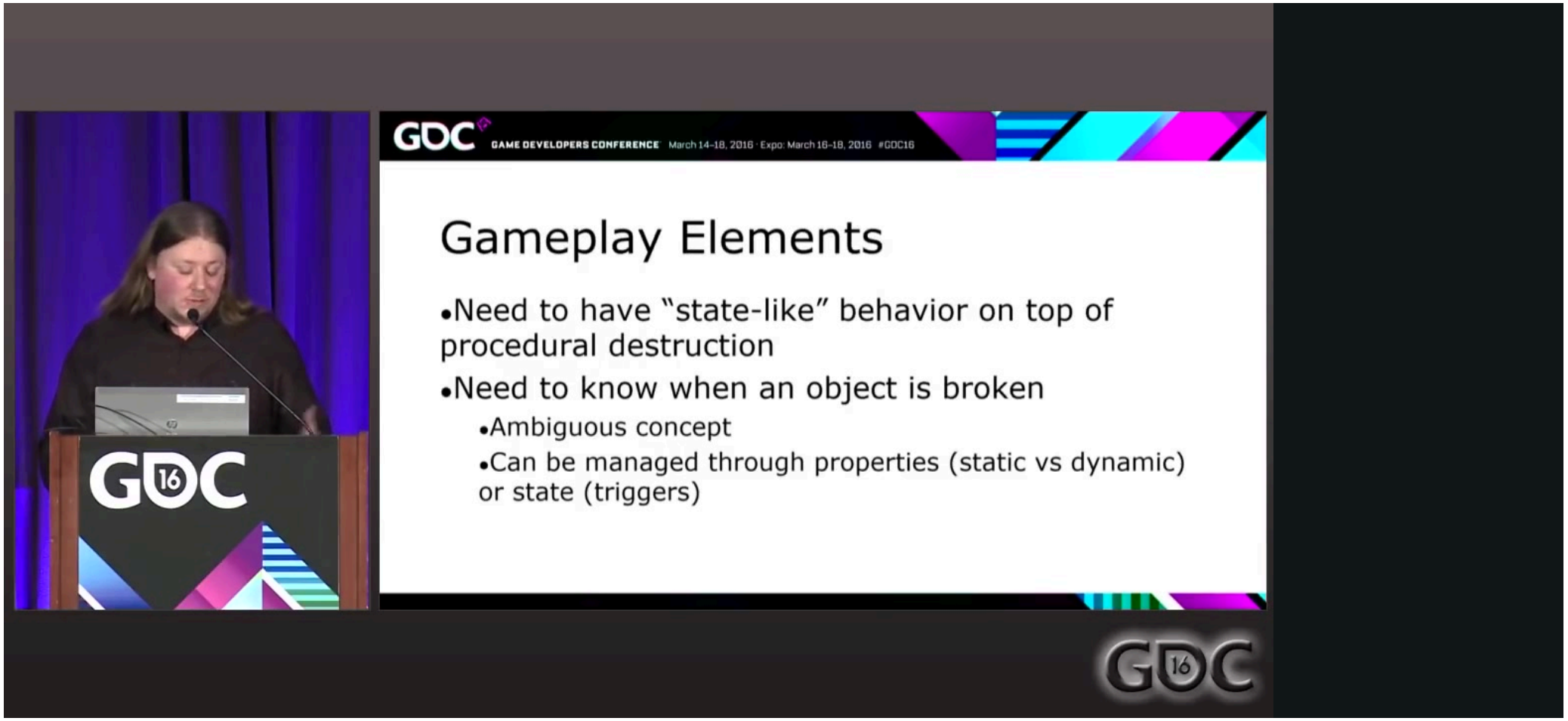


11:30–11:47

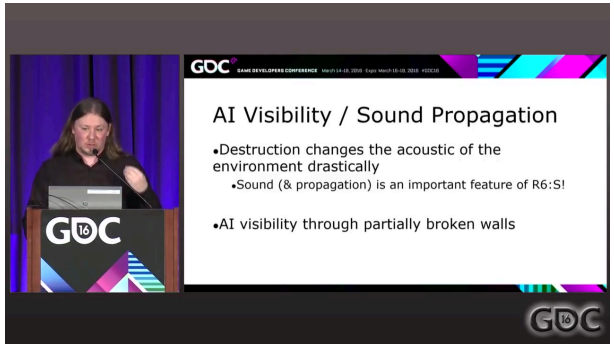
그 소리를 포함해 게임이 전달하는 정보가 중요한 겁니다. 그리고 게임플레이 프로그래머는 때로 확정적인 결과를 원합니다. 어떤 요소는 반드시 부서져야 하는 식이죠.

037. Gameplay Elements

영상 11:35–12:05 · 장면 캡처 11:37

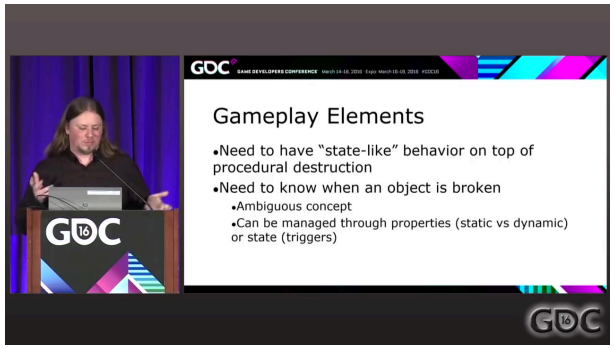


이 장면의 한국어 자막



11:30–11:47

그 소리를 포함해 게임이 전달하는 정보가 중요한 겁니다. 그리고 게임플레이 프로그래머는 때로 확정적인 결과를 원합니다. 어떤 요소는 반드시 부서져야 하는 식이죠.



11:47–12:04

우연에 의존하지 않고 원하는 결과를 보장해야 할 때가 있습니다. 결국 우리는 단순히 기술을 만드는 게 아니라 게임을 만드는 것이니까요. 이런 문제까지 고려해야 합니다.

12:04–12:22

그렇게 설계하면 충분히 관리할 수 있습니다. 이제 리얼블라스트 기술을 조금 더 설명하겠습니다. 높은 수준에서 개념을 소개할 텐데, 사용하는 파괴 기술에 따라 차이는 있을 겁니다.

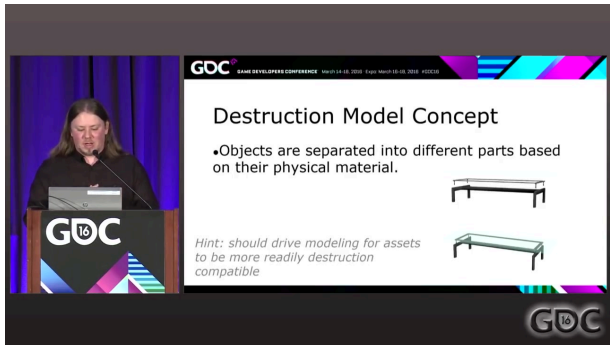
038. RealBlast Tech

영상 12:05–12:25 · 장면 캡처 12:07

이 장면의 한국어 자막

12:04–12:22

그렇게 설계하면 충분히 관리할 수 있습니다. 이제 리얼블라스트 기술을 조금 더 설명하겠습니다. 높은 수준에서 개념을 소개할 텐데, 사용하는 파괴 기술에 따라 차이는 있을 겁니다.



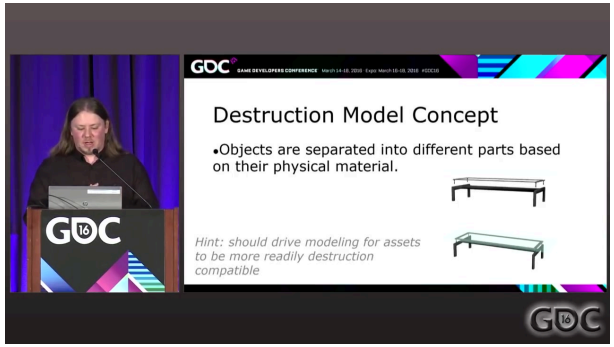
12:22–12:38

핵심 개념은 간단합니다. 서로 다른 부품으로 구성된 물체는 그 부품들을 별개로 모델링해야 한다는 겁니다. 당연해 보이죠.

039. Destruction Model Concept

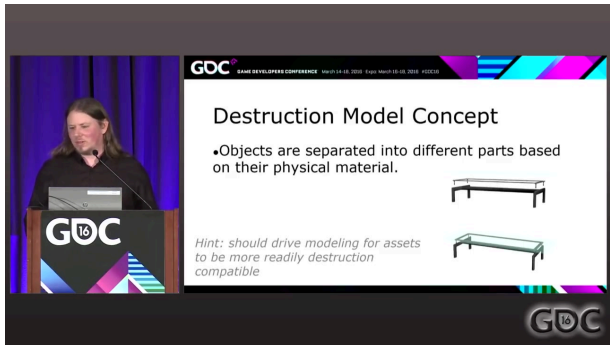
영상 12:25–13:00 · 장면 캡처 12:27

이 장면의 한국어 자막



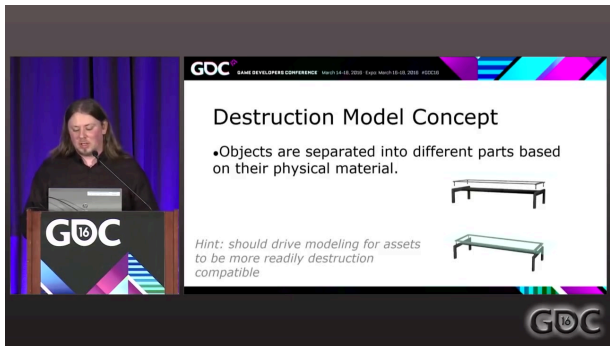
12:22–12:38

핵심 개념은 간단합니다. 서로 다른 부품으로 구성된 물체는 그 부품들을 별개로 모델링해야 한다는 겁니다. 당연해 보이죠.



12:38–12:54

유리가 나무에서 자라지는 않으니까요. 부품과 재질을 분리해 두면 각각의 특성에 맞는 처리 규칙을 풍부하게 넣을 수 있습니다. 조각이 나뉘는 방식도 물리적으로 자연스러워집니다.

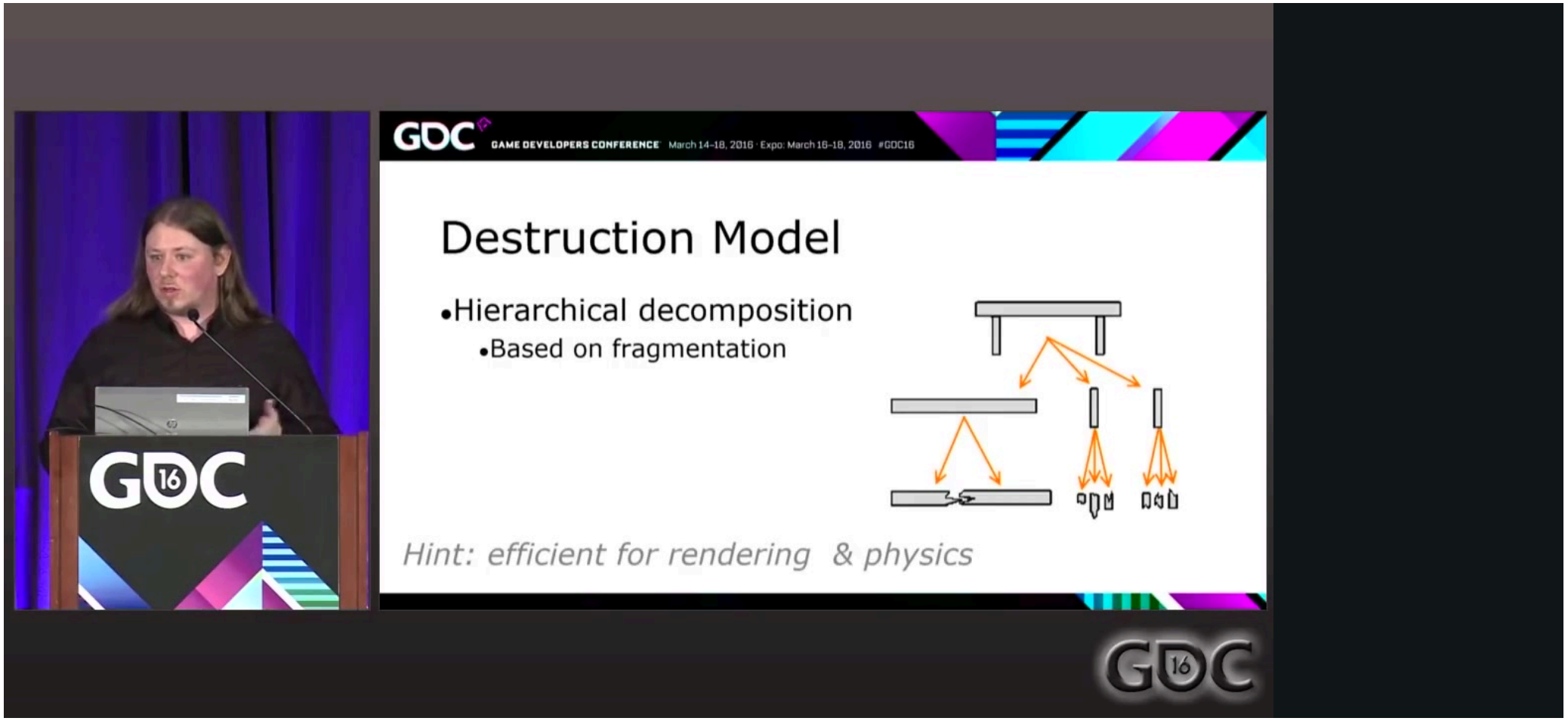


12:54–13:11

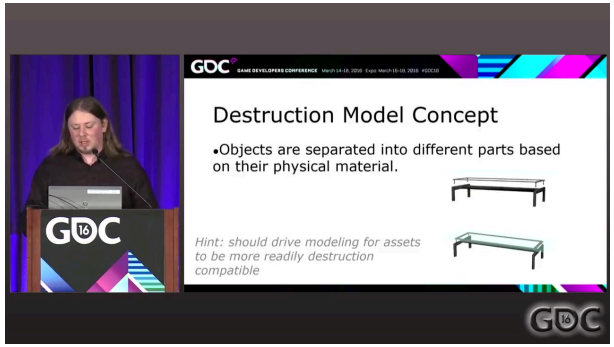
파괴로 조각을 나누면 계층 구조가 만들어집니다. 하나의 물체가 여러 조각으로 분할되는 중간 단계들을 보존해 두면, 렌더링이나 물리 계산에서 효율적으로 재활용할 수 있습니다.

040. Destruction Model: 계층 분해

영상 13:00–13:20 · 장면 캡처 13:02

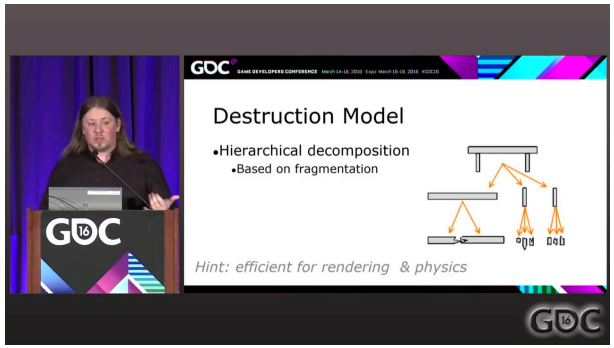


이 장면의 한국어 자막



12:54–13:11

파괴로 조각을 나누면 계층 구조가 만들어집니다. 하나의 물체가 여러 조각으로 분할되는 중간 단계들을 보존해 두면, 렌더링이나 물리 계산에서 효율적으로 재활용할 수 있습니다.

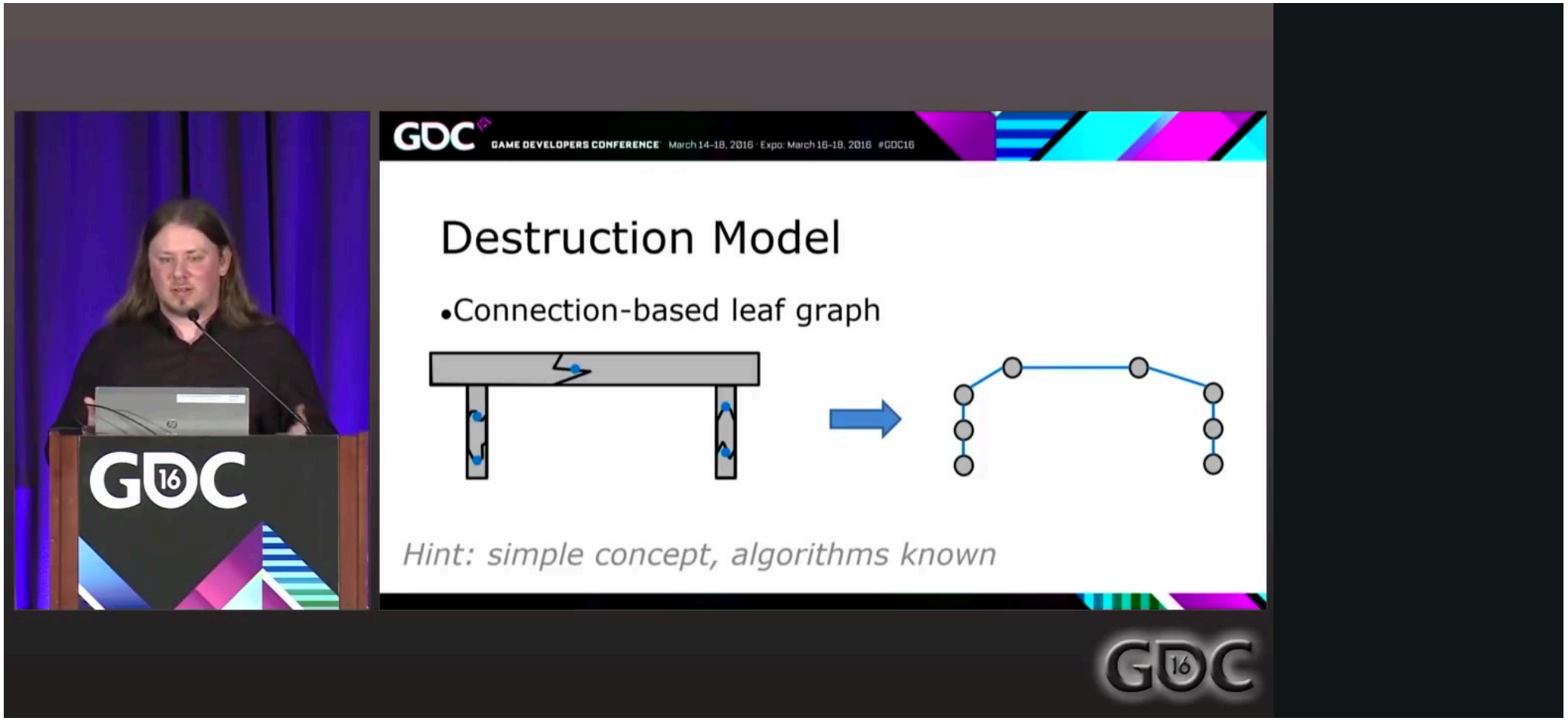


13:11–13:26

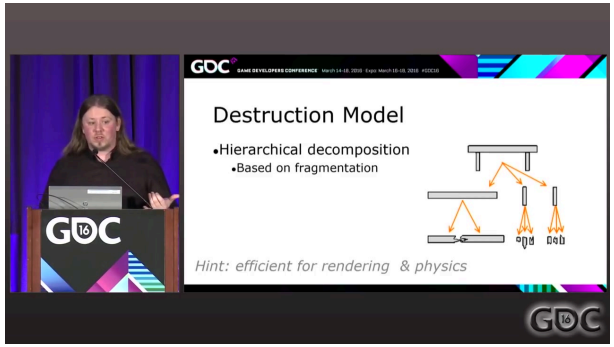
그리고 물체의 현재 상태를 표현할 파괴 모델이 필요합니다. 저희가 선택한 방법은 매우 단순합니다.

041. Destruction Model: 연결 그래프

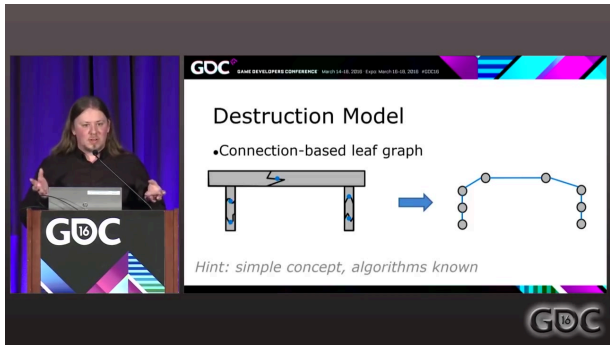
영상 13:20–14:00 · 장면 캡처 13:22



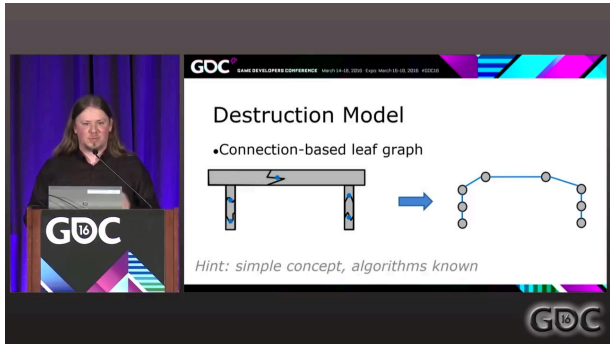
이 장면의 한국어 자막



13:11–13:26
그리고 물체의 현재 상태를 표현할 파괴 모델이 필요합니다. 저희가 선택한 방법은 매우 단순합니다.



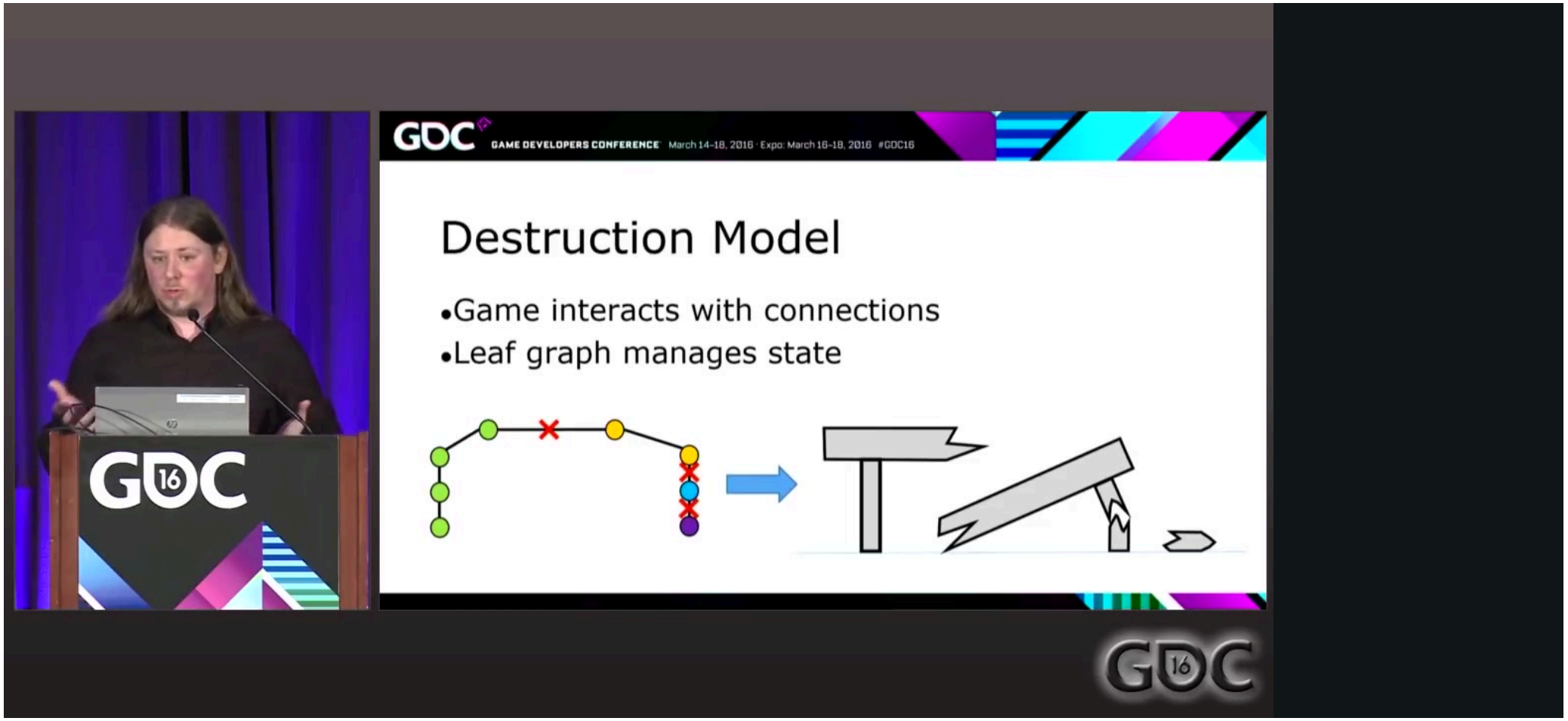
13:26–13:43
계층 구조의 말단 파편들을 그래프의 노드로 놓고, 파편 사이의 물리적 연결을 간선으로 표현합니다. 이렇게 하면 아주 유용한 그래프가 만들어지죠.



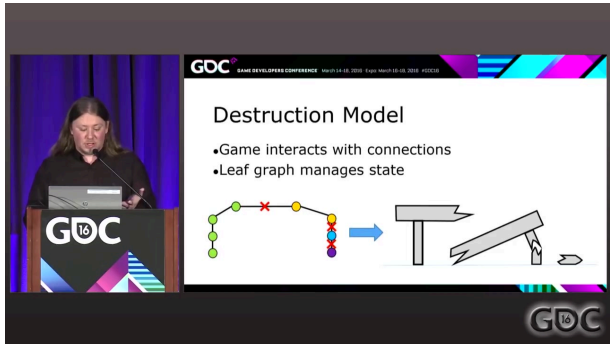
13:43–14:00
그래프에는 최소 컷, 최대 유량 같은 알고리즘을 포함해 수많은 기법을 적용할 수 있습니다. 이 구조로 물체의 상태를 관리할 수 있습니다.

042. Destruction Model: 상태 관리

영상 14:00–14:15 · 장면 캡처 14:02



이 장면의 한국어 자막

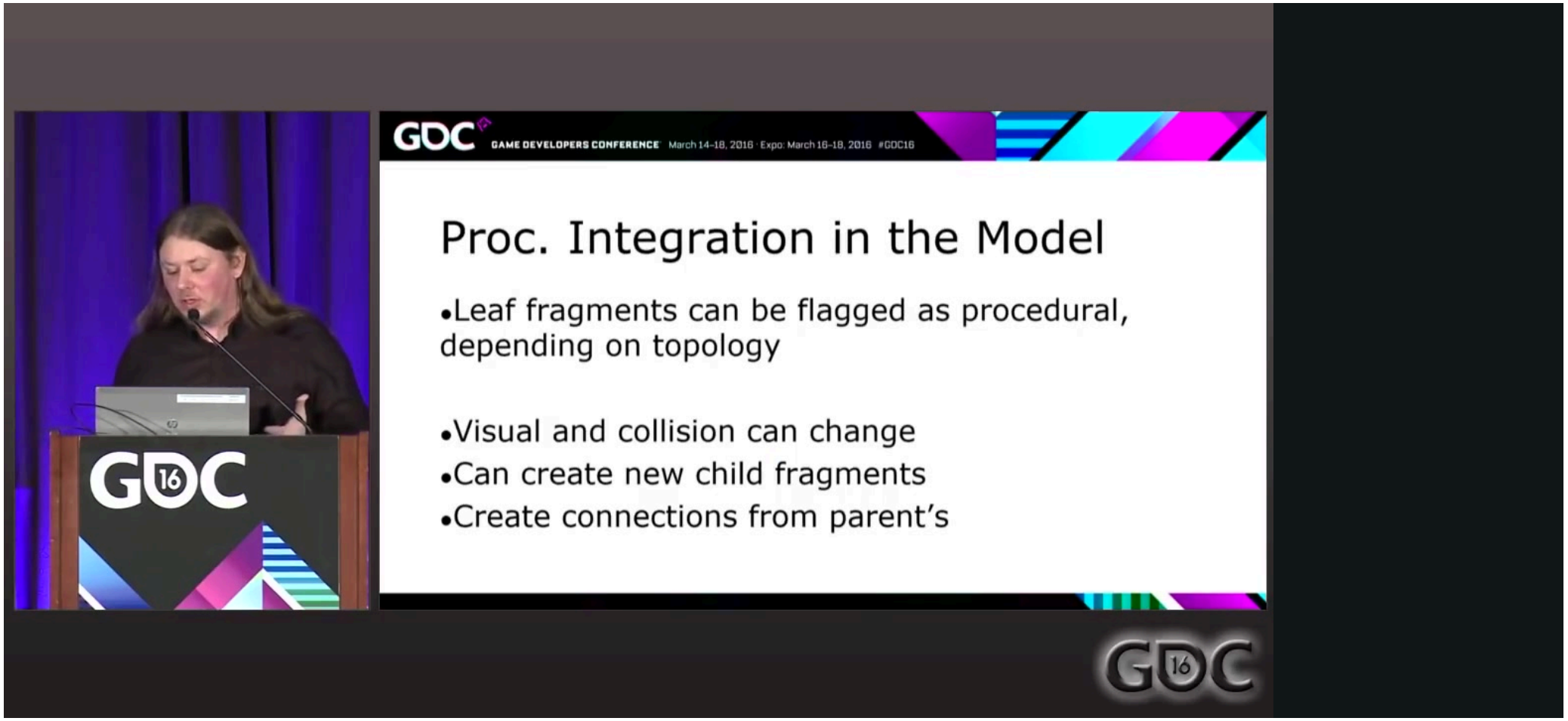


14:00–14:17

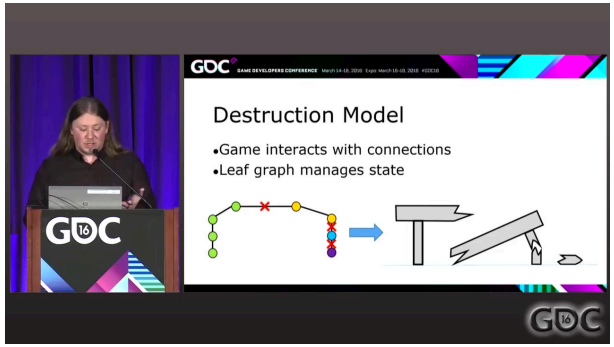
연결이 끊어지고 그래프에서 간선이 사라지면, 그 파편들은 더는 붙어 있지 않다는 뜻입니다. 게임 안에서 완전히 독립적인 오브젝트로 만들 수 있죠.

043. Procedural Integration in the Model

영상 14:15–14:40 · 장면 캡처 14:17

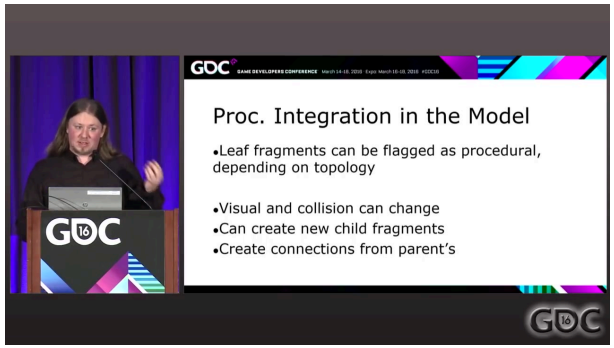


이 장면의 한국어 자막



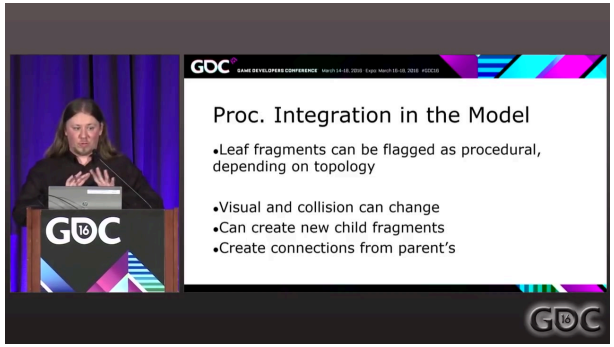
14:00–14:17

연결이 끊어지고 그래프에서 간선이 사라지면, 그 파편들은 더는 붙어 있지 않다는 뜻입니다. 게임 안에서 완전히 독립적인 오브젝트로 만들 수 있죠.



14:17–14:34

또한 이 모델에 절차적 파괴를 결합할 수 있습니다. 말단 조각을 다시 파괴 가능하게 설정하고, 토폴로지나 기법에 따라 렌더링 및 충돌 데이터를 바꾸거나 그래프의 조각을 교체할 수 있습니다.



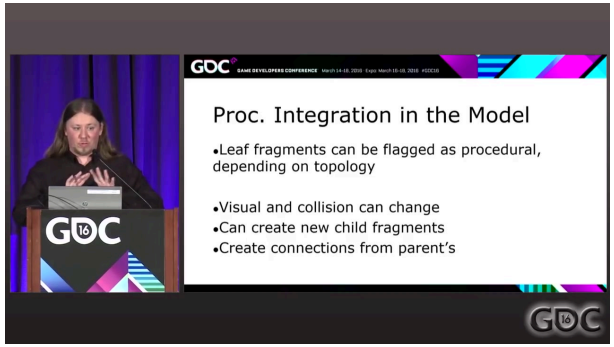
14:34–14:50

이런 방식으로 일관된 모델을 만들었습니다. 그럼 레인보우 식스에서 특별히 개발한 기술은 무엇일까요? '서피스 디스트럭션', 즉 표면 파괴 기술입니다. 출발점은 유리였습니다.

044. Surface Procedural Destruction Overview

영상 14:40–14:45 · 장면 캡처 14:42

이 장면의 한국어 자막

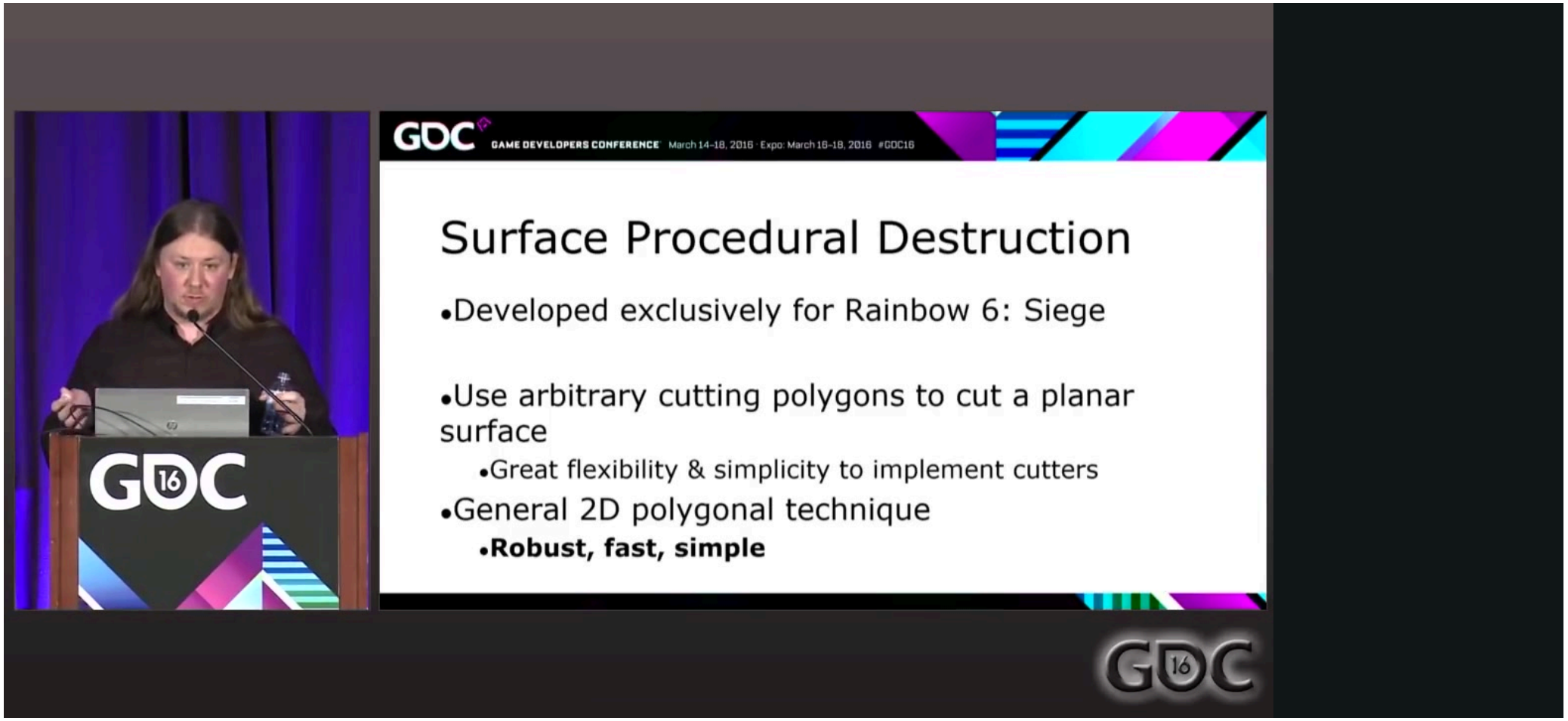


14:34–14:50

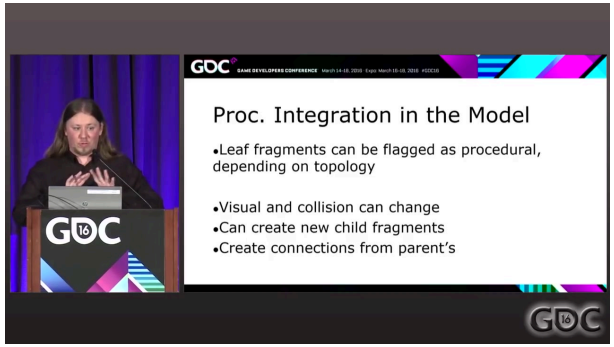
이런 방식으로 일관된 모델을 만들었습니다. 그럼 레인보우 식스에서 특별히 개발한 기술은 무엇일까요? '서피스 디스트럭션', 즉 표면 파괴 기술입니다. 출발점은 유리였습니다.

045. Surface Procedural Destruction

영상 14:45–15:35 · 장면 캡처 14:47

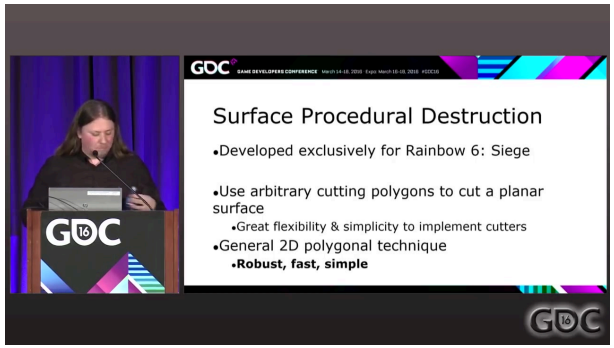


이 장면의 한국어 자막



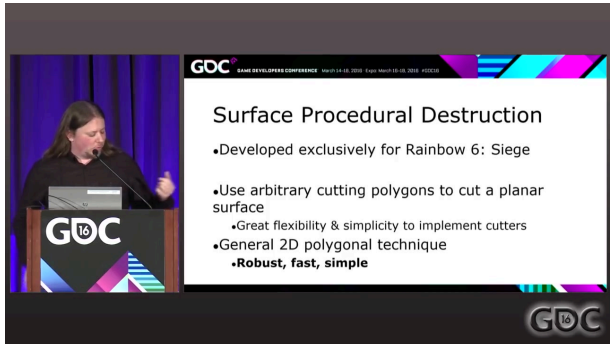
14:34–14:50

이런 방식으로 일관된 모델을 만들었습니다. 그럼 레인보우 식스에서 특별히 개발한 기술은 무엇일까요? '서피스 디스트럭션', 즉 표면 파괴 기술입니다. 출발점은 유리였습니다.



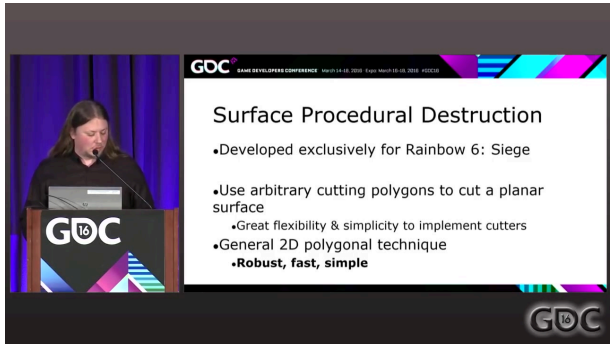
14:50–15:06

이를 다른 재질로 확장했습니다. 평면에 가까운 물체에만 쓸 수 있다는 한계가 있어 보이지만, 건물 내부와 벽, 집에는 평면이 아주 많습니다.



15:06–15:22

큰 장점은 원하는 커터를 자유롭게 사용할 수 있다는 겁니다. 아티스트가 절단 패턴을 직접 모델링하거나, 수학적 방법으로 2D 패턴을 생성하기도 쉽습니다.

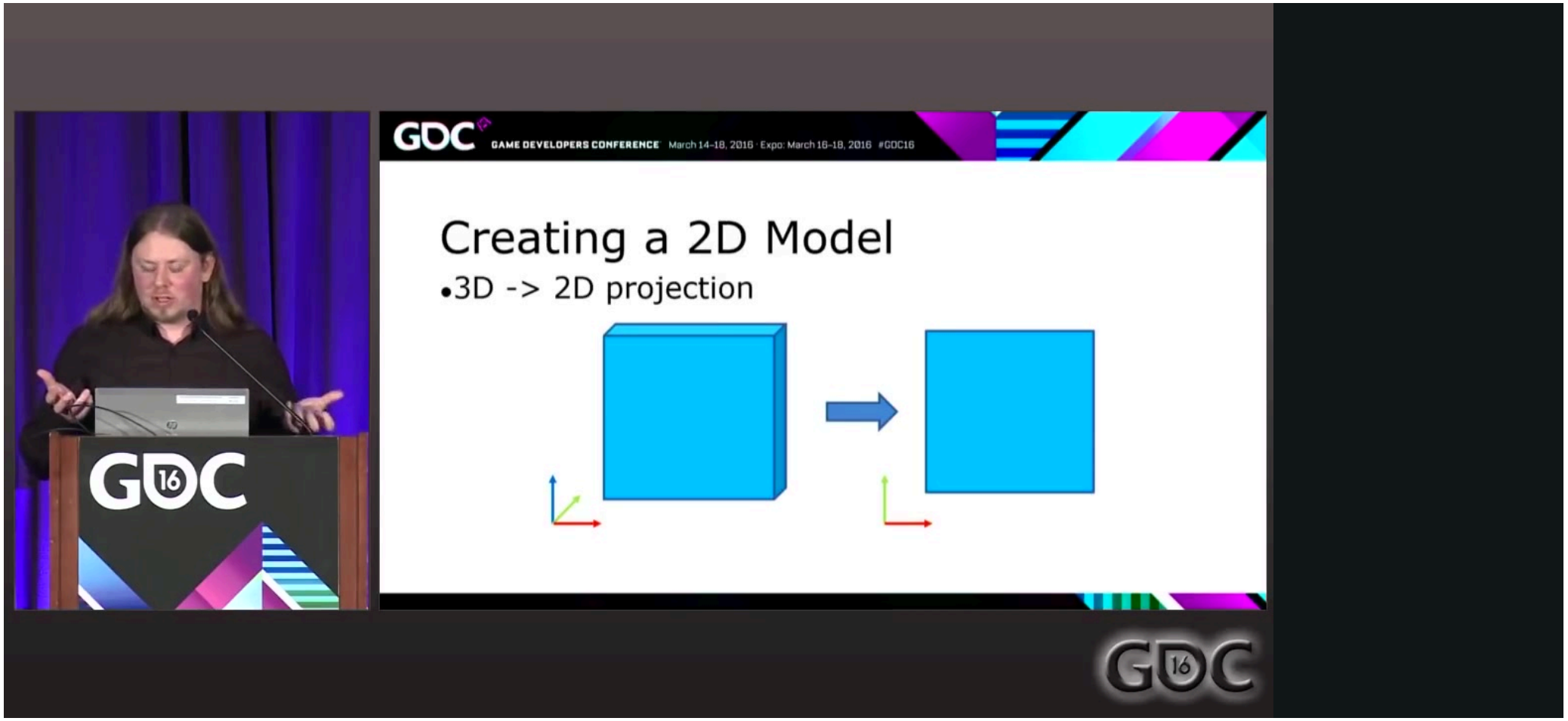


15:22–15:39

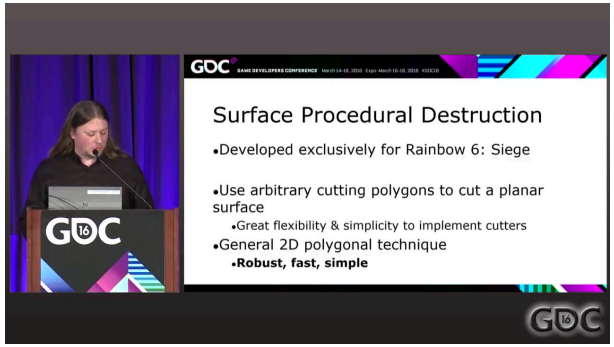
알고리즘도 전반적으로 빠르고 견고하며, 개념적으로 복잡하지 않습니다. 실제 처리 과정은 이렇습니다. 먼저 평면에 가까운 물체를 가져옵니다.

046. Creating a 2D Model

영상 15:35–15:45 · 장면 캡처 15:37

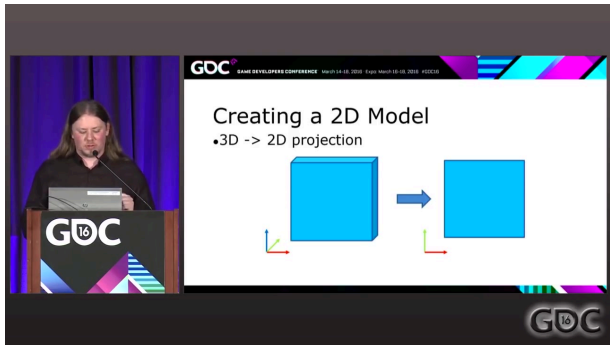


이 장면의 한국어 자막



15:22–15:39

알고리즘도 전반적으로 빠르고 견고하며, 개념적으로 복잡하지 않습니다. 실제 처리 과정은 이렇습니다. 먼저 평면에 가까운 물체를 가져옵니다.

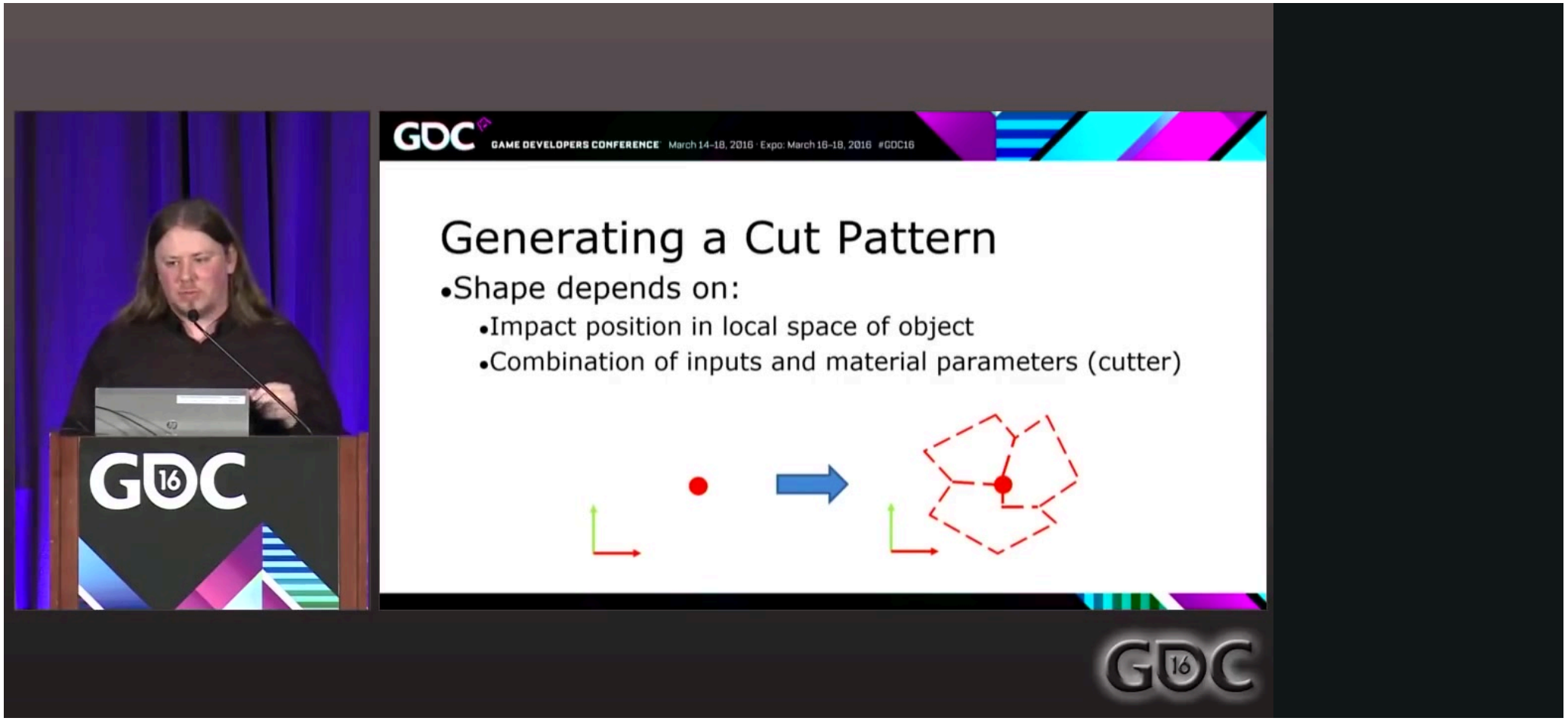


15:39–15:56

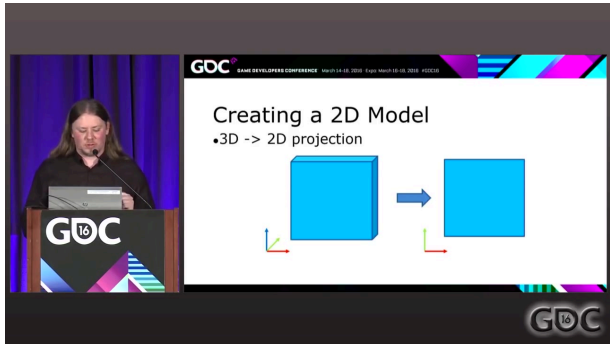
투영 기준 좌표계를 구하고 물체를 2D로 투영합니다. 이후 게임플레이에서 입력이 들어오면 재질과 입력을 비교해 어떤 파괴 패턴을 만들지 결정합니다.

047. Generating a Cut Pattern

영상 15:45–16:00 · 장면 캡처 15:47

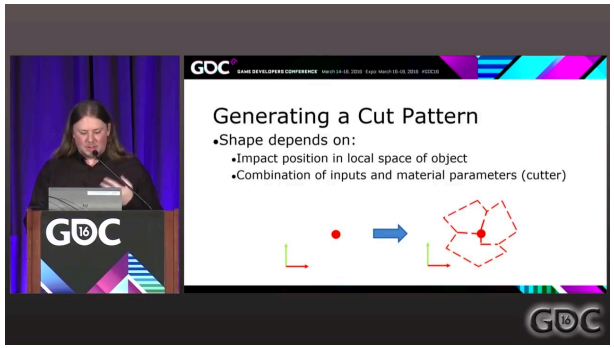


이 장면의 한국어 자막



15:39–15:56

투영 기준 좌표계를 구하고 물체를 2D로 투영합니다. 이후 게임플레이에서 입력이 들어오면 재질과 입력을 비교해 어떤 파괴 패턴을 만들지 결정합니다.

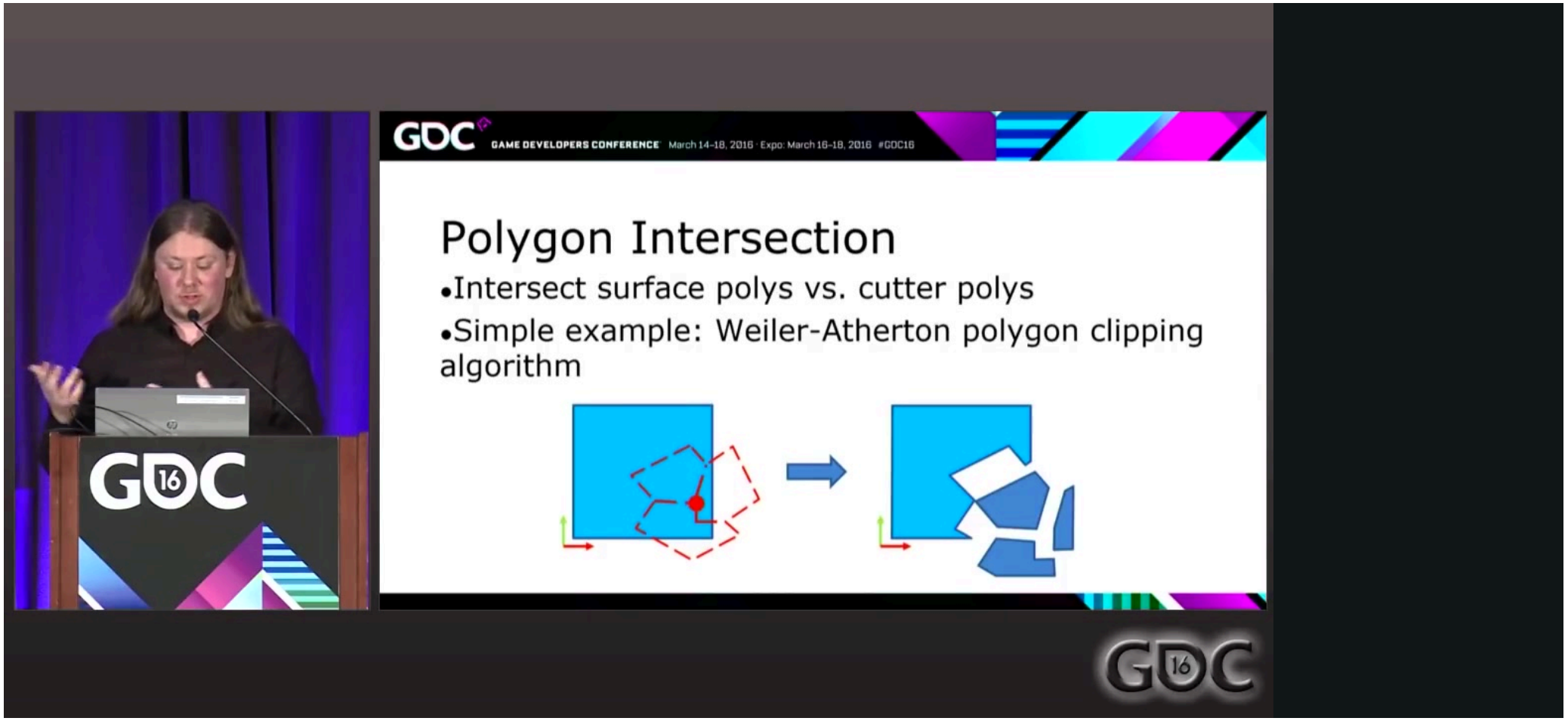


15:56–16:12

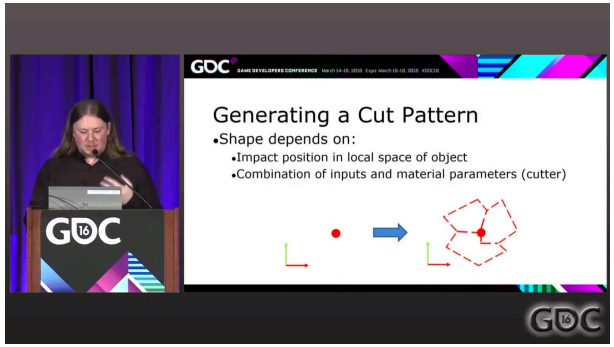
패턴을 생성해 새로운 폴리곤을 만들고, 2D 폴리곤과 커터의 교차 영역을 계산합니다. 이런 교차 연산 자체는 새로운 기술이 아니고 여러 알고리즘을 활용할 수 있습니다.

048. Polygon Intersection

영상 16:00–16:30 · 장면 캡처 16:02

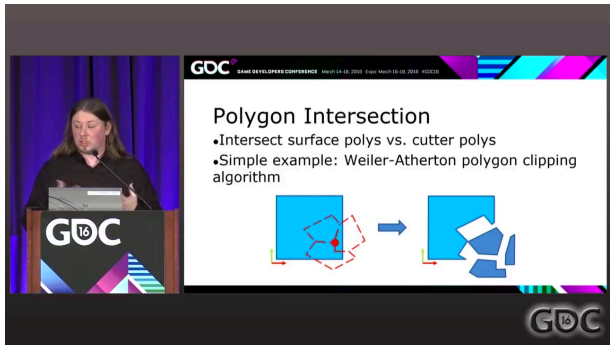


이 장면의 한국어 자막



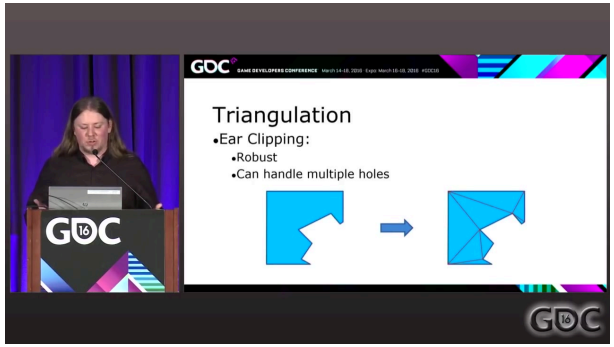
15:56–16:12

패턴을 생성해 새로운 폴리곤을 만들고, 2D 폴리곤과 커터의 교차 영역을 계산합니다. 이런 교차 연산 자체는 새로운 기술이 아니고 여러 알고리즘을 활용할 수 있습니다.



16:12–16:28

복잡한 폴리곤에도 적용할 수 있습니다. 교차 연산을 마친 뒤에는 UV 등 새로운 정점 속성을 보간합니다. 결국 물리와 렌더링에 사용할 지오메트리가 필요하므로 삼각형 분할을 진행하죠.

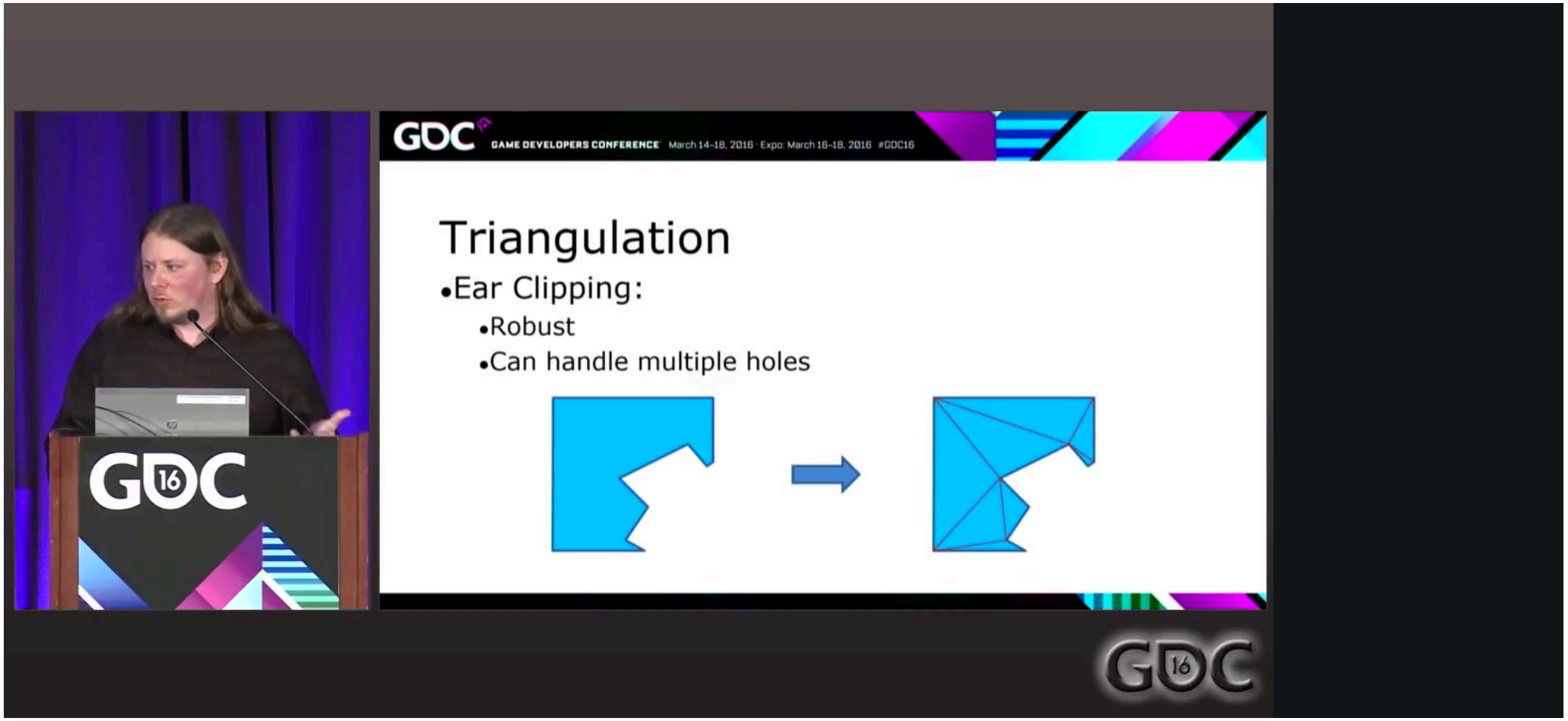


16:28–16:45

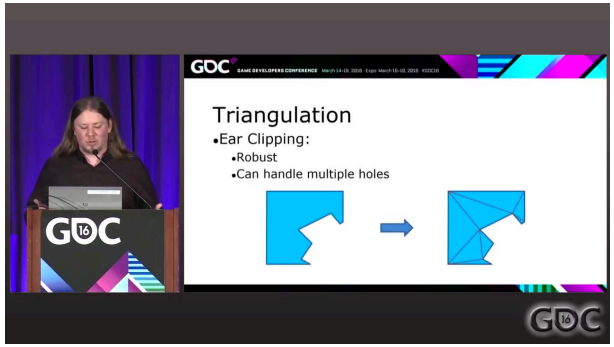
삼각형 분할에는 들로네 삼각분할 등 여러 알고리즘이 있습니다. 시즈의 파괴 시스템에서는 클리핑을 사용합니다. 최적의 알고리즘은 아니지만 견고하고, 흔한 상황을 잘 처리했습니다.

049. Triangulation

영상 16:30–16:55 · 장면 캡처 16:32

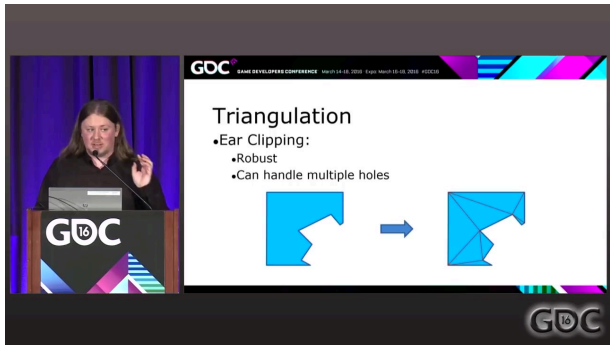


이 장면의 한국어 자막



16:28–16:45

삼각형 분할에는 들로네 삼각분할 등 여러 알고리즘이 있습니다. 시즈의 파괴 시스템에서는 클리핑을 사용합니다. 최적의 알고리즘은 아니지만 견고하고, 흔한 상황을 잘 처리했습니다.

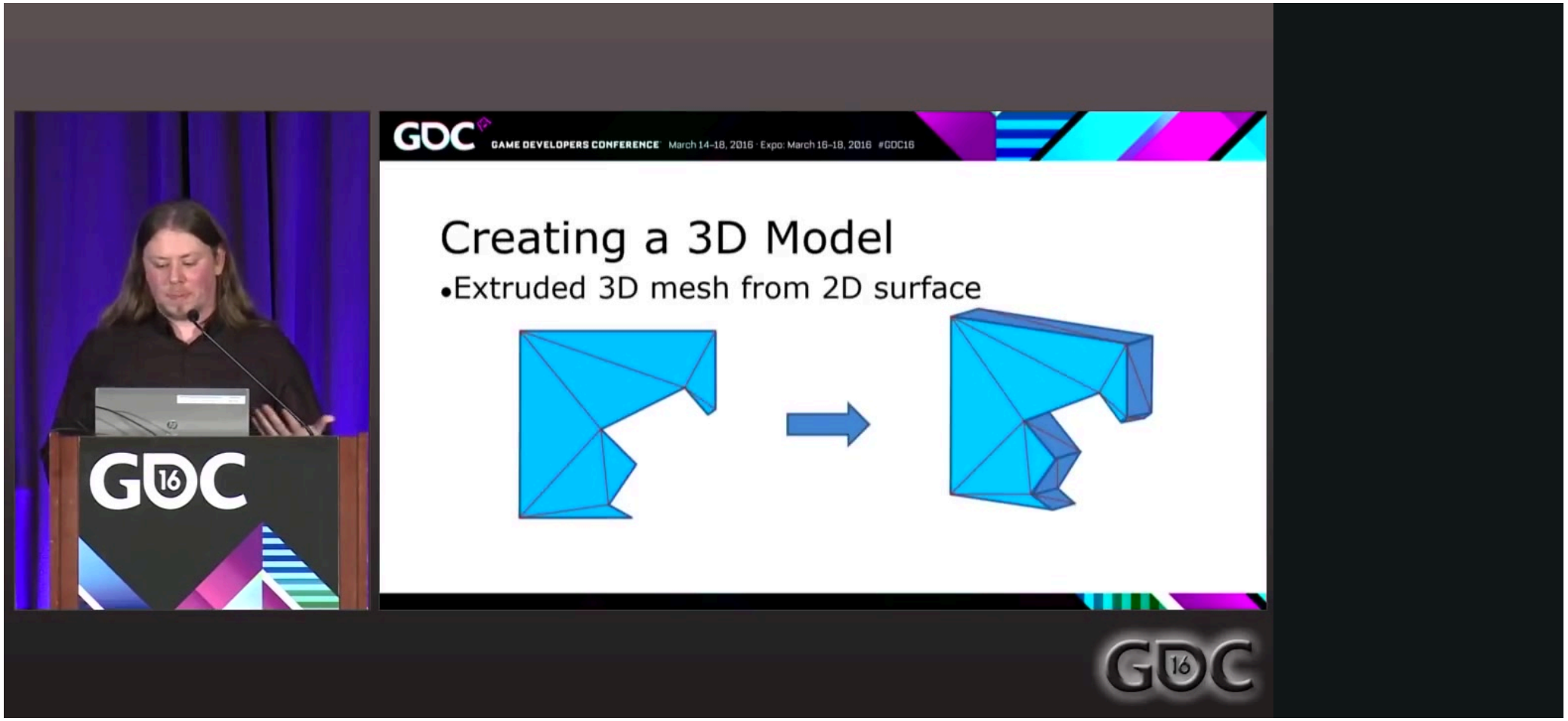


16:45–17:00

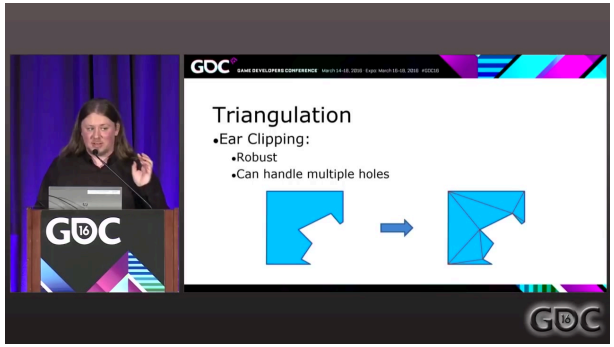
벽에 작은 구멍이 뚫리는 경우가 대표적입니다. 실제로 아주 효과적이었죠. 삼각형 분할을 마치면 3D 모델로 바꾸는 건 비교적 간단합니다.

050. Creating a 3D Model

영상 16:55–17:10 · 장면 캡처 16:57

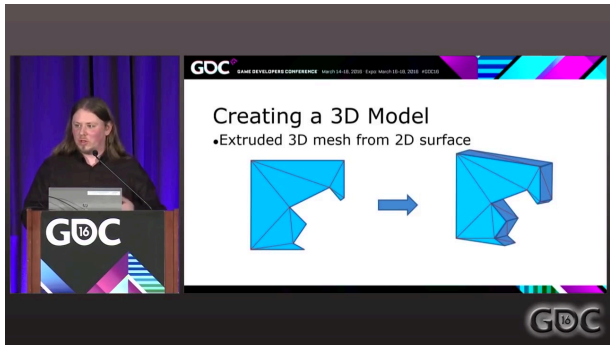


이 장면의 한국어 자막



16:45–17:00

벽에 작은 구멍이 뚫리는 경우가 대표적입니다. 실제로 아주 효과적이었죠. 삼각형 분할을 마치면 3D 모델로 바꾸는 건 비교적 간단합니다.

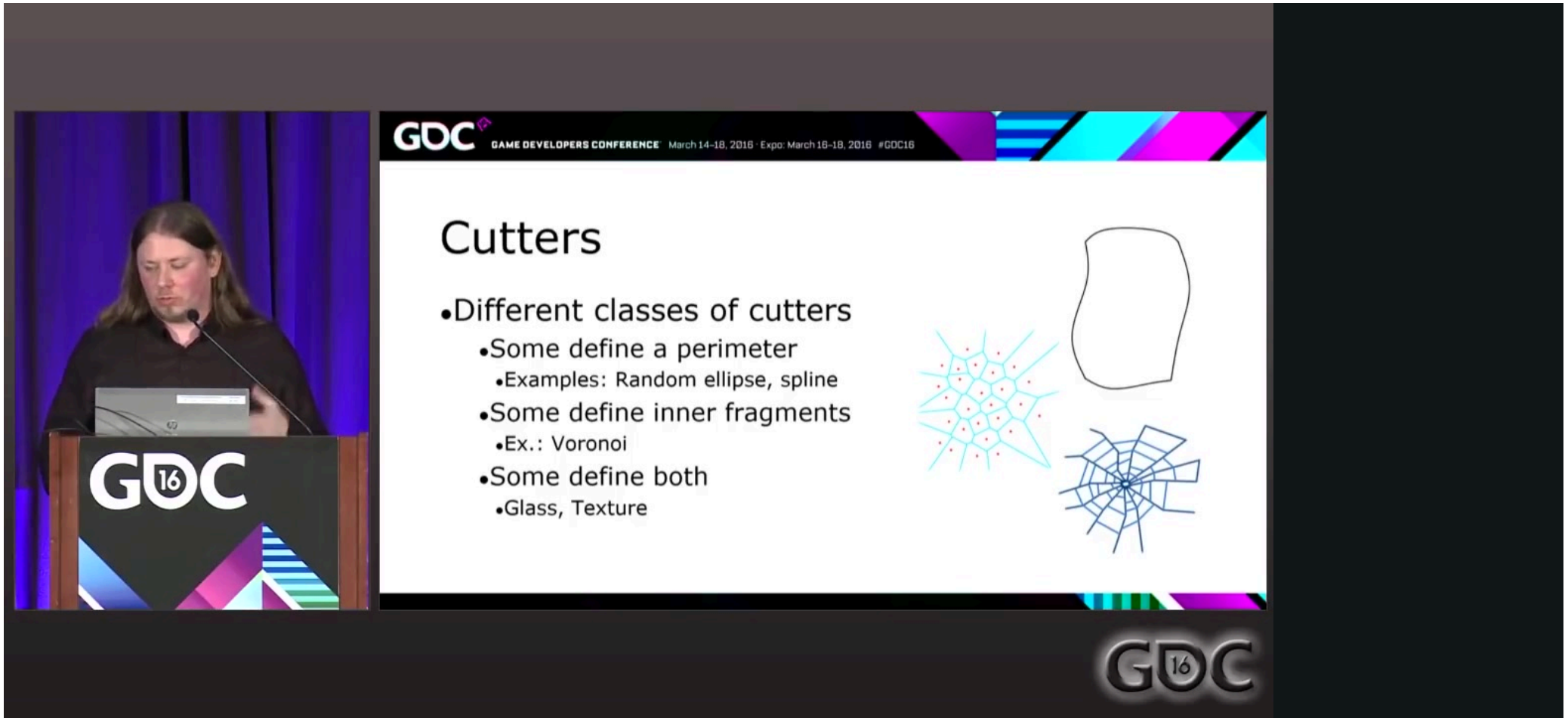


17:00–17:18

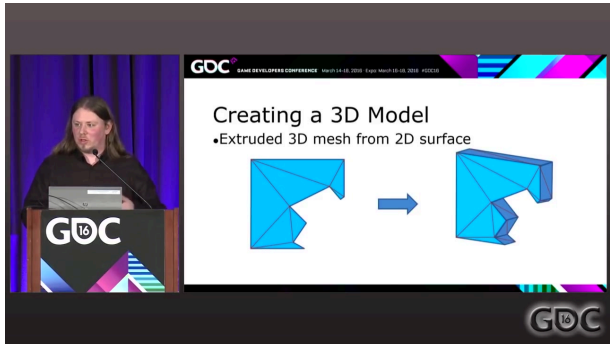
투영 기준 좌표계를 반대로 이용해 폴리곤을 익스트루드하면 3D 메시가 완성됩니다. 이제 커터 이야기를 해 보죠. 다양한 형태가 있지만 크게 세 종류로 나눌 수 있습니다.

051. Cutters

영상 17:10–17:35 · 장면 캡처 17:12

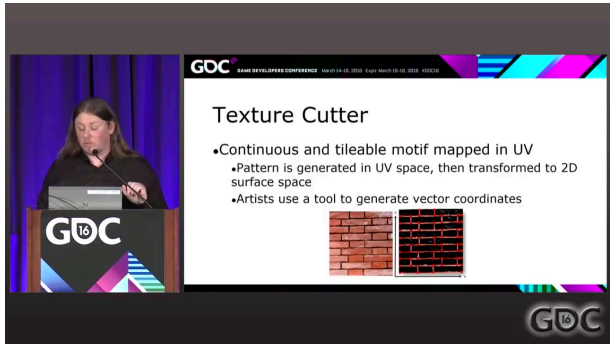


이 장면의 한국어 자막



17:00–17:18
투영 기준 좌표계를 반대로 이용해 폴리곤을 익스트루드하면 3D 메시가 완성됩니다. 이제 커터 이야기를 해 보죠. 다양한 형태가 있지만 크게 세 종류로 나눌 수 있습니다.

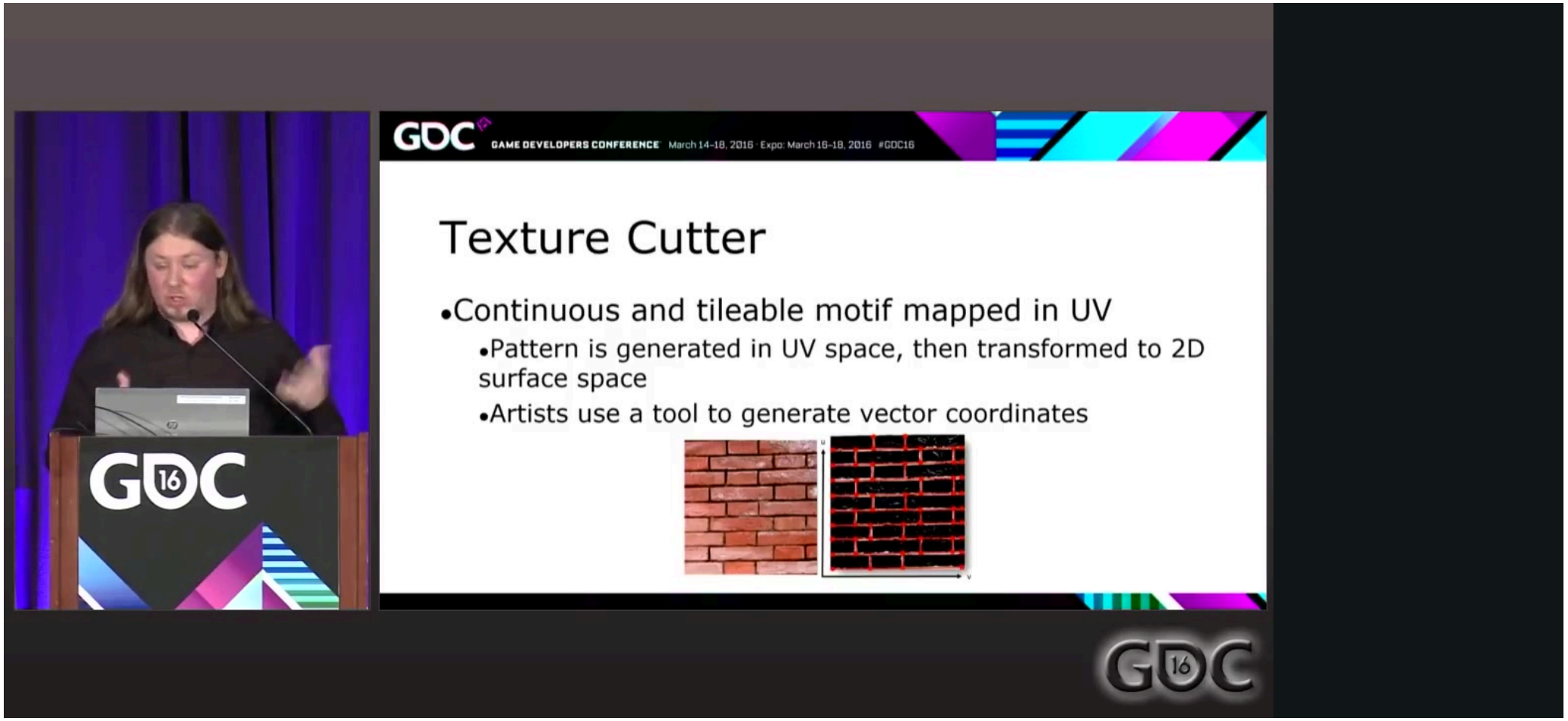
17:18–17:33
첫 번째는 타원이나 사각형처럼 매개변수로 정의되는 커터입니다. 두 번째는 노이즈처럼 불규칙한 형태를 만드는 커터고, 세 번째는 둘을 섞은 방식입니다. 유리용 커터가 예가 되겠네요.



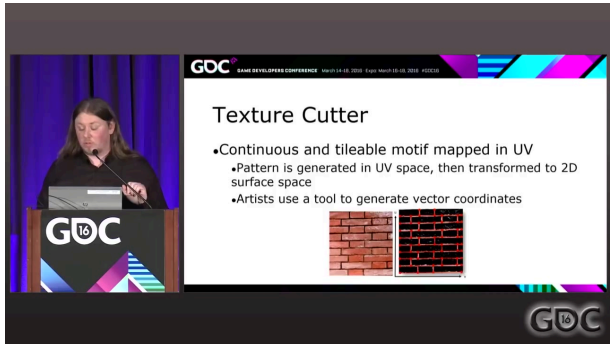
17:33–17:49
영상의 나무 파괴에는 텍스처 기반 커터를 사용했습니다. 정말 흥미로운 개념입니다. 아티스트가 UV 공간에 원하는 절단 모양을 직접 그릴 수 있죠.

052. Texture Cutter

영상 17:35–18:15 · 장면 캡처 17:37

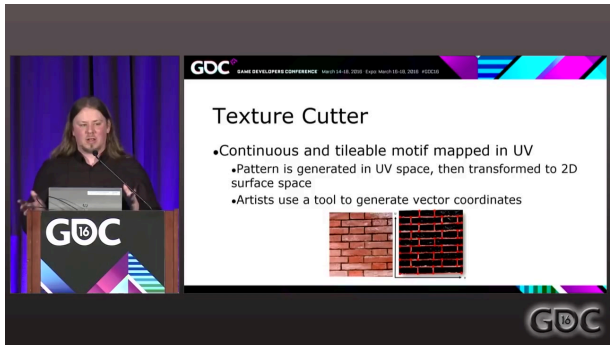


이 장면의 한국어 자막



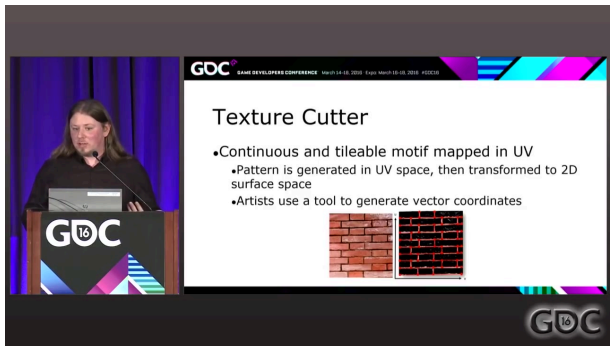
17:33–17:49

영상의 나무 파괴에는 텍스처 기반 커터를 사용했습니다. 정말 흥미로운 개념입니다. 아티스트가 UV 공간에 원하는 절단 모양을 직접 그릴 수 있죠.



17:49–18:06

게임플레이 입력이 들어오면 UV 공간에서 해당 지점을 찾아 패턴을 조회합니다. 이를 다시 2D 공간으로 변환해 교차 연산에 사용하면 됩니다.

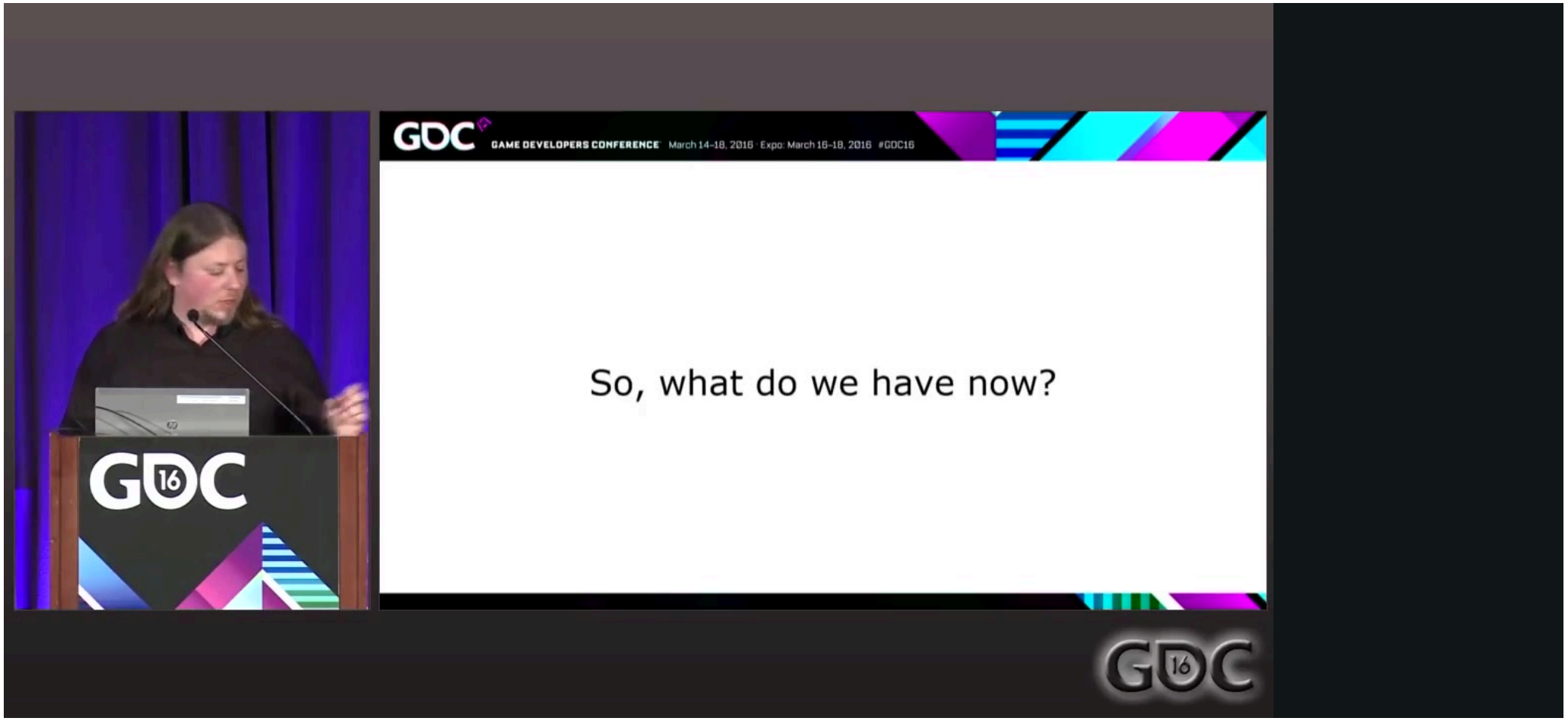


18:06–18:23

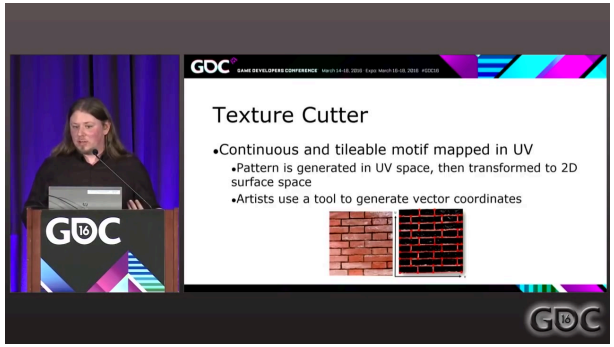
그러면 세밀하고 흥미로운 절단 형태를 만들 수 있습니다. 아티스트가 모든 결과를 직접 모델링할 필요도 없죠. 이렇게 만든 결과를 살펴봅시다.

053. So, what do we have now?

영상 18:15–18:25 · 장면 캡처 18:17



이 장면의 한국어 자막



18:06–18:23

그러면 세밀하고 흥미로운 절단 형태를 만들 수 있습니다. 아티스트가 모든 결과를 직접 모델링할 필요도 없죠. 이렇게 만든 결과를 살펴봅시다.

18:23–18:39

아직 프로그래머가 만든 임시 아트지만, 정점이 많고 유기적인 느낌이 납니다. 다만 외형을 개선할 요소가 더 필요합니다.

054. 표면 파괴 결과

영상 18:25–18:35 · 장면 캡처 18:27

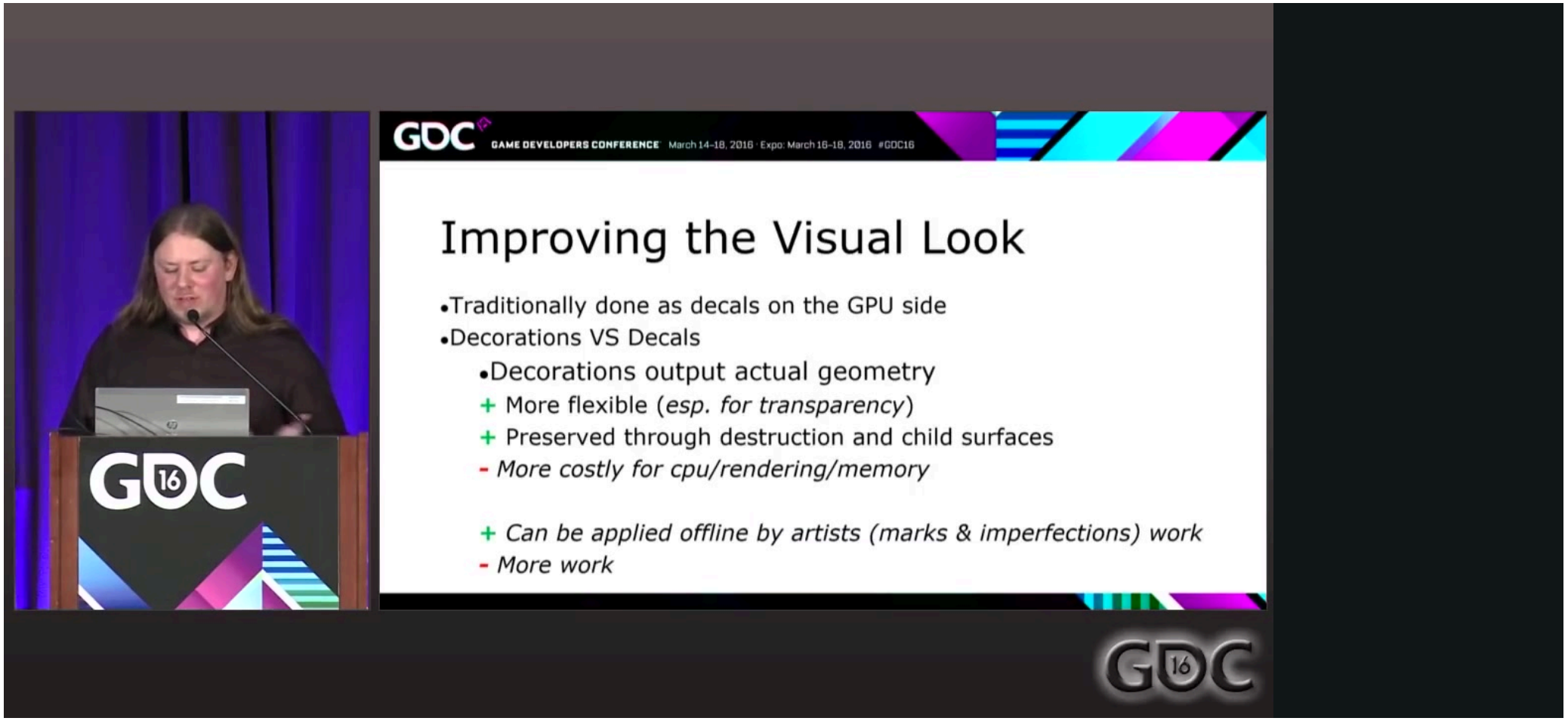
이 장면의 한국어 자막

18:23–18:39

아직 프로그래머가 만든 임시 아트지만, 정점이 많고 유기적인 느낌이 납니다. 다만 외형을 개선할 요소가 더 필요합니다.

055. Improving the Visual Look

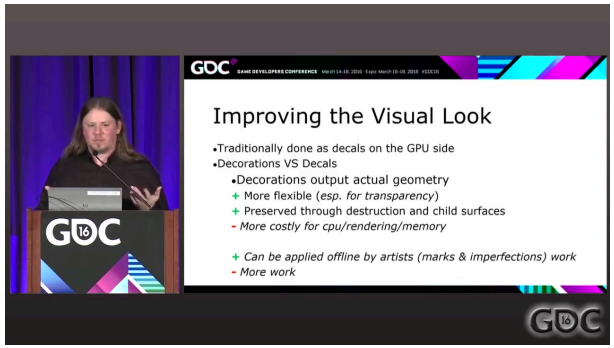
영상 18:35–19:45 · 장면 캡처 18:37



이 장면의 한국어 자막

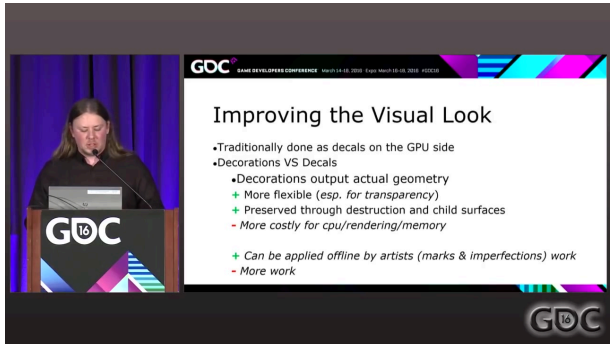
18:23–18:39

아직 프로그래머가 만든 임시 아트지만, 정점이 많고 유기적인 느낌이 납니다. 다만 외형을 개선할 요소가 더 필요합니다.



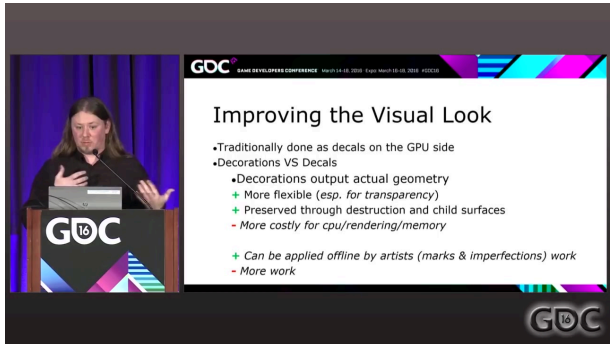
18:39–18:54

보통 슈팅 게임이라면 데칼을 사용할 겁니다. 하지만 저희는 조금 더 범용적인 방법을 원했고, 이를 '데코레이션'이라고 불렀습니다.

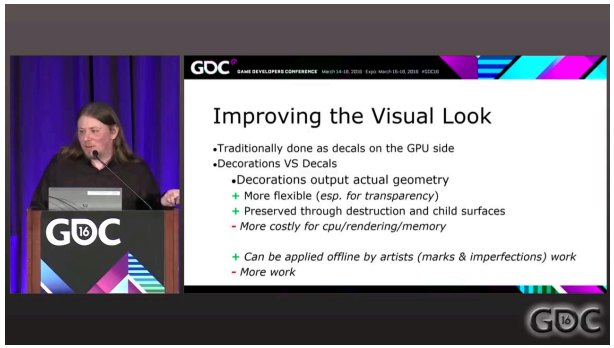


18:54–19:09

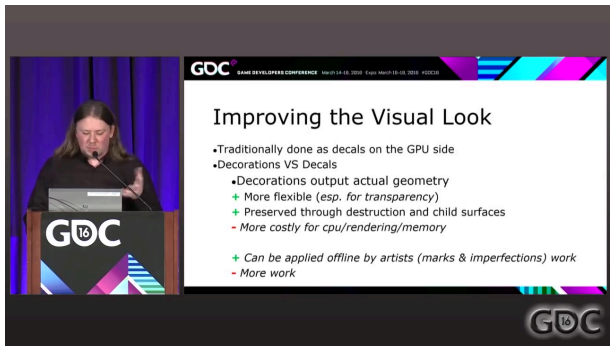
데코레이션은 실제 지오메트리로 구성됩니다. 2D 표면처럼 관리할 수도 있고, 표면 안쪽이나 앞뒤 등 원하는 위치에 배치할 수 있습니다. 투명 표현에도 응용할 수 있죠.



19:09–19:25
상당히 유연합니다. 벽이나 물체가 여러 부분으로 쪼개져도 그 생명주기 동안 데코레이션을 유지하고 관리할 수 있습니다.



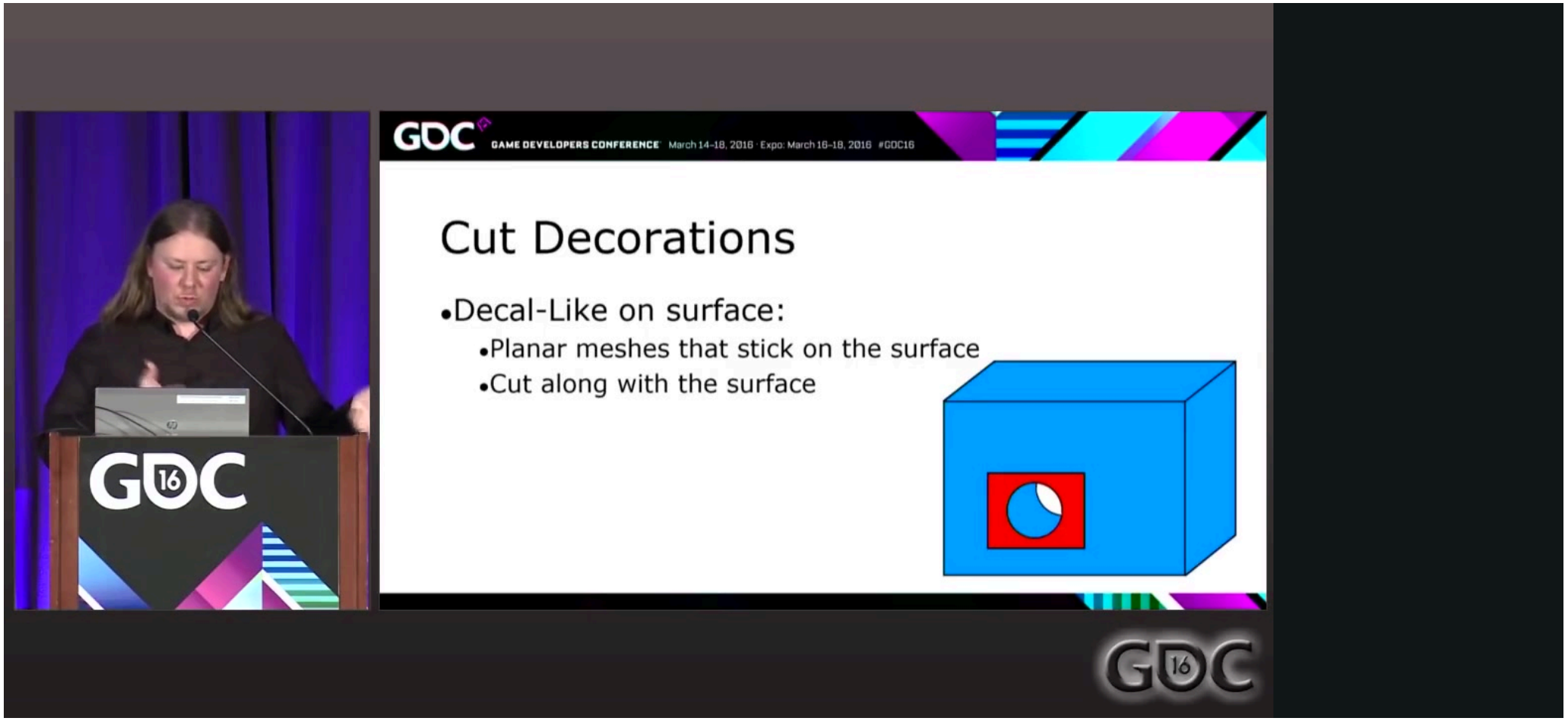
19:25–19:42
대신 CPU, 렌더링, 메모리 비용이 늘어납니다. 그래도 아티스트에게는 익숙한 작업 방식입니다. 원래도 데칼과 장식 요소를 벽 등에 배치해 외형을 다듬으니까요.



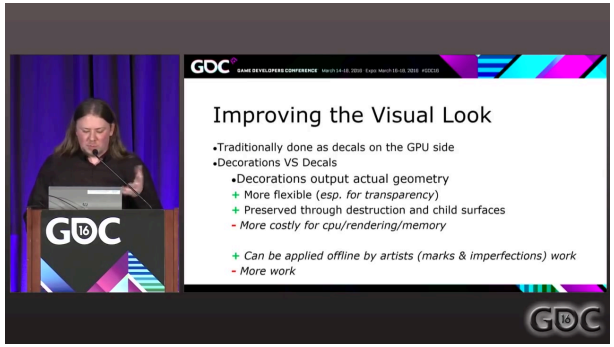
19:42–19:58
데코레이션은 기본적으로 두 종류입니다. 하나는 데칼처럼 표면에 붙는 장식입니다. 표면이 잘리면 장식도 함께 잘리므로 관리하기 쉽습니다.

056. Cut Decorations

영상 19:45–20:00 · 장면 캡처 19:47

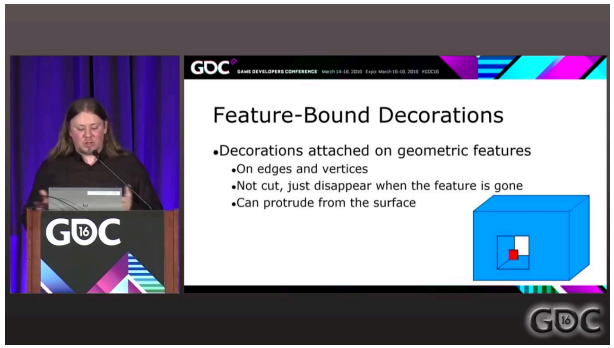


이 장면의 한국어 자막



19:42–19:58

데코레이션은 기본적으로 두 종류입니다. 하나는 데칼처럼 표면에 붙는 장식입니다. 표면이 잘리면 장식도 함께 잘리므로 관리하기 쉽습니다.

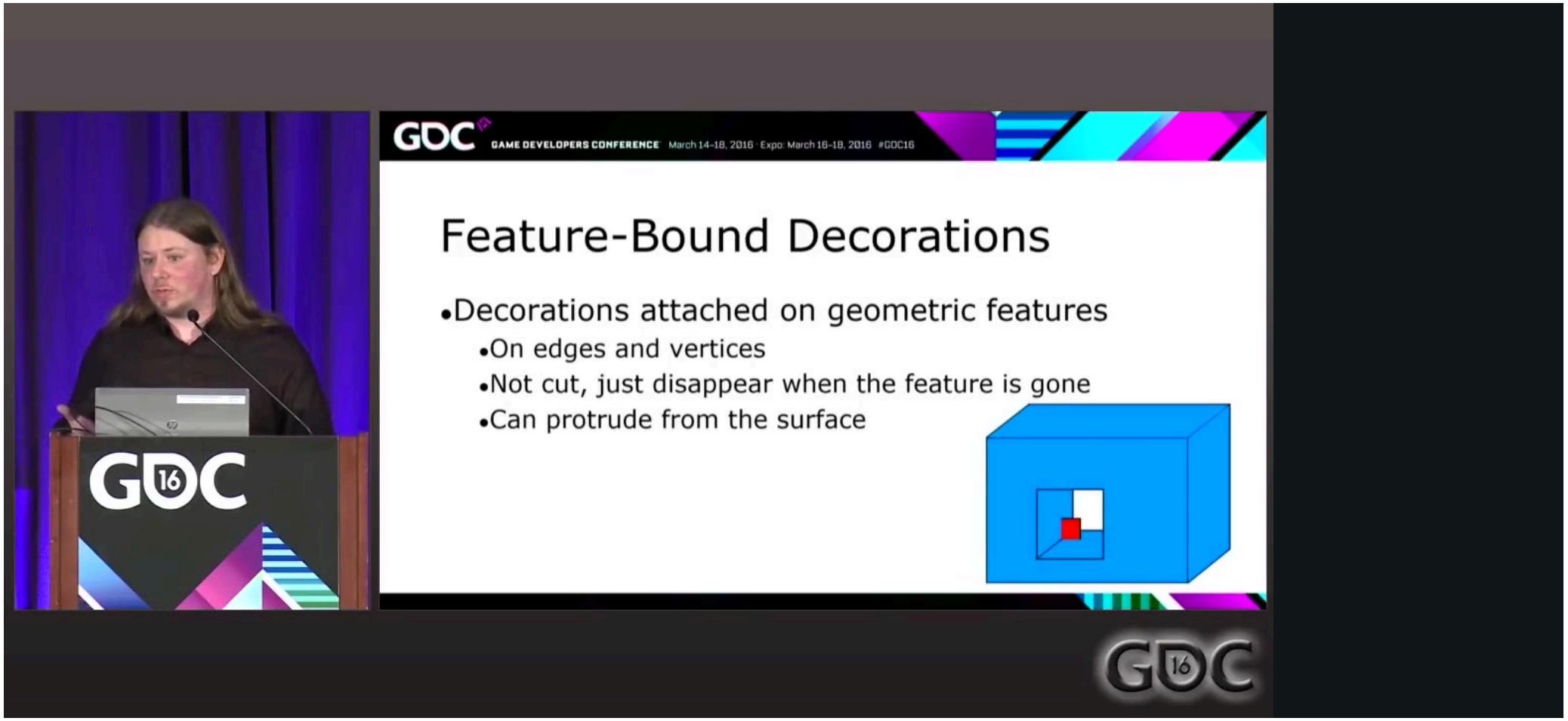


19:58–20:14

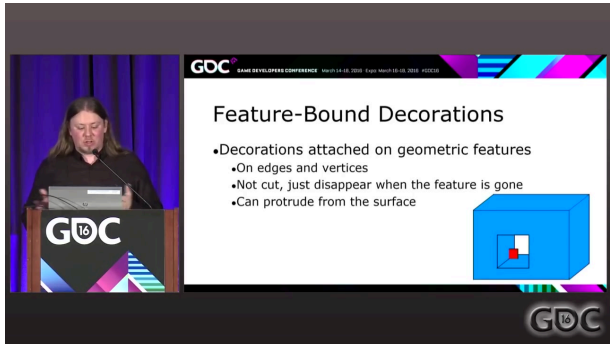
다른 하나는 정점이나 간선 같은 지오메트리 요소에 연결된 장식입니다. 내부에 무언가를 배치하거나 색을 입힐 수 있습니다. 연결된 요소가 없어지면 장식도 제거하면 됩니다.

057. Feature-Bound Decorations

영상 20:00–20:25 · 장면 캡처 20:02

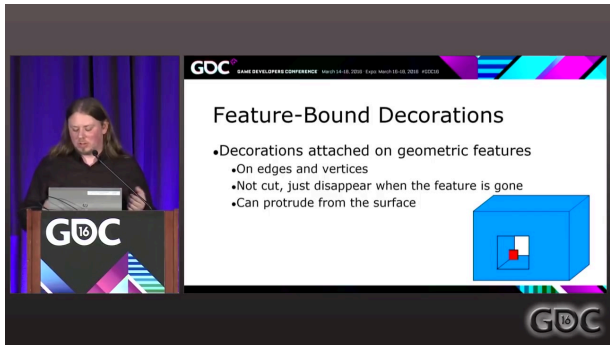


이 장면의 한국어 자막



19:58–20:14

다른 하나는 정점이나 간선 같은 지오메트리 요소에 연결된 장식입니다. 내부에 무언가를 배치하거나 색을 입힐 수 있습니다. 연결된 요소가 없어지면 장식도 제거하면 됩니다.



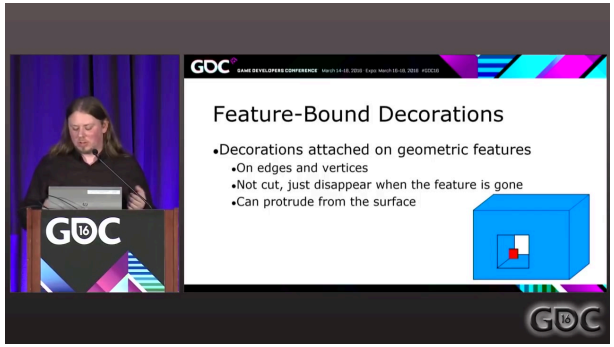
20:14–20:28

해당 간선이나 정점이 남아 있는지 확인하기 쉬우므로, 문제를 깔끔하게 해결할 수 있습니다. 결과를 보면 이런 모습이 됩니다.

058. 장식 결합 결과

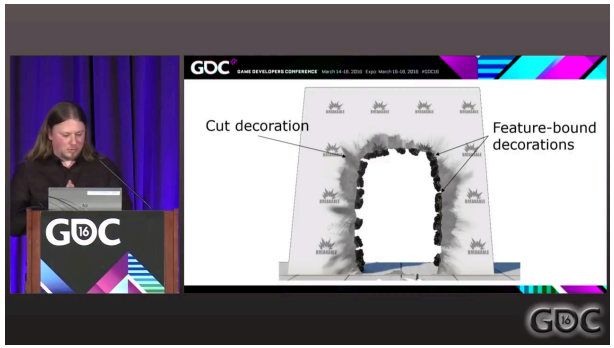
영상 20:25–20:45 · 장면 캡처 20:27

이 장면의 한국어 자막



20:14–20:28

해당 간선이나 정점이 남아 있는지 확인하기 쉬우므로, 문제를 깔끔하게 해결할 수 있습니다. 결과를 보면 이런 모습이 됩니다.



20:28–20:44

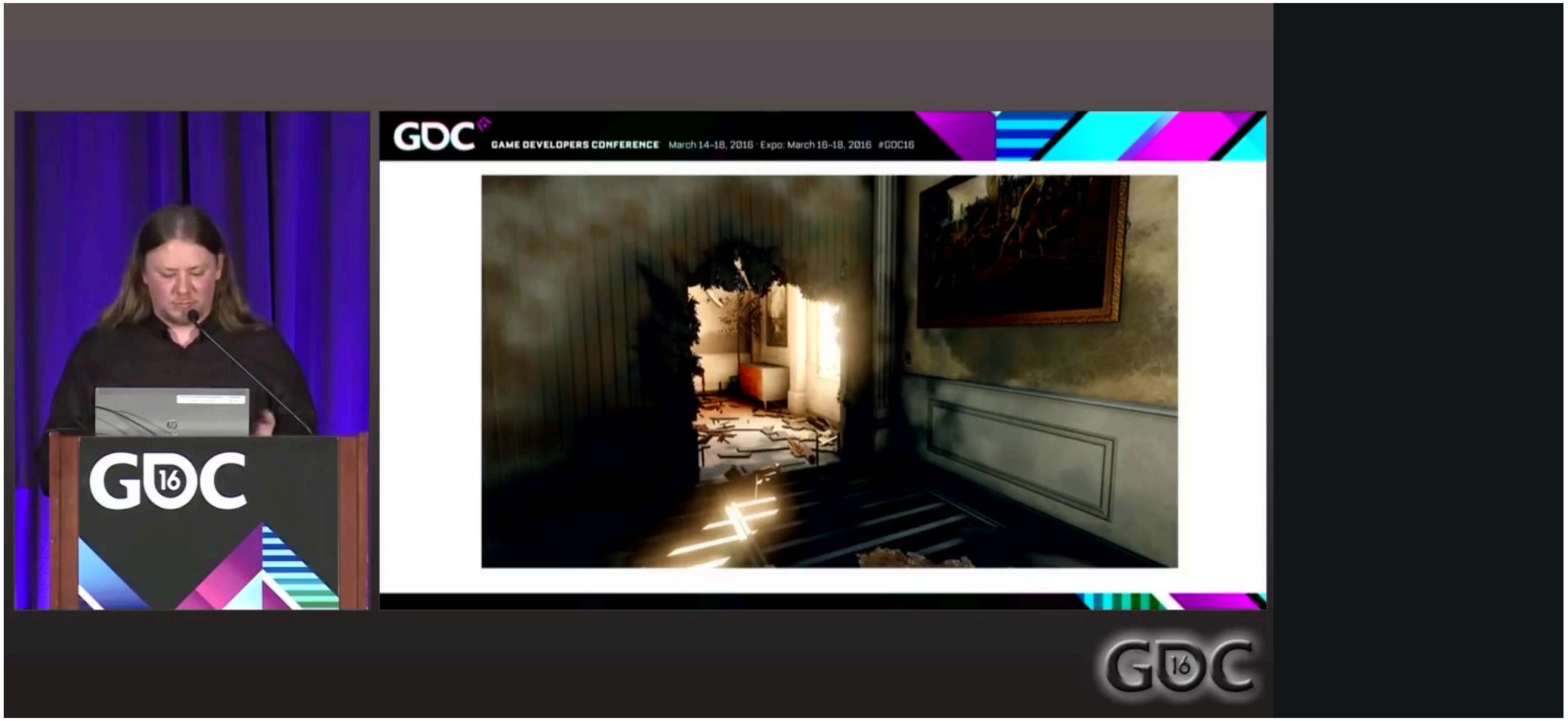
확실히 더 좋아졌죠? 아직 개발자용 임시 아트이지만 실제 텍스처를 적용했고, 절차적으로 부서진 자연스러운 느낌도 유지했습니다.

20:44–21:00

게임 안에서는 이렇게 보입니다. 이제 성능 문제로 넘어가죠. 레인보우 식스 시즈는 PvP에서 60FPS로 실행됩니다.

059. 파괴된 벽의 게임 화면

영상 20:45–20:50 · 장면 캡처 20:47



이 장면의 한국어 자막

20:44–21:00

게임 안에서는 이렇게 보입니다. 이제 성능 문제로 넘어가죠. 레인보우 식스 시즈는 PvP에서 60FPS로 실행됩니다.

060. Destruction & Performance

영상 20:50–21:10 · 장면 캡처 20:52

이 장면의 한국어 자막

20:44–21:00

게임 안에서는 이렇게 보입니다. 이제 성능 문제로 넘어가죠. 레인보우 식스 시즈는 PvP에서 60FPS로 실행됩니다.

21:00–21:17

파괴가 없어도 만만치 않은 목표입니다. 파괴 시스템을 추가하면 필요한 정보를 전달하더라도 다른 시스템에 영향이 생깁니다. AI, 내비게이션, 사운드 전파를 동적으로 처리해야 하니까요.

061. Impact on Other Systems

영상 21:10–21:55 · 장면 캡처 21:12

이 장면의 한국어 자막

21:00–21:17

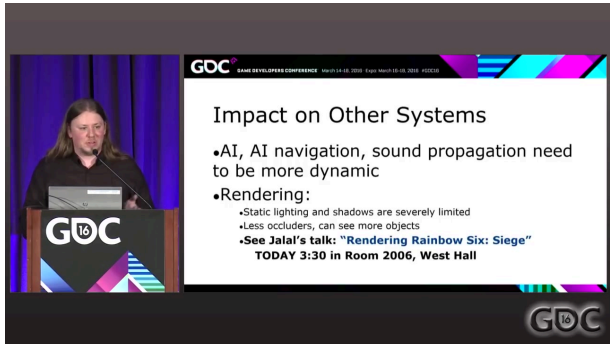
파괴가 없어도 만만치 않은 목표입니다. 파괴 시스템을 추가하면 필요한 정보를 전달하더라도 다른 시스템에 영향이 생깁니다. AI, 내비게이션, 사운드 전파를 동적으로 처리해야 하니까요.

21:17–21:34

렌더링 측면에서도 정적 라이팅을 쓰기 어려워지고 동적 그림자가 늘어납니다. 벽이 언제든 부서질 수 있으므로 오클루더가 줄어들고, 더 많은 오브젝트가 화면에 보이게 됩니다.

21:34–21:50

이것만으로도 복잡한 주제입니다. 관심 있는 분은 바로 다음에 열리는 질의 렌더링 관련 발표를 보셔도 좋겠습니다.

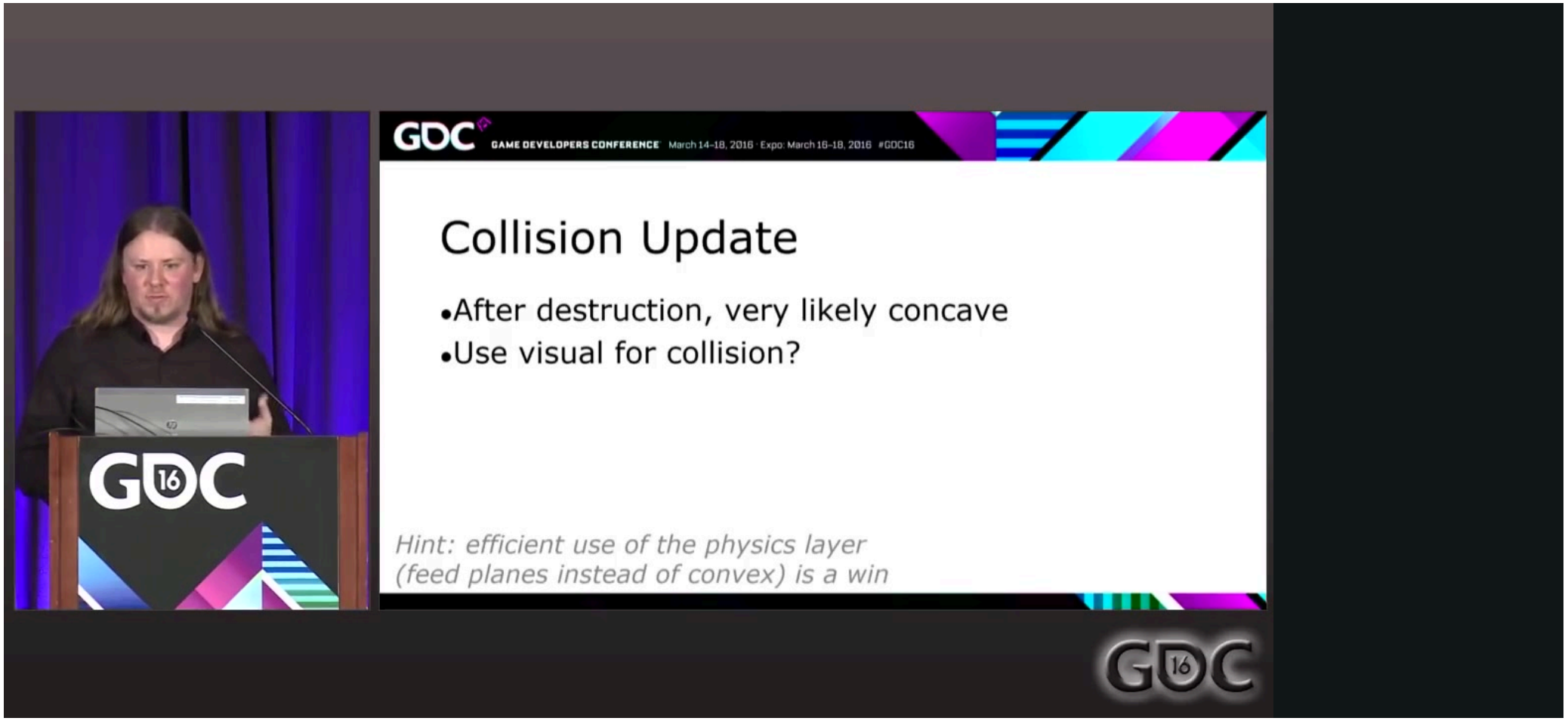


21:50–22:07

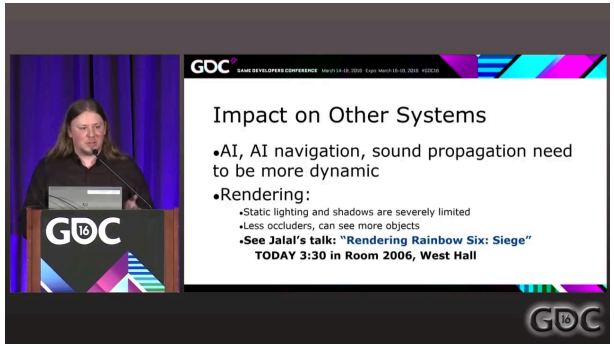
물리는 어떨까요? 물체를 파괴하면 충돌 형상이 오목한 형태가 되기 쉽습니다. 이를 그대로 물리 엔진에 맡겨 처리할 수도 있겠지만, 비용이 엄청나게 커질 수 있습니다.

062. Collision Update

영상 21:55–22:15 · 장면 캡처 21:57



이 장면의 한국어 자막



21:50–22:07

물리는 어떨까요? 물체를 파괴하면 충돌 형상이 오목한 형태가 되기 쉽습니다. 이를 그대로 물리 엔진에 맡겨 처리할 수도 있겠지만, 비용이 엄청나게 커질 수 있습니다.

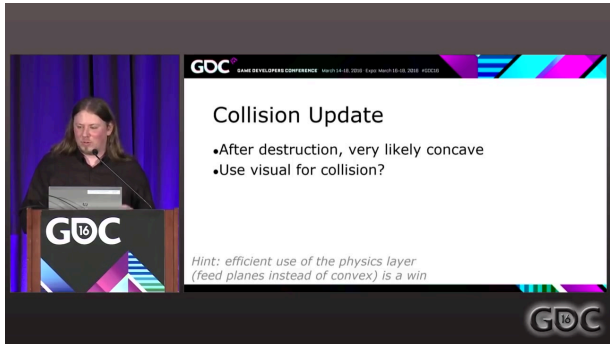
22:07–22:23

따라서 충돌 데이터를 적당히 단순화해야 합니다. 너무 작은 구멍은 제거할 수 있습니다. 캐릭터가 통과할 수도 없는 구멍은 물리 충돌에서 굳이 정밀하게 표현할 필요가 없죠.

063. Collision Update: Our Solution

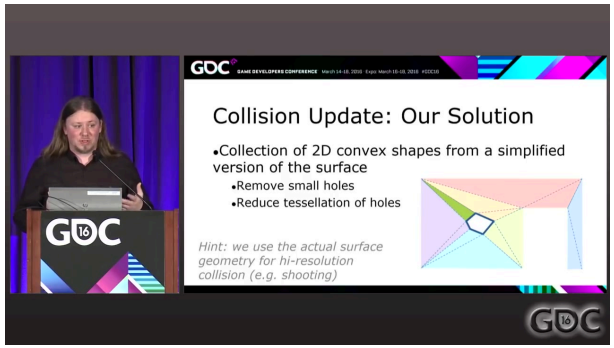
영상 22:15–22:50 · 장면 캡처 22:17

이 장면의 한국어 자막



22:07–22:23

따라서 충돌 데이터를 적당히 단순화해야 합니다. 너무 작은 구멍은 제거할 수 있습니다. 캐릭터가 통과할 수도 없는 구멍은 물리 충돌에서 굳이 정밀하게 표현할 필요가 없죠.



22:23–22:38

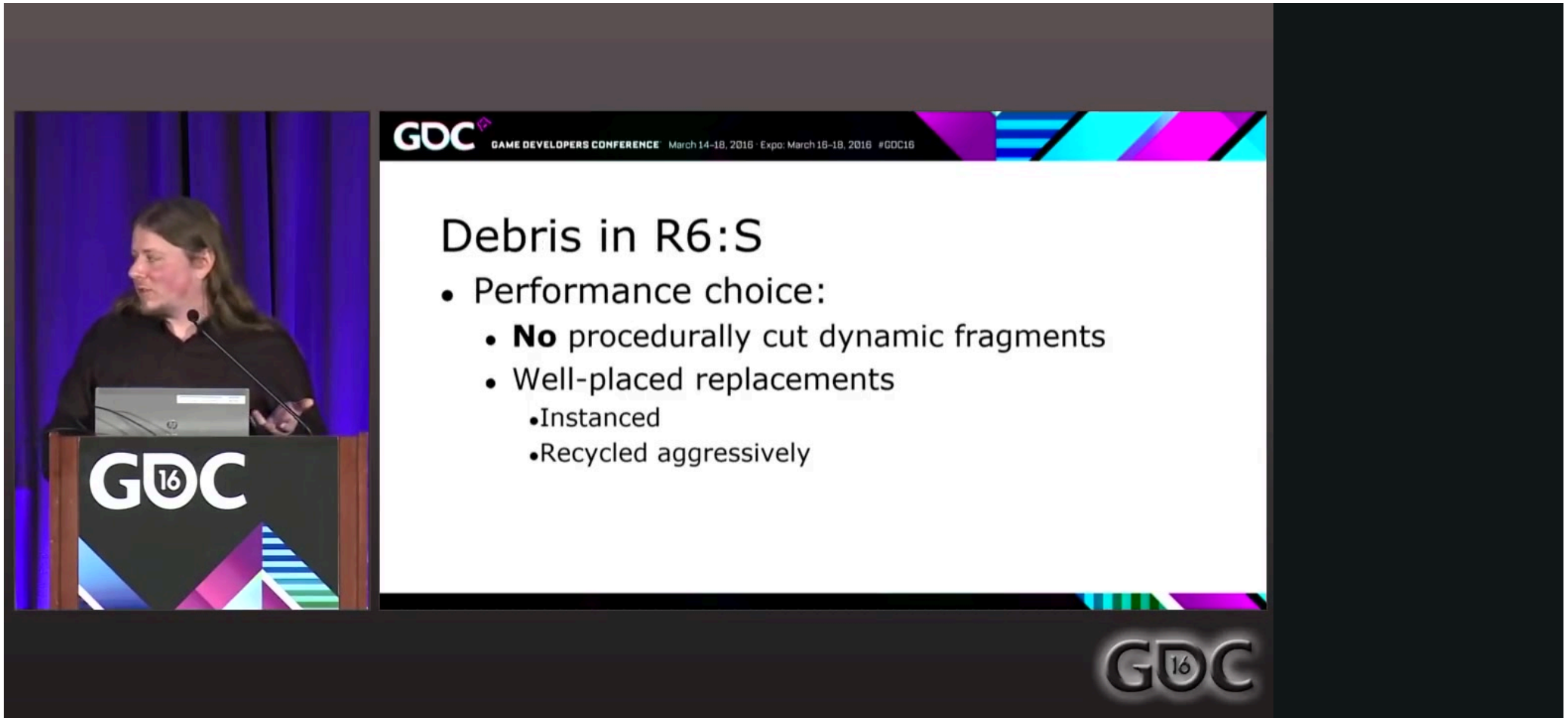
테셀레이션도 줄일 수 있지만 원래 표면의 형상을 해치지 않도록 주의해야 합니다. 저희는 여러 볼록 형상으로 충돌을 표현하는 방식을 선택했습니다.

22:38–22:55

앞면과 뒷면의 평면을 알고 있으므로 2D에서 볼록 껍질을 계산하면 됩니다. 훨씬 빠르죠. 그럼 파편은 어떨까요? 게임에는 수많은 잔해가 등장합니다.

064. Debris in R6:S: 재사용

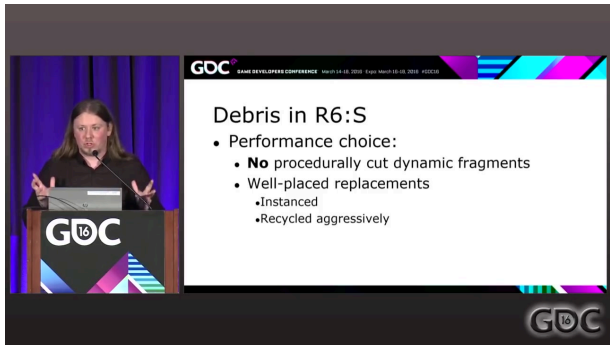
영상 22:50–23:35 · 장면 캡처 22:52



이 장면의 한국어 자막

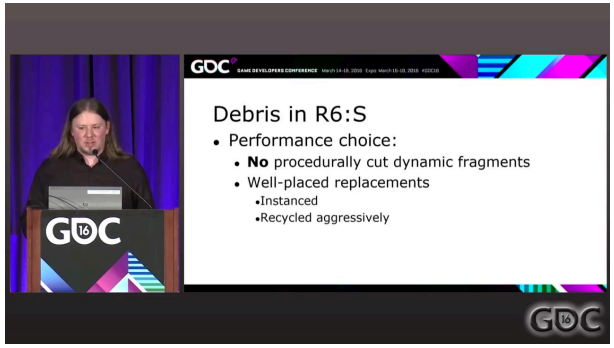
22:38–22:55

앞면과 뒷면의 평면을 알고 있으므로 2D에서 볼록 껍질을 계산하면 됩니다. 훨씬 빠르죠. 그럼 파편은 어떨까요? 게임에는 수많은 잔해가 등장합니다.



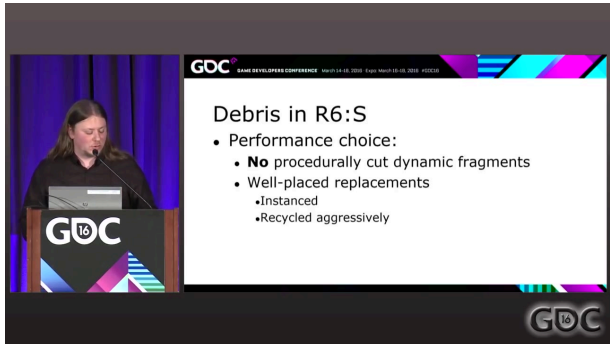
22:55–23:10

하지만 이 잔해들은 절차적으로 절단해 만든 것이 아닙니다. 프로젝트 초기에 그 비용은 피할 수 있다고 판단했죠. 앞서 말한 파괴 이벤트 출력 시스템을 활용했습니다.



23:10–23:28

충분히 자세한 출력 정보가 있으면 적절한 위치에 적절한 물리 특성을 가진 메시를 배치할 수 있습니다. 이 메시를 인스턴싱하고 적극적으로 재활용하면 물리 처리 비용도 줄일 수 있죠.



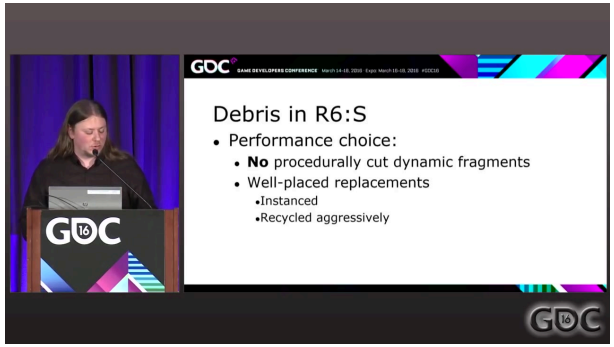
23:28–23:44

그 밖에도 작은 파편끼리는 충돌하지 않도록 할 수 있습니다. 폭발 한 번에 복잡한 파편 500개가 만들어지더라도, 화면에는 그보다 훨씬 적은 수만 보여 줘도 충분할 수 있습니다.

065. Debris in R6:S: 충돌과 효과

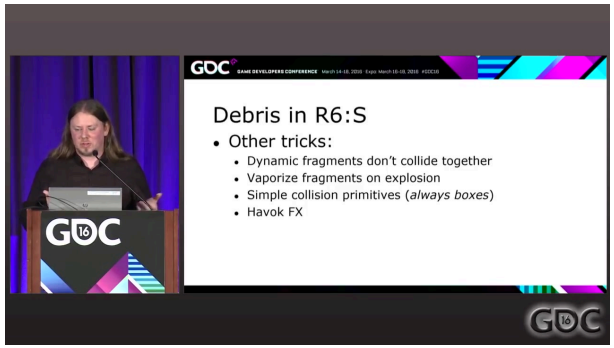
영상 23:35–23:55 · 장면 캡처 23:37

이 장면의 한국어 자막



23:28–23:44

그 밖에도 작은 파편끼리는 충돌하지 않도록 할 수 있습니다. 폭발 한 번에 복잡한 파편 500개가 만들어지더라도, 화면에는 그보다 훨씬 적은 수만 보여 줘도 충분할 수 있습니다.



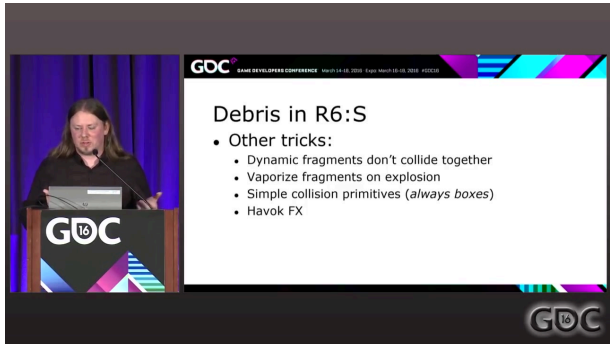
23:44–24:01

충돌체는 가능한 한 박스를 활용하고, 아주 작은 디테일은 VFX로 대신할 수도 있습니다. 이제 파괴 시스템 자체의 성능을 이야기해 보죠. 잠깐 물을 마시겠습니다.

066. Destruction Performance

영상 23:55–24:05 · 장면 캡처 23:57

이 장면의 한국어 자막



23:44–24:01

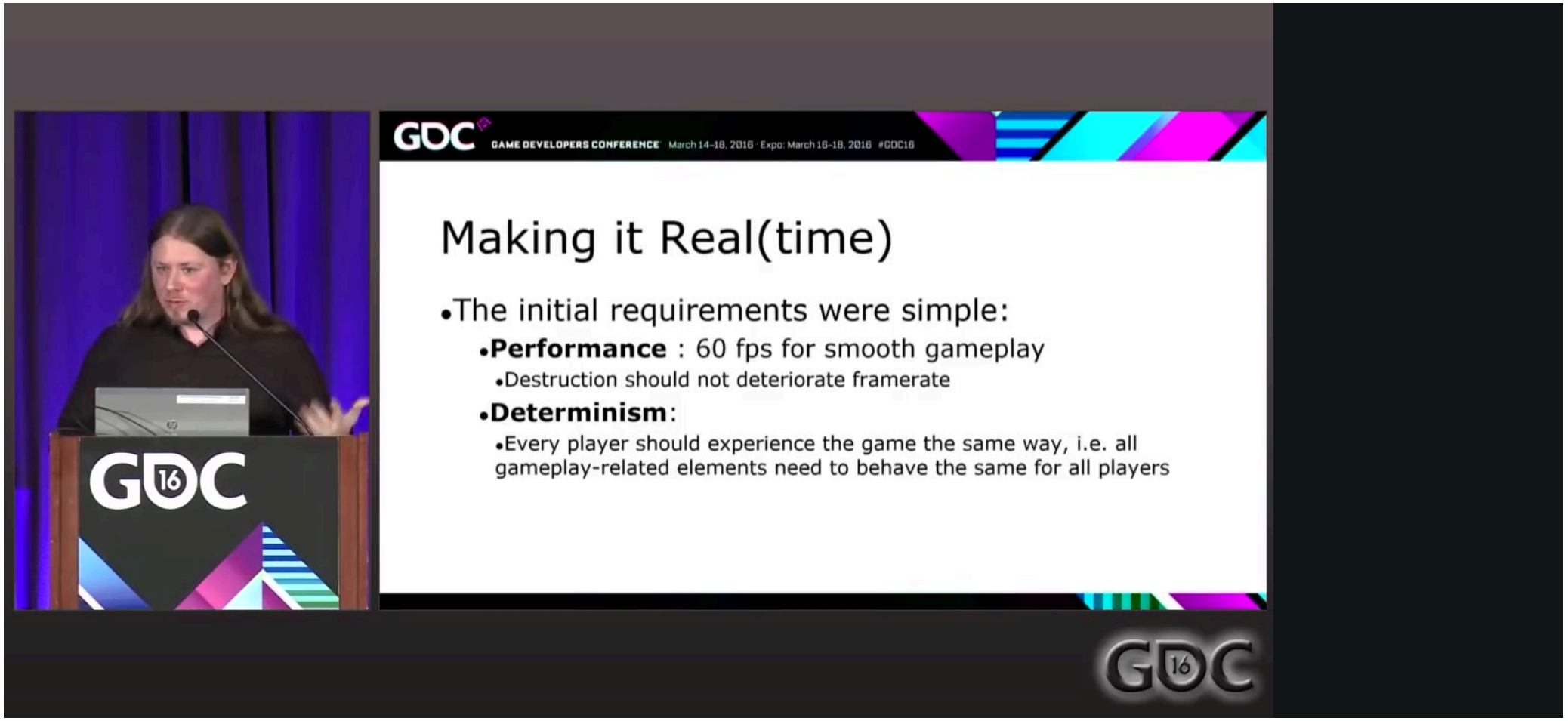
충돌체는 가능한 한 박스를 활용하고, 아주 작은 디테일은 VFX로 대신할 수도 있습니다. 이제 파괴 시스템 자체의 성능을 이야기해 보죠. 잠깐 물을 마시겠습니다.

24:01–24:15

60FPS는 타협할 수 없는 요구사항이었습니다. 그리고 파괴 때문에 게임 성능이 저하되지 않아야 했죠.

067. Making it Real(time)

영상 24:05–24:30 · 장면 캡처 24:07



이 장면의 한국어 자막

24:01–24:15

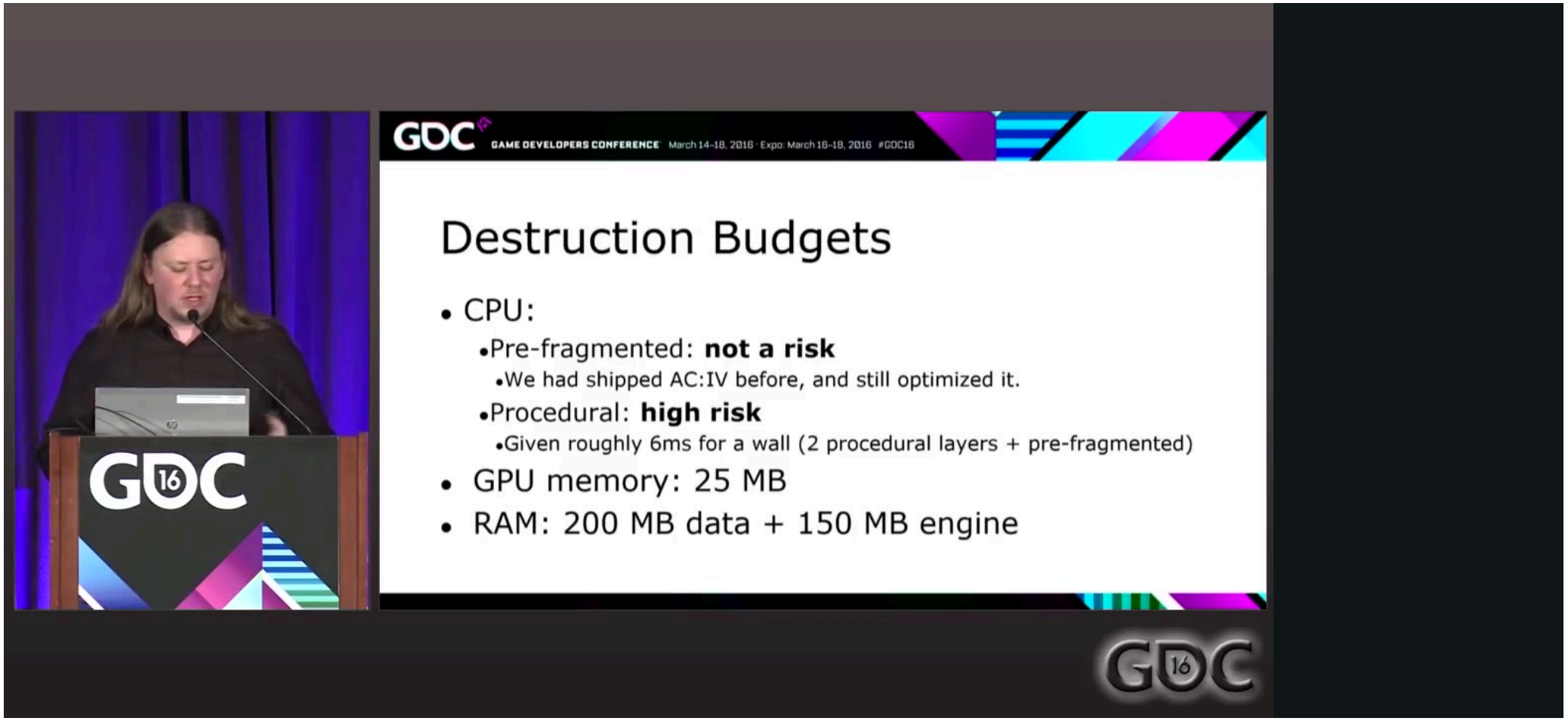
60FPS는 타협할 수 없는 요구사항이었습니다. 그리고 파괴 때문에 게임 성능이 저하되지 않아야 했죠.

24:16–24:32

말은 쉽지만 실제로는 어렵습니다. 이 부분이 바로 그런 문제에 대한 이야기입니다. 온라인 플레이를 위해 결정론적으로 동작해야 한다는 요구도 있었습니다. 모두 같은 파괴 결과를 봐야 하니까요.

068. Destruction Budgets

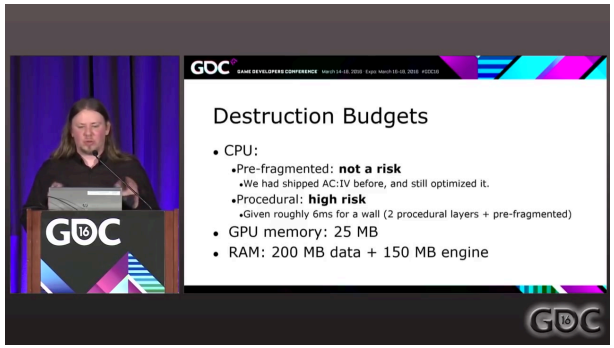
영상 24:30–25:00 · 장면 캡처 24:32



이 장면의 한국어 자막

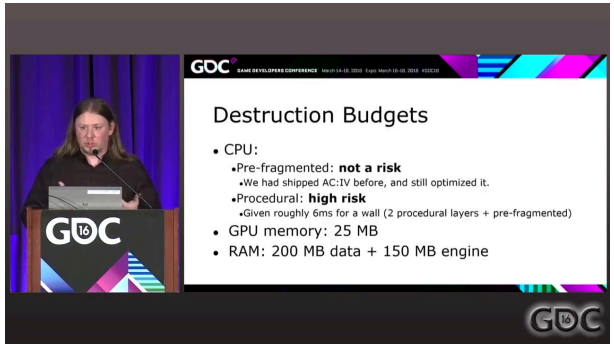
24:16–24:32

말은 쉽지만 실제로는 어렵습니다. 이 부분이 바로 그런 문제에 대한 이야기입니다. 온라인 플레이를 위해 결정론적으로 동작해야 한다는 요구도 있었습니다. 모두 같은 파괴 결과를 봐야 하니까요.



24:32–24:48

처음부터 파괴 처리의 명확한 예산이 정해진 것은 아니었습니다. 블랙 플래그에서는 비교적 관리하기 쉬웠지만, 절차적 파괴는 위험성이 훨씬 큰 기능이었으니까요. 대략적인 처리 시간 예산을 받았습니다.

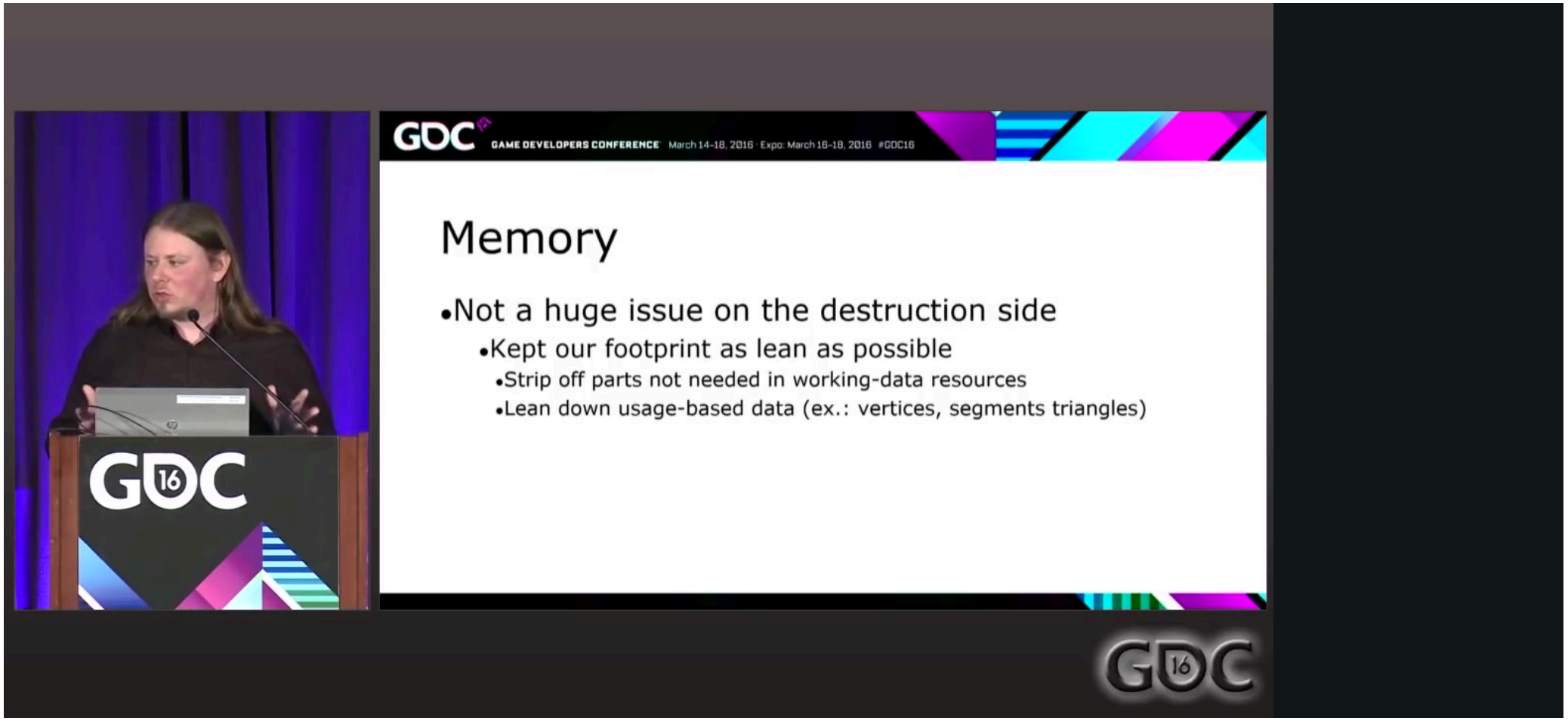


24:49–25:04

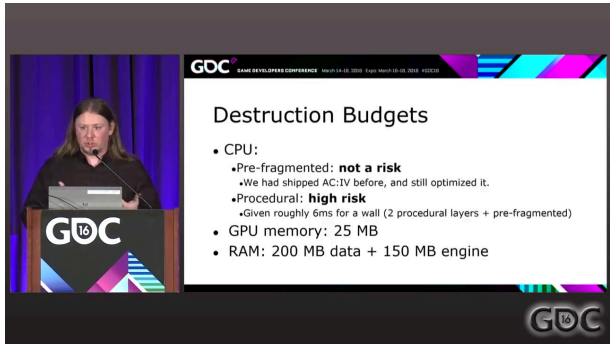
절차적 처리에 약 6밀리초 정도가 주어졌고, 메모리는 GPU 약 25MB, 그 외 약 350MB 수준이었습니다. 다만 메모리 자체는 개발 중 가장 큰 문제가 아니었습니다.

069. Memory

영상 25:00–25:10 · 장면 캡처 25:02



이 장면의 한국어 자막



24:49–25:04

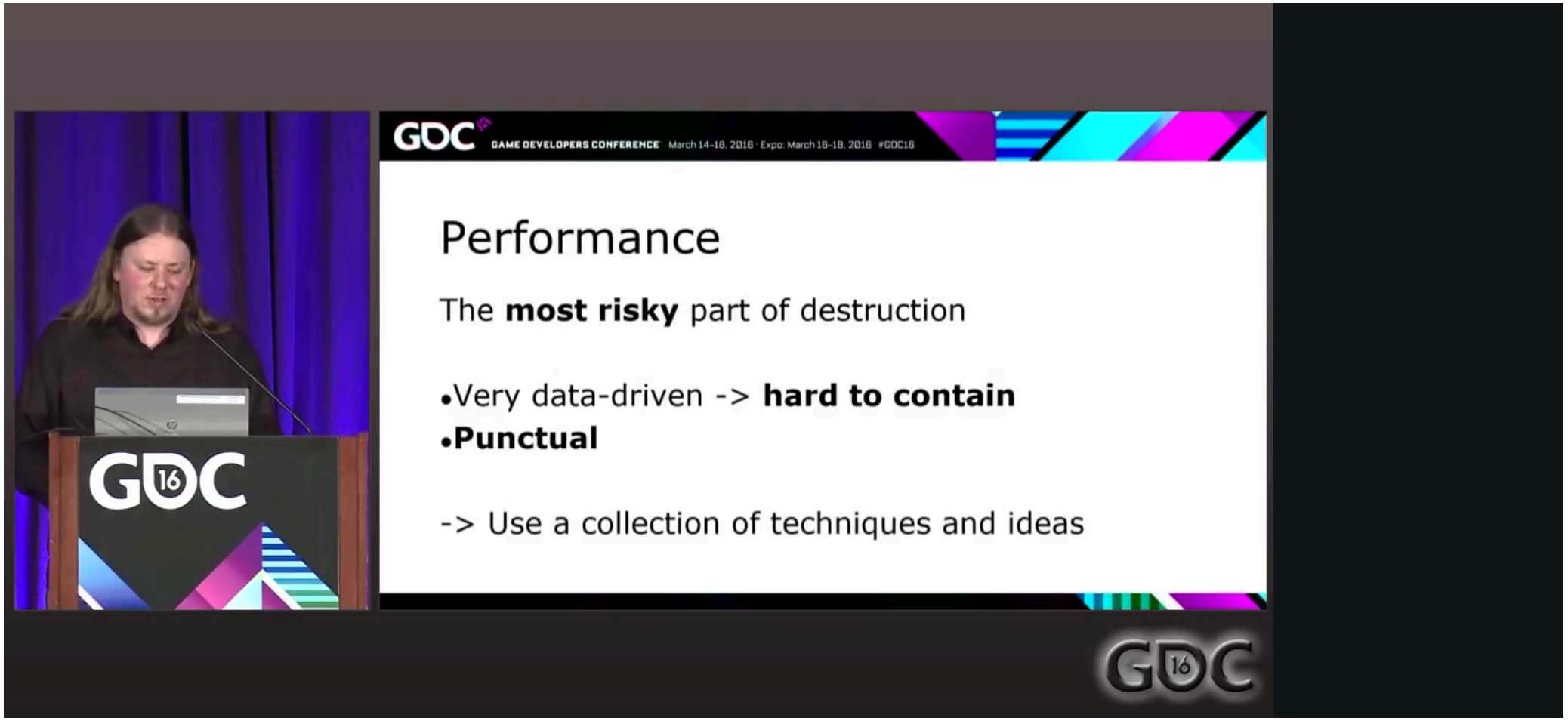
절차적 처리에 약 6밀리초 정도가 주어졌고, 메모리는 GPU 약 25MB, 그 외 약 350MB 수준이었습니다. 다만 메모리 자체는 개발 중 가장 큰 문제가 아니었습니다.

25:04–25:19

가능한 한 메모리 사용량을 작게 유지하려고 했지만, 가장 많은 노력을 들인 부분은 성능이었습니다. 프로젝트에서 위험성이 가장 큰 요소였거든요.

070. Performance

영상 25:10–25:35 · 장면 캡처 25:12



이 장면의 한국어 자막

25:04–25:19

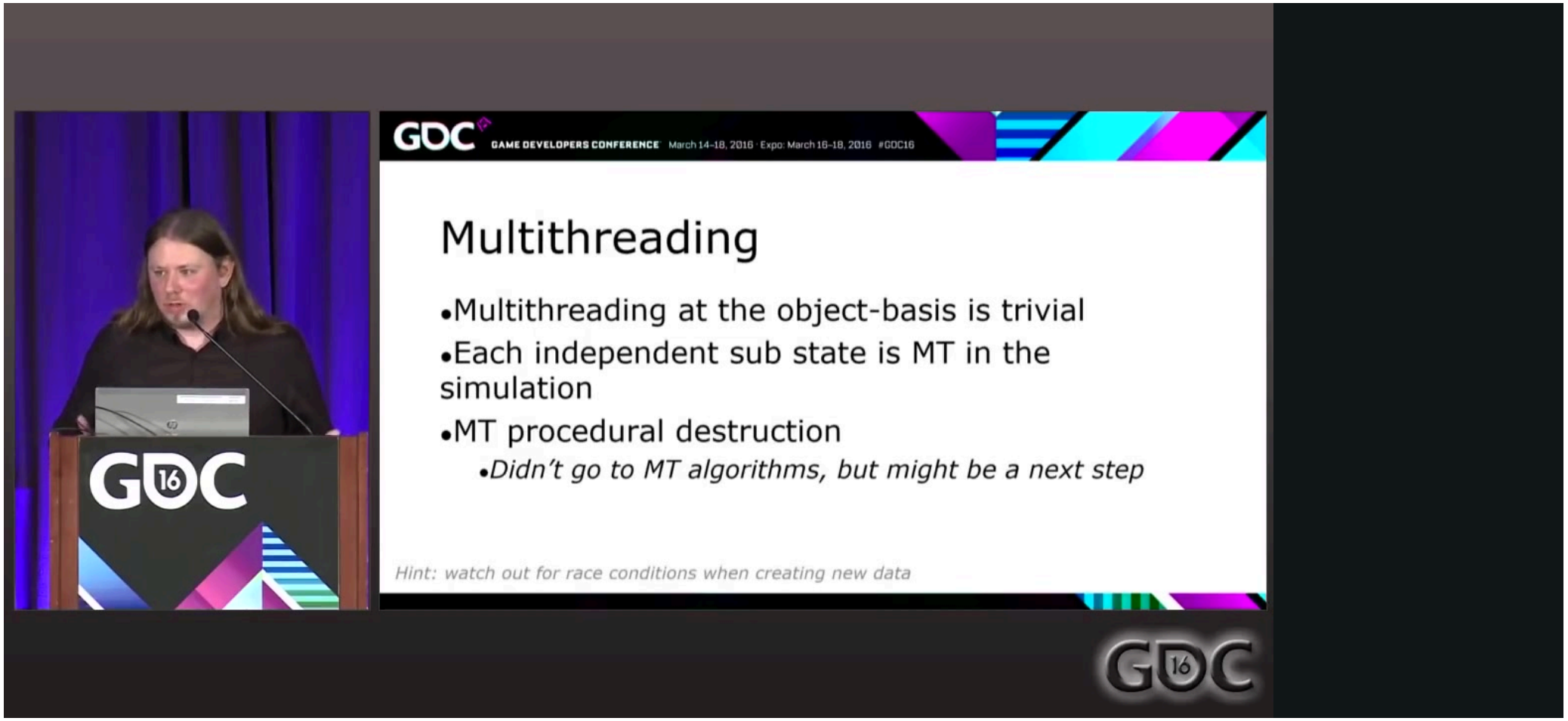
가능한 한 메모리 사용량을 작게 유지하려고 했지만, 가장 많은 노력을 들인 부분은 성능이었습니다. 프로젝트에서 위험성이 가장 큰 요소였거든요.

25:19–25:36

특히 데이터에 영향을 많이 받았습니다. 아티스트가 아주 복잡하고 멋진 결과물을 만들면 그만큼 비용이 올라갑니다. 이를 해결하기 위해 여러 기법을 사용했습니다.

071. Multithreading

영상 25:35–26:15 · 장면 캡처 25:37



이 장면의 한국어 자막

25:19–25:36

특히 데이터에 영향을 많이 받았습니다. 아티스트가 아주 복잡하고 멋진 결과물을 만들면 그만큼 비용이 올라갑니다. 이를 해결하기 위해 여러 기법을 사용했습니다.

25:36–25:53

처음 떠오르는 건 모든 걸 멀티스레딩하는 겁니다. 오브젝트 단위의 파괴는 서로 독립적으로 처리하기 쉽습니다. 파괴 후 여러 부분으로 분리된 오브젝트도 각자 시뮬레이션할 수 있죠.

25:53–26:08

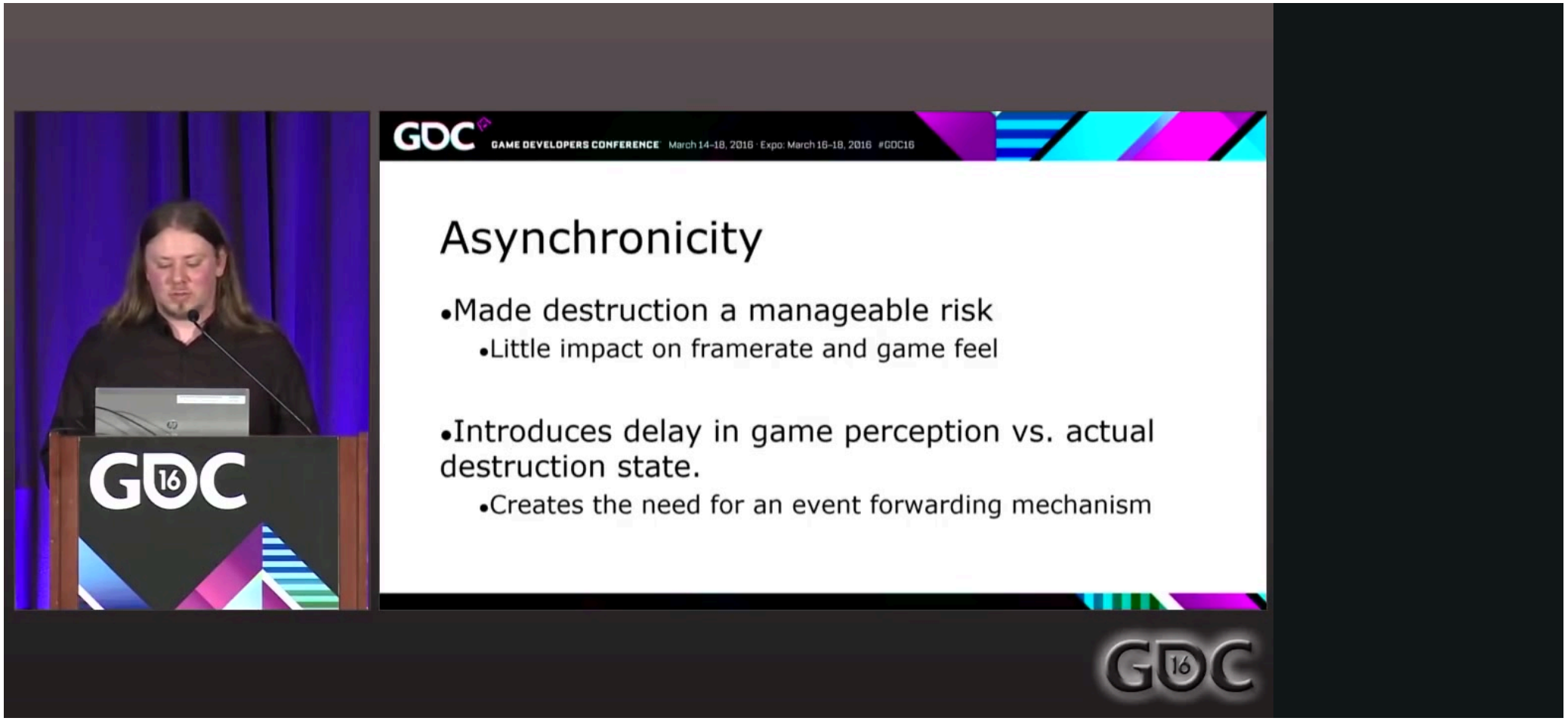
같은 오브젝트에 여러 파괴 입력이 들어와도 병렬화할 여지는 있습니다. 하지만 여전히 충분하지 않을 수 있고, 알고리즘 전체를 병렬화하는 일은 매우 복잡합니다.

26:08–26:23

저희는 그런 복잡한 방법 대신 파괴 처리 전체를 비동기식으로 만들었습니다. 이러면 파괴 계산이 무거워도 관리하기가 훨씬 쉬워집니다.

072. Asynchronicity

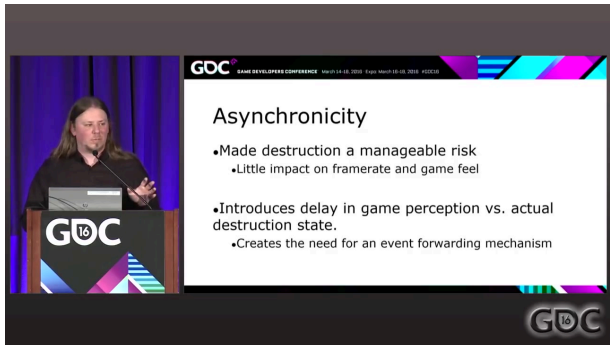
영상 26:15–27:00 · 장면 캡처 26:17



이 장면의 한국어 자막

26:08–26:23

저희는 그런 복잡한 방법 대신 파괴 처리 전체를 비동기식으로 만들었습니다. 이러면 파괴 계산이 무거워도 관리하기가 훨씬 쉬워집니다.



26:23–26:37

어떤 경우에는 파괴 계산이 100밀리초 걸리더라도 게임플레이 감각이나 렌더링에는 직접적인 영향이 없도록 할 수 있었습니다. 프로젝트 구성원들에게도 큰 부담을 덜어 주었죠.

26:37–26:51

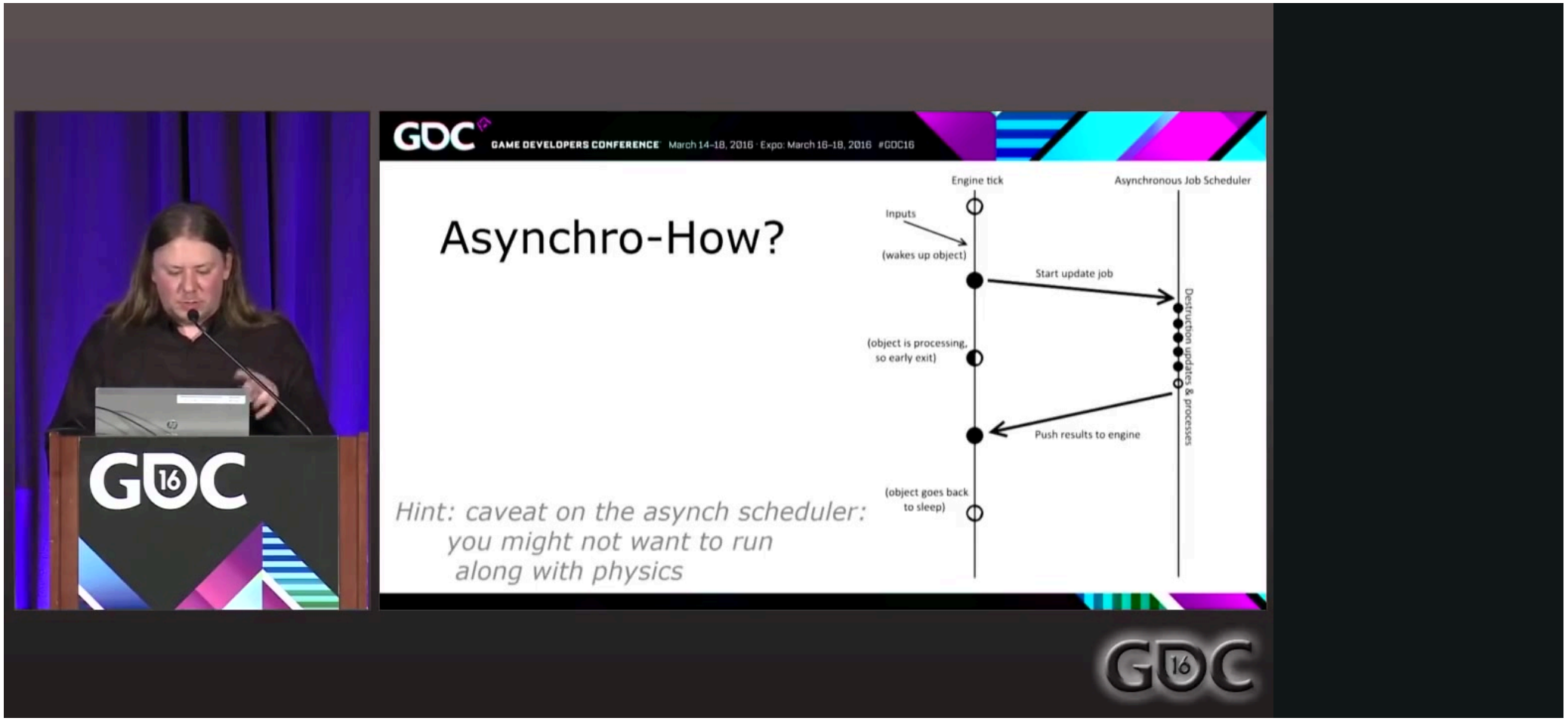
물론 단점은 있습니다. 실제 입력이 발생한 시점과 파괴가 화면에 반영되는 시점 사이에 지연이 생깁니다.

26:51–27:08

하지만 이펙트와 이벤트 전달 방식을 적절히 활용하면 그 지연을 감출 수 있습니다. 실제로 엔진은 늘 약간 이전 상태를 처리할 가능성이 있으니까요. 구현 과정을 살펴봅시다.

073. Asynchro-How?

영상 27:00–27:45 · 장면 캡처 27:02



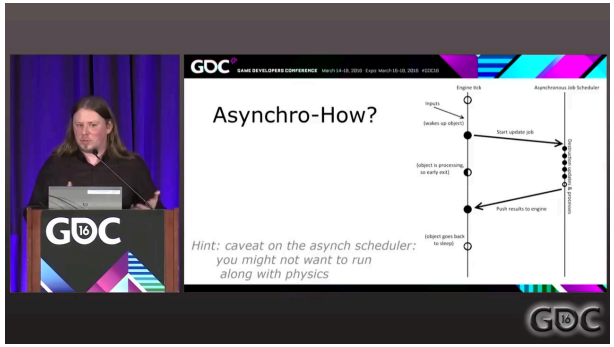
이 장면의 한국어 자막

26:51–27:08

하지만 이펙트와 이벤트 전달 방식을 적절히 활용하면 그 지연을 감출 수 있습니다. 실제로 엔진은 늘 약간 이전 상태를 처리할 가능성이 있으니까요. 구현 과정을 살펴봅시다.

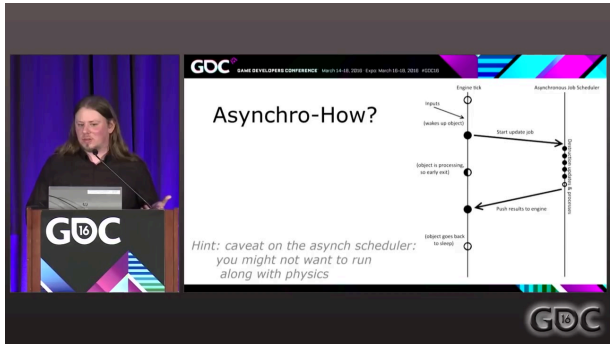
27:08–27:25

게임 엔진은 프레임마다 틱을 실행합니다. 파괴 입력이 들어오면 큰 작업을 시작하고, 나머지 엔진의 진행을 과도하게 방해하지 않는 비동기 작업 스케줄러에 넘깁니다.



27:25–27:41

엔진과 협의해 작업이 끝나기 전까지는 결과를 갱신할 필요가 없도록 합니다. 작업이 완료되면 그 결과를 엔진에 반영합니다. 그게 전부입니다.

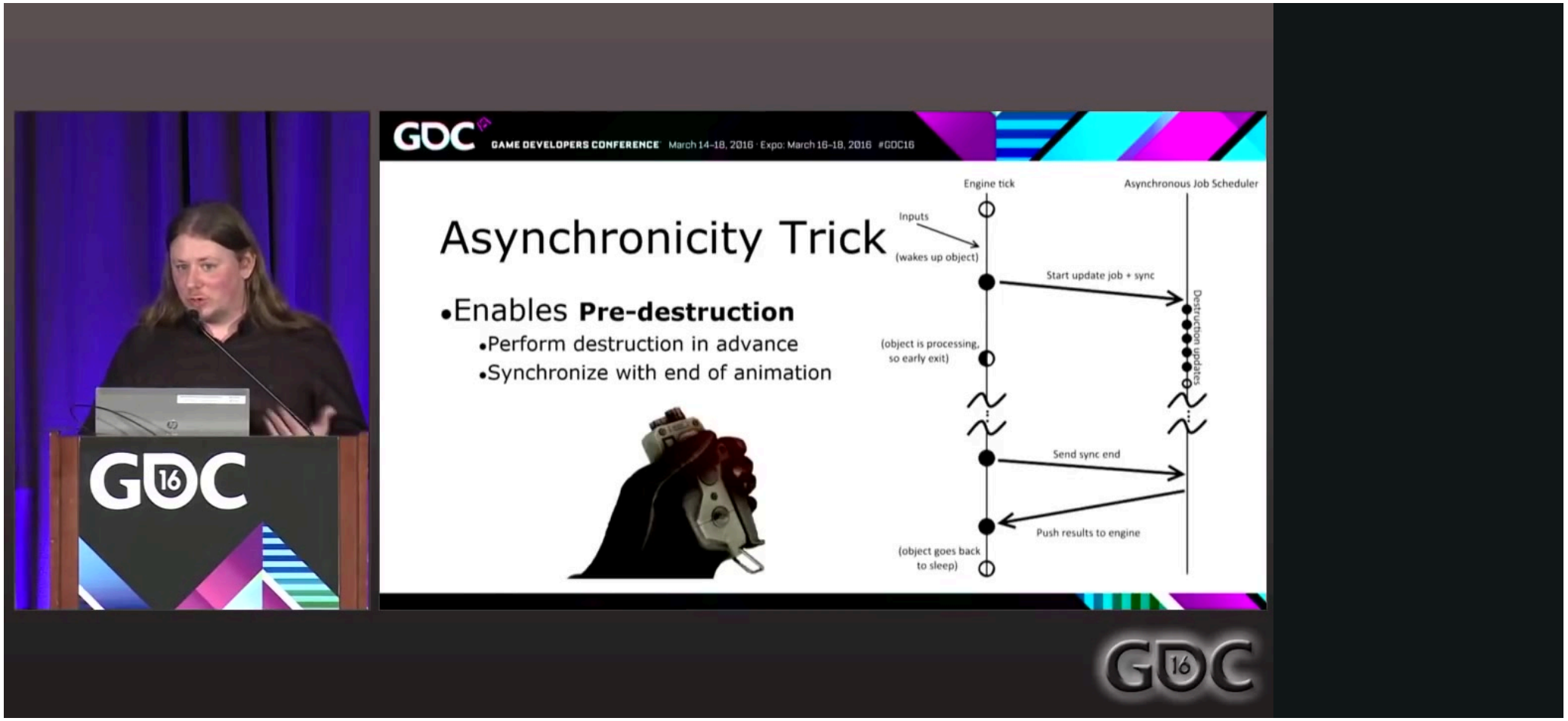


27:41–27:57

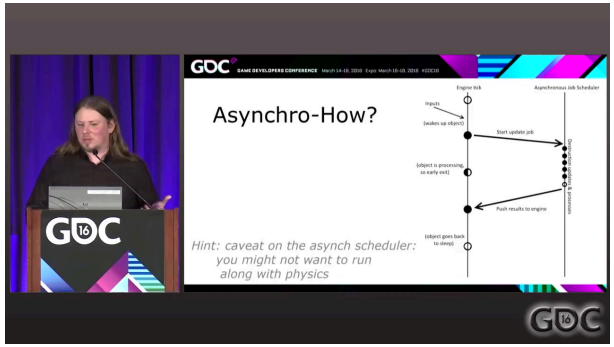
약간의 지연은 생기지만, 파괴 계산이 지나치게 느리지 않다면 거의 눈에 띄지 않습니다. 더 나아가 파괴가 일어날 시점을 예측할 수 있으면 미리 계산할 수도 있습니다.

074. Asynchronicity Trick

영상 27:45–28:35 · 장면 캡처 27:47

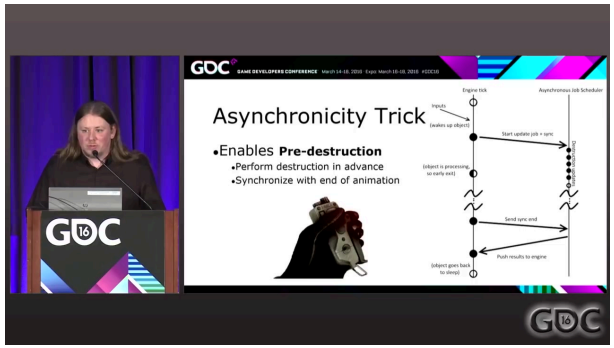


이 장면의 한국어 자막



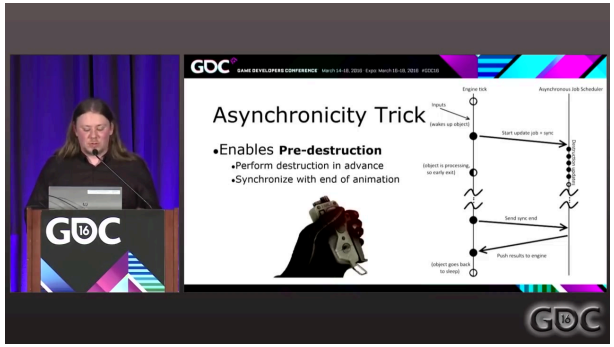
27:41–27:57

약간의 지연은 생기지만, 파괴 계산이 지나치게 느리지 않다면 거의 눈에 띄지 않습니다. 더 나아가 파괴가 일어날 시점을 예측할 수 있으면 미리 계산할 수도 있습니다.



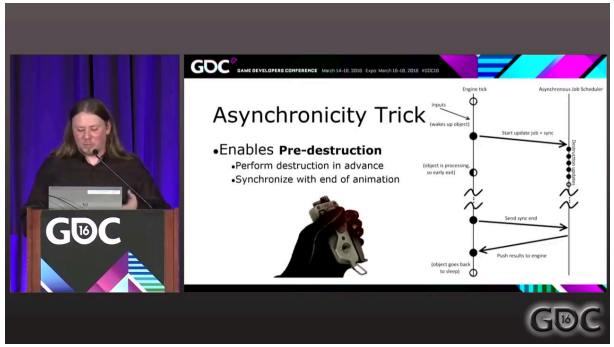
27:57–28:14

예를 들어 캐릭터의 애니메이션이 시작되어 10프레임 뒤에 폭발이 일어날 것을 안다면, 파괴 작업을 미리 실행할 수 있죠. 그러면 애니메이션이 끝나는 시점에 결과를 맞춰 반영할 수 있습니다.



28:14–28:29

게임플레이 쪽에서 미리 입력을 전달하고, 실제 결과가 필요한 순간에 받아 가는 겁니다. 시즈의 돌격 장약 같은 요소에 적용했고, 아주 잘 작동했습니다.

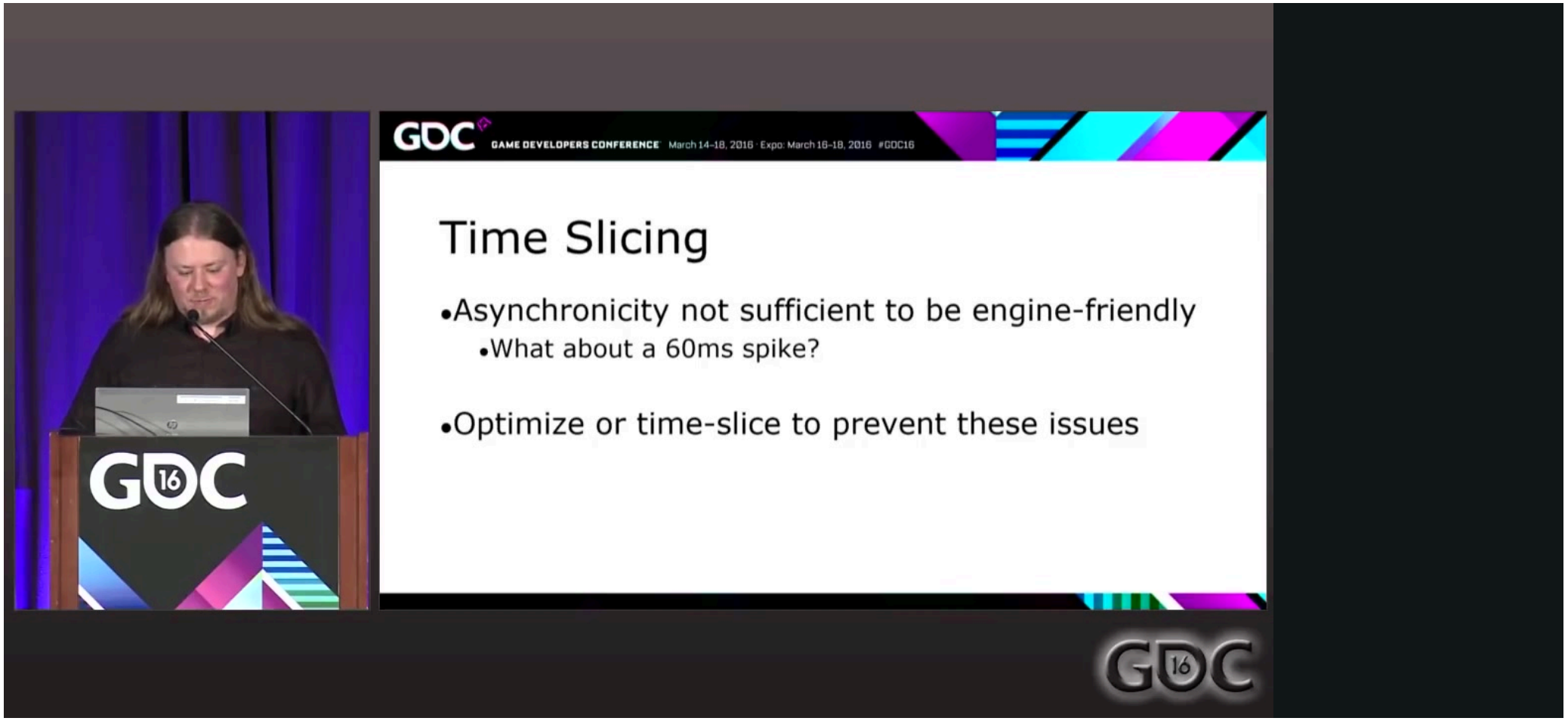


28:29–28:46

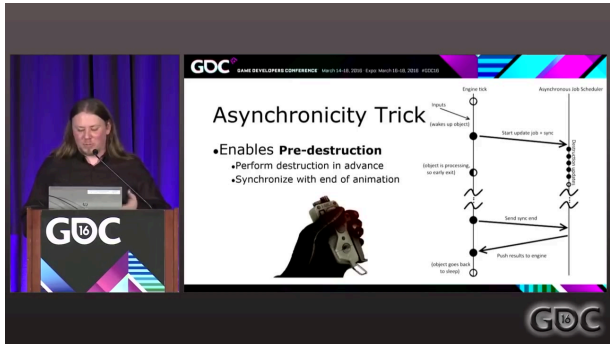
원한다면 파괴의 세부 표현을 더 늘릴 수도 있었지만 다른 시스템에 영향을 줄 수 있어 그러진 않았습니다. 다음은 작업을 시간 단위로 잘게 나누는 타임 슬라이싱입니다.

075. Time Slicing

영상 28:35–29:15 · 장면 캡처 28:37



이 장면의 한국어 자막



28:29–28:46

원한다면 파괴의 세부 표현을 더 늘릴 수도 있었지만 다른 시스템에 영향을 줄 수 있어 그러진 않았습니다. 다음은 작업을 시간 단위로 잘게 나누는 타임 슬라이싱입니다.

28:46–29:01

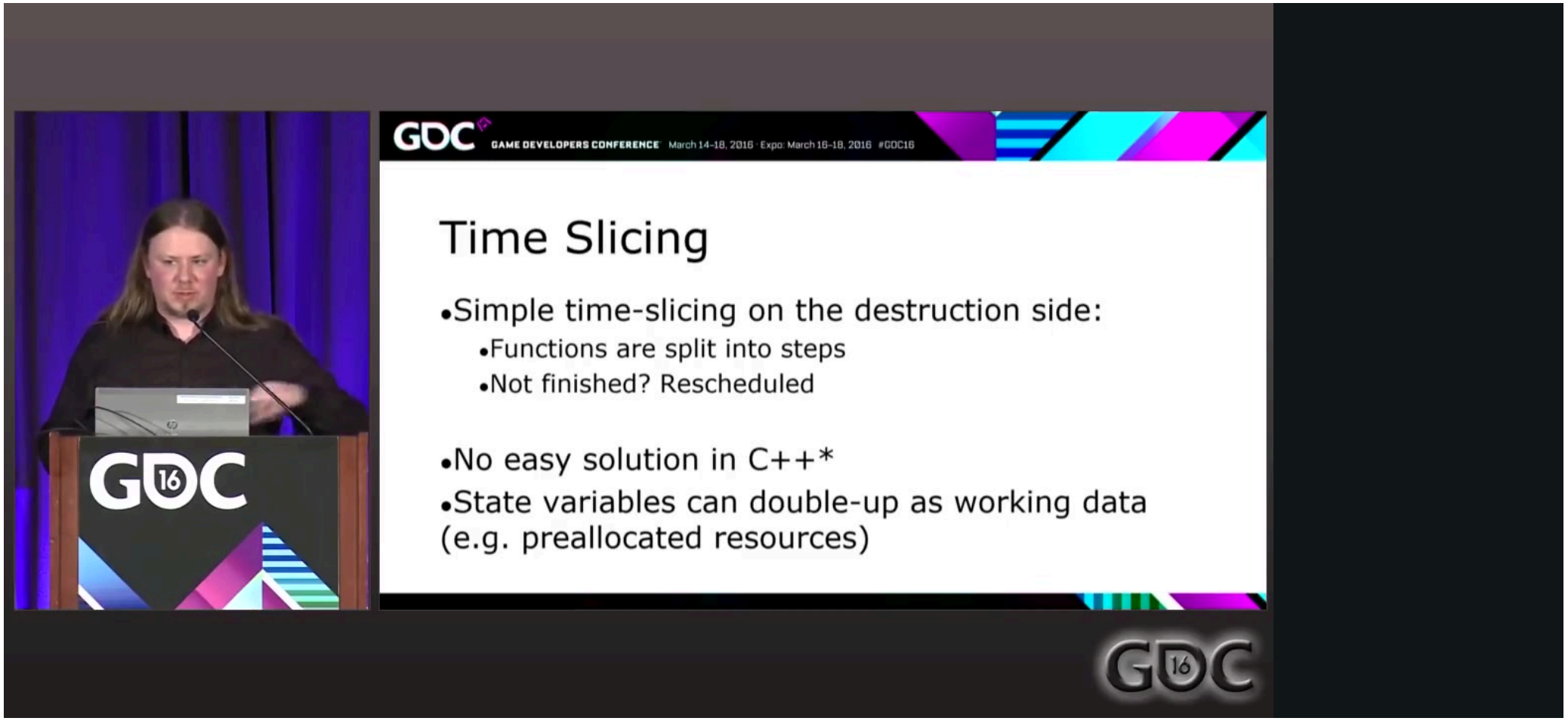
비동기 파괴 작업이라도 100밀리초씩 독점하면 다른 중요한 작업을 밀어내 프레임레이트를 떨어뜨릴 수 있습니다. 엔진 개발자들이 좋아할 리 없죠. 그래서 작업을 더 작게 쪼개야 합니다.

29:01–29:18

잘게 나누면 스케줄링 오버헤드 때문에 전체 완료 시간은 늘어날 수 있습니다. 그래도 프레임 안정성에는 큰 도움이 됩니다. 저희는 먼저 큰 단계로 나눈 다음,

076. Time Slicing: 작업 상태 보존

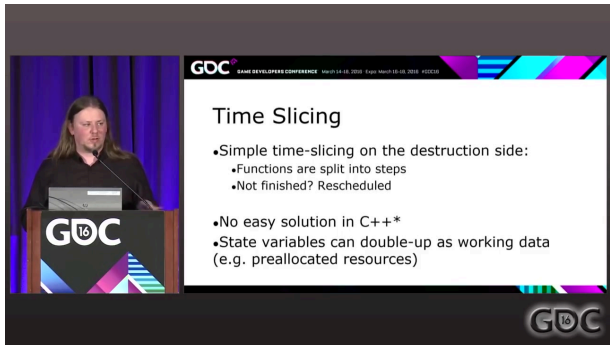
영상 29:15–30:05 · 장면 캡처 29:17



이 장면의 한국어 자막

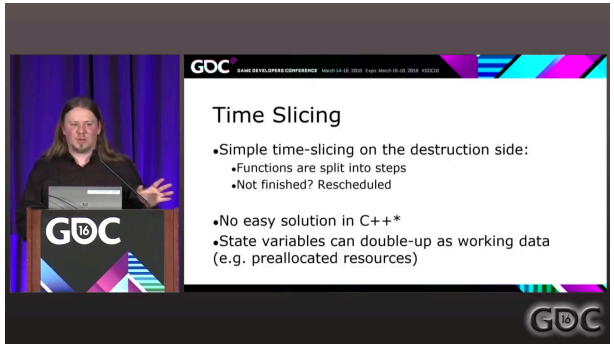
29:01–29:18

잘게 나누면 스케줄링 오버헤드 때문에 전체 완료 시간은 늘어날 수 있습니다. 그래도 프레임 안정성에는 큰 도움이 됩니다. 저희는 먼저 큰 단계로 나눈 다음,



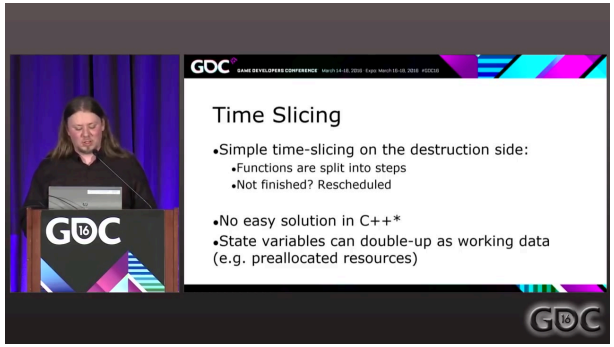
29:18–29:34

하위 단계에서는 더 세밀한 타임 슬라이싱을 적용했습니다. C++에서 이를 구현하는 뻔하고 간단한 해법은 없습니다. 퓨처를 쓰거나 스레드를 멈추는 방법도 있겠죠.



29:34–29:50

하지만 게임 엔진에서는 스케줄러와 조화롭게 동작해야 합니다. 그래서 저희는 현재 어떤 작업을 어디까지 진행했는지 기록하는 상태 변수를 사용했습니다.

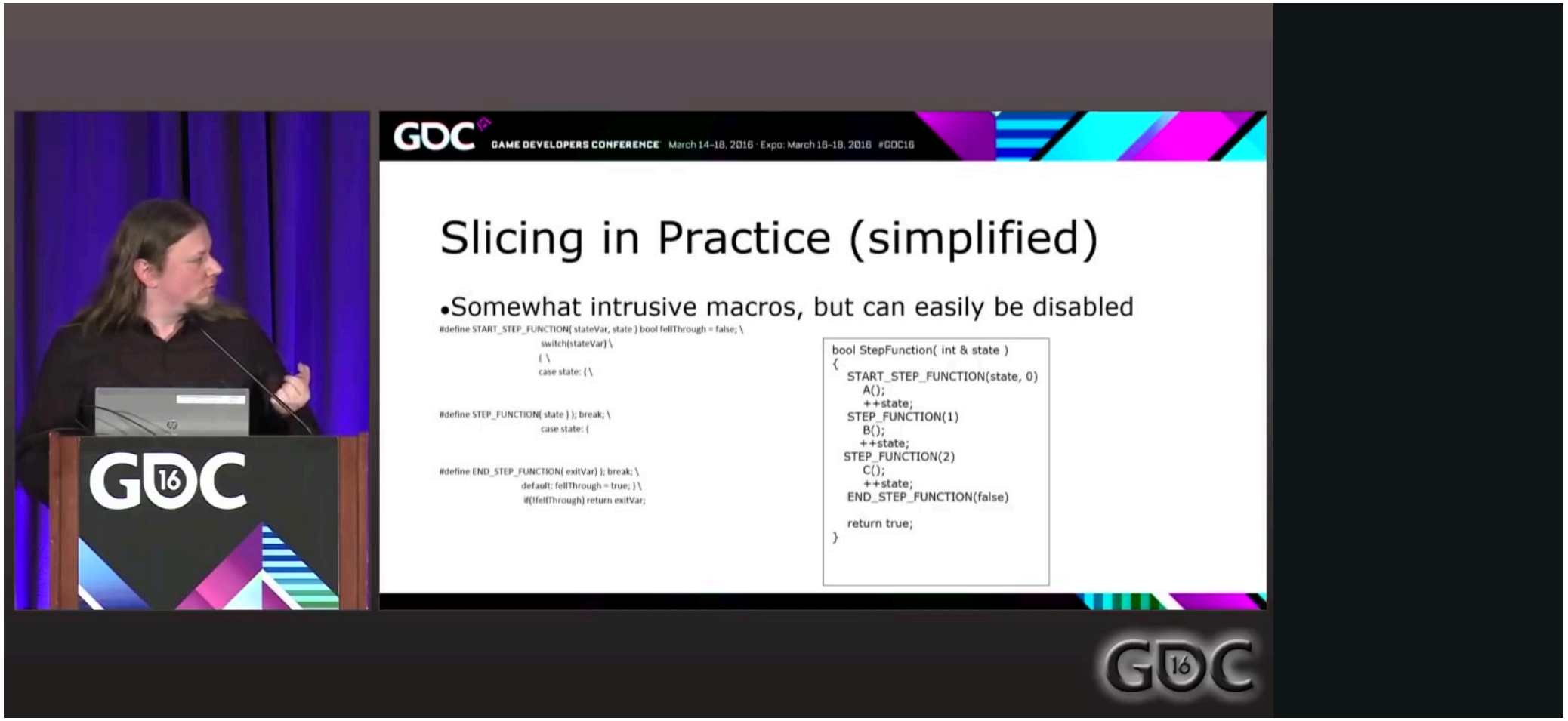


29:50–30:05

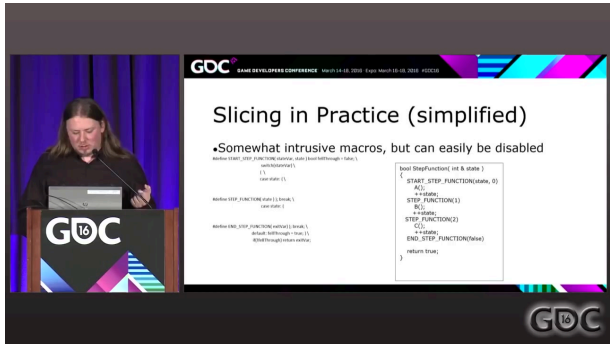
이 상태 구조체는 꽤 커질 수 있는데, 임시 배열 같은 작업용 데이터까지 넣어 두면 오히려 유리합니다. 메모리 할당 횟수를 크게 줄일 수 있으니까요.

077. Slicing in Practice

영상 30:05–30:40 · 장면 캡처 30:07

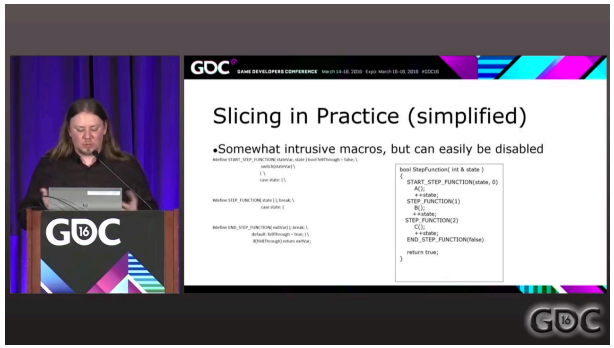


이 장면의 한국어 자막



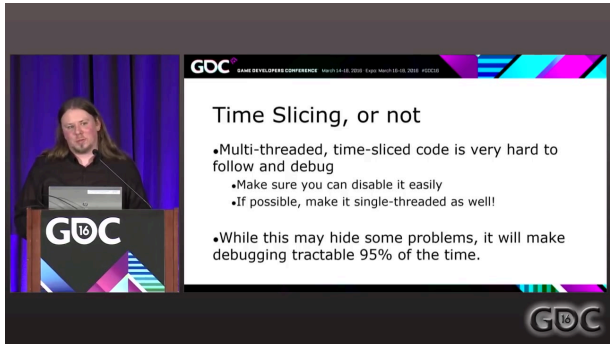
30:05–30:22

실제로는 이런 형태가 될 수 있습니다. 이 코드는 매우 단순화한 예시이고, 저희 코드와 똑같지는 않습니다. 쉽게 켜고 끌 수 있는 매크로와 상태 변수를 이용합니다.



30:22–30:39

상태를 관리하면서 남은 실행 시간을 확인해 작업을 계속할지 결정합니다. switch와 do-while을 조합하는 식으로 구현할 수 있고, 잘 작동합니다. 다만 디버깅이 어렵습니다.

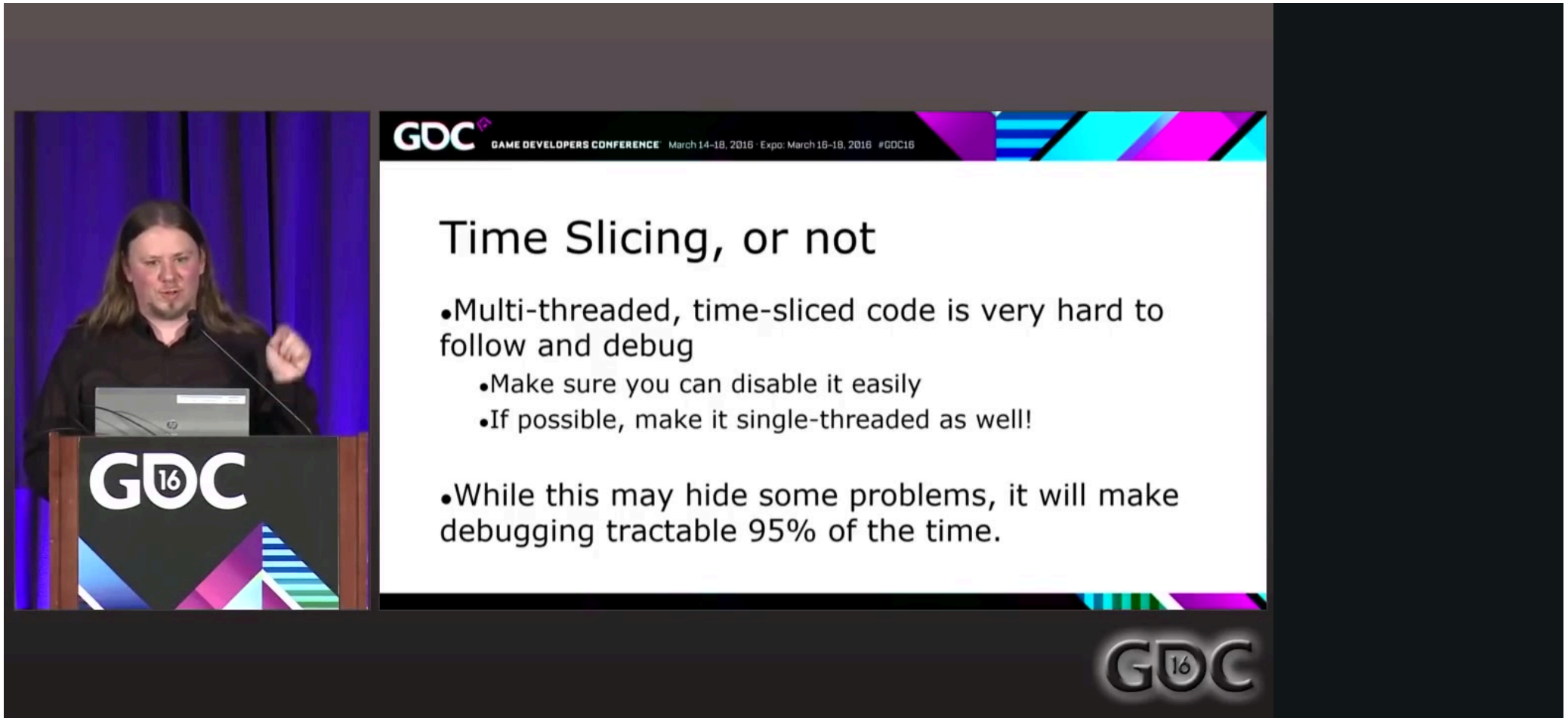


30:39–30:54

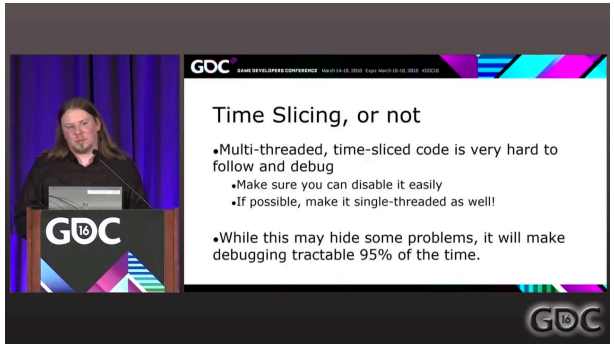
몇 단계만 실행한 뒤 함수에서 빠져나갔다가 다시 들어오는 일을 반복하기 때문입니다. 디버깅할 때는 정말 불편하죠. 그래서 매크로로 이 동작을 끌 수 있게 했습니다.

078. Time Slicing, or not

영상 30:40–31:35 · 장면 캡처 30:42

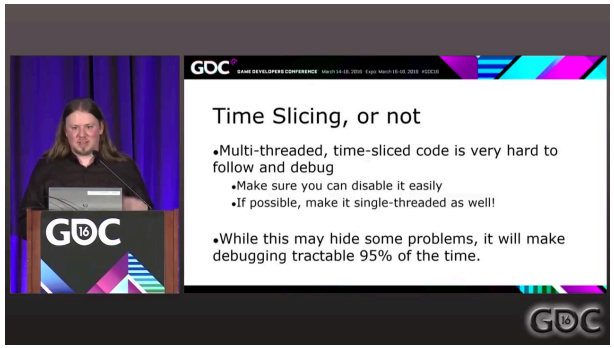


이 장면의 한국어 자막



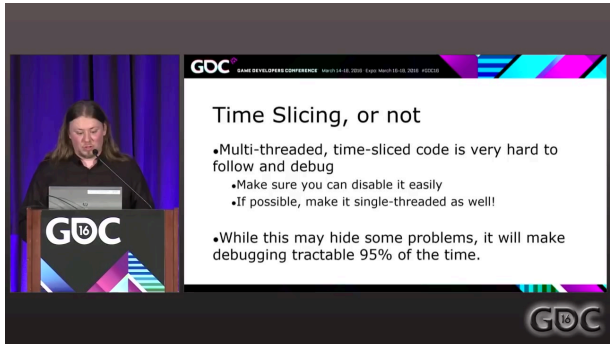
30:39–30:54

몇 단계만 실행한 뒤 함수에서 빠져나갔다가 다시 들어오는 일을 반복하기 때문입니다. 디버깅할 때는 정말 불편하죠. 그래서 매크로로 이 동작을 끌 수 있게 했습니다.



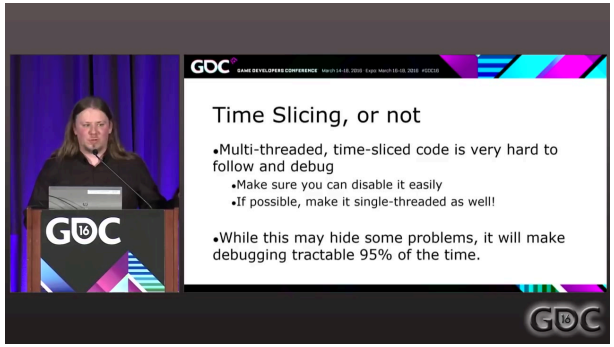
30:54–31:09

알고리즘을 디버깅할 때까지 직접 문맥 전환을 일일이 따라갈 필요는 없으니까요. 이런 장치가 문제를 찾는 데 크게 도움이 됩니다.



31:09–31:25

지금까지 비동기 처리와 타임 슬라이싱을 이야기했습니다. 둘 다 제대로 작동하도록 만들면 실제 효과가 상당합니다.

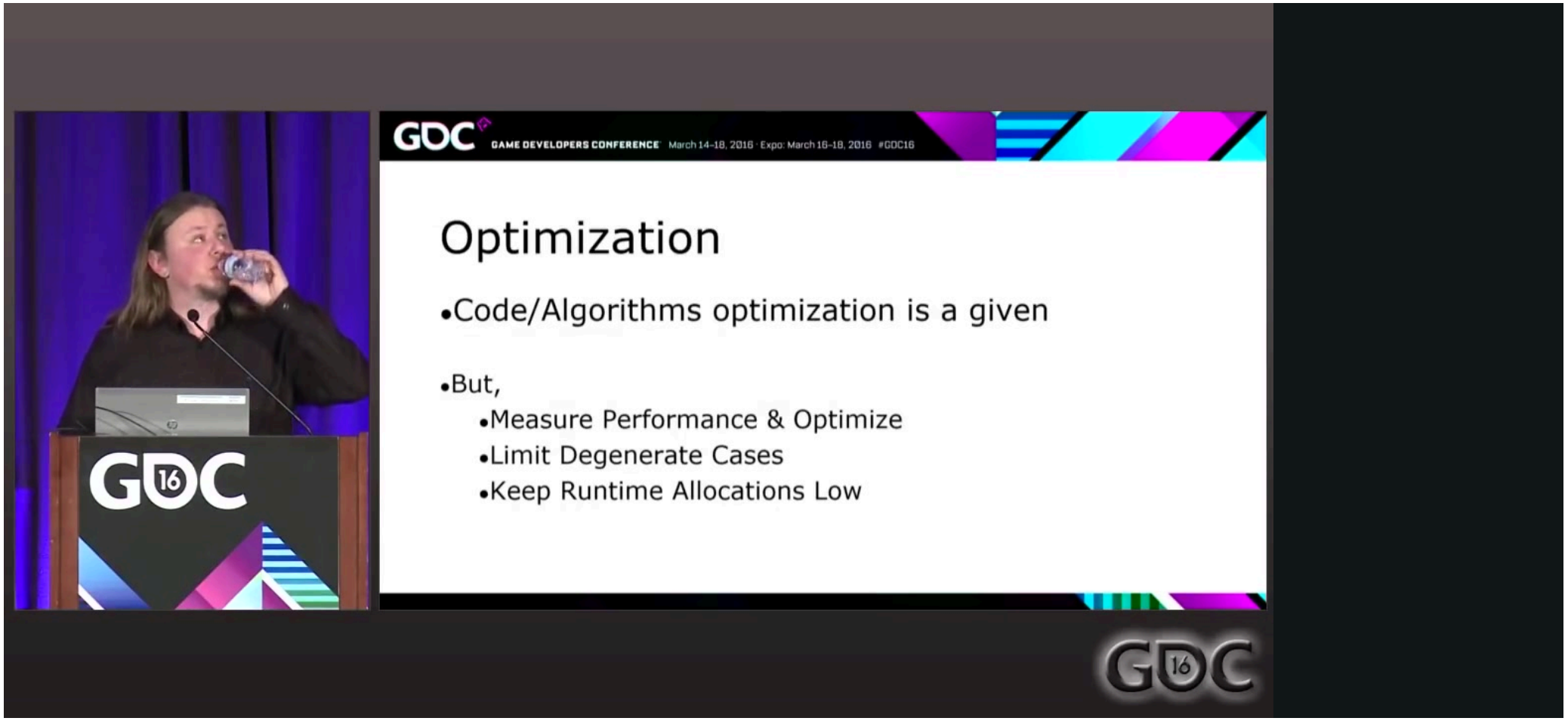


31:25–31:42

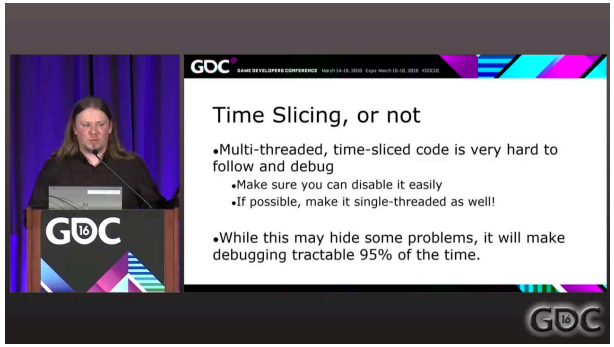
릴리스 모드로 전체 엔진을 실행하면서 파괴 코드만 디버그 모드로 돌려도 거의 차이를 느끼지 못할 정도입니다. 그만큼 유용한 방법이라는 뜻이죠.

079. Optimization

영상 31:35–32:00 · 장면 캡처 31:37

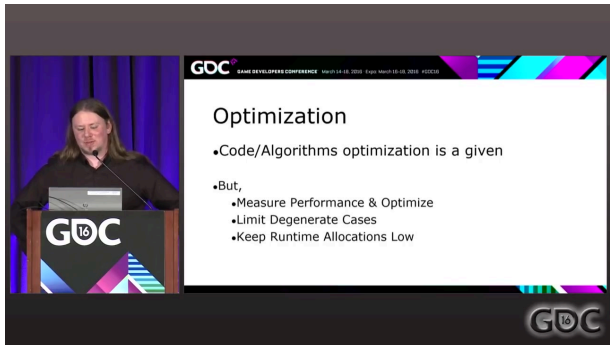


이 장면의 한국어 자막



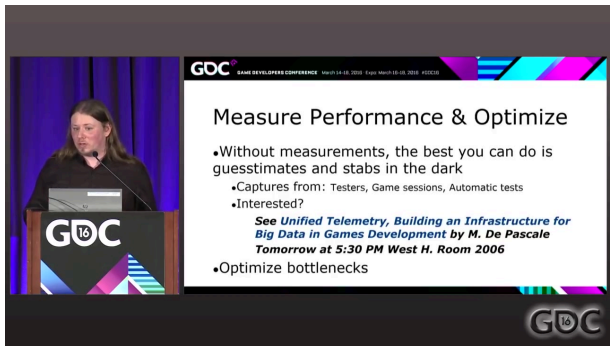
31:25–31:42

릴리스 모드로 전체 엔진을 실행하면서 파괴 코드만 디버그 모드로 돌려도 거의 차이를 느끼지 못할 정도입니다. 그만큼 유용한 방법이라는 뜻이죠.



31:42–31:59

이제 최적화 이야기입니다. 상사나 엔진 팀에서는 늘 더 빨라질 수 없냐고 묻겠죠. 게임 업계에서 최적화는 당연히 해야 하는 일입니다. 하지만 무작정 고치지 말고,

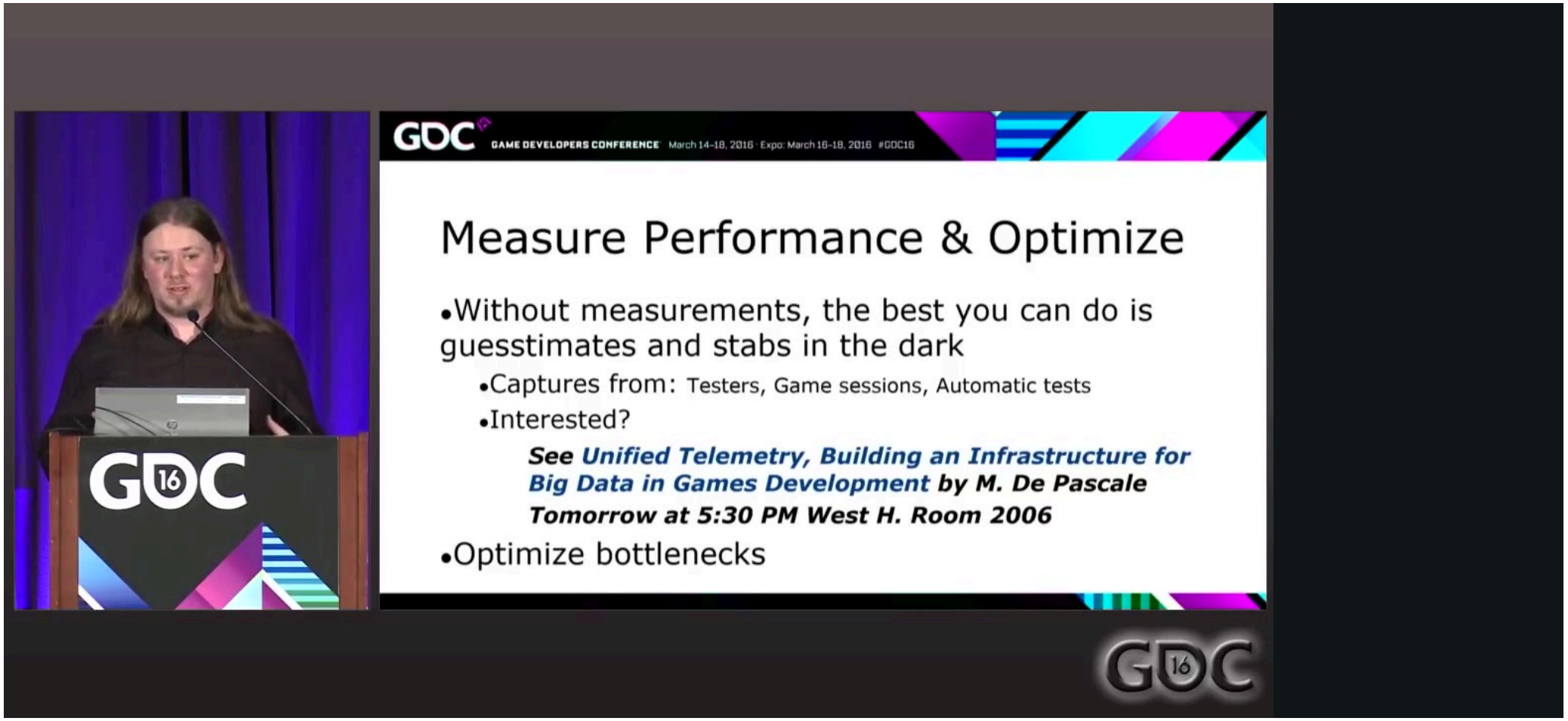


31:59–32:15

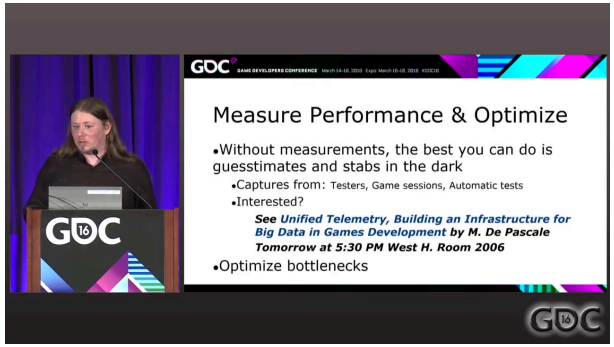
먼저 성능을 측정하고 병목을 찾아 최적화해야 합니다. 시즈에서는 테스트 세션의 텔레메트리와 캡처 데이터를 수집하는 훌륭한 기술 팀의 도움을 받았습니다.

080. Measure Performance & Optimize

영상 32:00–32:40 · 장면 캡처 32:02

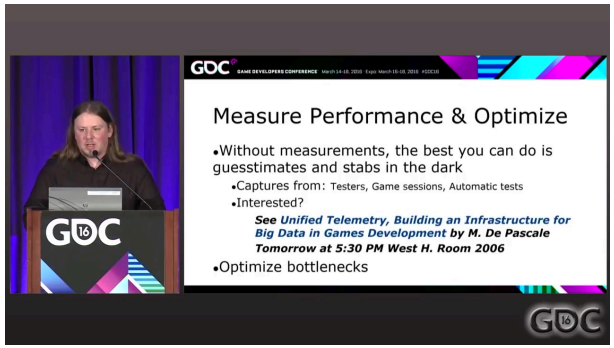


이 장면의 한국어 자막



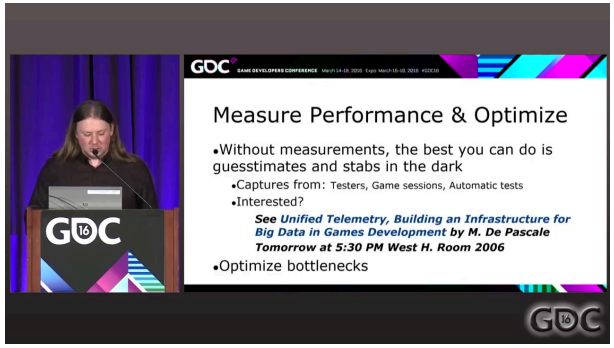
31:59–32:15

먼저 성능을 측정하고 병목을 찾아 최적화해야 합니다. 시즈에서는 테스트 세션의 텔레메트리와 캡처 데이터를 수집하는 훌륭한 기술 팀의 도움을 받았습니다.



32:15–32:31

자동화 테스트까지 활용할 수 있어서 문제를 찾기가 훨씬 수월했습니다. 관련 내용이 궁금하다면 내일 열리는 게임 개발용 빅데이터·텔레메트리 인프라 발표를 추천합니다.

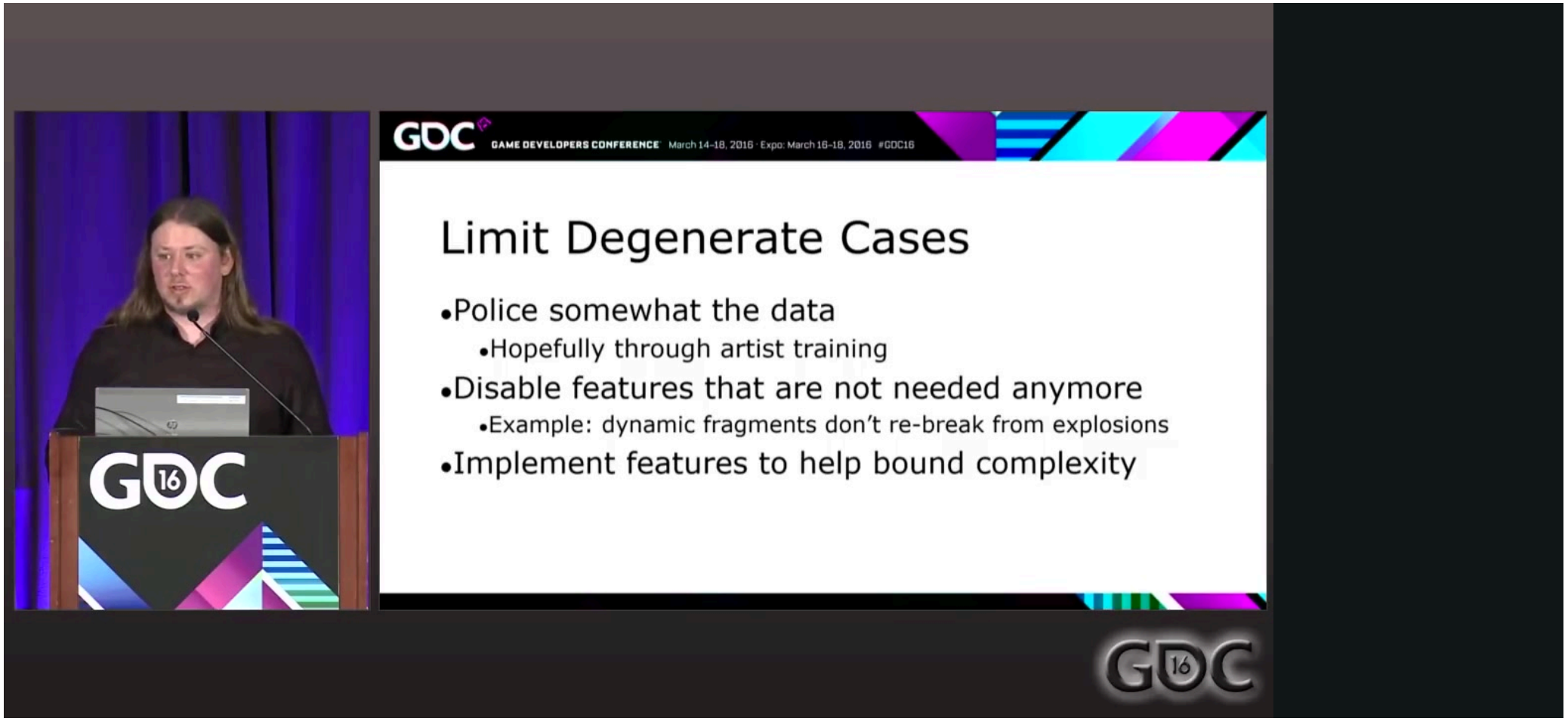


32:31–32:46

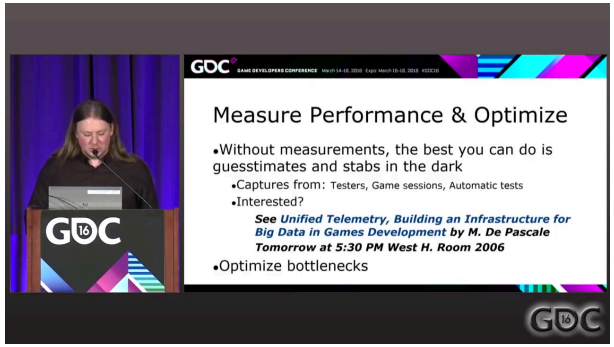
관련 발표자가 아주 훌륭한 분이기도 합니다. 다음으로는 극단적인 사례를 제한하는 방법을 보죠. 당연한 이야기처럼 들리지만 실제로는 중요합니다.

081. Limit Degenerate Cases

영상 32:40–33:15 · 장면 캡처 32:42



이 장면의 한국어 자막



32:31–32:46

관련 발표자가 아주 훌륭한 분이기도 합니다. 다음으로는 극단적인 사례를 제한하는 방법을 보죠. 당연한 이야기처럼 들리지만 실제로는 중요합니다.

32:46–33:02

아티스트와 긴밀히 대화하고 교육할 필요가 있습니다. 서로 같은 기술 언어를 쓰지 않을 수 있거든요. 필요하지 않은 기능은 꺼도 됩니다. 예를 들어 폭발 이후 바닥에 떨어진 조각들이라면,

33:02–33:18

계속 복잡하게 반응할 필요 없이 물리력에 밀려나기만 해도 충분할 수 있습니다. 또 파괴의 복잡도를 제한하면서 보기 좋은 결과를 만드는 규칙을 설계할 수도 있죠.

082. Self-Destruction

영상 33:15–33:20 · 장면 캡처 33:17

이 장면의 한국어 자막

33:02–33:18

계속 복잡하게 반응할 필요 없이 물리력에 밀려나기만 해도 충분할 수 있습니다. 또 파괴의 복잡도를 제한하면서 보기 좋은 결과를 만드는 규칙을 설계할 수도 있죠.

33:18–33:33

예를 들어 여러 총알이 가까운 위치에 맞았을 때 구멍을 각각 남기면 치즈처럼 보입니다. 그래서 구멍들을 합쳐 더 큰 구멍으로 만드는 방식을 택했습니다.

083. Self-Destruction: 작은 구멍

영상 33:20–33:30 · 장면 캡처 33:22

이 장면의 한국어 자막

33:18–33:33

예를 들어 여러 총알이 가까운 위치에 맞았을 때 구멍을 각각 남기면 치즈처럼 보입니다. 그래서 구멍들을 합쳐 더 큰 구멍으로 만드는 방식을 택했습니다.

084. Self-Destruction: 큰 구멍으로 병합

영상 33:30–33:40 · 장면 캡처 33:32

이 장면의 한국어 자막

33:18–33:33

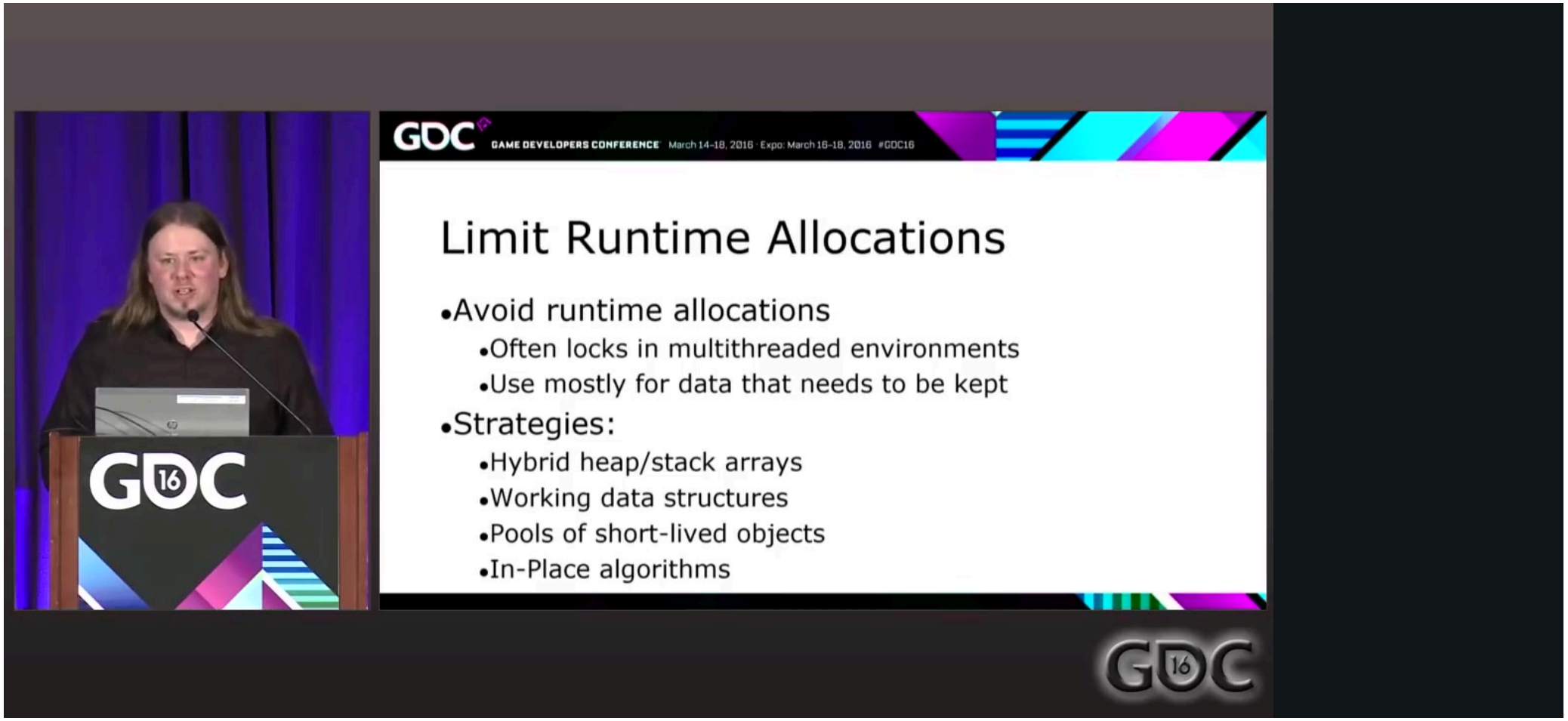
예를 들어 여러 총알이 가까운 위치에 맞았을 때 구멍을 각각 남기면 치즈처럼 보입니다. 그래서 구멍들을 합쳐 더 큰 구멍으로 만드는 방식을 택했습니다.

33:33–33:49

더 보기 좋고 디자이너와 플레이어도 선호하며 성능에도 유리했습니다. 다음은 런타임 메모리 할당입니다. 프로젝트 후반에 집중적으로 개선한 영역입니다.

085. Limit Runtime Allocations

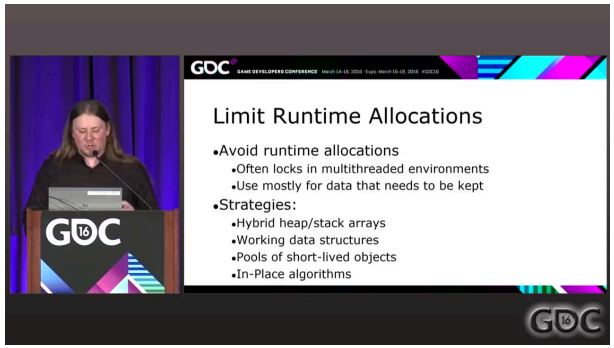
영상 33:40–34:35 · 장면 캡처 33:42



이 장면의 한국어 자막

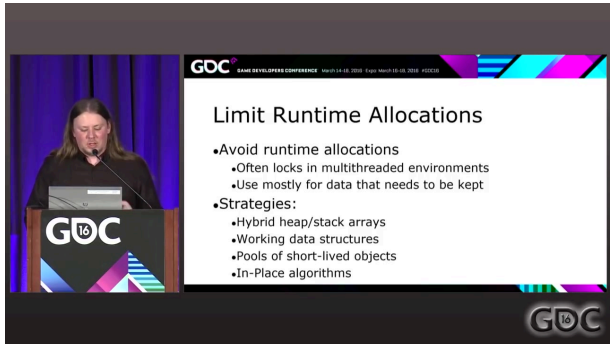
33:33–33:49

더 보기 좋고 디자이너와 플레이어도 선호하며 성능에도 유리했습니다. 다음은 런타임 메모리 할당입니다. 프로젝트 후반에 집중적으로 개선한 영역입니다.



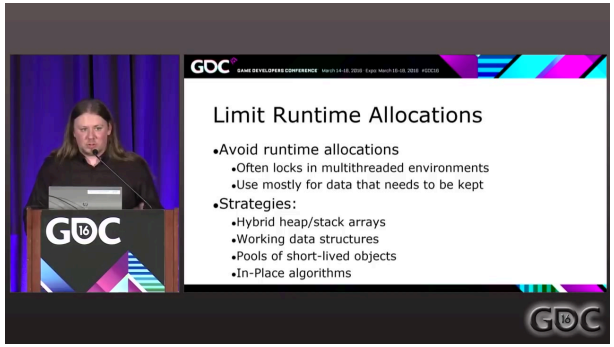
33:49–34:04

처음에는 지나치게 조심하지 않았지만, 런타임 할당에도 비용이 듭니다. 특히 멀티스레드 환경에서는 할당자가 락을 잡는 일이 많기 때문입니다. 물론 오래 유지할 데이터라면 할당이 필요합니다.



34:04–34:21

그래도 더 나은 전략이 있습니다. 스택과 힙을 혼합한 배열을 쓰거나, 평소 크기가 작은 작업 데이터를 스택에 두고, 재사용할 큰 작업 버퍼를 확보할 수 있습니다.

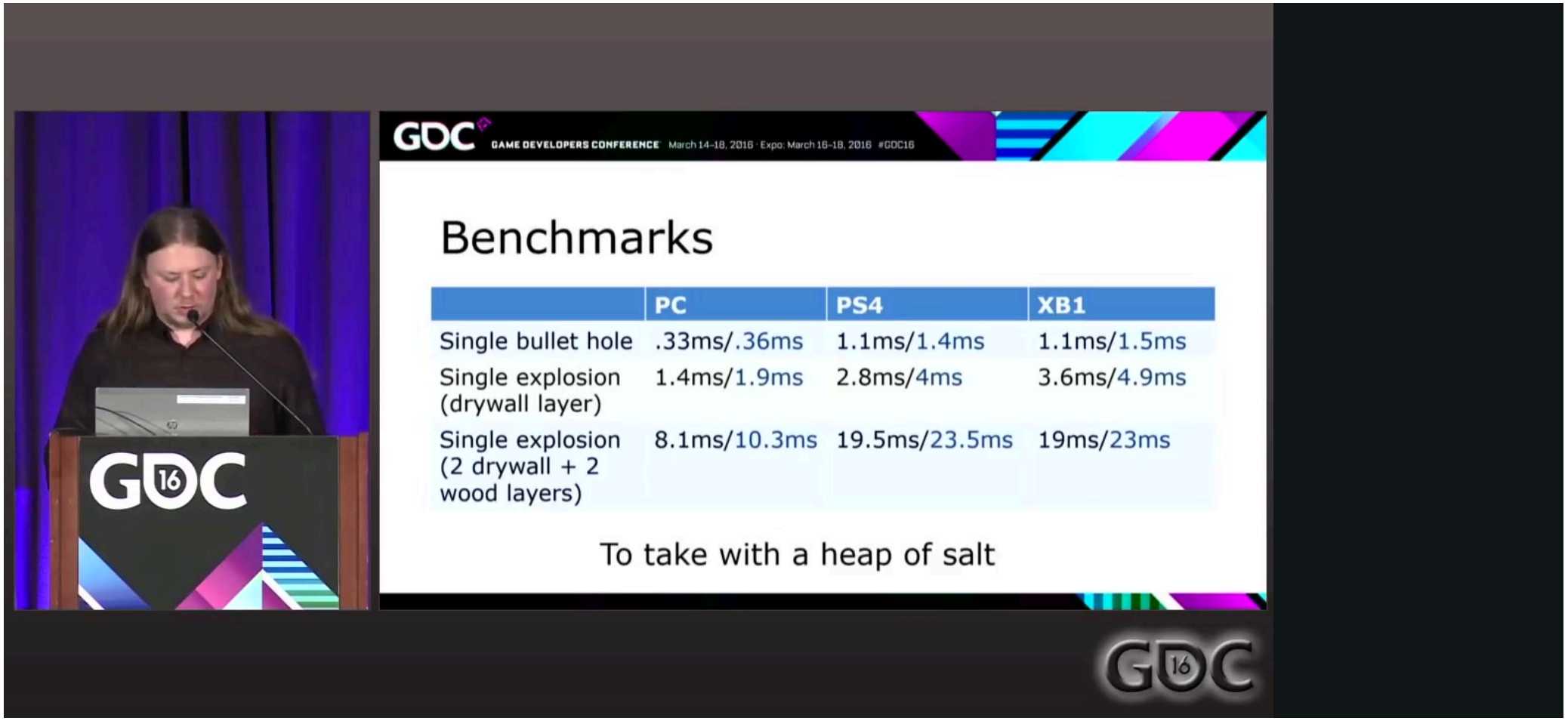


34:21–34:36

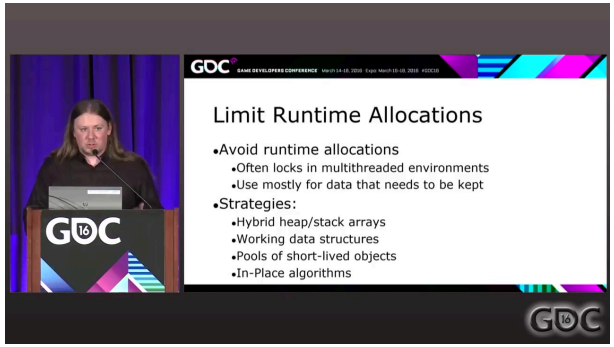
수명이 짧은 오브젝트 풀도 유용합니다. 앞서 설명한 파괴 이벤트는 몇 프레임 정도만 존재하므로 매번 생성하고 삭제할 필요가 없습니다. 풀에서 가져와 재활용하면 되죠.

086. Benchmarks

영상 34:35–35:45 · 장면 캡처 34:37

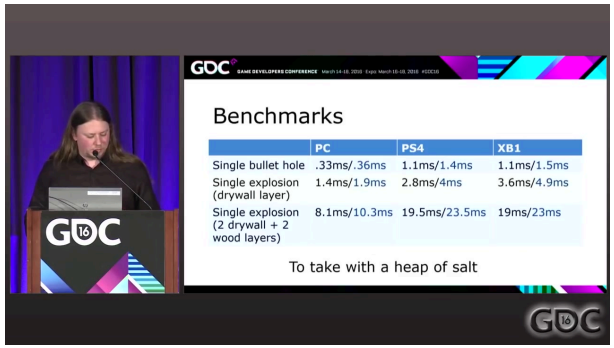


이 장면의 한국어 자막



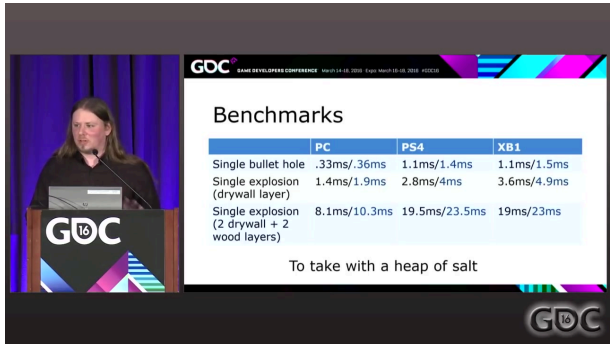
34:21–34:36

수명이 짧은 오브젝트 풀도 유용합니다. 앞서 설명한 파괴 이벤트는 몇 프레임 정도만 존재하므로 매번 생성하고 삭제할 필요가 없습니다. 풀에서 가져와 재활용하면 되죠.



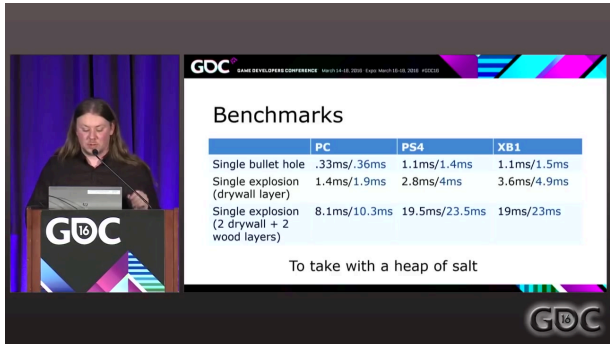
34:36–34:50

이제 벤치마크를 보겠습니다. 다만 이 수치들은 절대적인 기준으로 받아들이지 마세요. 제한된 몇 가지 사례에서 측정한 결과입니다.



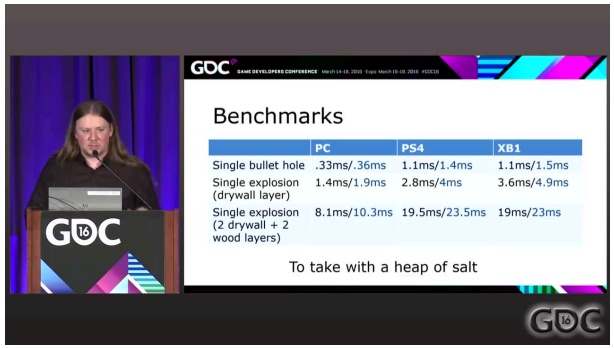
34:50–35:05

수치는 참고만 해 주세요. PC에서는 비교적 빠르고, 총알에 의한 파괴는 처리 비용이 낮습니다. 첫 번째 수치는 절차적 파괴 계산 시간이고, 두 번째는 시각 및 물리 갱신 시간입니다.



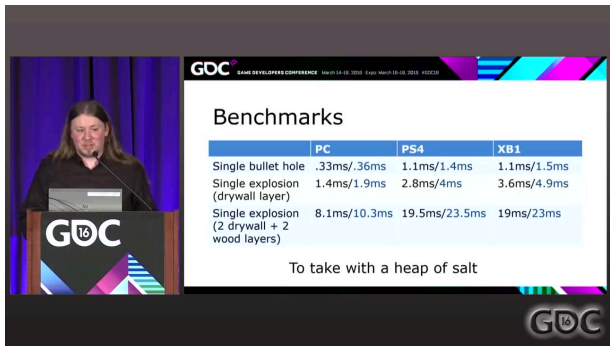
35:05–35:20

총알 구멍은 물리 형상을 바꾸지 않으므로 저렴합니다. 폭발은 훨씬 많은 요소를 변경하므로 비용이 높습니다. 그래도 PC에서는 대략 3밀리초 기준보다 꽤 낮았습니다.



35:20–35:37

여러 겹의 나무로 덮인 벽처럼 복잡한 구조를 더하면 비용이 급격히 늘 수 있습니다. 하지만 비동기로 처리해 2~3프레임 뒤에 결과를 반영하므로 엔진 팀도 감당할 수 있었죠.



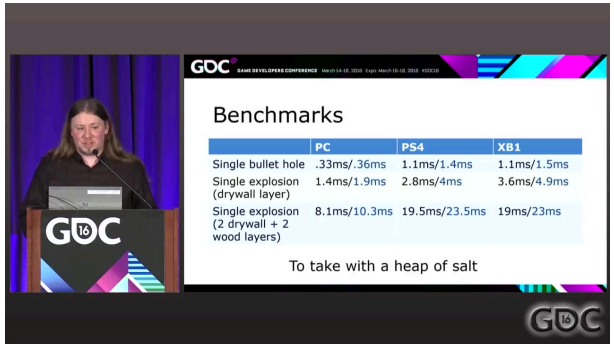
35:37–35:52

이제 온라인 처리에 대해 이야기하겠습니다. 앞서 말씀드렸듯 시즈의 파괴는 게임플레이 기능입니다.

087. Online

영상 35:45–35:50 · 장면 캡처 35:47

이 장면의 한국어 자막



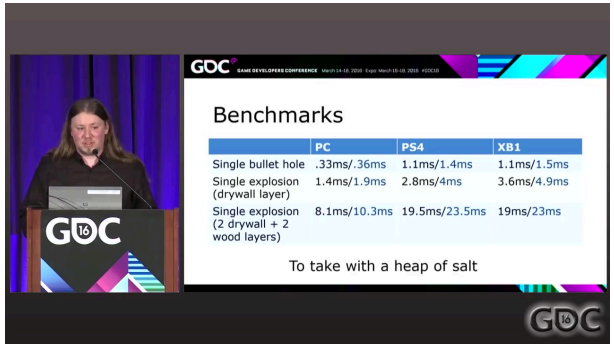
35:37–35:52

이제 온라인 처리에 대해 이야기하겠습니다. 앞서 말씀드렸듯 시즈의 파괴는 게임플레이 기능입니다.

088. Determinism and Replication

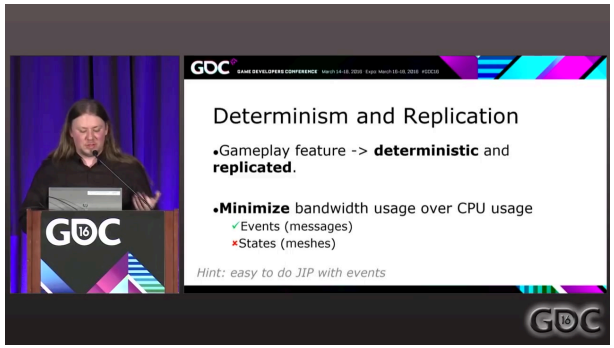
영상 35:50–36:30 · 장면 캡처 35:52

이 장면의 한국어 자막



35:37–35:52

이제 온라인 처리에 대해 이야기하겠습니다. 앞서 말씀드렸듯 시즈의 파괴는 게임플레이 기능입니다.



35:52–36:09

모든 플레이어가 같은 결과를 보거나 경험해야 하므로 결정론적으로 동작해야 합니다. 당연히 네트워크 복제도 필요하죠. 생성된 메시 전체를 보내면 될 것 같지만 현실적이지 않습니다.

36:09–36:25

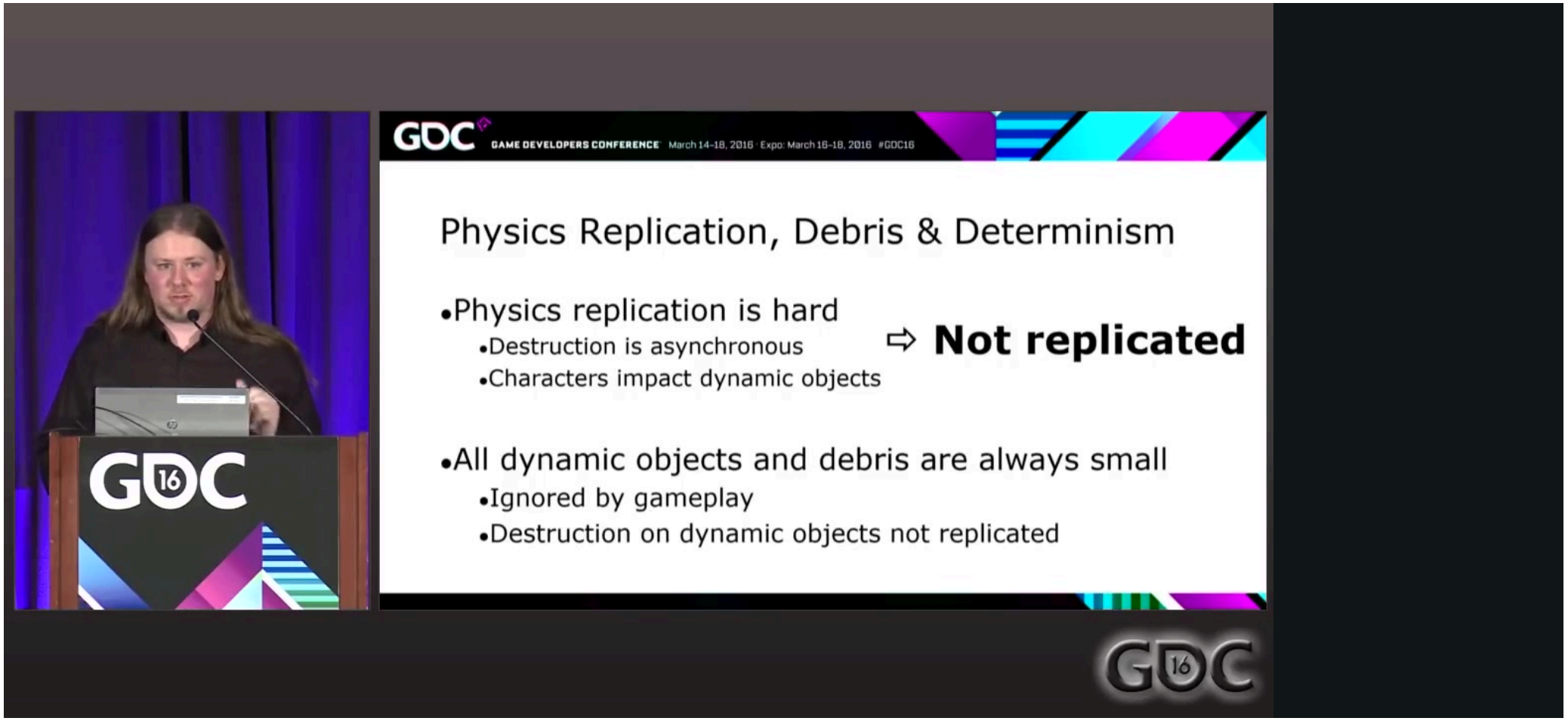
네트워크 대역폭은 언제나 아껴야 합니다. 그래서 메시 대신 파괴 이벤트만 네트워크로 전송하고, 모든 클라이언트의 파괴 시스템이 정확히 같은 결과를 생성하도록 했습니다.

36:25–36:40

이 결정론을 확보하는 데에도 어려움이 있습니다. 그건 잠시 뒤 설명하죠. 물리 시뮬레이션의 네트워크 동기화는 가능하면 하지 않는 것이 좋다는 게 저희 원칙입니다.

089. Physics Replication, Debris & Determinism

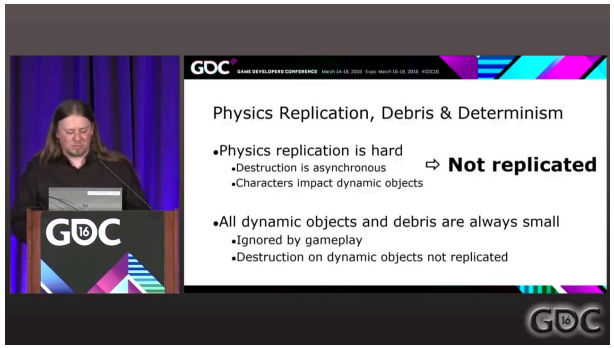
영상 36:30–37:10 · 장면 캡처 36:32



이 장면의 한국어 자막

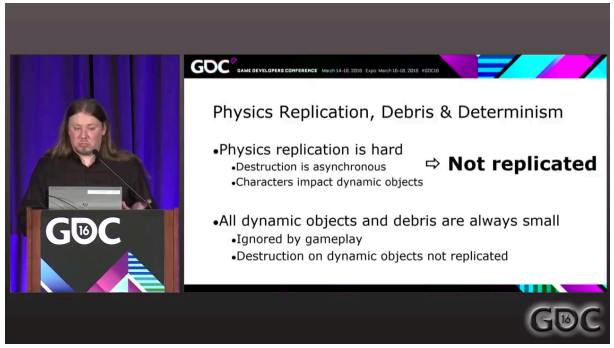
36:25–36:40

이 결정론을 확보하는 데에도 어려움이 있습니다. 그건 잠시 뒤 설명하죠. 물리 시뮬레이션의 네트워크 동기화는 가능하면 하지 않는 것이 좋다는 게 저희 원칙입니다.



36:40–36:56

파괴는 비동기적으로 처리되므로 언제 시작하고 끝나는지 정확히 맞추기 어렵고, 캐릭터가 동적 물체에 영향을 주는 문제도 있습니다. 꼭 필요하지 않으면 물리 상태까지 복제하지 않는 편이 좋습니다.

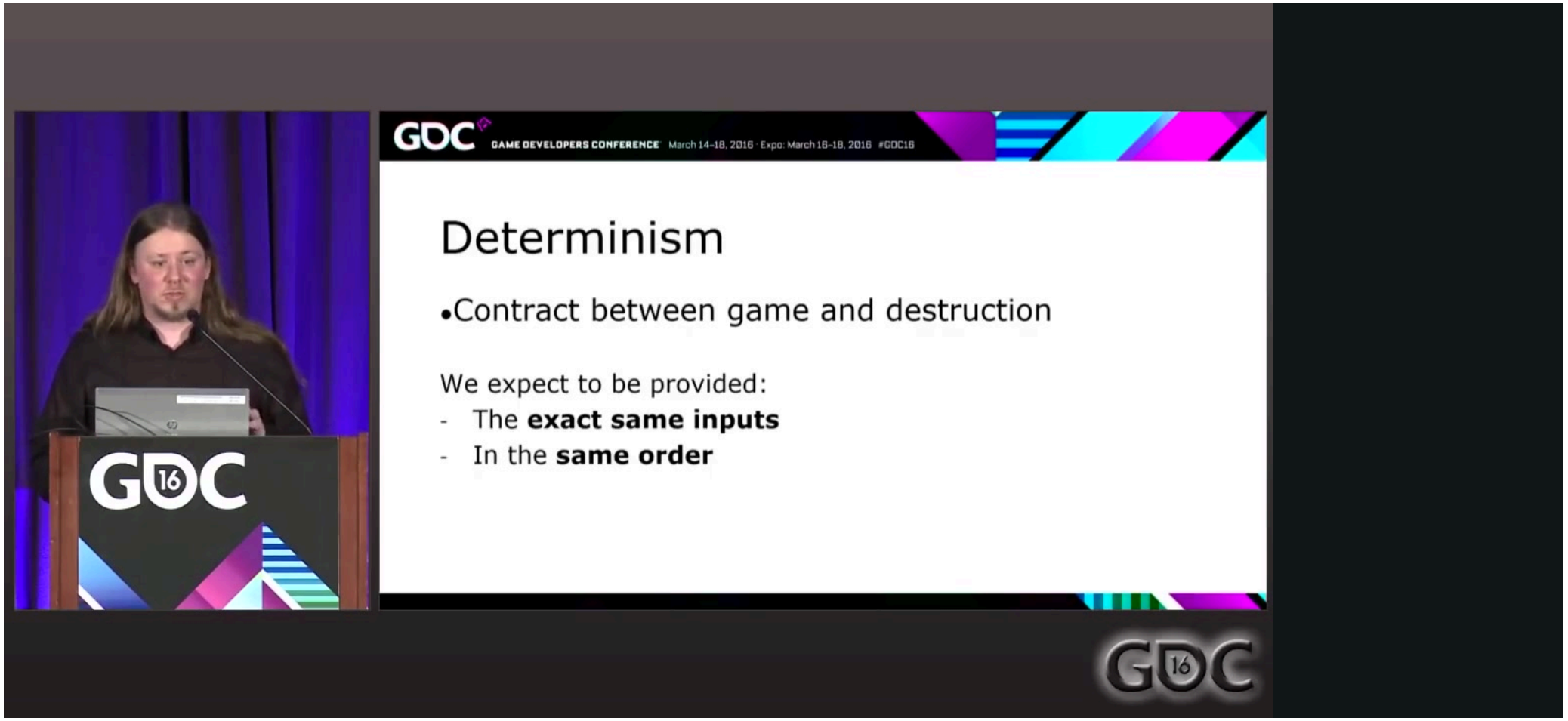


36:56–37:11

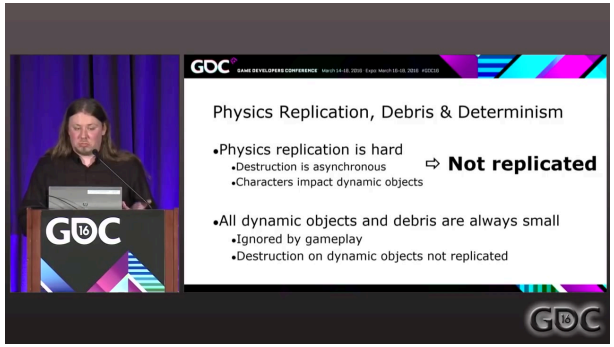
시즈에서 동적 파편은 대부분 작고 게임플레이에 큰 영향을 주지 않습니다. 이런 물체들의 물리 상태는 아예 복제하지 않기도 합니다. 이제 결정론으로 돌아가 보겠습니다.

090. Determinism

영상 37:10–37:25 · 장면 캡처 37:12



이 장면의 한국어 자막



36:56–37:11

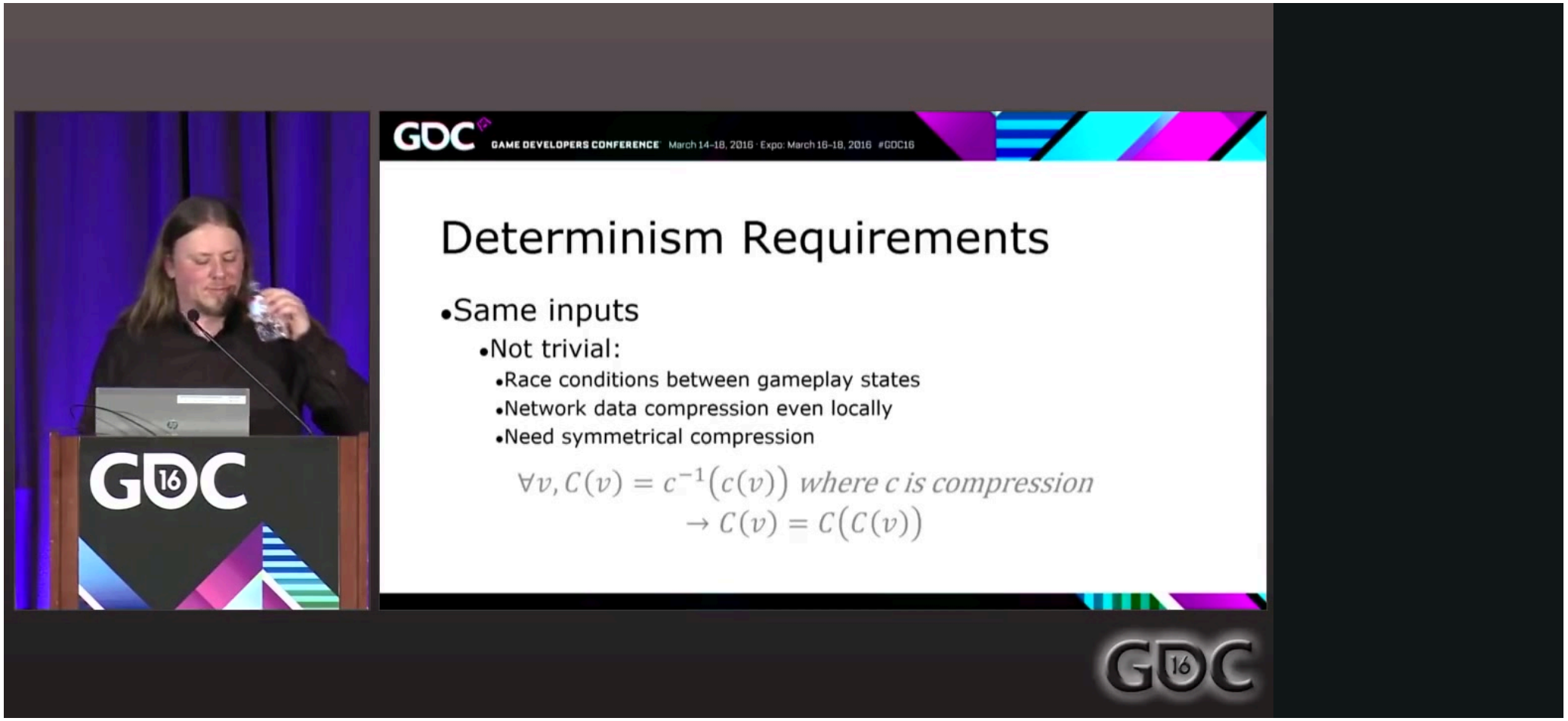
시즈에서 동적 파편은 대부분 작고 게임플레이에 큰 영향을 주지 않습니다. 이런 물체들의 물리 상태는 아예 복제하지 않기도 합니다. 이제 결정론으로 돌아가 보겠습니다.

37:11–37:27

저희는 게임 시스템과 파괴 시스템 사이에 일종의 계약을 정의했습니다. 모든 곳에서 완전히 같은 입력을 같은 순서로 전달하도록 하고, 시스템도 그 조건에서 같은 결과를 내게 했습니다.

091. Determinism Requirements: 같은 입력

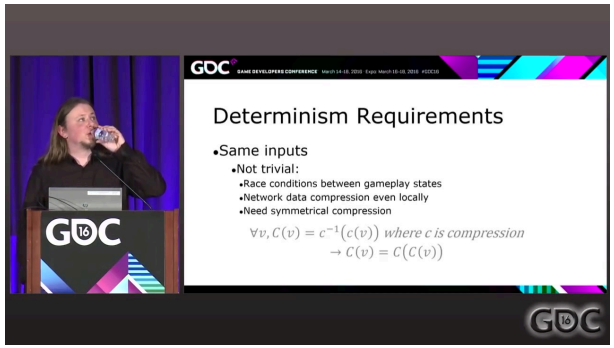
영상 37:25–38:10 · 장면 캡처 37:27



이 장면의 한국어 자막

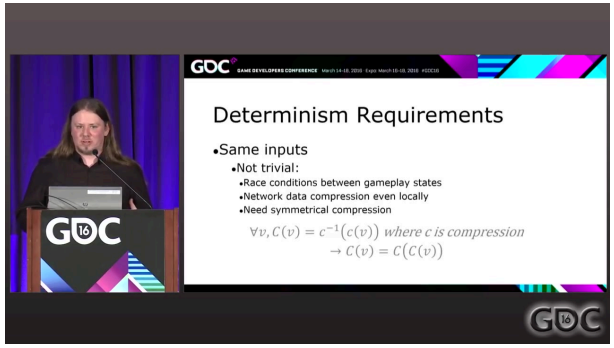
37:11–37:27

저희는 게임 시스템과 파괴 시스템 사이에 일종의 계약을 정의했습니다. 모든 곳에서 완전히 같은 입력을 같은 순서로 전달하도록 하고, 시스템도 그 조건에서 같은 결과를 내게 했습니다.



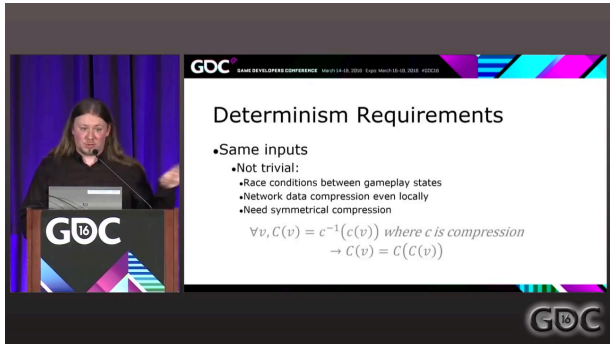
37:27–37:43

하지만 '같은 입력'이라는 조건이 생각만큼 단순하지 않습니다. 네트워크에서는 전송량을 줄이려고 값을 압축하는 경우가 많습니다. 그렇다면 로컬에서도 같은 압축 과정을 적용해야 하죠.



37:43–37:59

클라이언트가 데이터를 압축해 호스트에 보내고, 호스트가 이를 처리한 뒤 다른 클라이언트로 다시 보내는 과정에서도 모든 참여자가 동일한 값을 사용해야 합니다.

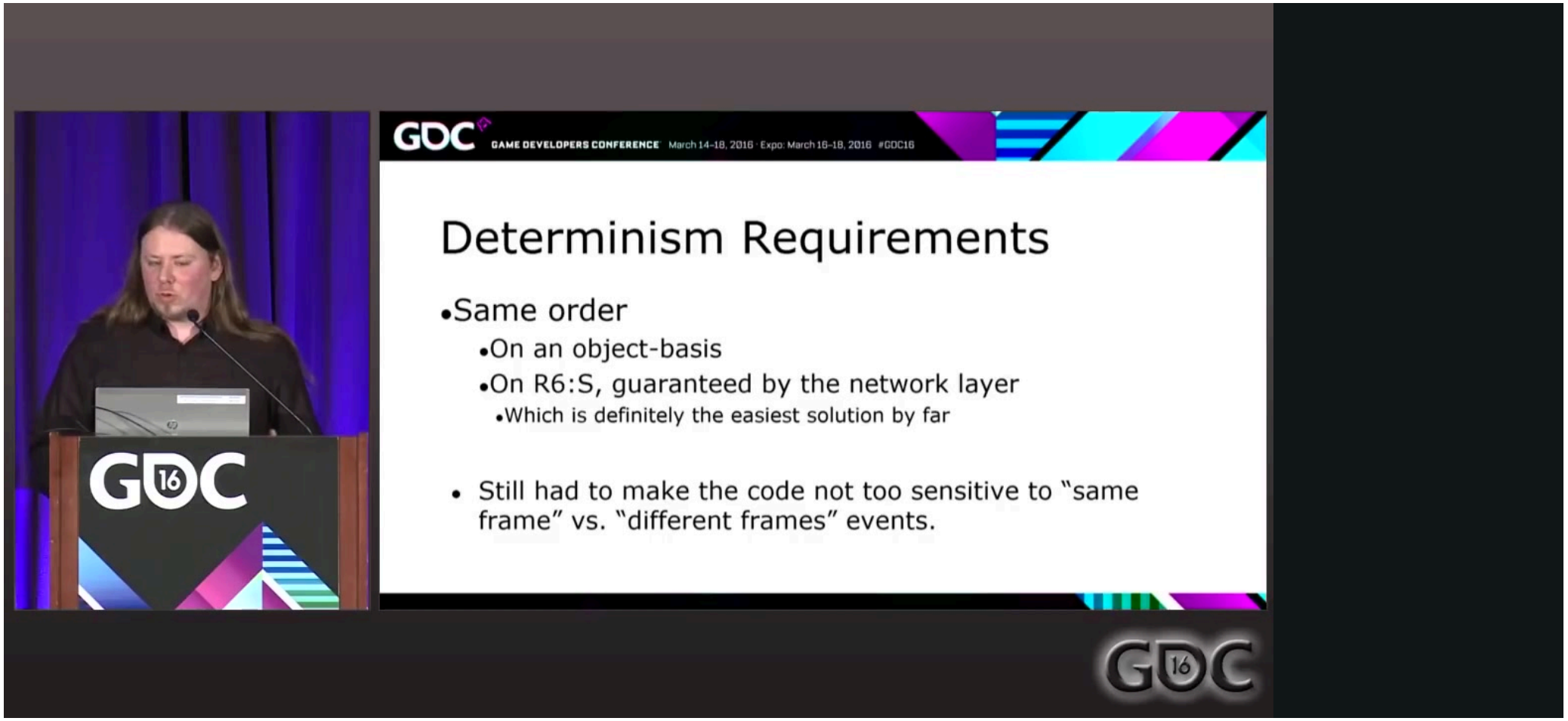


37:59–38:14

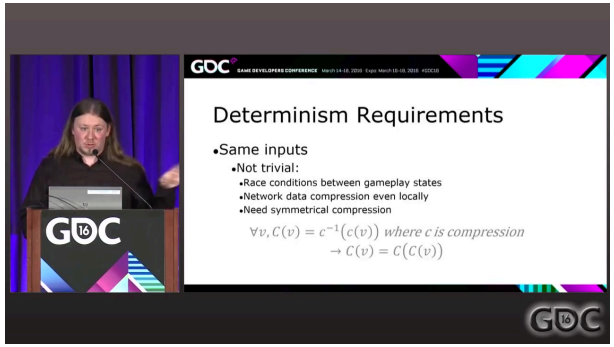
이를 위해 대칭적인 압축 방식을 사용합니다. 반드시 무손실일 필요는 없습니다. 최초 압축에서 정보가 조금 손실되어도 이후 압축된 값이 일관되게 유지되면 됩니다. 다음은 입력 순서입니다.

092. Determinism Requirements: 같은 순서

영상 38:10–38:40 · 장면 캡처 38:12

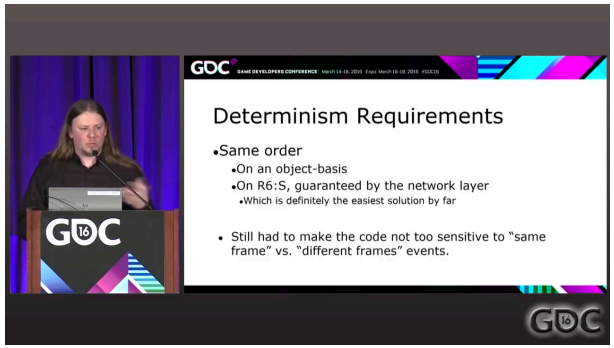


이 장면의 한국어 자막



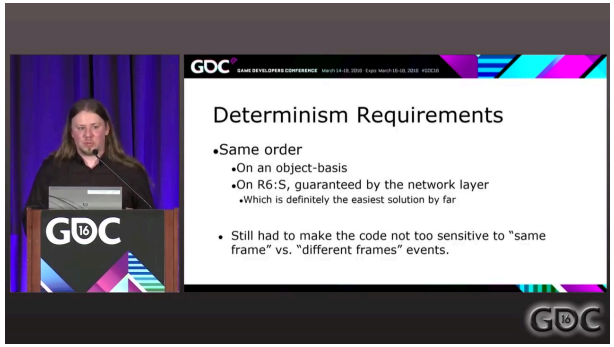
37:59–38:14

이를 위해 대칭적인 압축 방식을 사용합니다. 반드시 무손실일 필요는 없습니다. 최초 압축에서 정보가 조금 손실되어도 이후 압축된 값이 일관되게 유지되면 됩니다. 다음은 입력 순서입니다.



38:14–38:29

시즈의 네트워크 계층은 입력 순서를 보장해 주었기 때문에 큰 문제가 없었습니다. 하지만 네트워크 입력이 한 프레임에 몰려 들어올 수는 있습니다. 파괴 결과가 그런 상황에 영향을 받지 않게 해야 합니다.

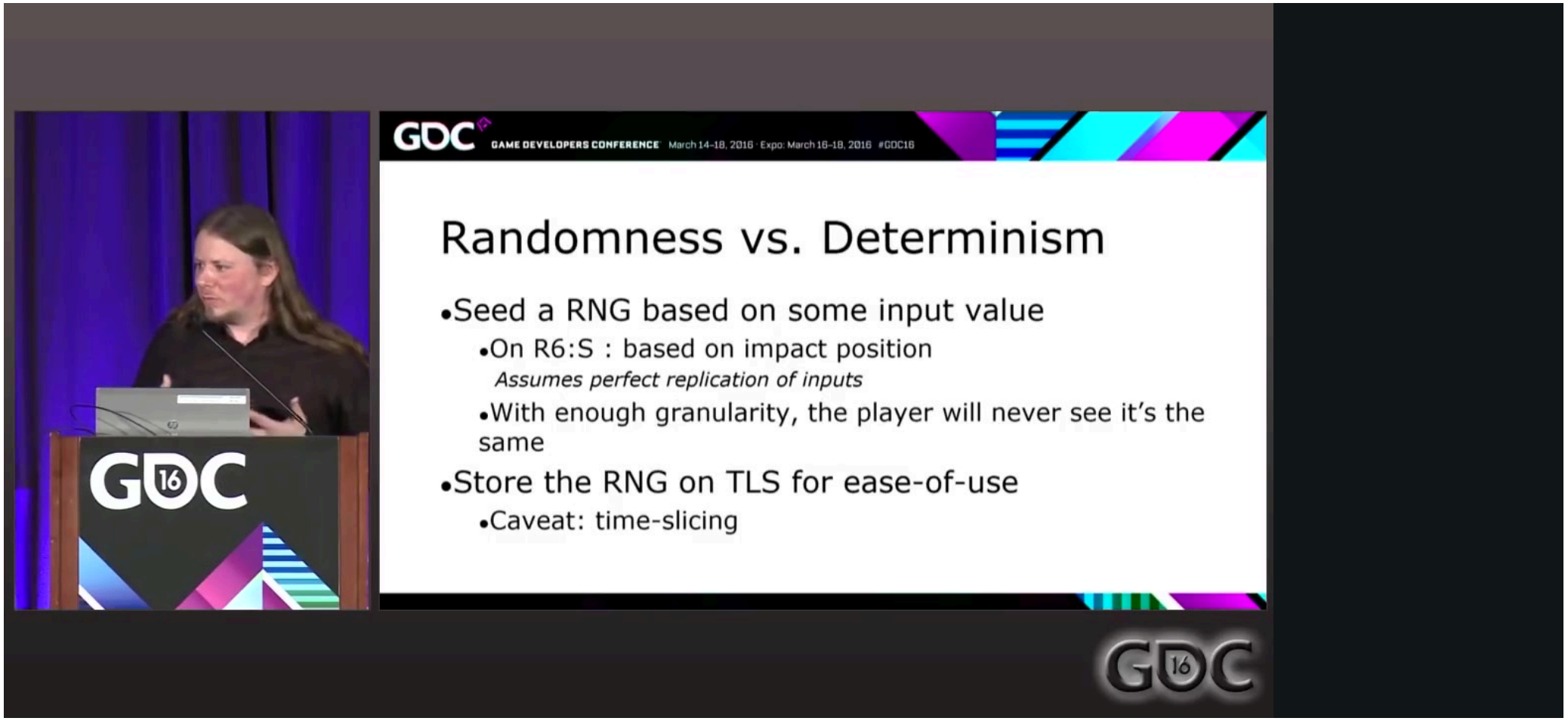


38:29–38:46

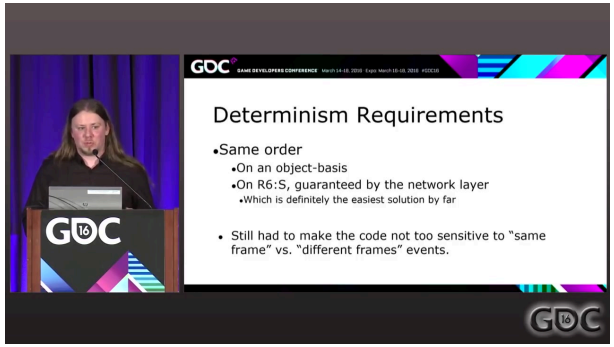
예를 들어 매 프레임 입력 하나씩 순서대로 처리하도록 만들 수 있습니다. 그리고 절차적 파괴에는 무작위성이 필요해 보이죠. 입력마다 고유한 결과를 만들고 싶으니까요.

093. Randomness vs. Determinism

영상 38:40–39:35 · 장면 캡처 38:42

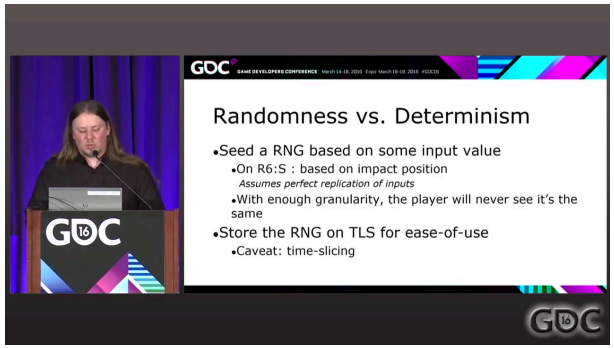


이 장면의 한국어 자막



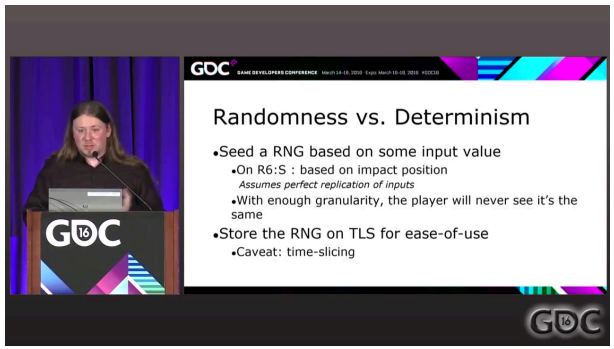
38:29–38:46

예를 들어 매 프레임 입력 하나씩 순서대로 처리하도록 만들 수 있습니다. 그리고 절차적 파괴에는 무작위성이 필요해 보이죠. 입력마다 고유한 결과를 만들고 싶으니까요.



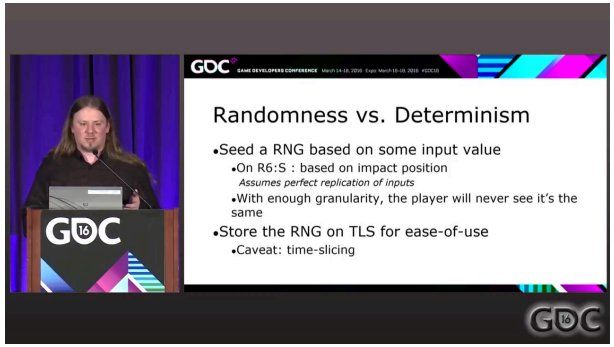
38:46–39:02

그런 무작위 느낌을 유지하면서도 결과를 일치시키려면 공통값으로 난수 생성기의 시드를 정하면 됩니다. 시즈에서는 모든 클라이언트에 복제되는 충돌 위치를 사용할 수 있습니다.



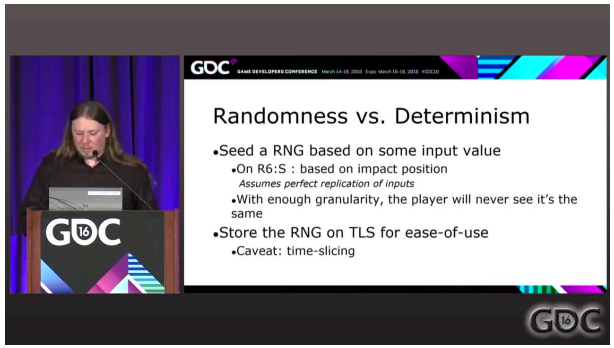
39:02–39:17

입력 위치가 동일하다면 그 값을 난수 생성기의 시드로 사용하면 됩니다. 난수 생성기가 충분히 좋다면 결과가 완전히 무작위가 아니라는 사실을 아무도 알아차리지 못할 겁니다.



39:17–39:33

C++에서는 난수 생성기를 스레드 로컬 스토리지에 둘 수도 있습니다. 사용하기 편하지만 타임 슬라이싱과 함께 쓸 때는 주의해야 합니다.

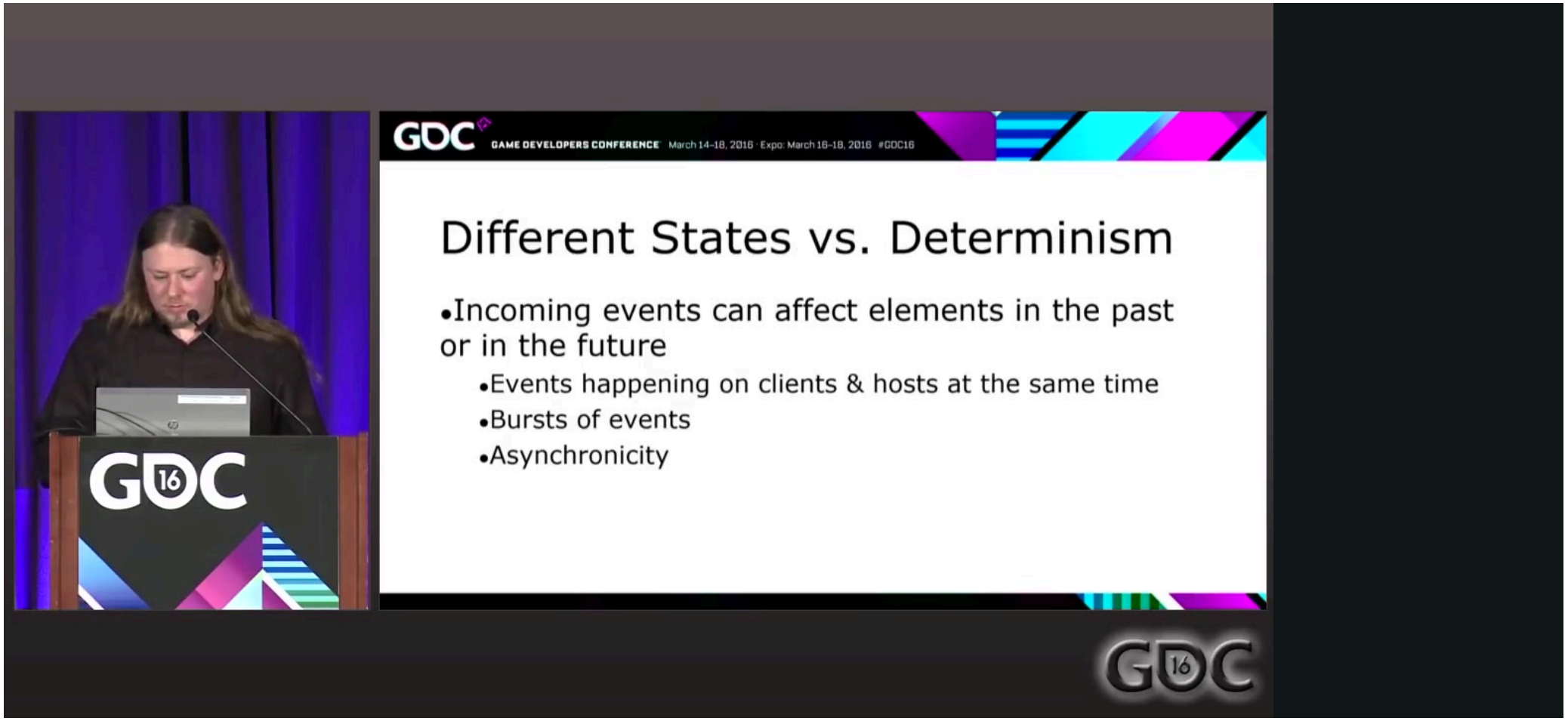


39:33–39:49

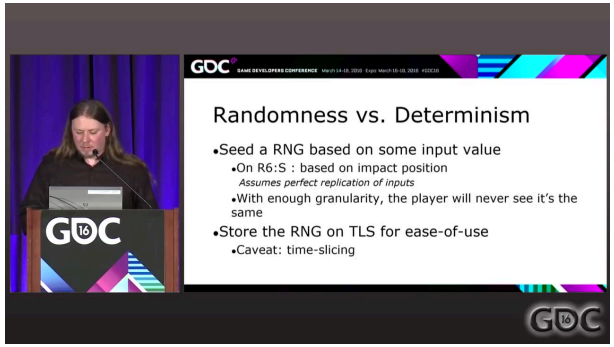
이제 서로 다른 상태의 문제를 보죠. 다른 클라이언트에서 온 과거 시점의 메시지를 받을 수 있습니다. 파괴 처리가 비동기이기 때문일 수도 있죠. 반대로 미래 시점의 메시지가 들어올 수도 있습니다.

094. Different States vs. Determinism

영상 39:35–40:15 · 장면 캡처 39:37

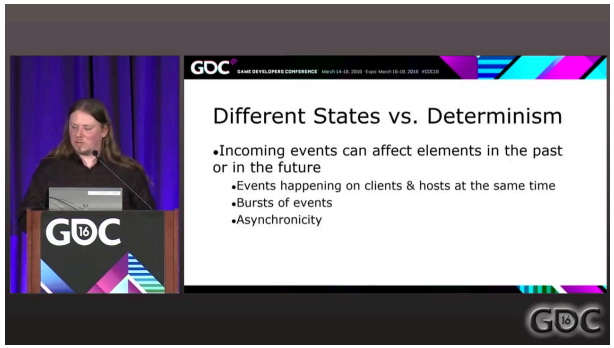


이 장면의 한국어 자막



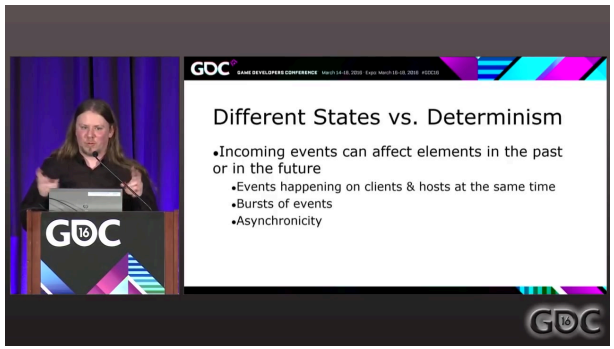
39:33–39:49

이제 서로 다른 상태의 문제를 보죠. 다른 클라이언트에서 온 과거 시점의 메시지를 받을 수 있습니다. 파괴 처리가 비동기이기 때문일 수도 있죠. 반대로 미래 시점의 메시지가 들어올 수도 있습니다.



39:49–40:07

호스트가 나보다 앞서 이벤트를 처리했을 수 있으니까요. 시스템은 이런 차이에 견고해야 합니다. 저희는 과거 이벤트의 기록을 유지해 입력의 적용 대상을 다시 조정하는 등의 처리를 합니다.

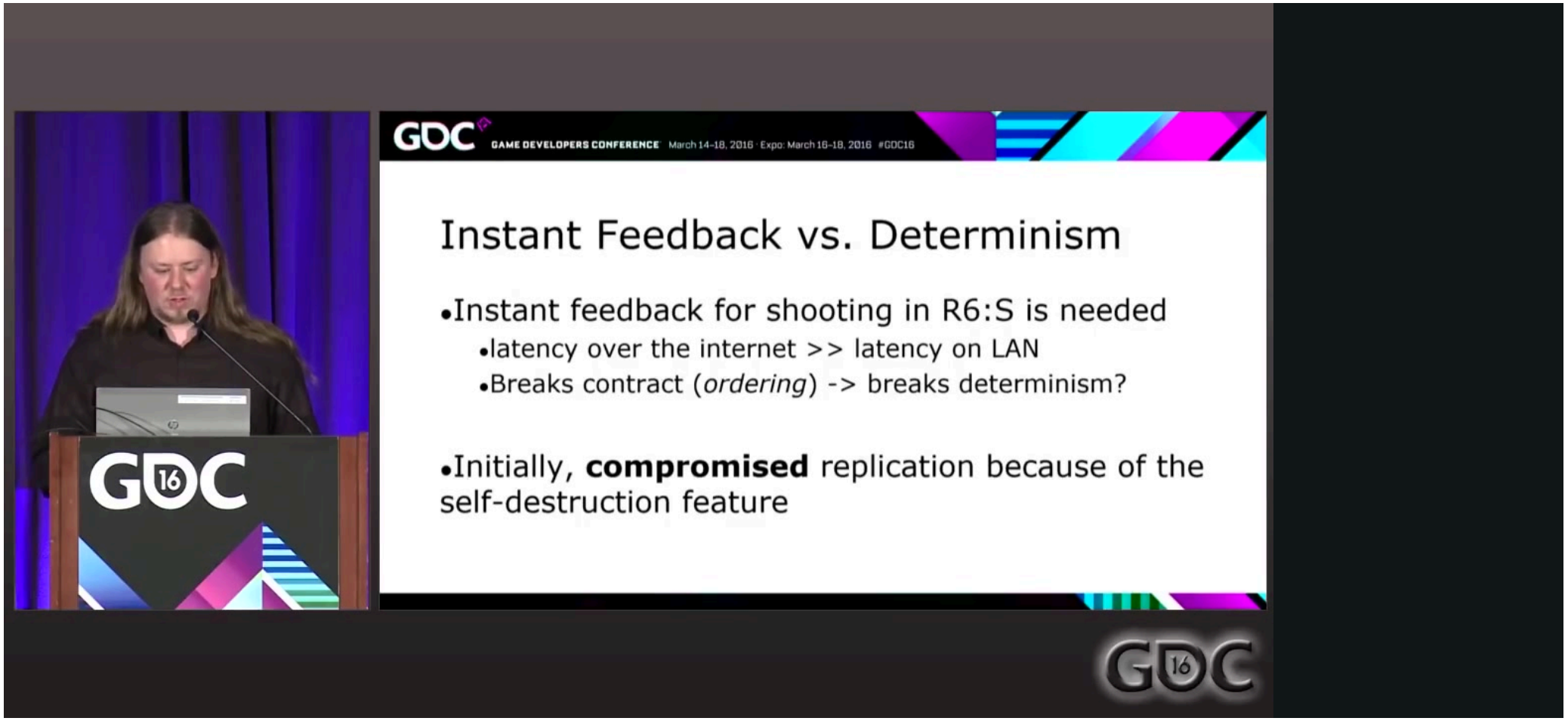


40:07–40:21

이전 상태를 고려할 수 있는 보정 장치도 마련했습니다. 다음은 사용자에게 주는 피드백입니다. 플레이어는 총을 쏘면 즉각적인 반응을 기대합니다.

095. Instant Feedback vs. Determinism

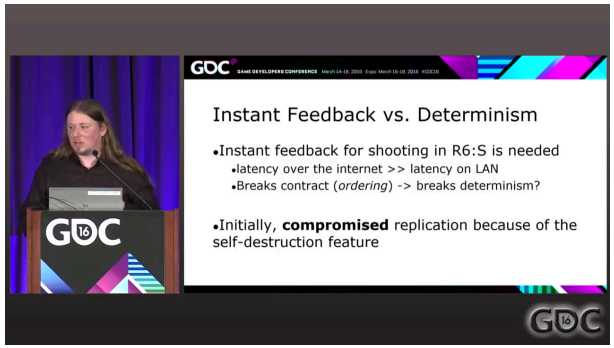
영상 40:15–41:15 · 장면 캡처 40:17



이 장면의 한국어 자막

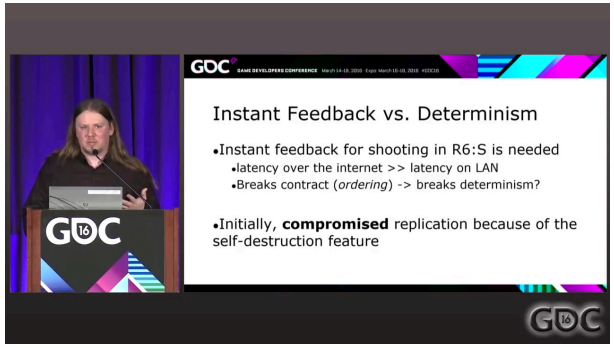
40:07–40:21

이전 상태를 고려할 수 있는 보정 장치도 마련했습니다. 다음은 사용자에게 주는 피드백입니다. 플레이어는 총을 쏘면 즉각적인 반응을 기대합니다.



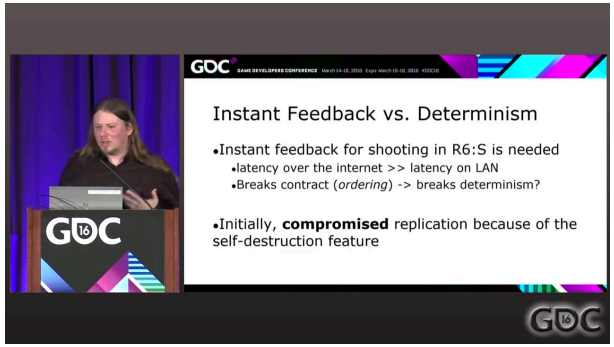
40:21–40:37

시즈라면 총알이 파괴 가능한 표면에 맞았을 때 바로 구멍이 생겨야 합니다. 하지만 온라인에서 호스트에 메시지를 보내고 응답을 기다리면 지연이 생깁니다.



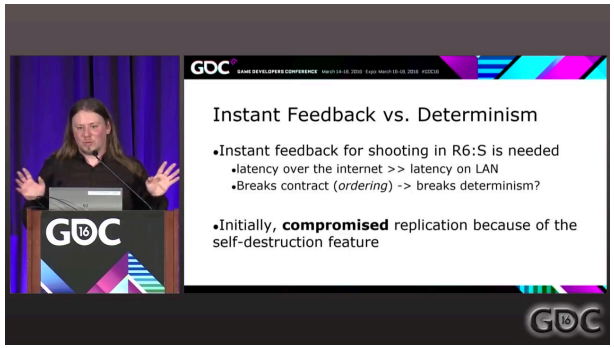
40:37–40:54

인터넷 지연까지 고려하면 호스트 응답만 기다릴 수는 없습니다. 그렇다고 로컬에서 즉시 파괴를 처리하면 입력 순서에 대한 앞서의 계약이 깨질 수 있죠.



40:54–41:09

이를 어떻게 해결할까요? 앞에서 본 구멍 병합 기능 때문에 특히 문제가 됩니다. 작은 구멍들이 합쳐져 큰 구멍이 되는 과정은 입력 순서가 바뀌면 완전히 다른 결과가 될 수 있거든요.

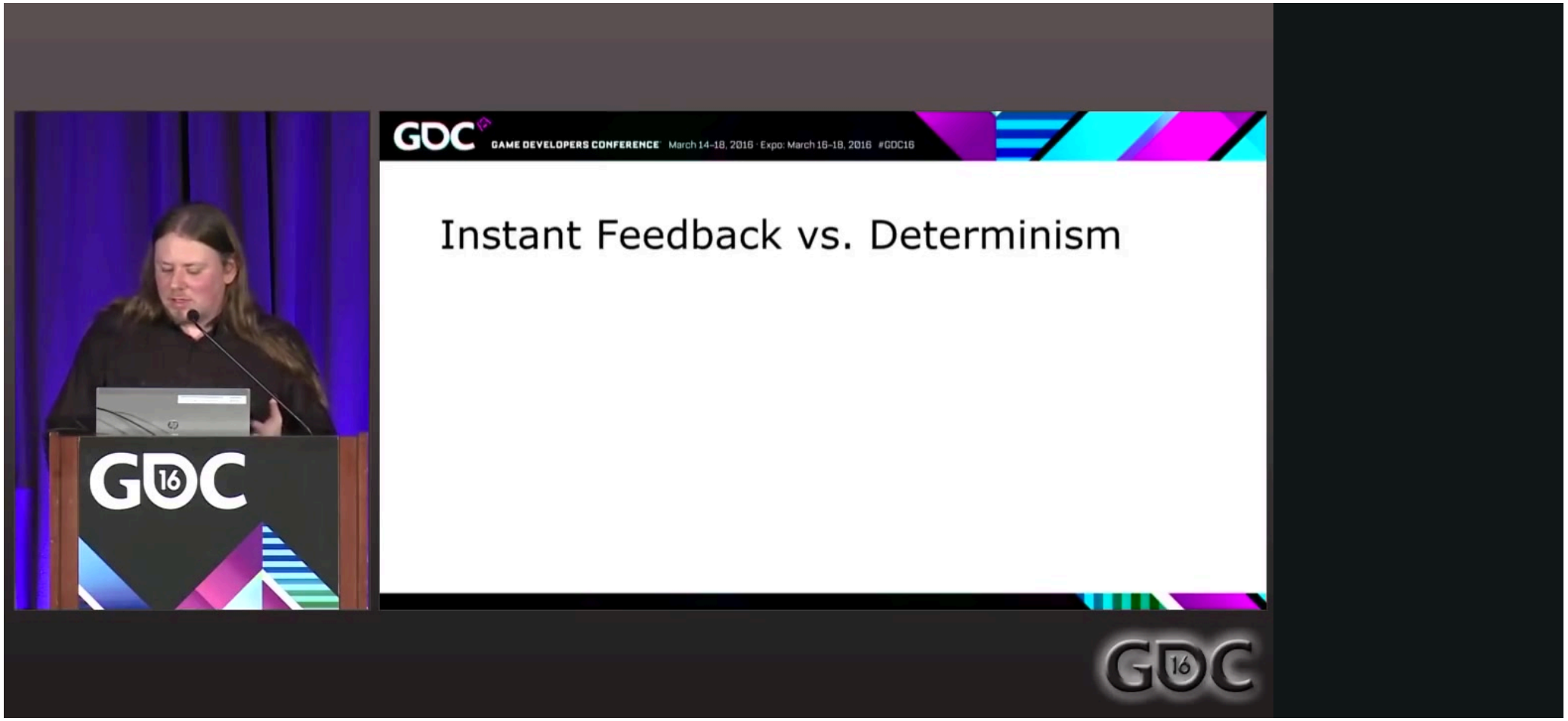


41:10–41:27

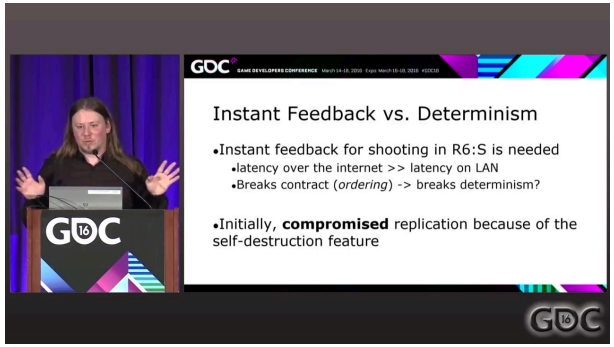
클라이언트마다 구멍 모양이 달라지면 결국 게임이 어긋납니다. 그래서 해당 기능을 두 부분으로 나눴습니다. 즉시 나타나는 작은 구멍과 표면에 누적되는 실제 피해를 분리한 겁니다.

096. Instant Feedback: 처리 흐름

영상 41:15–41:25 · 장면 캡처 41:17



이 장면의 한국어 자막

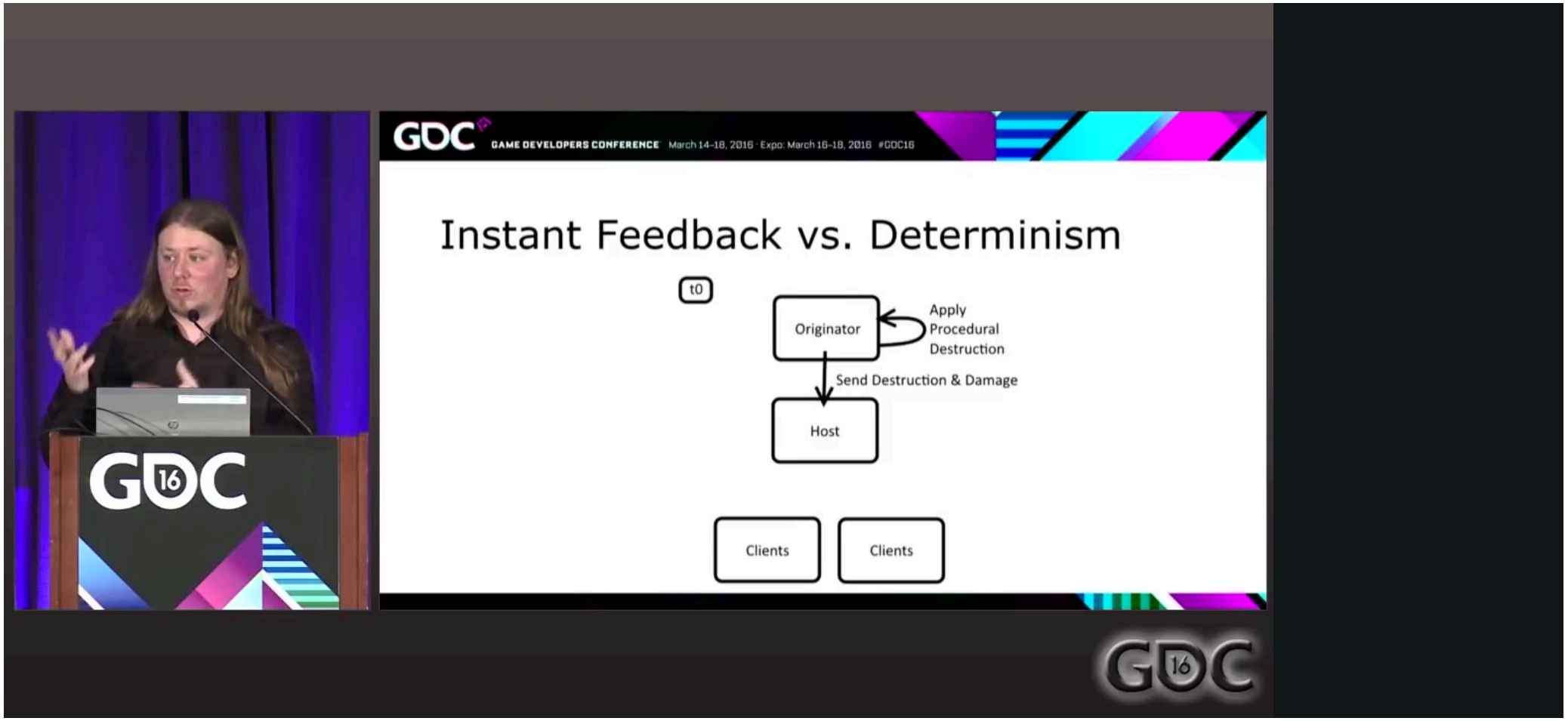


41:10–41:27

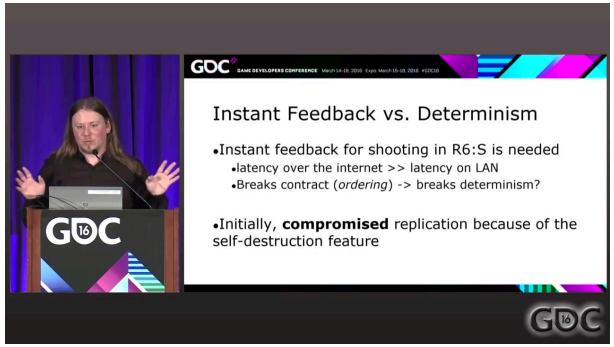
클라이언트마다 구멍 모양이 달라지면 결국 게임이 어긋납니다. 그래서 해당 기능을 두 부분으로 나눴습니다. 즉시 나타나는 작은 구멍과 표면에 누적되는 실제 피해를 분리한 겁니다.

097. Instant Feedback: 발사자와 호스트

영상 41:25–41:40 · 장면 캡처 41:27

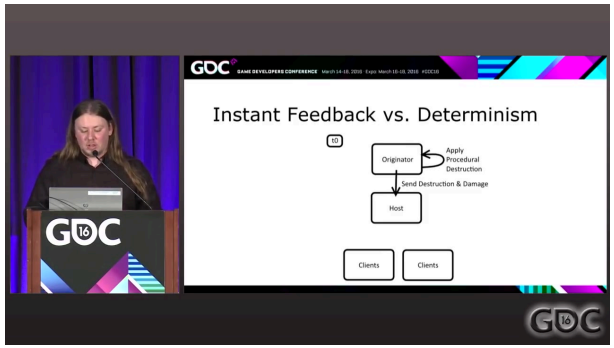


이 장면의 한국어 자막



41:10–41:27

클라이언트마다 구멍 모양이 달라지면 결국 게임이 어긋납니다. 그래서 해당 기능을 두 부분으로 나눴습니다. 즉시 나타나는 작은 구멍과 표면에 누적되는 실제 피해를 분리한 겁니다.

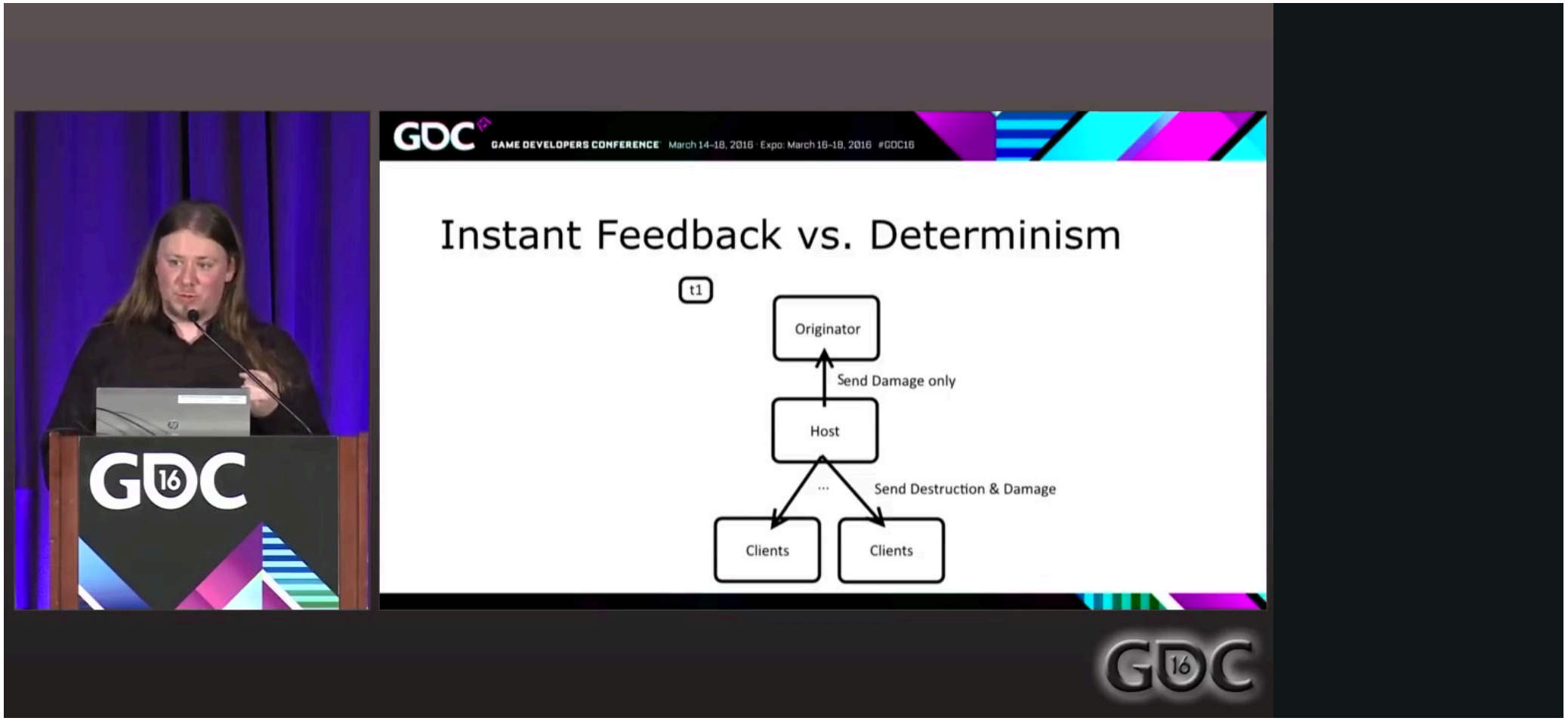


41:27–41:42

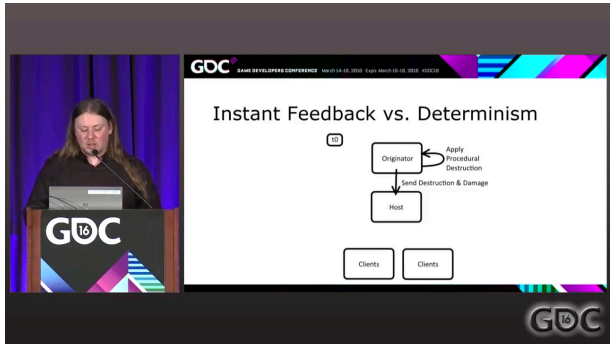
총을 쏜 클라이언트는 우선 로컬에 작은 구멍을 표시하고, 전체 정보를 호스트에 전달합니다. 호스트는 그 이벤트를 다른 플레이어들에게 배포합니다.

098. Instant Feedback: 이벤트 분기

영상 41:40–42:00 · 장면 캡처 41:42

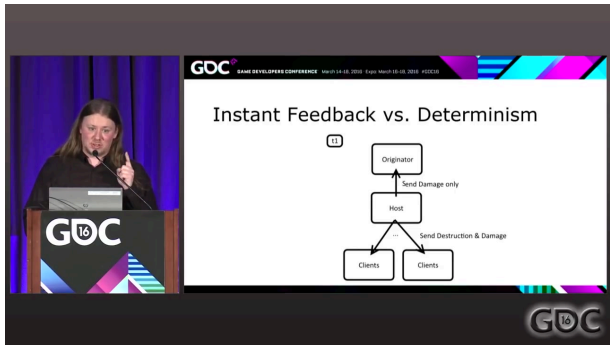


이 장면의 한국어 자막



41:27–41:42

총을 쏜 클라이언트는 우선 로컬에 작은 구멍을 표시하고, 전체 정보를 호스트에 전달합니다. 호스트는 그 이벤트를 다른 플레이어들에게 배포합니다.



41:42–41:57

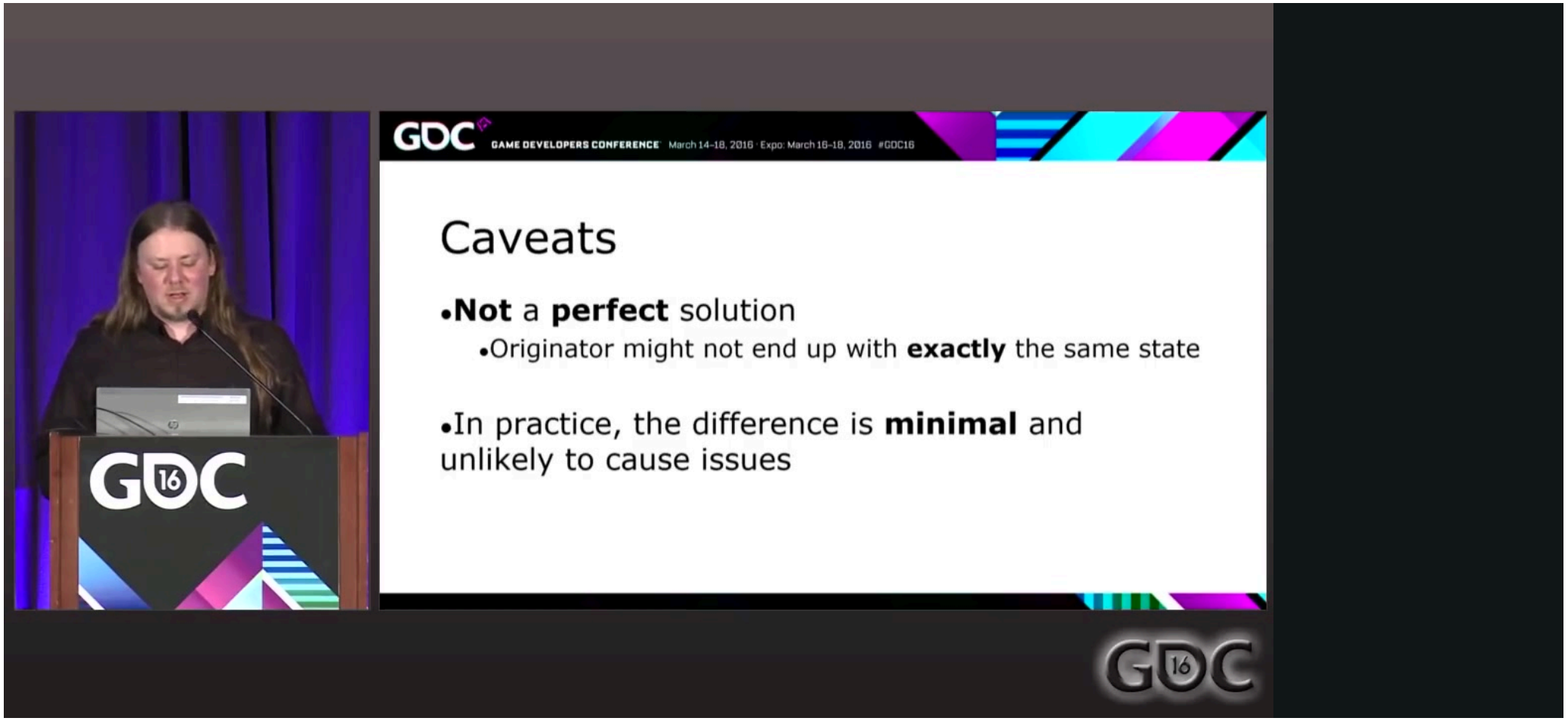
발사자에게는 특별한 형태로 결과를 전달해 큰 구멍을 위한 실제 피해만 적용하도록 합니다. 표면의 누적 피해는 항상 호스트가 정한 순서대로 처리됩니다.

41:57–42:14

완벽하지는 않습니다. 발사자의 화면에 작은 차이가 남을 수도 있고, 구멍의 형태가 약간 달라질 가능성도 있습니다. 하지만 실제로는 큰 문제가 되지 않았습니다.

099. Caveats

영상 42:00–42:25 · 장면 캡처 42:02



이 장면의 한국어 자막

41:57–42:14

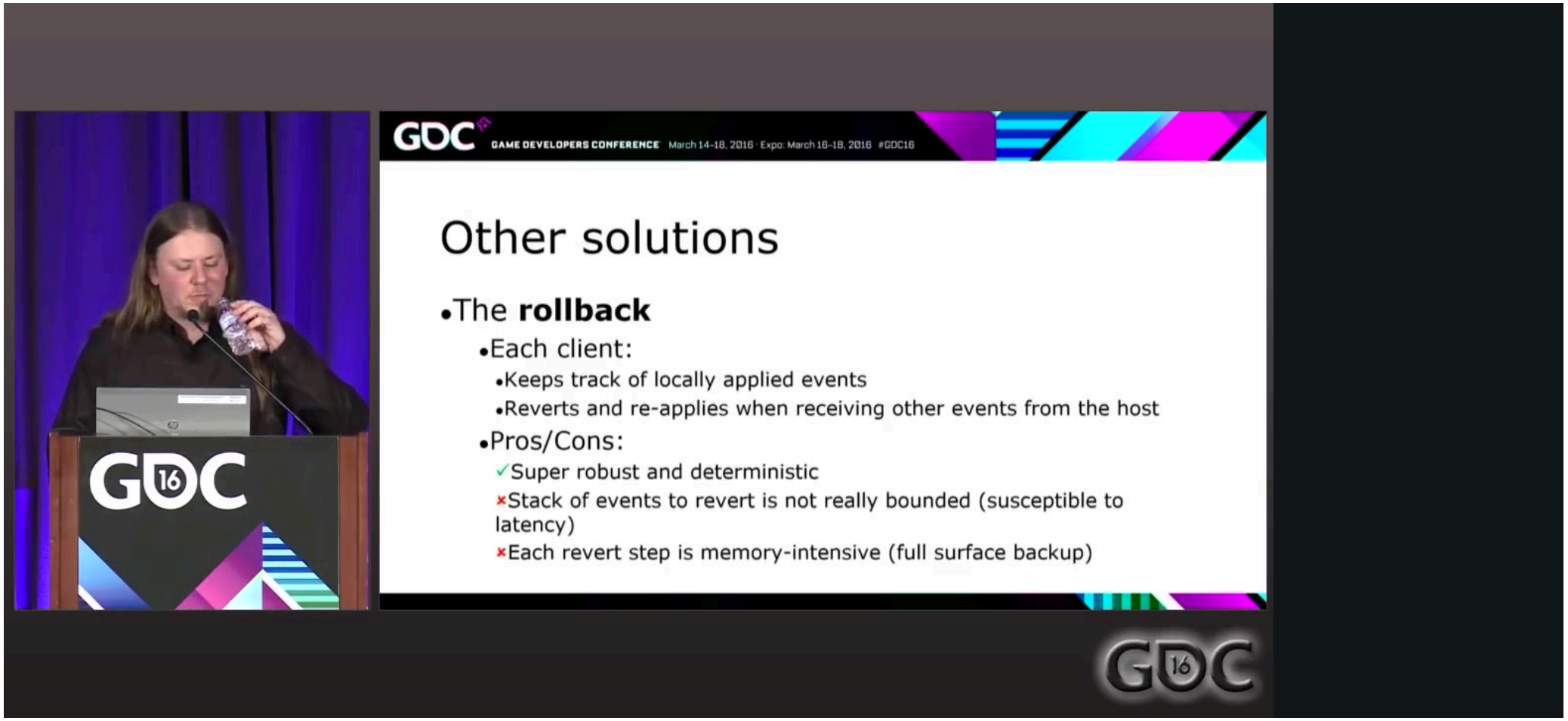
완벽하지는 않습니다. 발사자의 화면에 작은 차이가 남을 수도 있고, 구멍의 형태가 약간 달라질 가능성도 있습니다. 하지만 실제로는 큰 문제가 되지 않았습니다.

42:14–42:32

많은 테스트를 해 보니 정점의 개수와 위치가 거의 동일하게 나왔습니다. 미세한 차이가 있더라도 무시할 수 있는 수준이었습니다. 물론 다른 해결책도 검토했습니다.

100. Other Solutions: Rollback

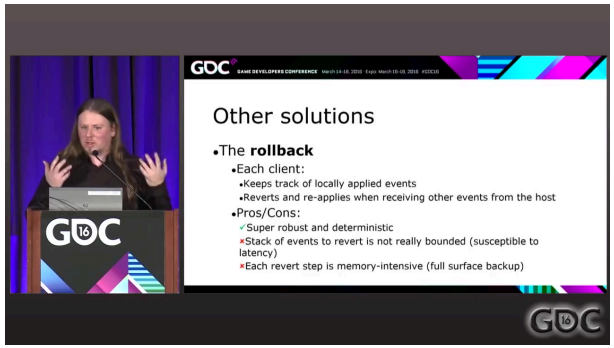
영상 42:25–43:35 · 장면 캡처 42:27



이 장면의 한국어 자막

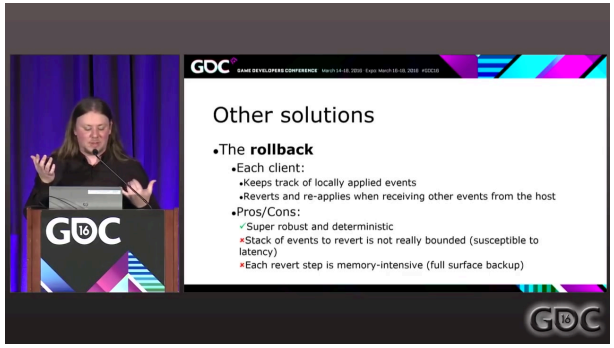
42:14–42:32

많은 테스트를 해 보니 정점의 개수와 위치가 거의 동일하게 나왔습니다. 미세한 차이가 있더라도 무시할 수 있는 수준이었습니다. 물론 다른 해결책도 검토했습니다.



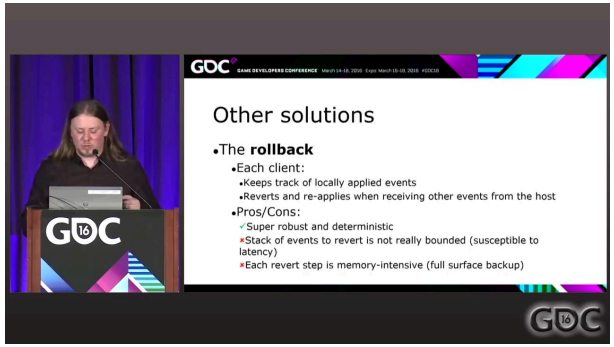
42:32–42:49

그중 하나는 롤백입니다. 상당히 견고한 해법이 될 수 있습니다. 클라이언트가 로컬에서 수행한 동작을 기록하고, 호스트의 메시지가 도착하면 그 기록과 비교합니다.



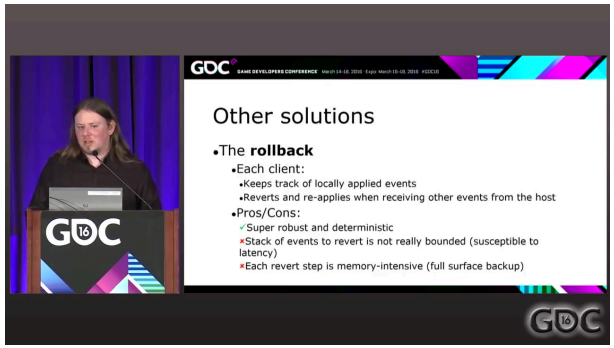
42:49–43:05

호스트 메시지가 로컬에서 수행한 작업과 다르면 로컬 변경을 되돌리고, 호스트 메시지를 적용한 다음 로컬 입력을 다시 실행합니다. 자신이 보낸 입력이 확인되면 기록에서 제거하면 되죠.



43:05–43:21

논리적으로는 훌륭한 방법입니다. 지연이 작다면 잘 작동할 겁니다. 하지만 네트워크 지연이 길면 기록이 아주 많이 쌓이고, 되돌렸다가 재실행하는 비용도 커집니다.



43:21–43:35

그 과정에서 낭비되는 계산량이 문제가 됩니다. 그래서 실제로는 사용하지 않았지만 향후 LAN 환경에서는 고려할 수 있을 겁니다. 제가 말이 너무 빠르죠? 시간이 거의 다 됐네요.

101. Future Development & Conclusion

영상 43:35–43:40 · 장면 캡처 43:37

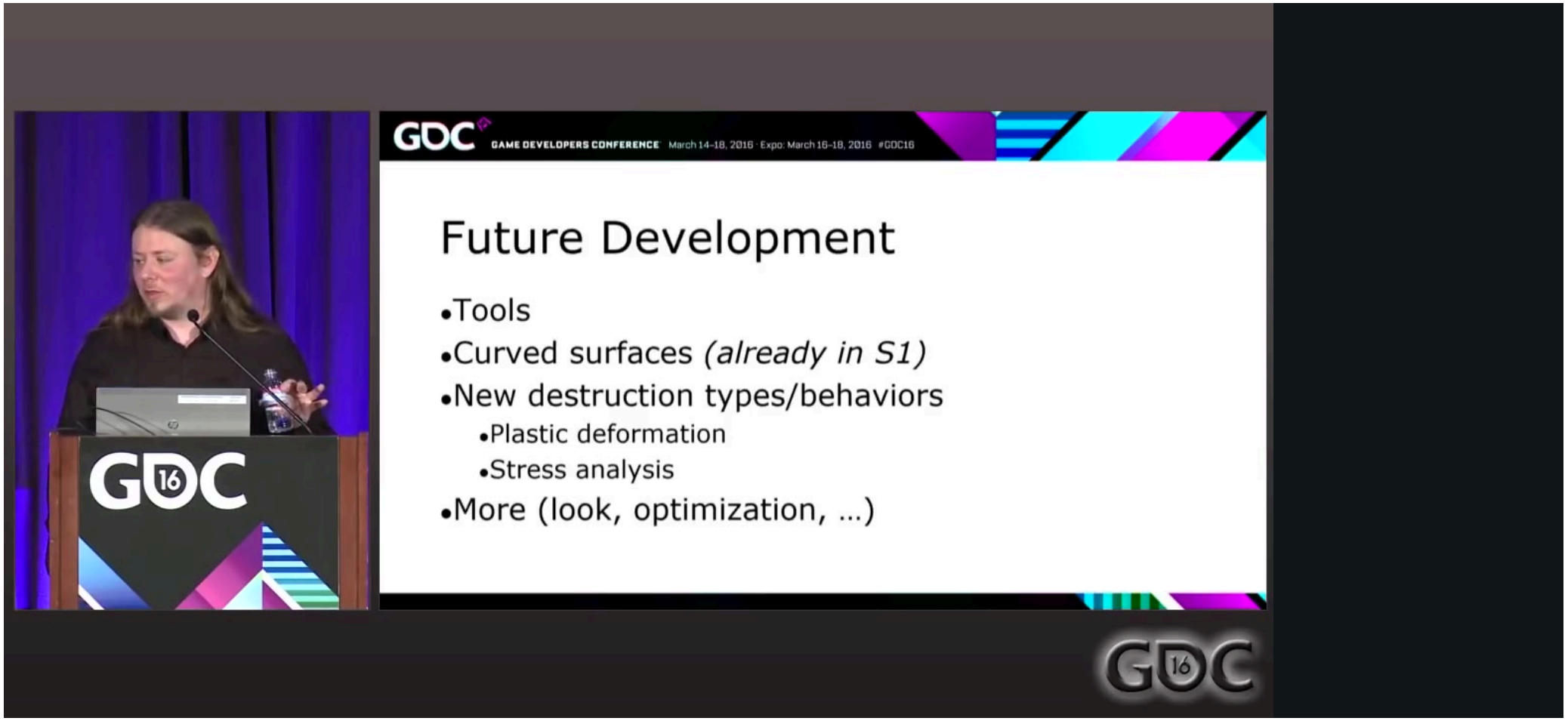
이 장면의 한국어 자막

43:35–43:52

미래 계획을 이야기하겠습니다. 도구 얘기는 잠깐 했지만, 아티스트가 더 쉽고 재미있게 작업할 수 있도록 계속 개발 중입니다. 입력마다 결과가 달라질 수 있는데,

102. Future Development

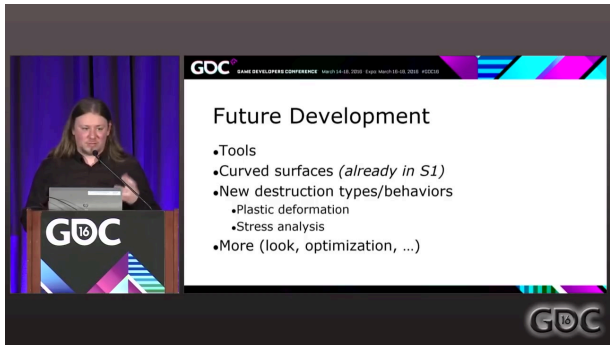
영상 43:40–44:30 · 장면 캡처 43:42



이 장면의 한국어 자막

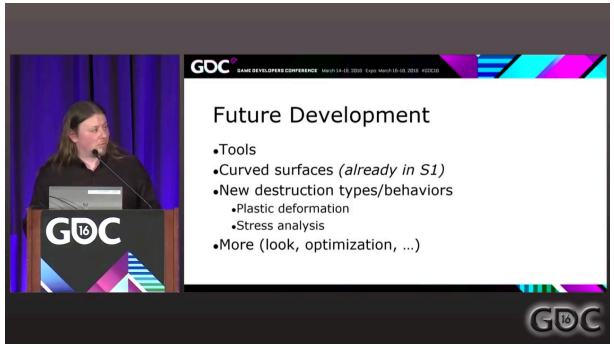
43:35–43:52

미래 계획을 이야기하겠습니다. 도구 얘기는 잠깐 했지만, 아티스트가 더 쉽고 재미있게 작업할 수 있도록 계속 개발 중입니다. 입력마다 결과가 달라질 수 있는데,



43:52–44:08

이를 아티스트에게 어떻게 명확하게 보여 주고 편집하게 할지가 큰 과제입니다. 또 곡면으로 표면 파괴 개념을 확장해 휘어진 벽도 지원하도록 개발했습니다.



44:08–44:26

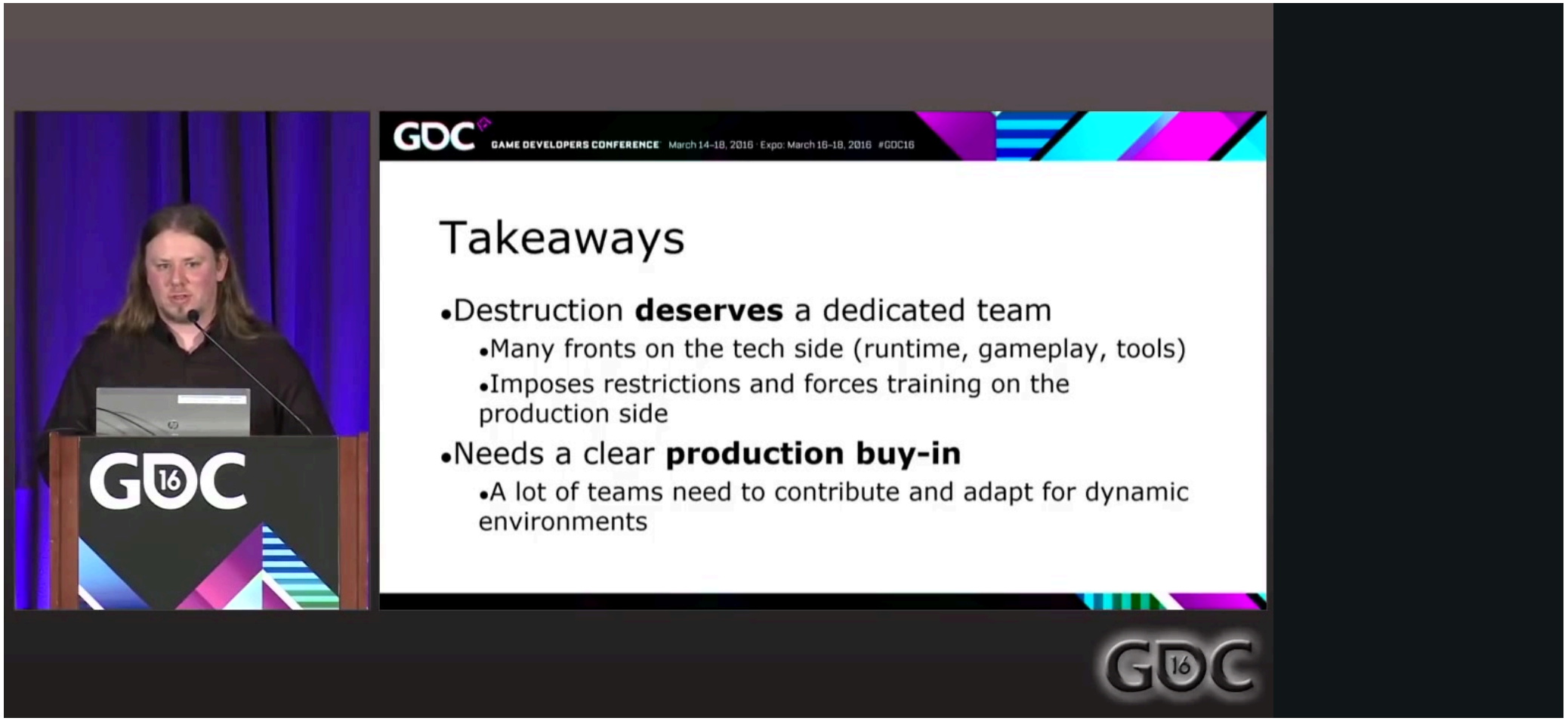
이 기능은 시즈 시즌 1에 이미 출시했습니다. 앞으로는 소성 변형, 응력 해석, 완전한 3D 불리언 기법 등 다른 파괴 방식도 고려하고, 기존 기술도 꾸준히 최적화하고 외형을 개선할 예정입니다.

44:26–44:43

내용이 복잡하죠? 핵심 교훈을 정리하겠습니다. 파괴 기술을 개발하려면 꼭 전담 팀을 두세요. 런타임 파괴 계산뿐 아니라 해결해야 할 기술적 문제가 매우 많습니다.

103. Takeaways: 팀과 제작 지원

영상 44:30–45:15 · 장면 캡처 44:32



이 장면의 한국어 자막

44:26–44:43

내용이 복잡하죠? 핵심 교훈을 정리하겠습니다. 파괴 기술을 개발하려면 꼭 전담 팀을 두세요. 런타임 파괴 계산뿐 아니라 해결해야 할 기술적 문제가 매우 많습니다.

44:43–44:58

게임플레이와 다른 시스템에 미치는 영향, 제작 도구까지 생각하면 다양한 분야의 인력이 필요합니다. 아티스트에게 기술적인 조언과 교육도 제공해야 합니다.

44:58–45:14

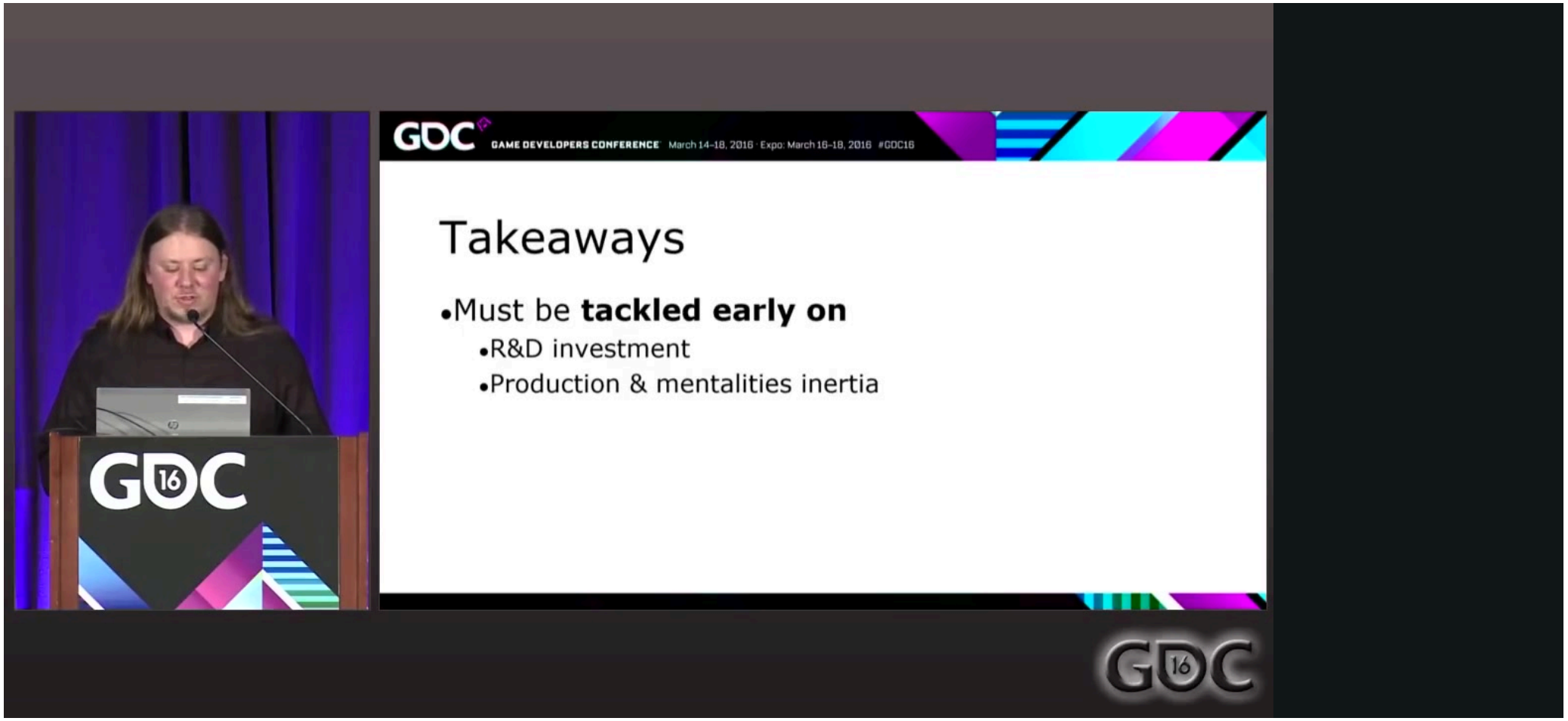
또 제작팀의 목표와 요구사항이 명확해야 합니다. 아무리 좋은 기술을 만들어도 제작팀이 시간이나 예산이 없다고 하면 실제 게임에 들어가지 못합니다.

45:14–45:30

파괴 가능한 환경을 만들 생각이라면 개발 초기에 시작해야 합니다. 큰 투자가 필요하고, 기존 제작 방식이 쉽게 바뀌지 않기 때문입니다.

104. Takeaways: 초기에 시작하기

영상 45:15–45:30 · 장면 캡처 45:17



이 장면의 한국어 자막

45:14–45:30

파괴 가능한 환경을 만들 생각이라면 개발 초기에 시작해야 합니다. 큰 투자가 필요하고, 기존 제작 방식이 쉽게 바뀌지 않기 때문입니다.

105. Conclusion

영상 45:30–45:50 · 장면 캡처 45:32

이 장면의 한국어 자막

45:30–45:45

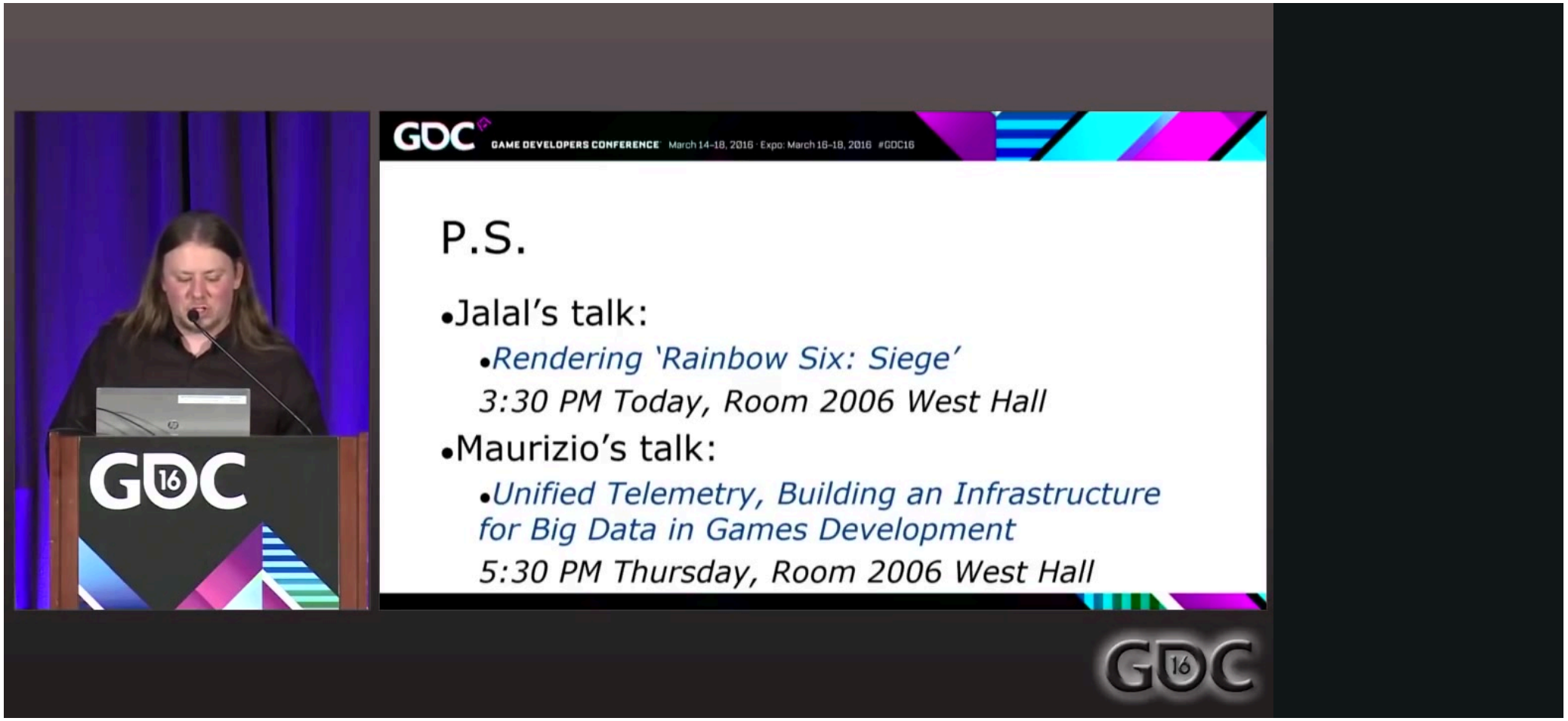
마지막으로 파괴 가능한 환경과 살아 있는 동적 환경은 앞으로도 계속 중요해질 겁니다. 완전히 정적인 세계로 되돌아가진 않을 테니까요. 과감히 받아들이세요. 파괴는 정말 멋지기도 하고요!

45:45–46:02

혁신과 변화를 지지하고, 어려워도 도전해 보세요. 내일 진행되는 다른 발표도 기억해 주시고요. 질문이나 연락은 유비소프트 이메일로 보내 주시면 됩니다.

106. P.S.: 관련 발표

영상 45:50–46:00 · 장면 캡처 45:52



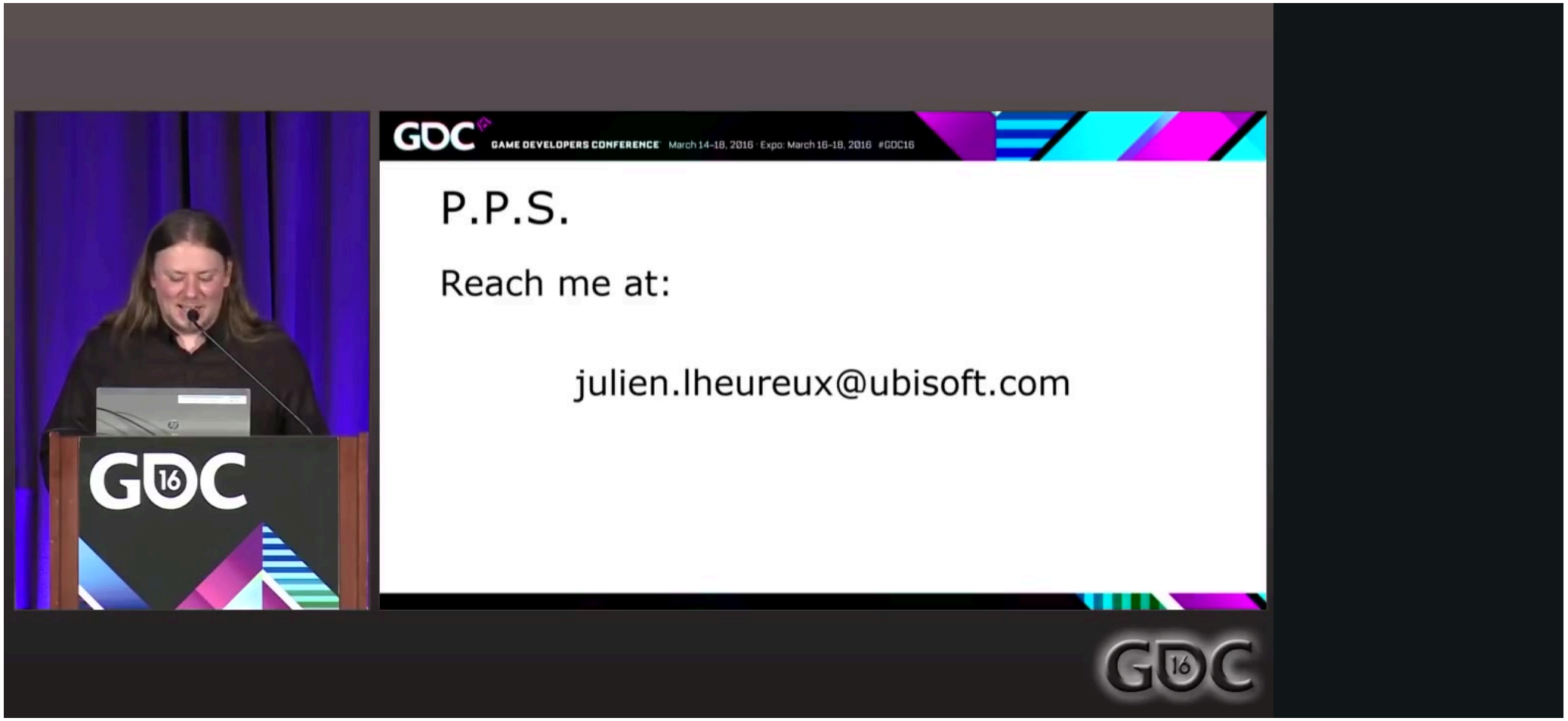
이 장면의 한국어 자막

45:45–46:02

혁신과 변화를 지지하고, 어려워도 도전해 보세요. 내일 진행되는 다른 발표도 기억해 주시고요. 질문이나 연락은 유비소프트 이메일로 보내 주시면 됩니다.

107. P.P.S.: 연락처

영상 46:00–46:10 · 장면 캡처 46:02



이 장면의 한국어 자막

45:45–46:02

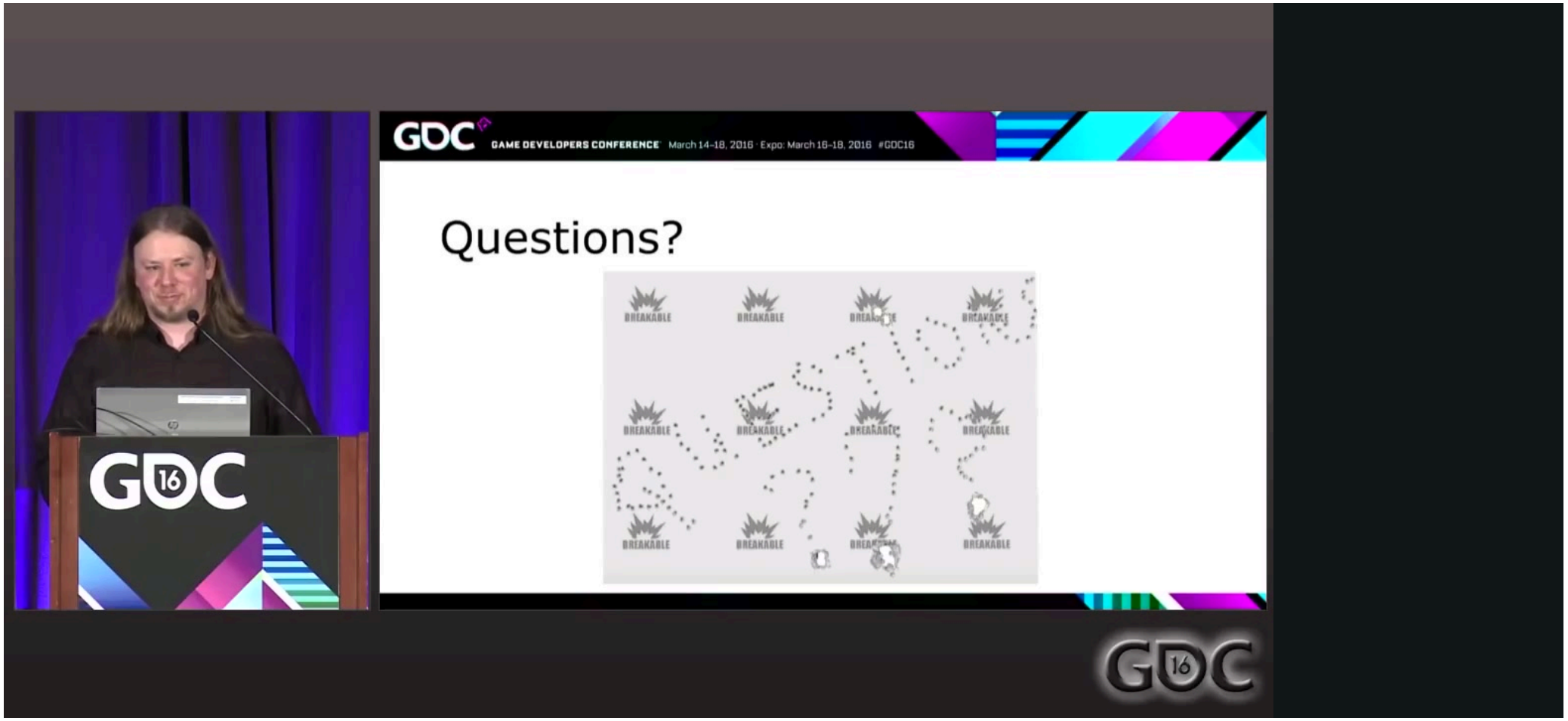
혁신과 변화를 지지하고, 어려워도 도전해 보세요. 내일 진행되는 다른 발표도 기억해 주시고요. 질문이나 연락은 유비소프트 이메일로 보내 주시면 됩니다.

46:02–46:13

이메일이든 사랑의 편지든 환영합니다. 질문 있으신가요?

108. Questions?

영상 46:10–46:25 · 장면 캡처 46:12



이 장면의 한국어 자막

46:02–46:13

이메일이든 사랑의 편지든 환영합니다. 질문 있으신가요?

109. 질의응답

영상 46:25–47:45 · 장면 캡처 46:27

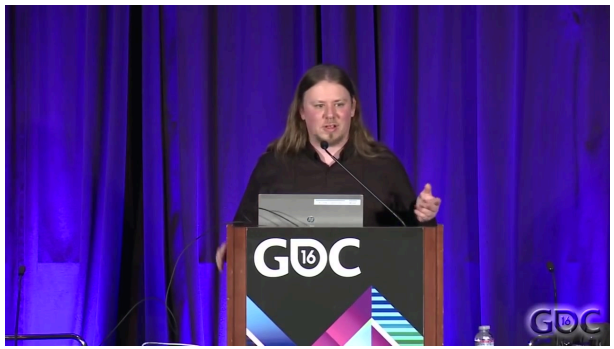
이 장면의 한국어 자막

46:30–46:44

[청중 질문] 희소 복셀은 어떻게 생각하시나요? 발표하신 기술과 비교하면 성능과 구현 난이도는 어떤가요? [답변] 아주 흥미로운 질문입니다. 복셀은 분명 훌륭한 기술입니다.

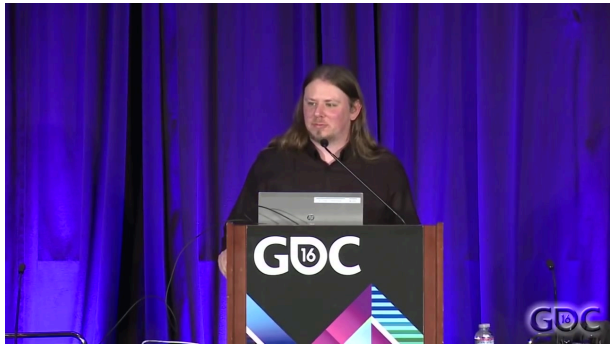
46:44–47:01

하지만 원하는 시각적 결과를 정확히 얻지 못할 수도 있습니다. 무엇보다 기존 게임 엔진을 복셀 기반으로 바꾸는 것은 아주 큰 변화입니다.



47:01–47:16

이미 게임을 개발 중인 팀에 복셀을 도입하려면 상당한 저항과 부담을 넘어야 합니다. 특히 규모가 큰 회사에서는 더욱 그렇습니다. 반면 기존 지오메트리나 메시를 사용하면,



47:16–47:23

새로운 기술을 받아들이는 부담이 훨씬 작고 도입하기도 쉽습니다.

47:27–47:41

저는 잠시 더 여기 있을 예정이고, 오후 4시에 유비소프트 부스에서 저와 알렉시스를 만나셔도 됩니다. 도구에 관해 이야기하고 질문에 답하겠습니다. 감사합니다.

110. 영상 마무리

영상 47:45–47:53 · 장면 캡처 47:47

이 장면의 한국어 자막

제공된 TXT에 이 구간의 자막이 없습니다.

원본 영상: https://www.youtube.com/watch?v=SjkQxowsL0I